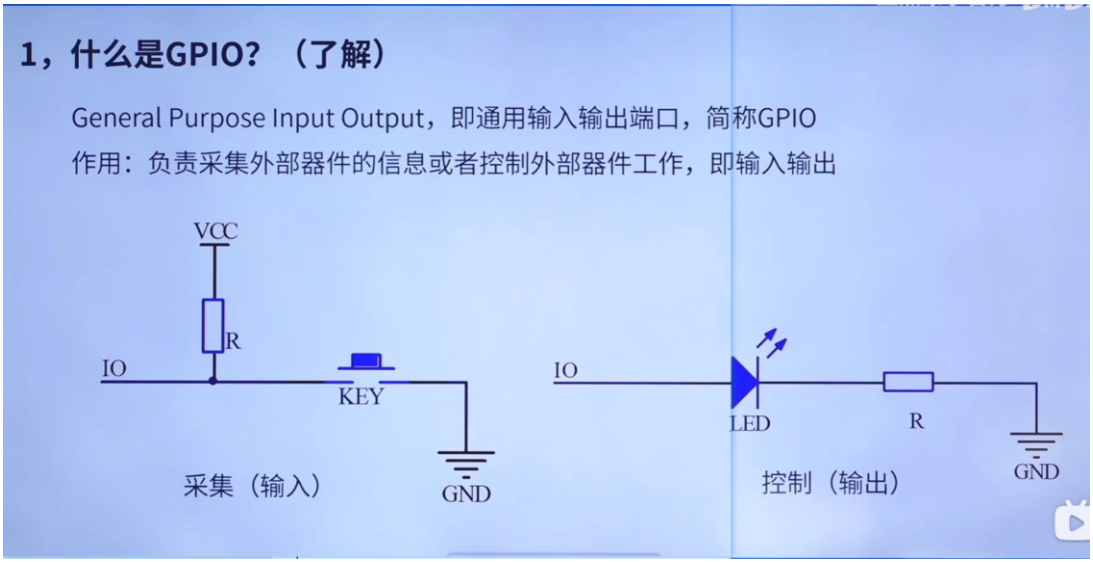
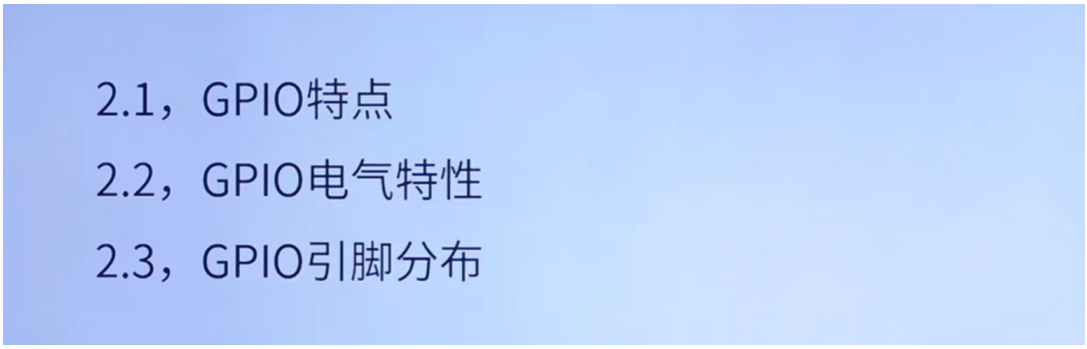
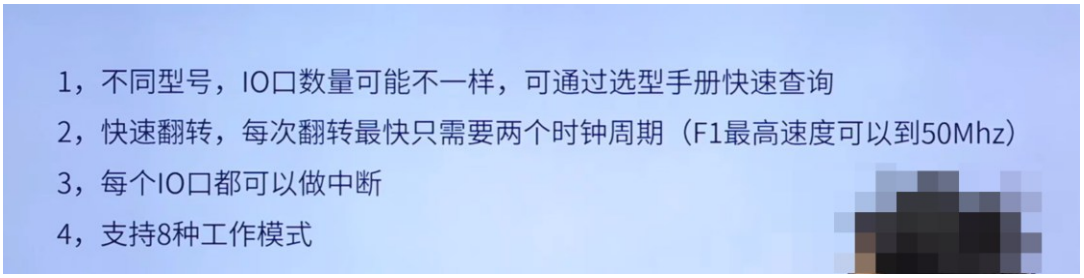
1 GPIO



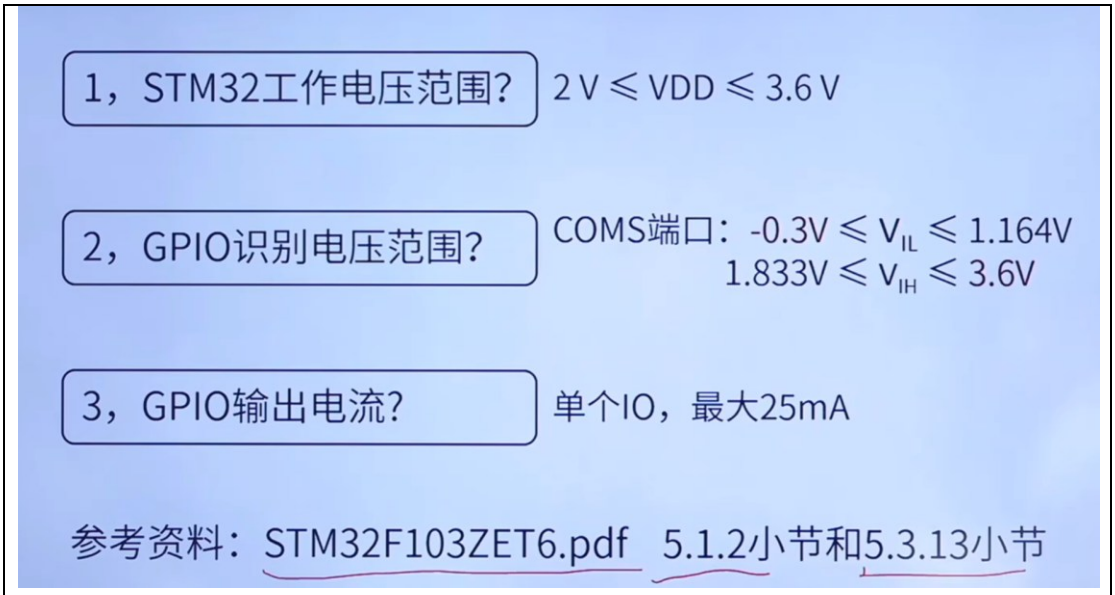
1.1 STM32 GPIO 简介



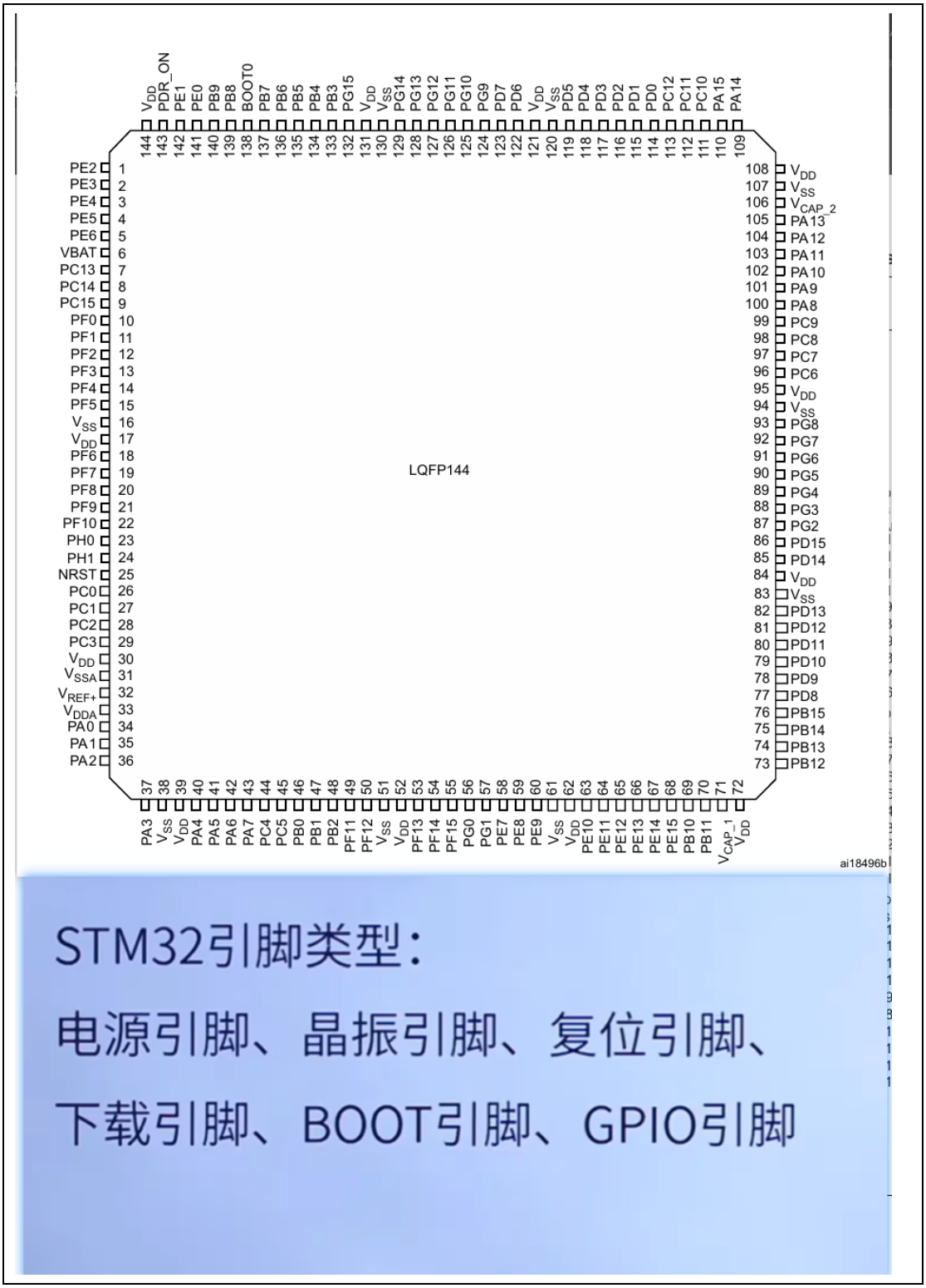
1.1.1 GPIO 特点



1.1.2 GPIO 电气特性



1.1.3 GPIO 引脚分布



1. 电源引脚 (Power Pins) —— “供电接口”

- **代表引脚：** `VDD`，`VSS`，`VDDA`，`VSSA`，`VBAT`。
- **作用：** 给芯片提供能量和参考地。
- **软件视角：** 通常不需要写代码控制，但需要在初始化时关注电源状态（比如检查 `PWR` 寄存器看电压是否稳定）。
- **注意：** `VDDA` 是给 ADC/DAC 供电的，如果它不稳，你采集的数据就会乱跳。

2. 晶振引脚 (Oscillator Pins) —— “心跳接口”

- **代表引脚：** `OSC_IN/OUT` (HSE), `OSC32_IN/OUT` (LSE)。
- **作用：** 连接外部石英晶体，为系统提供精准的时钟源。
- **软件视角：** 在 `SystemClock_Config()` 中配置。如果你用内部晶振（HSI/LSI），这些引脚可以释放出来当普通 GPIO 用；如果用外部晶振，它们就被“占用”了。

3. 复位引脚 (Reset Pin) —— “重启接口”

- **代表引脚：** `NRST`。
- **作用：** 低电平有效，把芯片恢复到初始状态。
- **软件视角：** 软件里可以通过 `NVIC_SystemReset()` 触发软复位，但硬件上这个引脚通常接一个按键，用于“死机时强制重启”。

4. 下载/调试引脚 (Debug/SWD Pins) —— “维护接口”

- **代表引脚：** `SWDIO` (PA13), `SWCLK` (PA14)。
- **作用：** 连接 ST-Link 或 J-Link，用于烧录程序和断点调试。
- **软件视角：** **极其重要！** 在 Keil 里点“Download”就是靠这两个引脚。如果你在代码里把它们强行配置成普通 GPIO 输出，下次你就连不上芯片了（俗称“锁死”），这时候就得靠按住复位键来抢救。

5. BOOT 引脚 (Boot Mode Pins) —— “启动选择接口”

- **代表引脚：** `BOOT0` (PA0), `BOOT1` (PB2)。
- **作用：** 决定芯片上电后从哪里开始运行程序（Flash、SRAM 还是 System Memory）。
- **软件视角：**
  - **正常模式：** `BOOT0 = 0`，从 Flash 启动（跑你写的程序）。
  - **串口下载模式：** `BOOT0 = 1`，进入内置 Bootloader，可以通过串口（USART1）下载程序。

6. GPIO 引脚 (General Purpose I/O) —— “业务接口”

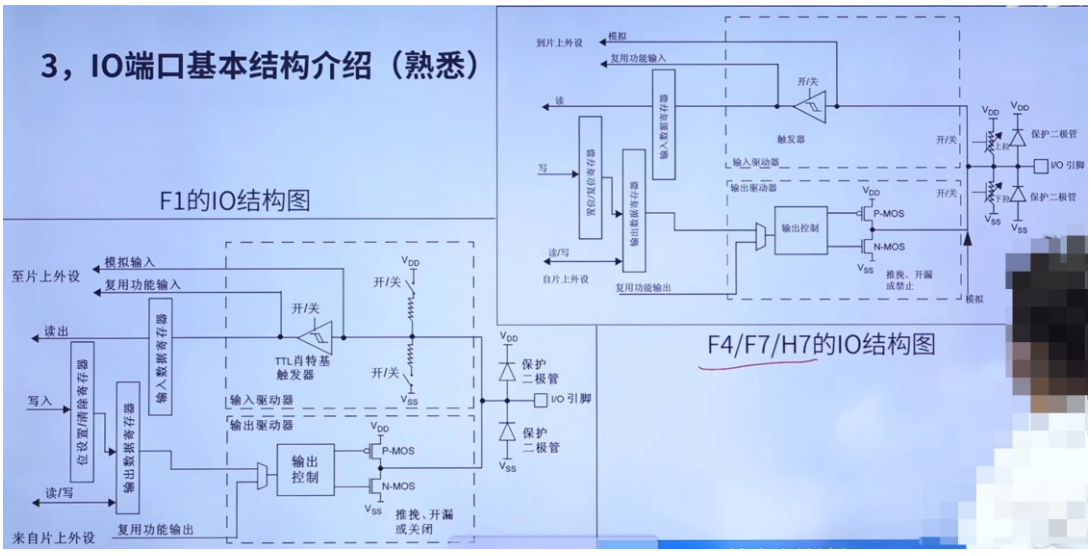
- **代表引脚：** `PA0-PA15`，`PB0-PB15` 等（除了上述特殊功能外的所有引脚）。
- **作用：** 最灵活的引脚，可以配置成输入、输出、复用功能（UART, SPI, I2C, PWM, ADC 等）。
- **软件视角：** 这是你写代码的主战场。90% 的业务逻辑（点灯、读传感器、发数据）都是通过这些引脚完成的。

不同芯片引脚分布情况

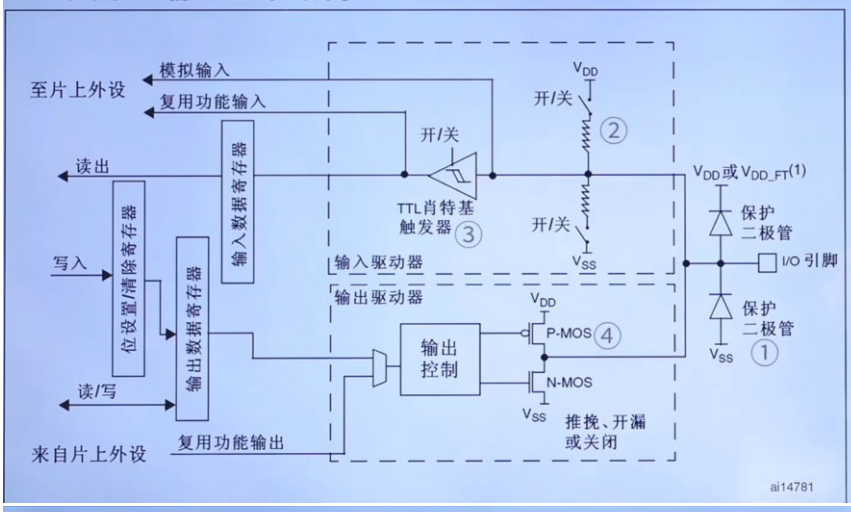
正点原子开发板	芯片型号	IO数	GPIO分组情况
Mini	STM32F103RCT6	51	GPIOA~GPIOC、GPIOD：PD0~PD2
精英	STM32F103ZET6	112	GPIOA~GPIOG
战舰	STM32F103ZET6	112	GPIOA~GPIOG
探索者	STM32F407ZGT6	114	GPIOA~GPIOG、GPIOH：PH0~PH1
阿波罗F429	STM32F429IGT6	140	GPIOA~GPIOH、GPIOI：PI0~PI11
阿波罗F767	STM32F767IGT6	140	GPIOA~GPIOH、GPIOI：PI0~PI11
阿波罗H743	STM32H743IIT6	140	GPIOA~GPIOH、GPIOI：PI0~PI11
北极星F750	STM32F750N8H6	168	GPIOA~GPIOJ、GPIOK：PK0~PK11
北极星H750	STM32H750XBH6	168	GPIOA~GPIOJ、GPIOK：PK0~PK11
MINI PRO H750	STM32H750VBT6	82	GPIOA~GPIOE、GPIOH：PH0~PH1

IO引脚分布特点：按组存在、组数视芯片而定、每组最多16个IO引脚

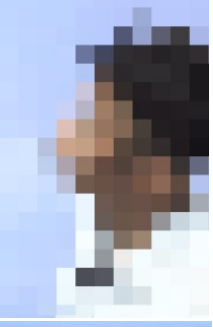
1.2 IO 端口基本结构介绍



F1系列IO端口基本结构



① 保护二极管
② 内部上拉、下拉电阻
③ 施密特触发器
④ P-MOS & N-MOS管



施密特触发器简介

施密特触发器就是一种整形电路，可以将非标准方波，整形成方波

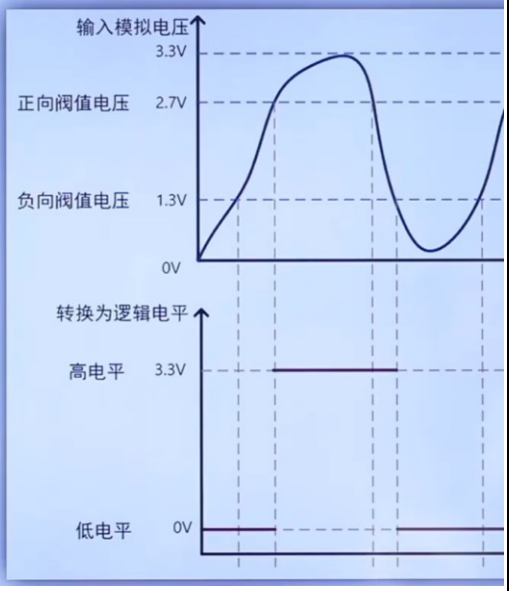
特点：

当输入电压高于正向阈值电压，输出为高；

当输入电压低于负向阈值电压，输出为低；

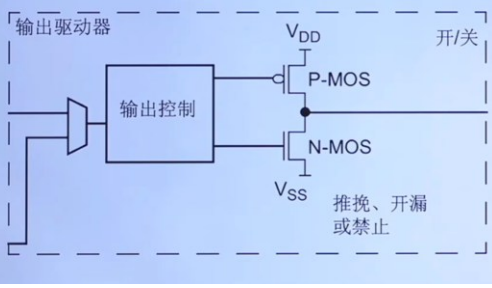
当输入在正负向阈值电压之间，输出不改变。

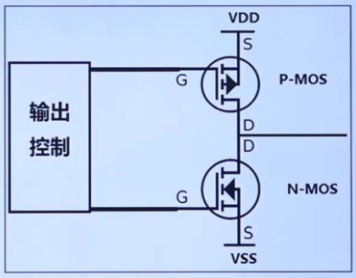
作用：整形！如正弦波转方波



P-MOS & N-MOS管简介

MOS管是压控型元件，通过控制栅源电压（Vgs）来实现导通或关闭。





G：栅极
S：源极
D：漏极

P：Vgs<0，导通
N：Vgs>0，导通

1.3 GPIO 的八种模式分析

GPIO八种模式	特点及应用
输入浮空	输入用，完全浮空，状态不定
输入上拉	输入用，用内部上拉，默认是高电平
输入下拉	输入用，用内部下拉，默认是低电平
模拟功能	ADC、DAC
开漏输出	软件IIC的SDL、SCL等
推挽输出	驱动能力强，25mA（max），通用输出
开漏式复用功能	片上外设功能（硬件IIC 的SDL、SCL引脚等）
推挽式复用功能	片上外设功能（SPI 的SCK、MISO、MOSI引脚等）

一、 核心硬件结构：GPIO 的“内脏”

STM32 的每个 GPIO 引脚内部主要包含以下模块：

1. **保护二极管**：防止电压过高（>VDD+0.7V）或过低（<VSS-0.7V）烧毁芯片。
2. **上下拉电阻**：可选的内部电阻（通常 40kΩ 左右），用于提供默认电平。
3. **施密特触发器 (Schmitt Trigger)**：将模拟电压信号整形成数字信号（0 或 1），具有滞回特性，抗干扰。
4. **输出驱动器**：由一对互补的 **P-MOS** 和 **N-MOS** 管组成。
5. **复用功能选择器**：决定信号是来自普通 GPIO 寄存器，还是来自片上外设（如 UART, TIM）。

二、 输入模式原理 (Input Modes)

输入模式的核心在于：**外部信号如何进入 CPU，以及引脚在没有外部信号时的状态。**

1. 输入浮空 (Floating Input)

- **原理**：
  - **上下拉电阻断开**：内部开关 S1 和 S2 都断开，引脚处于高阻抗状态（High-Z）。
  - **施密特触发器工作**：外部电压直接进入触发器，转换为数字信号存入输入数据寄存器（`IDR`）。
- **现象**：如果外部悬空，引脚电平会随周围电磁环境随机跳动（噪声）。
- **Java 类比**：`Integer value = null;` 此时读取 `value` 会抛出异常或得到不可预测的结果。
- **用途**：标准通信协议（如 I2C 的数据线在空闲时）、↓ ；采集前的预处理。

2. 输入上拉 (Pull-Up Input)

- **原理**：
  - **上拉电阻接通**：内部开关 S1 闭合，将引脚通过电阻连接到 VDD (3.3V)。
  - **下拉电阻断开**：S2 断开。
- **现象**：当外部没有信号（悬空）时，电流从 VDD 流过电阻，引脚被“拉”到高电平 (1)。只有当外部强行接地时，引脚才变低。
- **Java 类比**：`boolean flag = true;` 除非外部显式调用 `setFlag(false)`，否则默认为真。
- **用途**：按键检测（按键一端接地，按下即为低电平）。

3. 输入下拉 (Pull-Down Input)

- **原理**：
  - **下拉电阻接通**：内部开关 S2 闭合，将引脚通过电阻连接到 VSS (GND)。
  - **上拉电阻断开**：S1 断开。
- **现象**：当外部悬空时，引脚被“拉”到低电平 (0)。只有当外部接高电平时，引脚才变高。
- **Java 类比**：`boolean flag = false;` 除非外部显式调用 `setFlag(true)`，否则默认为假。
- **用途**：某些传感器输出、使能信号（Enable）。

三、 输出模式原理 (Output Modes)

输出模式的核心在于：**CPU 如何驱动外部负载。** 关键在于 **P-MOS** 和 **N-MOS** 这两个晶体管的配合。

- **P-MOS**：栅极给低电平导通，连接 VDD（输出高电平）。
- **N-MOS**：栅极给高电平导通，连接 VSS（输出低电平）。

4. 推挽输出 (Push-Pull Output)

- **原理**：
  - **输出 1 (High)**：P-MOS 导通，N-MOS 关闭。电流从 VDD **推 (Push)** 向负载。
  - **输出 0 (Low)**：N-MOS 导通，P-MOS 关闭。电流从负载 **挽 (Pull)** 入 VSS。
- **特点**：无论高低电平，都有很强的驱动能力（通常可达 20mA）。开关速度快，因为不存在电容充电的“软启动”过程。
- **Java 类比**：标准的 `return` 语句，强势返回结果，不依赖外部环境。
- **用途**：LED、蜂鸣器、SPI 时钟线、UART 发送端。

5. 开漏输出 (Open-Drain Output)

- **原理：**
  - **P-MOS 永远关闭**：内部切断了通往 VDD 的路径。
  - **输出 0 (Low)**：N-MOS 导通，引脚接地。
  - **输出 1 (High)**：N-MOS 关闭，引脚处于**高阻态 (Floating)**。此时引脚电平完全取决于**外部电路**。
- **关键点**：必须外接**上拉电阻**才能输出高电平。
- **Java 类比**：一个只能执行 `clear()` 操作的对象。如果想让它变"满"，得靠外部环境（上拉电阻）来填充。
- **用途：**
  - **电平转换**：STM32 (3.3V) 控制 5V 设备。开漏输出 + 5V 上拉，高电平时引脚就是 5V，不会烧坏 3.3V 的 P-MOS。
  - **"线与" (Wired-AND)**：多个开漏引脚连在一起，只要有一个拉低，总线就是低。用于 I2C。

四、 复用功能模式 (Alternate Function, AF)

当引脚被片上外设（如 UART, SPI, I2C, TIM）占用时，GPIO 的控制权从 `ODR` 寄存器移交给了外设模块。

6. 推挽复用 (AF Push-Pull)

- **原理**：外设信号直接驱动推挽输出级（P-MOS + N-MOS）。
- **现象**：外设产生的波形（如 PWM、UART 数据位）具有强驱动能力。
- **用途**：UART TX, SPI MOSI/SCK, TIM PWM 输出。

7. 开漏复用 (AF Open-Drain)

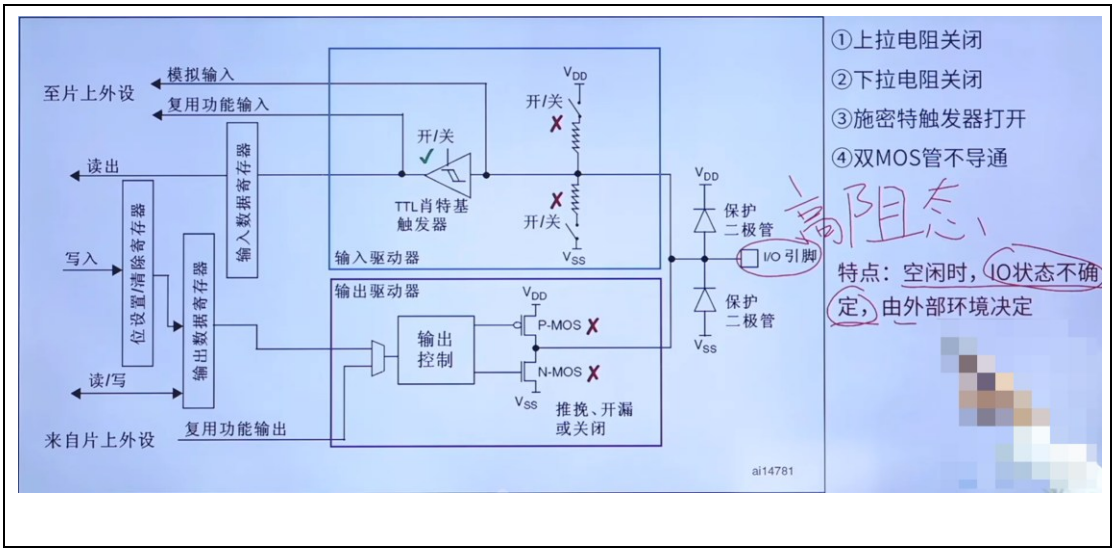
- **原理**：外设信号只驱动 N-MOS，P-MOS 禁用。
- **现象**：外设只能拉低电平，高电平靠外部上拉。
- **用途**：**I2C SDA/SCL**。这是 I2C 协议的物理层要求，支持多主仲裁和时钟同步。

五、 模拟模式原理 (Analog Mode)

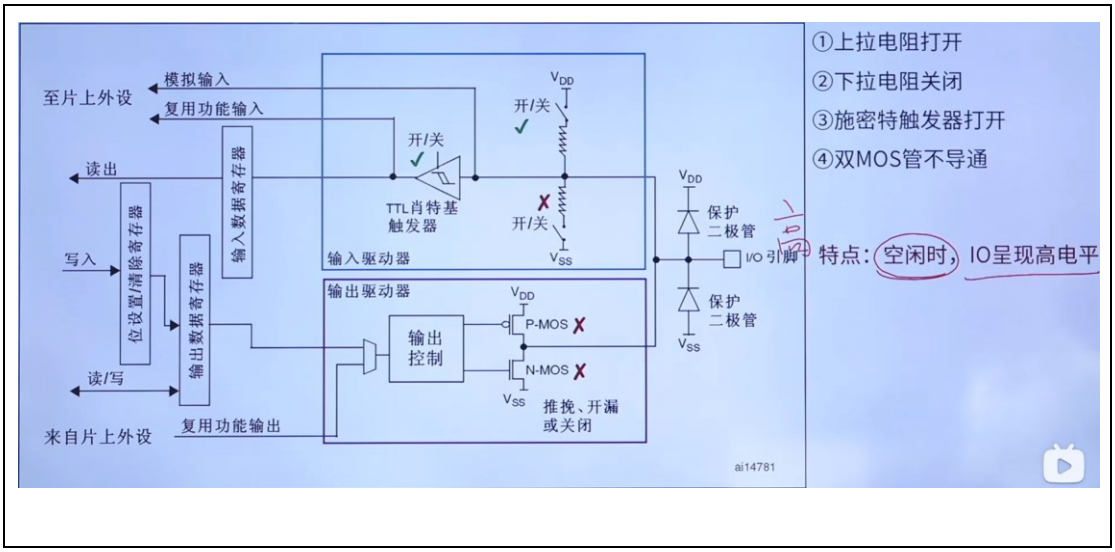
8. 模拟功能 (Analog)

- **原理：**
  - **断开数字通路**：关闭施密特触发器，防止模拟电压在阈值附近波动时产生大量的数字噪声电流。
  - **直通 ADC/DAC**：引脚直接连接到内部的 ADC（模数转换器）或 DAC（数模转换器）模块。
  - **高阻抗**：通常也表现为浮空状态，但目的是为了让外部电压无损耗地进入模拟前端。
- **Java 类比：** bypass 所有的业务逻辑校验，直接把原始字节流 (`byte[]`) 传给底层处理引擎。
- **用途**：采集电池电压、温度传感器、音频信号。

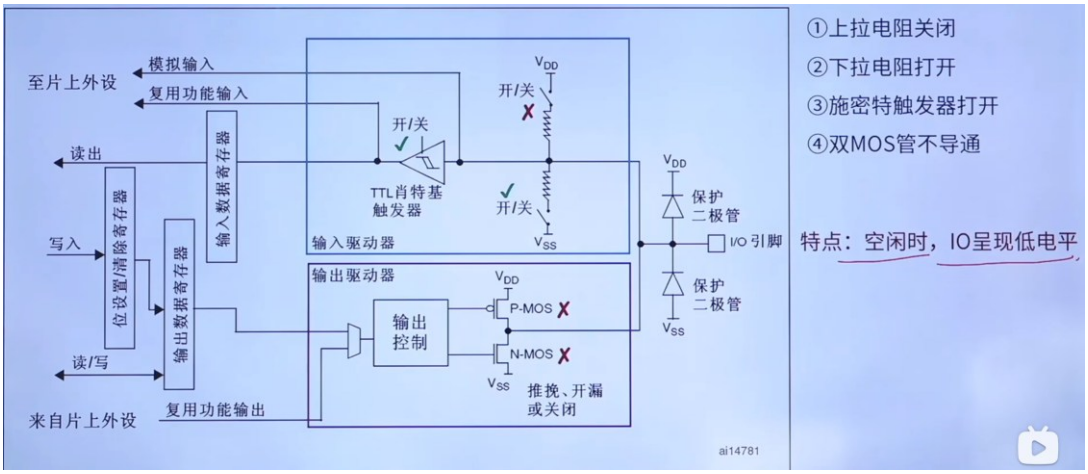
1.3.1 输入浮空



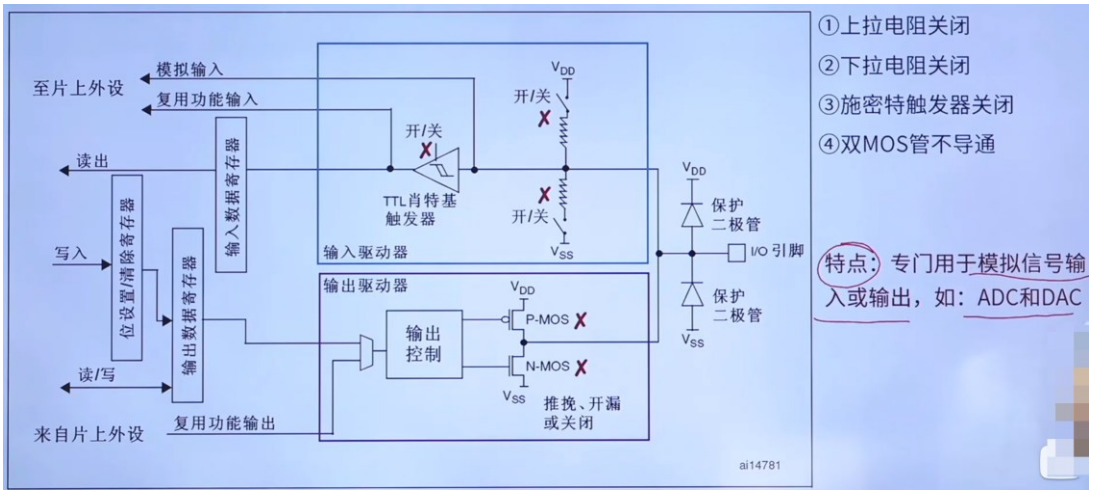
1.3.2 输入上拉



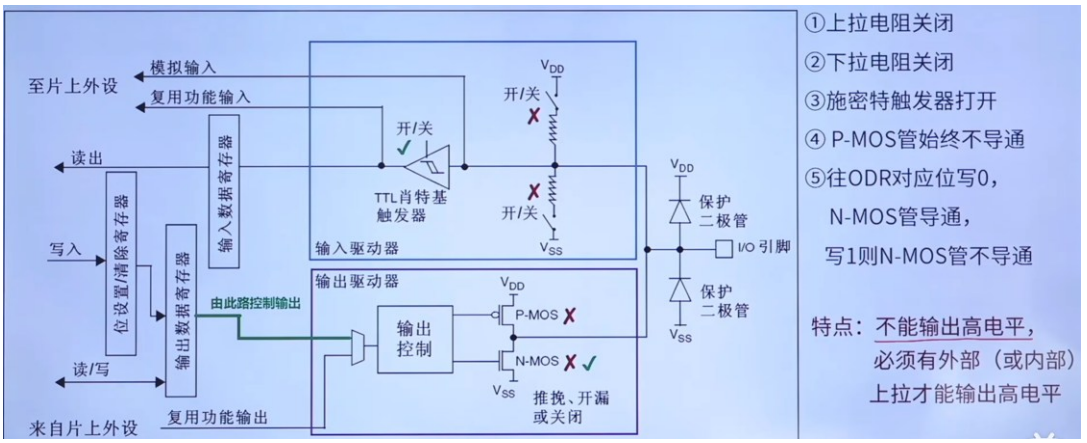
1.3.3 输入下拉



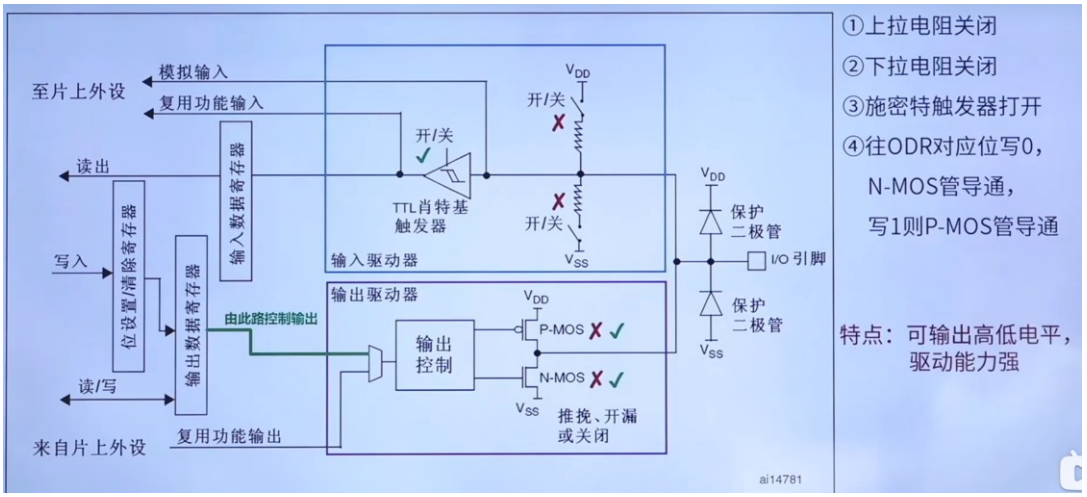
1.3.4 模拟功能



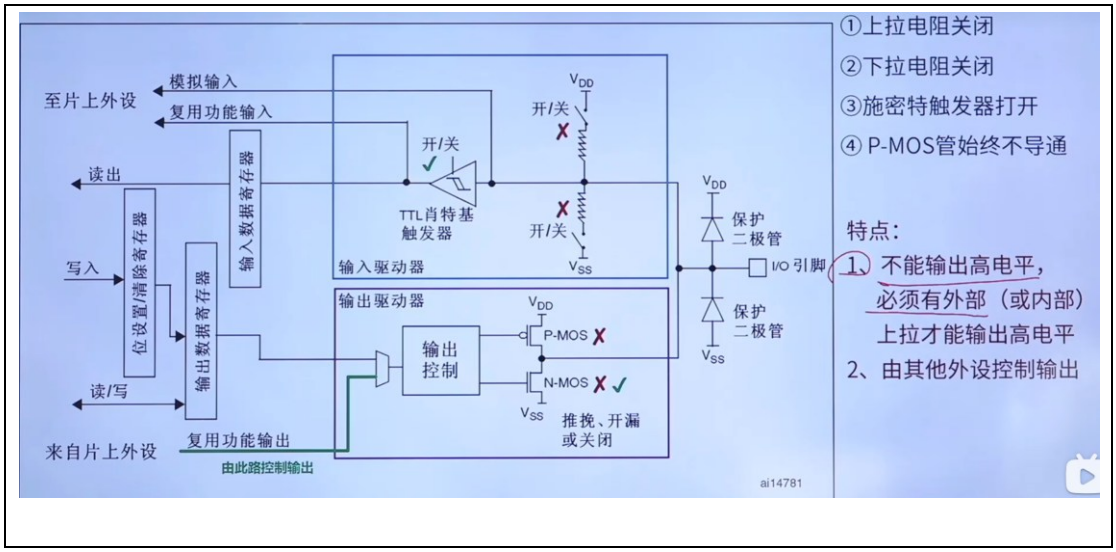
1.3.5 开漏输出



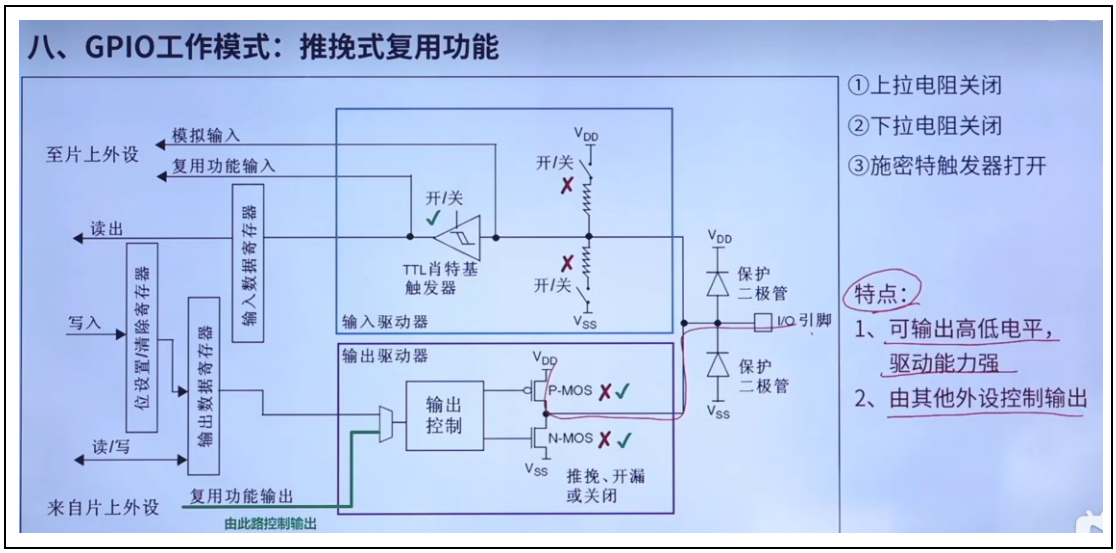
1.3.6 推挽输出



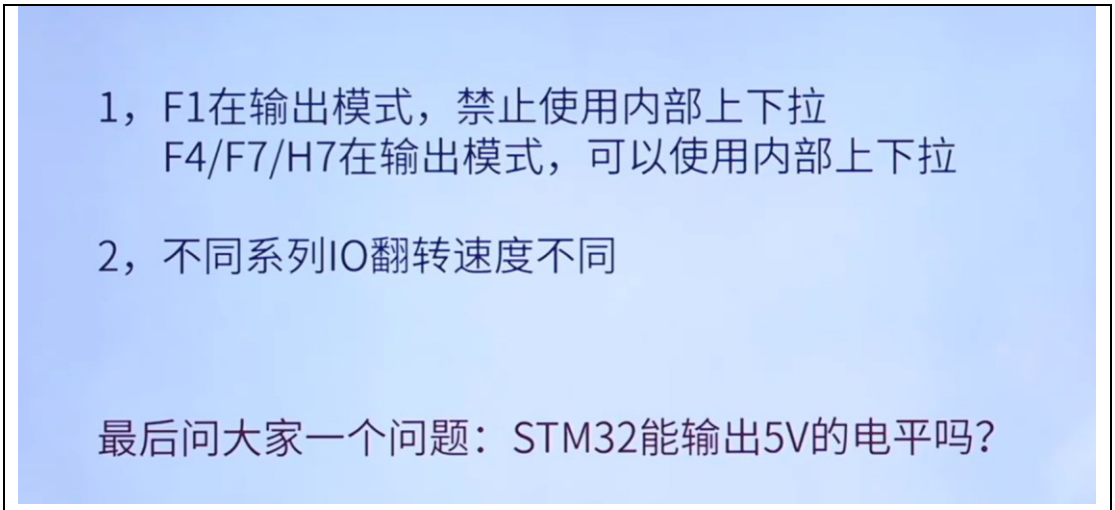
1.3.7 开漏式复用功能



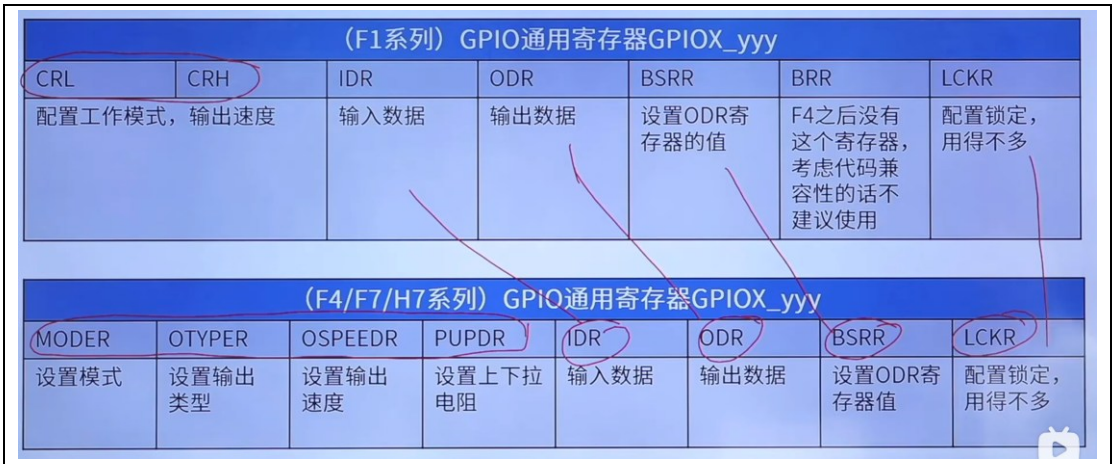
1.3.8 推挽式复用功能



1.3.9 F4/F7/H7 系列和 F1 系列的 GPIO 差异点



1.4 GPIO 寄存器介绍



（F1系列）GPIO通用寄存器GPIOX_yyy

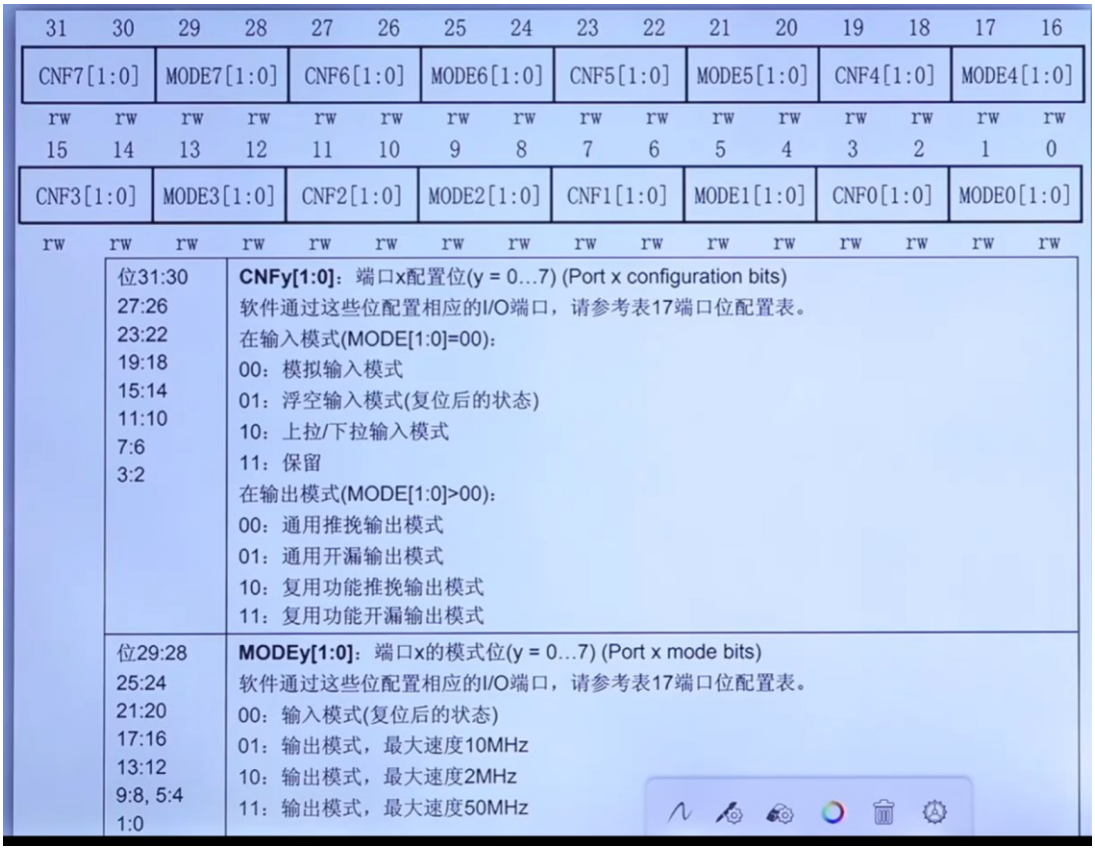
CRL	CRH	IDR	ODR	BSRR	BRR	LCKR
配置工作模式，输出速度		输入数据	输出数据	设置ODR寄存器的值	F4之后没有这个寄存器，考虑代码兼容性的话不建议使用	配置锁定，用得不多

（F4/F7/H7系列）GPIO通用寄存器GPIOX_yyy

MODER	OTYPER	OSPEEDR	PUPDR	IDR	ODR	BSRR	LCKR
设置模式	设置输出类型	设置输出速度	设置上下拉电阻	输入数据	输出数据	设置ODR寄存器值	配置锁定，用得不多

1.4.1 F1

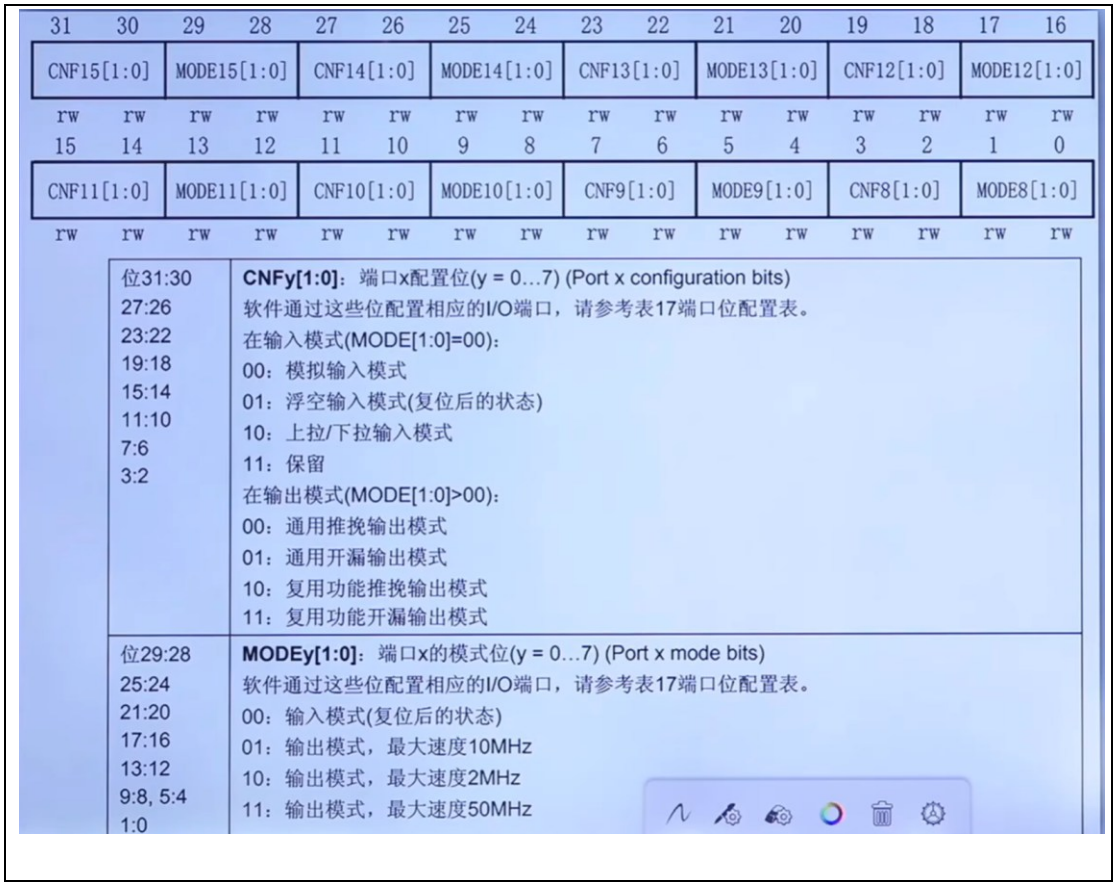
1.4.1.1 端口配置低寄存器(CRL)



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:30 27:26 23:22 19:18 15:14 11:10 7:6 3:2	**CNFy[1:0]**：端口x配置位(y = 0…7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 在输入模式(MODE[1:0]=00)： 00：模拟输入模式 01：浮空输入模式(复位后的状态) 10：上拉/下拉输入模式 11：保留 在输出模式(MODE[1:0]>00)： 00：通用推挽输出模式 01：通用开漏输出模式 10：复用功能推挽输出模式 11：复用功能开漏输出模式
位29:28 25:24 21:20 17:16 13:12 9:8, 5:4 1:0	**MODEy[1:0]**：端口x的模式位(y = 0…7) (Port x mode bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 00：输入模式(复位后的状态) 01：输出模式，最大速度10MHz 10：输出模式，最大速度2MHz 11：输出模式，最大速度50MHz

1.4.1.2 端口配置高寄存器(CRH)



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF15[1:0]		MODE15[1:0]		CNF14[1:0]		MODE14[1:0]		CNF13[1:0]		MODE13[1:0]		CNF12[1:0]		MODE12[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF11[1:0]		MODE11[1:0]		CNF10[1:0]		MODE10[1:0]		CNF9[1:0]		MODE9[1:0]		CNF8[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:30 27:26 23:22 19:18 15:14 11:10 7:6 3:2	**CNFy[1:0]**：端口x配置位(y = 0…7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 在输入模式(MODE[1:0]=00)： 00：模拟输入模式 01：浮空输入模式(复位后的状态) 10：上拉/下拉输入模式 11：保留 在输出模式(MODE[1:0]>00)： 00：通用推挽输出模式 01：通用开漏输出模式 10：复用功能推挽输出模式 11：复用功能开漏输出模式
位29:28 25:24 21:20 17:16 13:12 9:8, 5:4 1:0	**MODEy[1:0]**：端口x的模式位(y = 0…7) (Port x mode bits) 软件通过这些位配置相应的I/O端口，请参考表17端口位配置表。 00：输入模式(复位后的状态) 01：输出模式，最大速度10MHz 10：输出模式，最大速度2MHz 11：输出模式，最大速度50MHz

1.4.1.3 工作模式配置补充

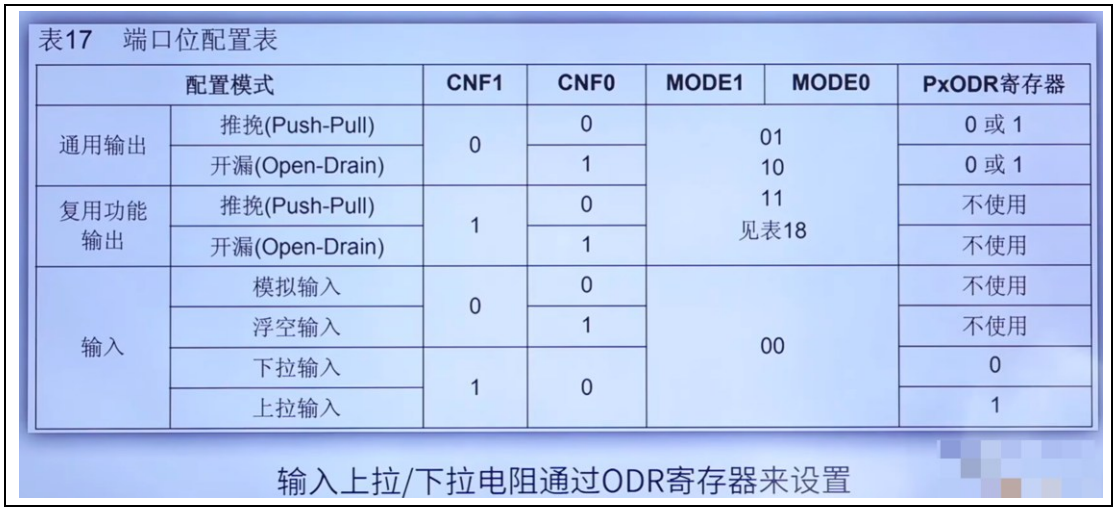
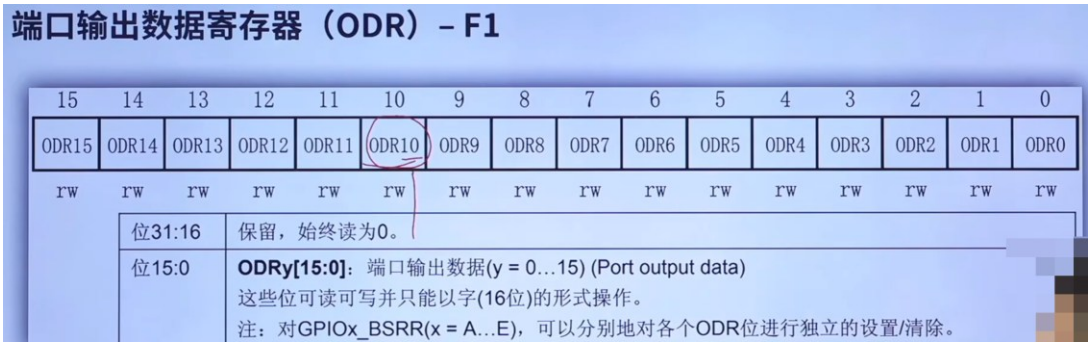


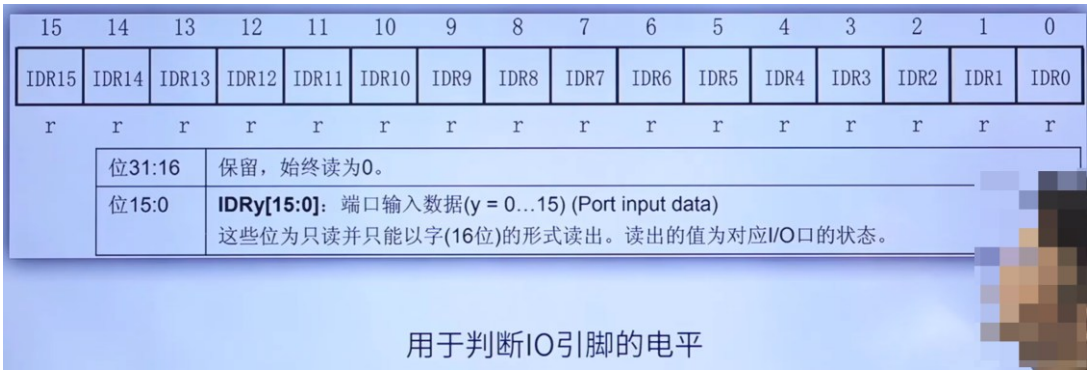
表17　端口位配置表

配置模式		CNF1	CNF0	MODE1	MODE0	PxODR寄存器
通用输出	推挽(Push-Pull)	0	0	01 10 11 见表18		0 或 1
	开漏(Open-Drain)		1			0 或 1
复用功能输出	推挽(Push-Pull)	1	0			不使用
	开漏(Open-Drain)		1			不使用
输入	模拟输入	0	0	00		不使用
	浮空输入		1			不使用
	下拉输入	1	0			0
	上拉输入					1

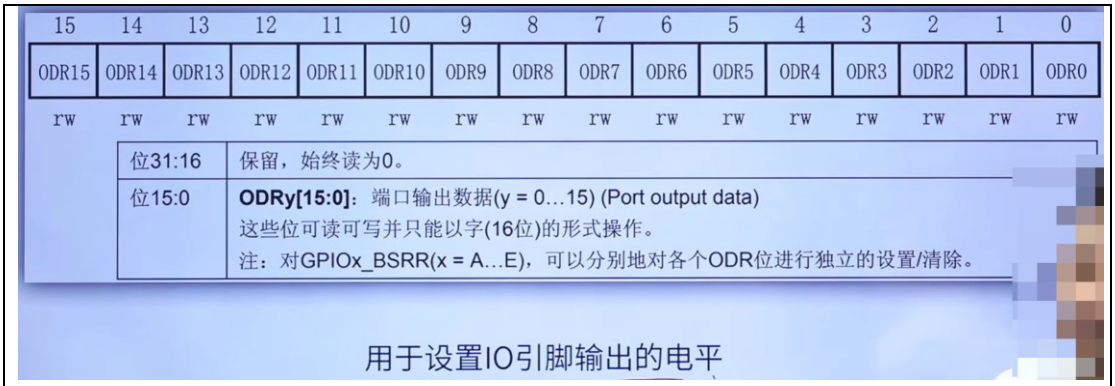
输入上拉/下拉电阻通过ODR寄存器来设置



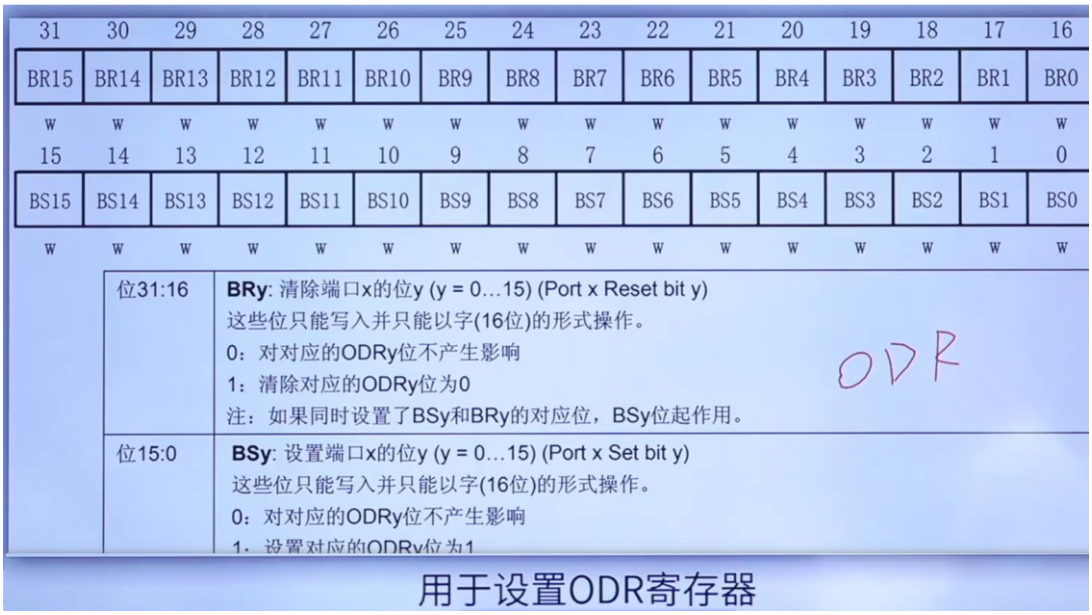
1.4.1.4 端口输入数据寄存器(IDR)



1.4.1.5 端口输出数据寄存器(ODR)



1.4.1.6 端口位设置/清除寄存器（BSRR）



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

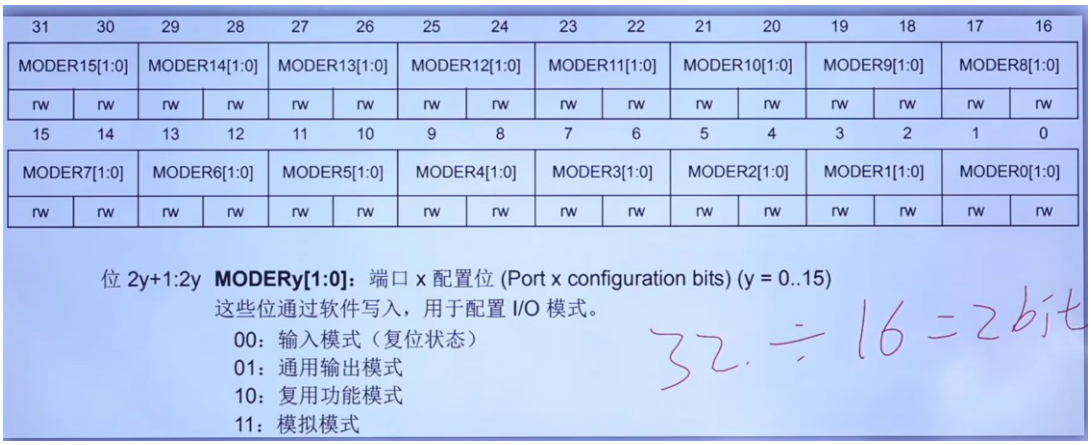
位31:16	**BRy**: 清除端口x的位y (y = 0…15) (Port x Reset bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：清除对应的ODRy位为0 注：如果同时设置了BSy和BRy的对应位，BSy位起作用。
位15:0	**BSy**: 设置端口x的位y (y = 0…15) (Port x Set bit y) 这些位只能写入并只能以字(16位)的形式操作。 0：对对应的ODRy位不产生影响 1：设置对应的ODRy位为1

ODR

用于设置ODR寄存器

1.4.2 F4/F7/H7

1.4.2.1 GPIO 端口模式寄存器（MODER）

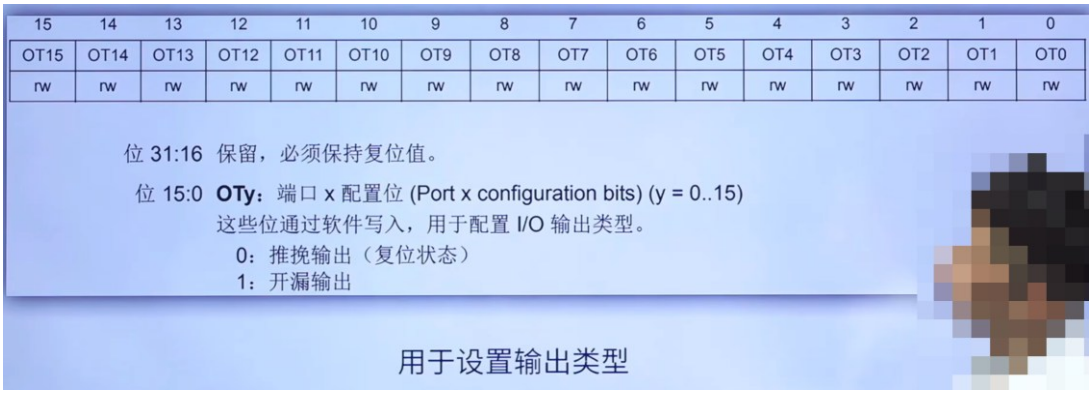


31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **MODERy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 模式。
00：输入模式（复位状态）
01：通用输出模式
10：复用功能模式
11：模拟模式

32÷16=2bit

1.4.2.2 GPIO 端口输出类型寄存器（OTYPER）



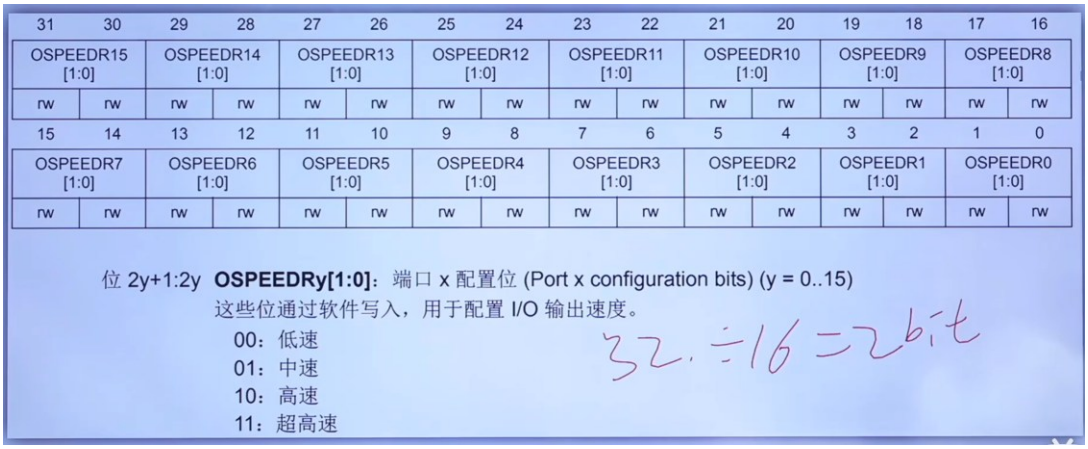
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **OTy**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 输出类型。
0：推挽输出（复位状态）
1：开漏输出

用于设置输出类型

1.4.2.3 GPIO 端口输出速度寄存器（OSPEEDR）



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **OSPEEDRy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 输出速度。
00：低速
01：中速
10：高速
11：超高速

32÷16=2bit

1.4.2.4 GPIO 端口上拉/下拉寄存器（PUPDR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **PUPDRy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 上拉或下拉。
00：无上拉或下拉
01：上拉
10：下拉
11：保留

用于设置上拉/下拉电阻

1.4.2.5 8 中工作模式对应的配置

GPIO 工作模式	模式寄存器 MODER[0:1]	输出类型寄存器 OTYPER	输出速度寄存器 OSPEEDR[0:1]	上拉/下拉寄存器 PUPDR[0:1]
输入浮空	00-输入模式	无效	无效	00-无上拉或下拉
输入上拉				01-上拉
输入下拉				10-下拉
模拟功能	11-模拟模式			00-无上拉或下拉
开漏输出	01-通用输出	1-开漏输出	00-低速	00-无上拉或下拉
推挽输出		0-推挽输出	01-中速	01-上拉
开漏式复用功能	10-复用功能	1-开漏输出	10-高速	10-下拉
推挽式复用功能		0-推挽输出	11-超高速	11-保留

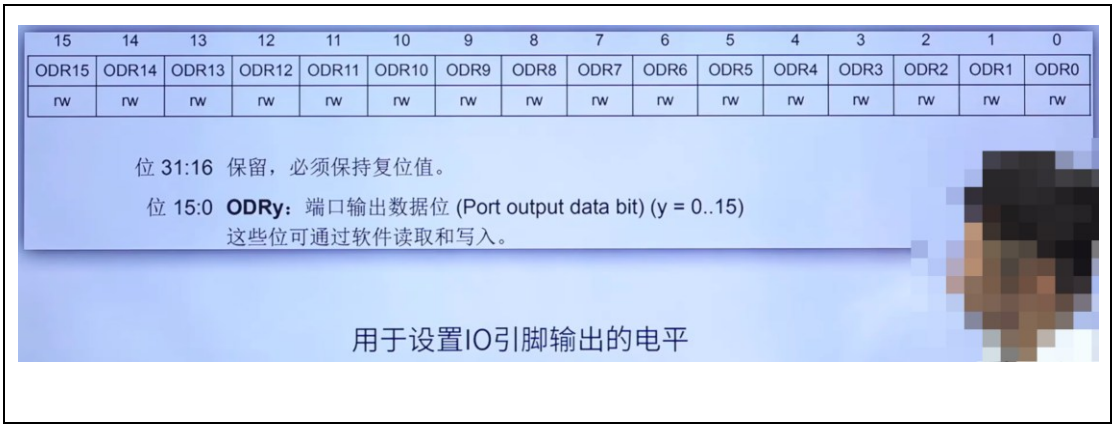
1.4.2.6 GPIO 端口输入数据寄存器（IDR）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 31:16 保留，必须保持复位值。
位 15:0 **IDRy**：端口输入数据位 (Port input data bit) (y = 0..15)
这些位为只读。它们包含相应 I/O 端口的输入值。

用于读取IO引脚的电平

1.4.2.7 GPIO 端口输出数据寄存器（ODR）



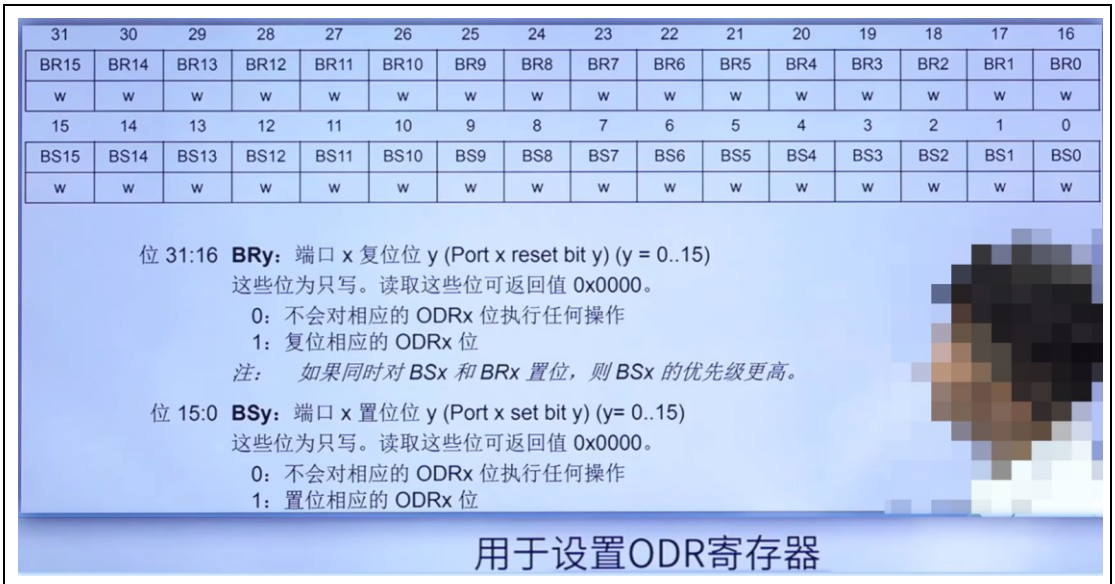
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **ODRy**：端口输出数据位 (Port output data bit) (y = 0..15)
这些位可通过软件读取和写入。

用于设置IO引脚输出的电平

1.4.2.8 端口置位/复位寄存器（BSRR）



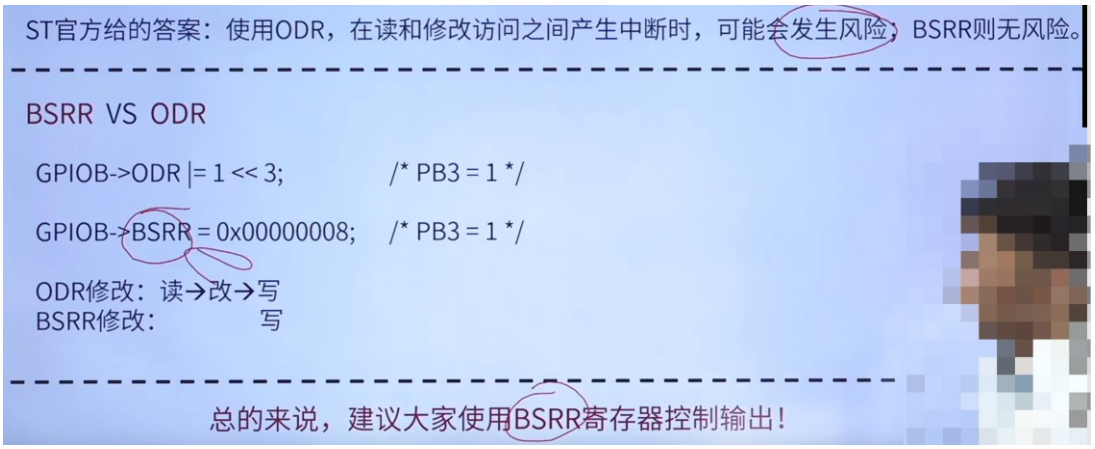
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**：端口 x 复位位 y (Port x reset bit y) (y = 0..15)
这些位为只写。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：复位相应的 ODRx 位
注：　如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

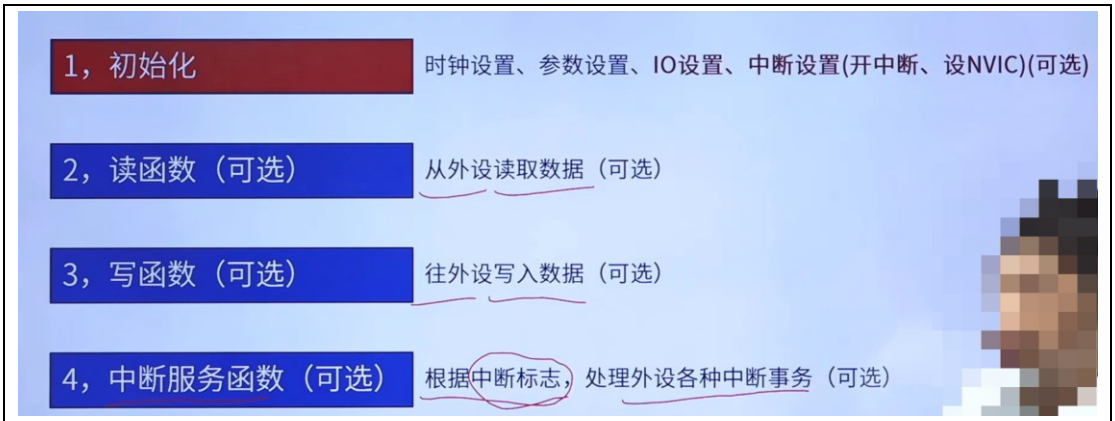
位 15:0 **BSy**：端口 x 置位位 y (Port x set bit y) (y= 0..15)
这些位为只写。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：置位相应的 ODRx 位

用于设置ODR寄存器

1.4.2.9 ODR 和 BSRR 寄存器控制输出的区别



1.5 通用外设驱动模型（四步法）



1.6 GPIO 配置步骤

1，使能时钟 __HAL_RCC_GPIOx_CLK_ENABLE()

2，设置工作模式 HAL_GPIO_Init()

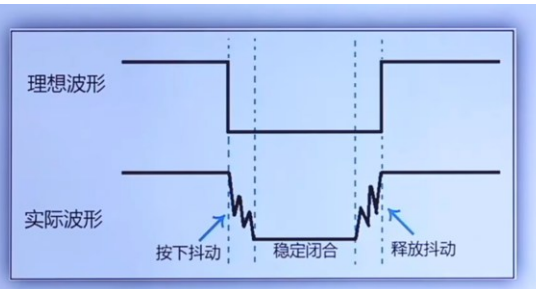
3，设置输出状态（可选） HAL_GPIO_WritePin()
HAL_GPIO_TogglePin()

4，读取输入状态（可选） HAL_GPIO_ReadPin()

相关HAL库函数简介

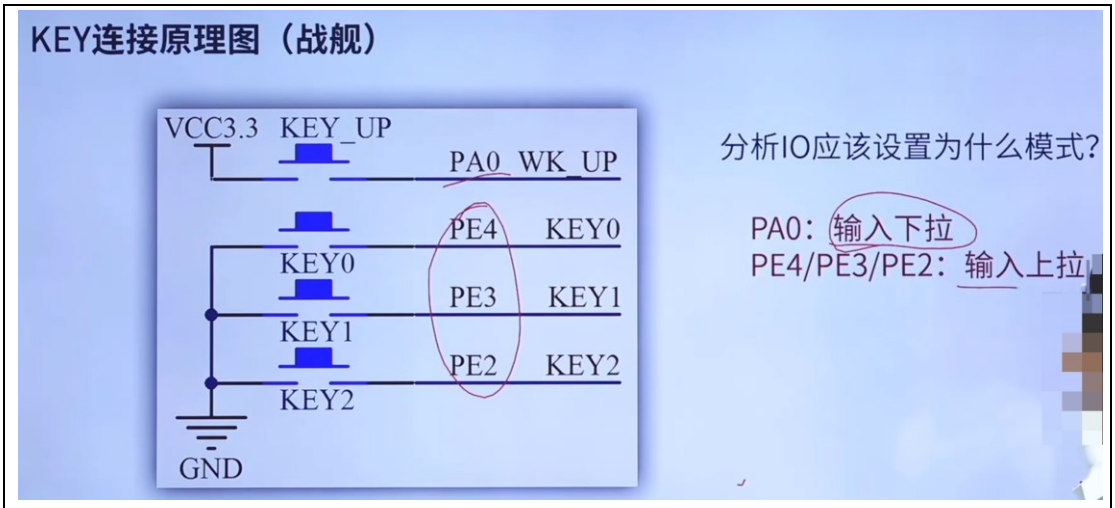
HAL库驱动函数	主要寄存器	功能
__HAL_RCC_GPIOx_CLK_ENABLE()	F1：RCC_APB2ENR F4：RCC_AHB1ENR F7：RCC_AHB1ENR H7：RCC_AHB4ENR	开启GPIO时钟
HAL_GPIO_Init(...)	F1：CRL、CRH、ODR F4/F7/H7：MODER、OTYPER、OSPEEDR、PUPDR	初始化GPIO
HAL_GPIO_WritePin(...)	BSRR	控制IO输出高/低电平
HAL_GPIO_TogglePin(...)	BSRR	每次调用IO输出电平翻转一次
HAL_GPIO_ReadPin(...)	IDR	读取IO电平

1.7 GPIO 输入



独立按键抖动波形图

软件消抖：通过延时跳过抖动的时间段，再判断IO输入电平。



1.7.1 按键控制 LED 灯

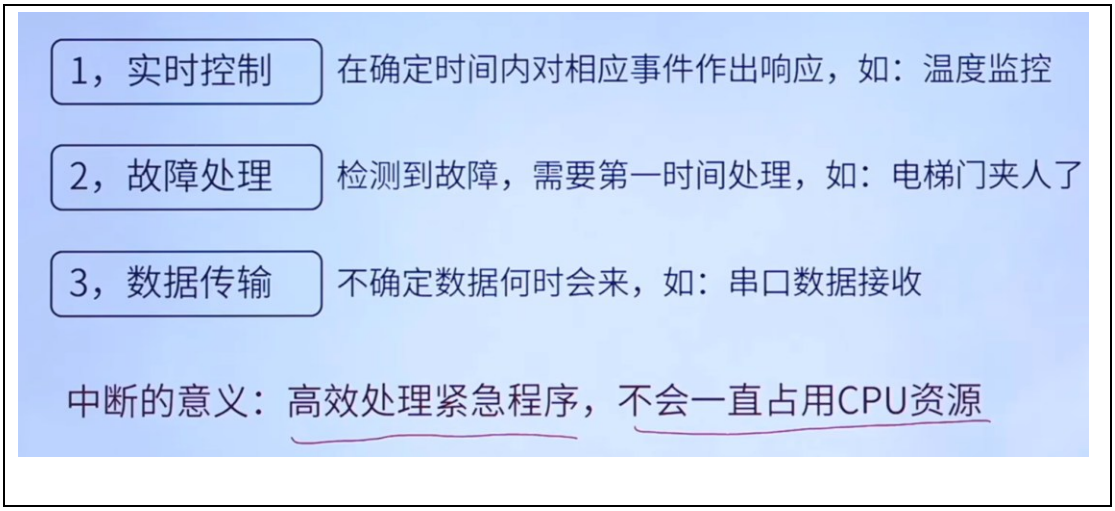
已完成案例:
D:\embedded\projects\OLD\06-HAL 库-外部按键控制 LED 灯实验

2 中断

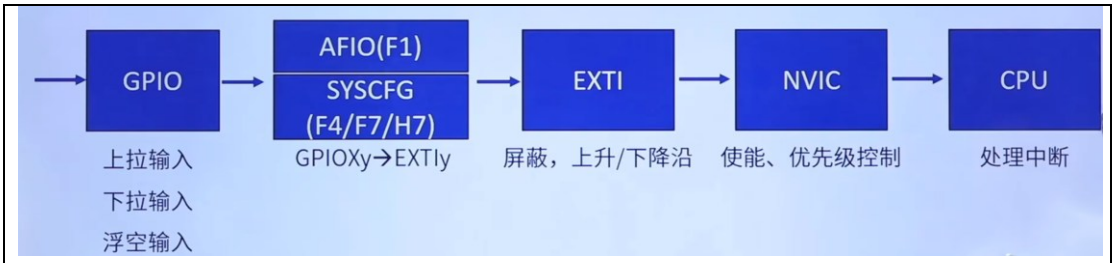
2.1 什么是中断

打断CPU执行正常的程序，转而处理紧急程序，然后返回原暂停的程序继续运行，就叫中断

2.2 中断的作用和意义

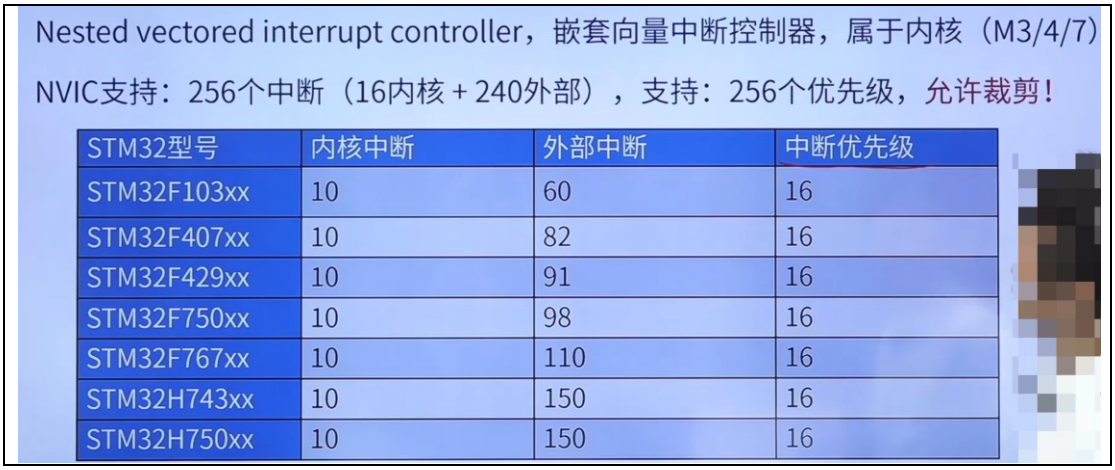


2.3 STM32 GPIO 外部中断简图

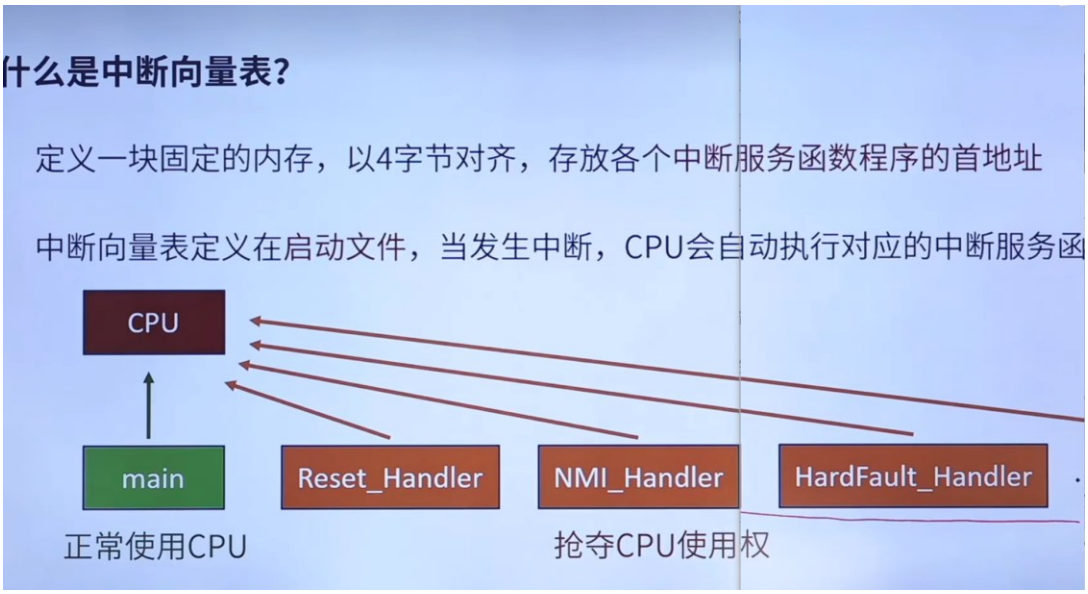


2.4 NVIC

2.4.1 NVIC 的基本概念



STM32型号	内核中断	外部中断	中断优先级
STM32F103xx	10	60	16
STM32F407xx	10	82	16
STM32F429xx	10	91	16
STM32F750xx	10	98	16
STM32F767xx	10	110	16
STM32H743xx	10	150	16
STM32H750xx	10	150	16

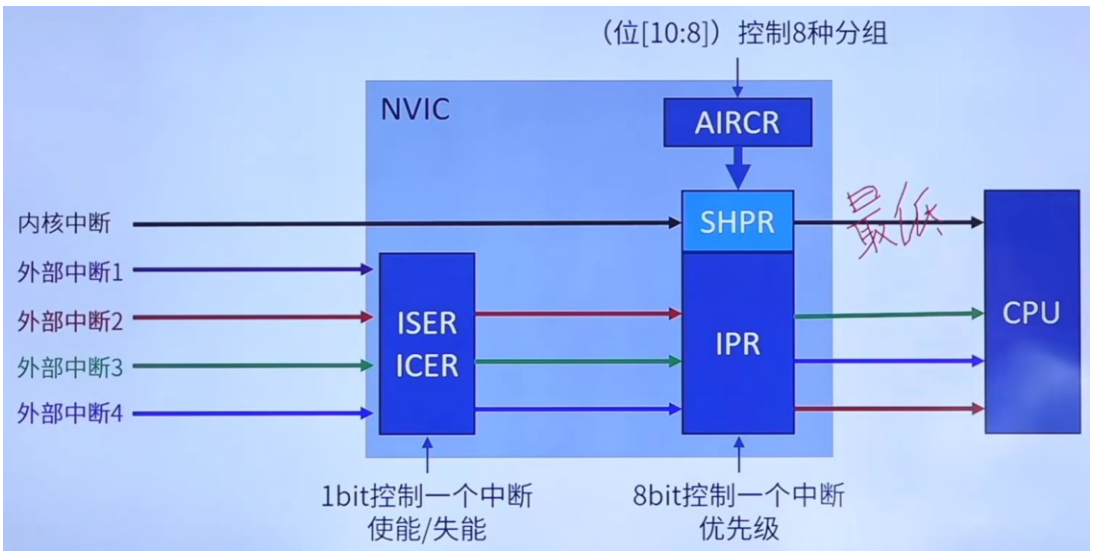


2.4.2 NVIC 相关寄存器介绍

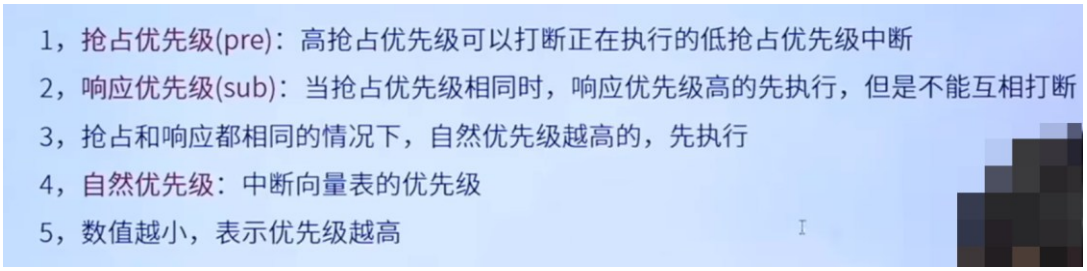
NVIC相关寄存器	位数	寄存器个数	备注
中断使能寄存器（ISER）	32	8	每个位控制一个中断
中断除能寄存器（ICER）	32	8	每个位控制一个中断
应用程序中断及复位控制寄存器（AIRCR）	32	1	位[10:8]控制优先级分组
中断优先级寄存器（IPR）	8	240	8个位对应一个中断，而STM32只使用高4位

NVIC还有：中断挂起，解挂，激活标志等非常用功能，不做介绍！

2.4.3 NVIC 工作原理



2.4.4 STM32 中断优先级基本概念



自然优先级：

表 45. STM32F405xx/07xx 和 STM32F415xx/17xx 的向量表

位置	优先级	优先级类型	名称	说明	地址
	-	-	-	保留	0x0000 0000
	-3	固定	Reset	复位	0x0000 0004
	-2	固定	NMI	不可屏蔽中断。RCC 时钟安全系统 (CSS) 连接到 NMI 向量。	0x0000 0008
	-1	固定	HardFault	所有类型的错误	0x0000 000C
	0	可设置	MemManage	存储器管理	0x0000 0010
	1	可设置	BusFault	预取指失败，存储器访问失败	0x0000 0014
	2	可设置	UsageFault	未定义的指令或非法状态	0x0000 0018
	-	-	-	保留	0x0000 001C - 0x0000 002B
	3	可设置	SVCall	通过 SWI 指令调用的系统服务	0x0000 002C
	4	可设置	Debug Monitor	调试监控器	0x0000 0030
	-	-	-	保留	0x0000 0034
	5	可设置	PendSV	可挂起的系统服务	0x0000 0038
	6	可设置	SysTick	系统嘀嗒定时器	0x0000 003C
0	7	可设置	WWDG	窗口看门狗中断	0x0000 0040
1	8	可设置	PVD	连接到 EXTI 线的可编程电压检测 (PVD) 中断	0x0000 0044
2	9	可设置	TAMP_STAMP	连接到 EXTI 线的入侵和时间戳中断	0x0000 0048
3	10	可设置	RTC_WKUP	连接到 EXTI 线的 RTC 唤醒中断	0x0000 004C
4	11	可设置	FLASH	Flash 全局中断	0x0000 0050
5	12	可设置	RCC	RCC 全局中断	0x0000 0054
6	13	可设置	EXTI0	EXTI 线 0 中断	0x0000 0058
7	14	可设置	EXTI1	EXTI 线 1 中断	0x0000 005C
8	15	可设置	EXTI2	EXTI 线 2 中断	0x0000 0060
9	16	可设置	EXTI3	EXTI 线 3 中断	0x0000 0064
10	17	可设置	EXTI4	EXTI 线 4 中断	0x0000 0068

2.4.5 STM32 中断优先级分组

优先级分组	AIRCR[10:8]	IPRx bit[7:4]分配	分配结果
0	111	None ： [7:4]	0位抢占优先级，4位响应优先级
1	110	[7] ： [6:4]	1位抢占优先级，3位响应优先级
2	101	[7:6] ： [5:4]	2位抢占优先级，2位响应优先级
3	100	[7:5] ： [4]	3位抢占优先级，1位响应优先级
4	011	[7:4] ： None	4位抢占优先级，0位响应优先级

特别提示：一个工程中，一般只设置一次中断优先级分组。

STM32中断优先级举例（假设分组是2）

编号	自然优先级	对应外设	抢占	响应	执行顺序
3	10	RTC	2	1	2
6	13	EXTI0	3	0	4
7	14	EXTI1	2	0	1
-1	6	Systick	3	0	3

EXTI1和RTC可以打断：EXTI0和Systick的中断，获得优先执行！

2.4.6 STM32 NVIC 的使用

1，设置中断分组　AIRCR[10:8]，HAL_NVIC_SetPriorityGrouping

2，设置中断优先级　IPRx bit[7:4]，HAL_NVIC_SetPriority

3，使能中断　ISERx，HAL_NVIC_EnableIRQ

特别提醒：例程中，中断分组已经在HAL_Init()函数中设置了分组2

了解寄存器：

1，SCB_AIRCR

2，NVIC_IPRx

3，NVIC_ISER

3 EXTI

3.1 EXTI 基本概念

External(Extended) interrupt/event Controller，外部(扩展)中断事件控制器

包含20个产生事件/中断请求的边沿检测器，即总共：20条EXTI线（F1）

中断和事件的理解：

中断：要进入NVIC，有相应的中断服务函数，需要CPU处理

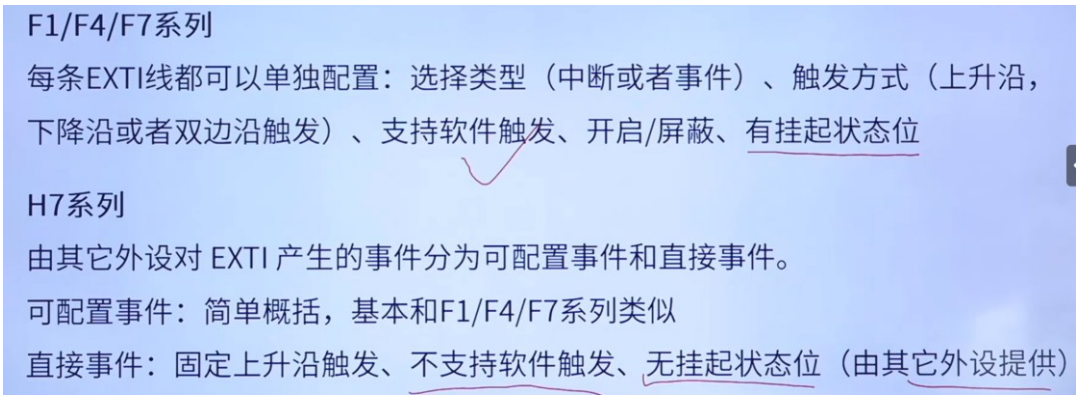
事件：不进入NVIC，仅用于内部硬件自动控制的，如：TIM、DMA、ADC

EXTI支持的外部中断/事件请求

中断线	F1	F4	F7	H7
EXTI线0~15：对应GPIO PIN 0~15	✔	✔	✔	✔
EXTI线16：PVD输出	✔	✔	✔	参考H7参考手册（中文版）657页
EXTI线17：RTC闹钟事件	✔	✔	✔	
EXTI线18：USB OTG FS唤醒事件	✔	✔	✔	
EXTI线19：以太网唤醒事件		✔	✔	
EXTI线20：USB OTG HS唤醒事件		✔	✔	
EXTI线21：RTC 入侵和时间戳事件		✔	✔	
EXTI线22：RTC 唤醒事件		✔	✔	
EXTI线23：LPTIM1 异步事件			✔	
…				

3.2 EXTI 主要特性

3.2.1 F1/F4/F7/H7 系列



F1/F4/F7系列

每条EXTI线都可以单独配置：选择类型（中断或者事件）、触发方式（上升沿，下降沿或者双边沿触发）、支持软件触发、开启/屏蔽、有挂起状态位

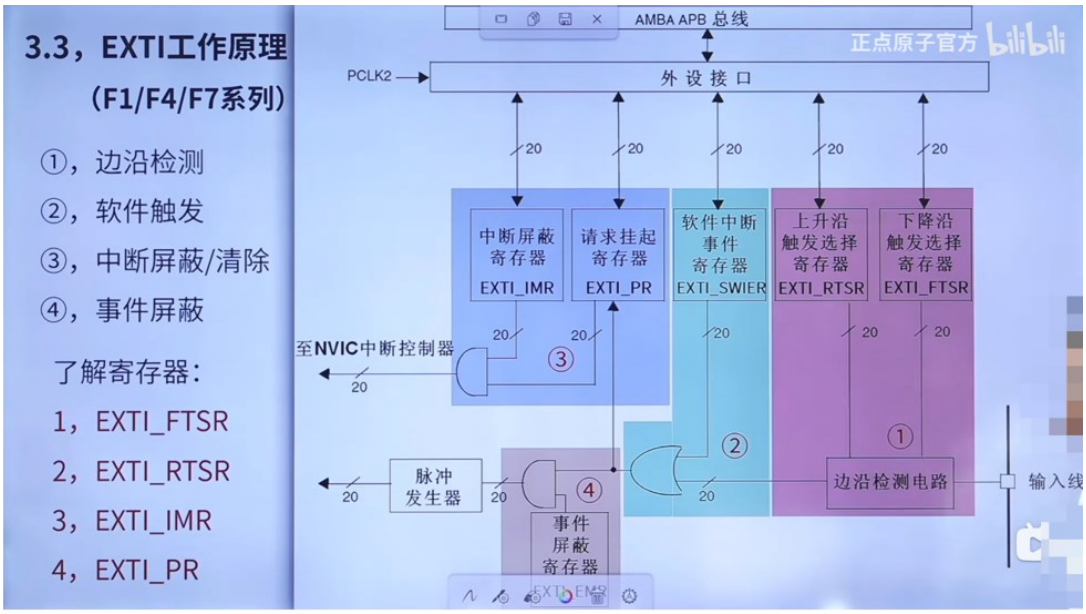
H7系列

由其它外设对 EXTI 产生的事件分为可配置事件和直接事件。

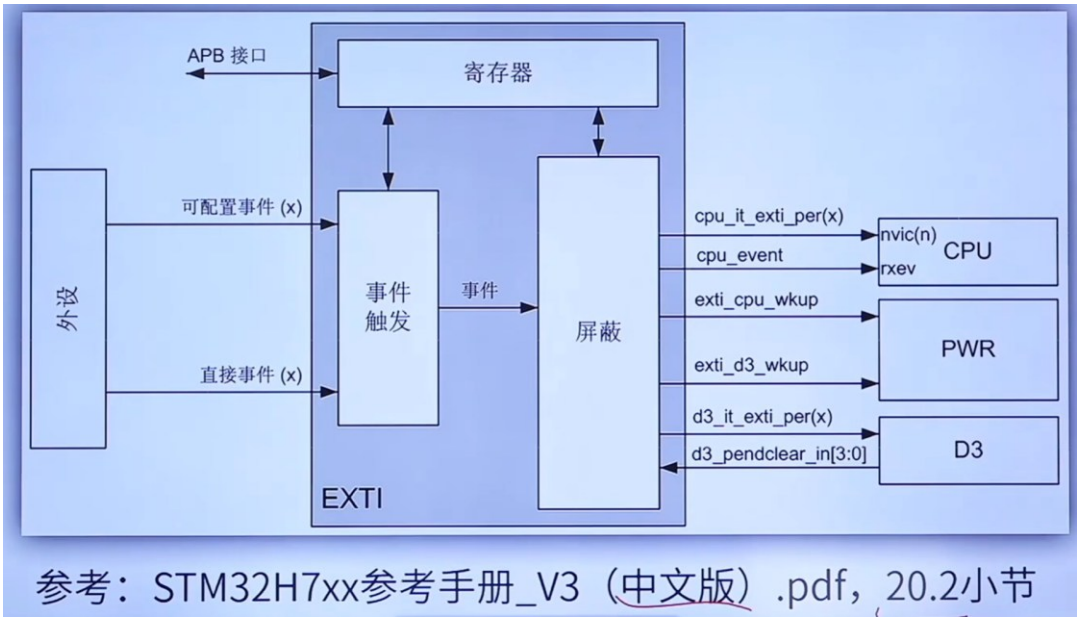
可配置事件：简单概括，基本和F1/F4/F7系列类似

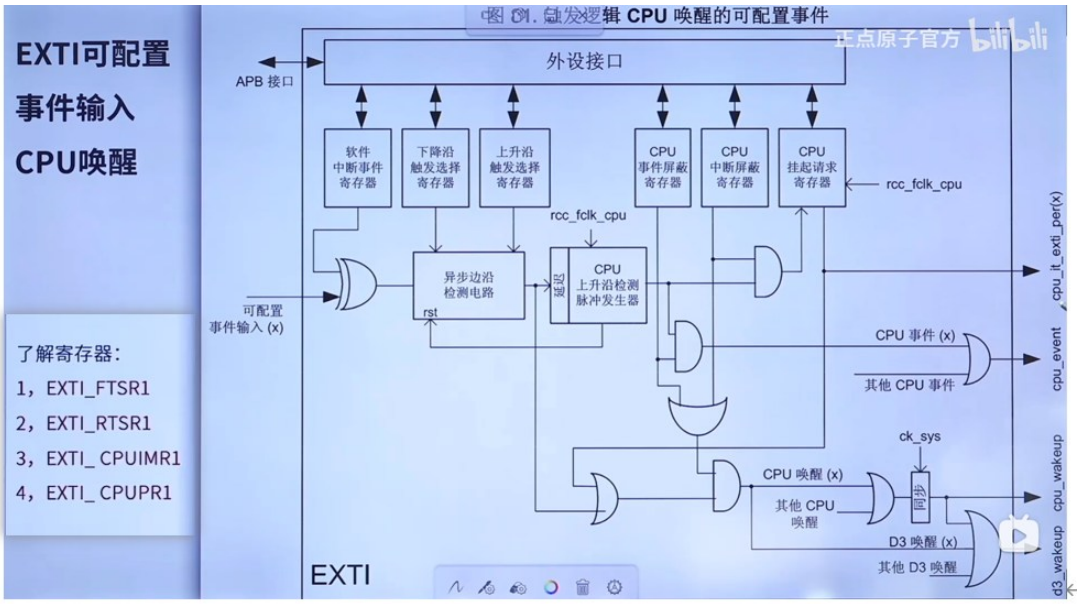
直接事件：固定上升沿触发、不支持软件触发、无挂起状态位（由其它外设提供）

3.2.2 EXTI 工作原理（F1/F4/F7）



3.2.3 EXTI 工作原理（H7）

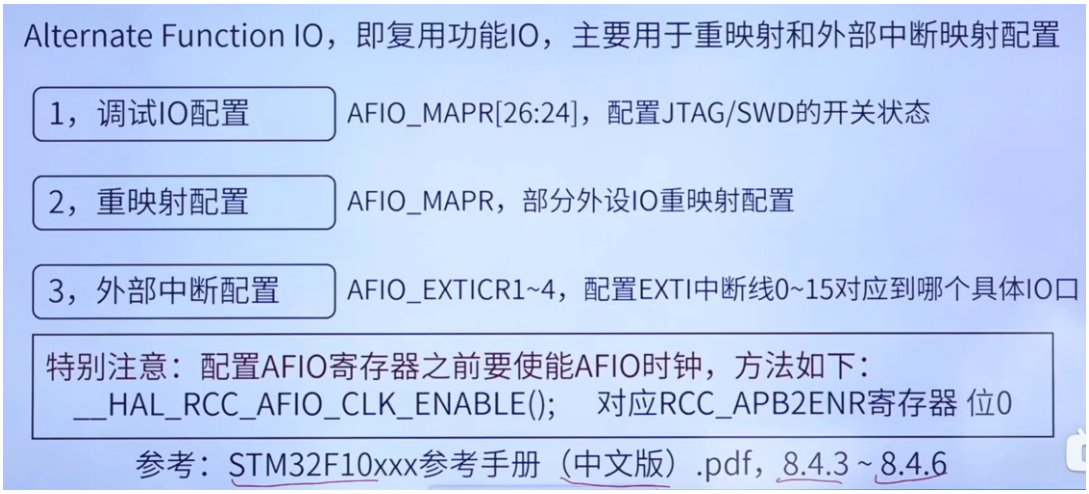




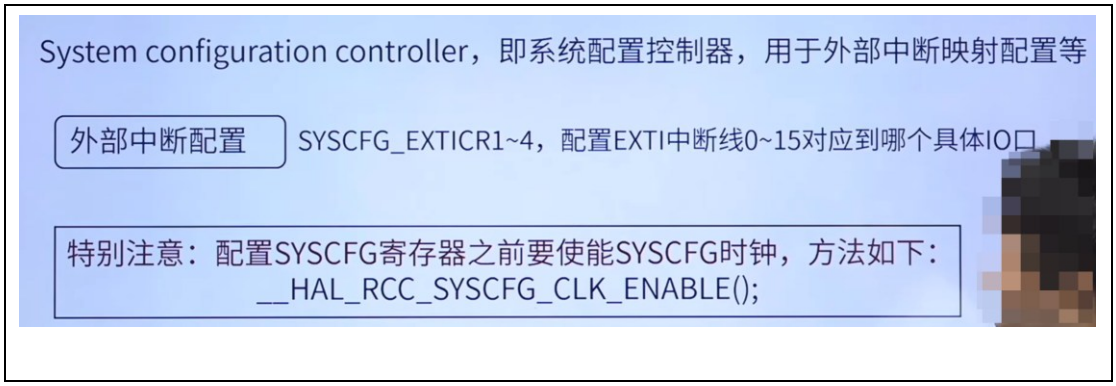
：“与”，：“或”

3.3 EXFTI 和 IO 的映射关系

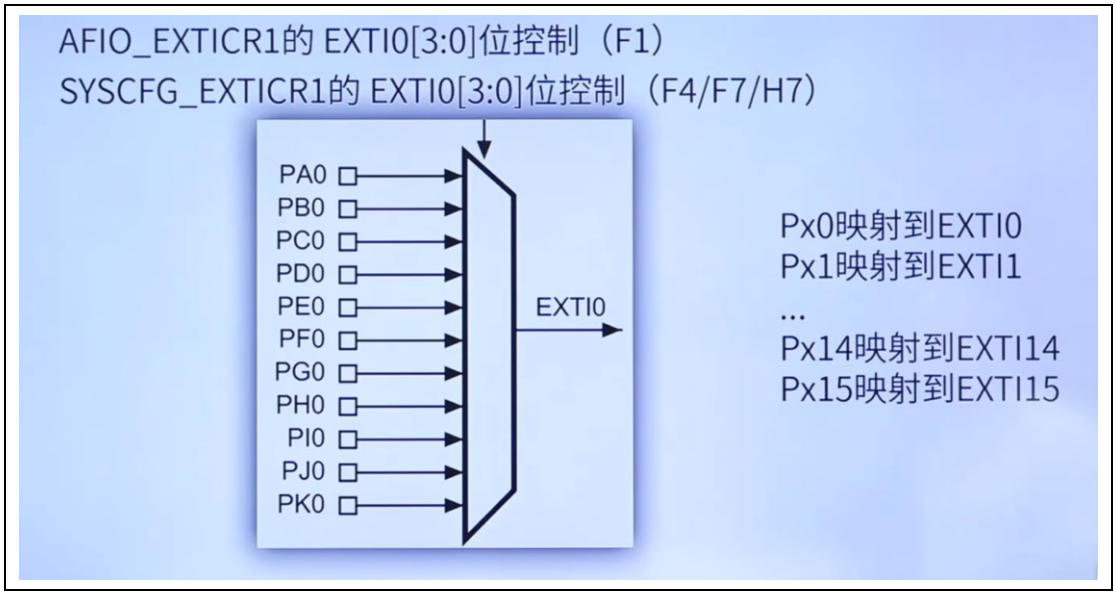
3.3.1 AFIO 简介（F1）



3.3.2 SYSCFG 简介（F4/F7/H7）



3.3.3 EXTI 与 IO 对应关系



SYSCFG external interrupt configuration register 1

偏移地址：0x08

复位值：0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

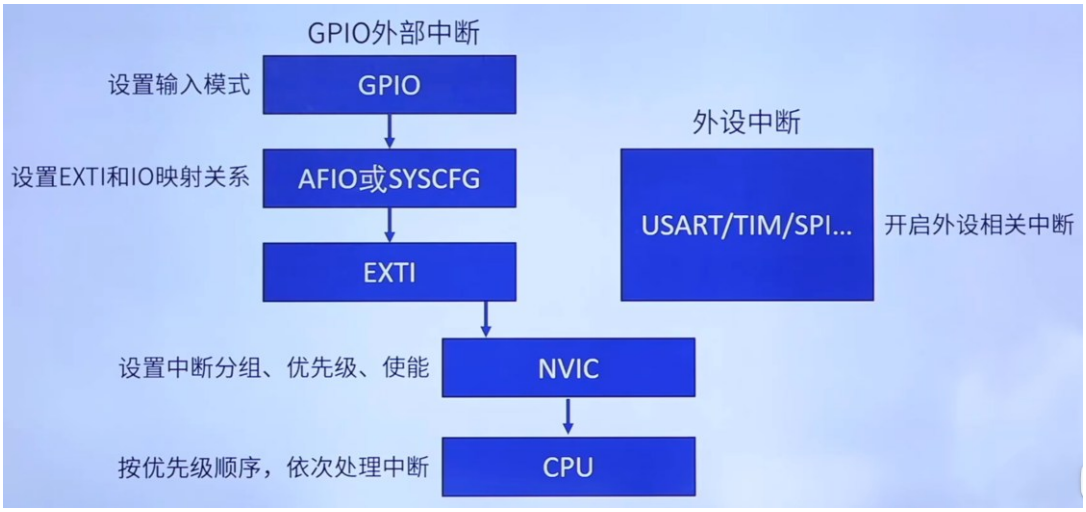
位 31:16 保留，必须保持复位值。

位 15:0 **EXTIx[3:0]**：EXTI x 配置（x = 0 到 3）(EXTI x configuration (x = 0 to 3))
这些位通过软件写入，以选择 EXTIx 外部中断的源输入。
0000：PA[x] 引脚
0001：PB[x] 引脚
0010：PC[x] 引脚
0011：PD[x] 引脚
0100：PE[x] 引脚
0101：PF[C] 引脚
0110：PG[x] 引脚
0111：PH[x] 引脚
1000：PI[x] 引脚

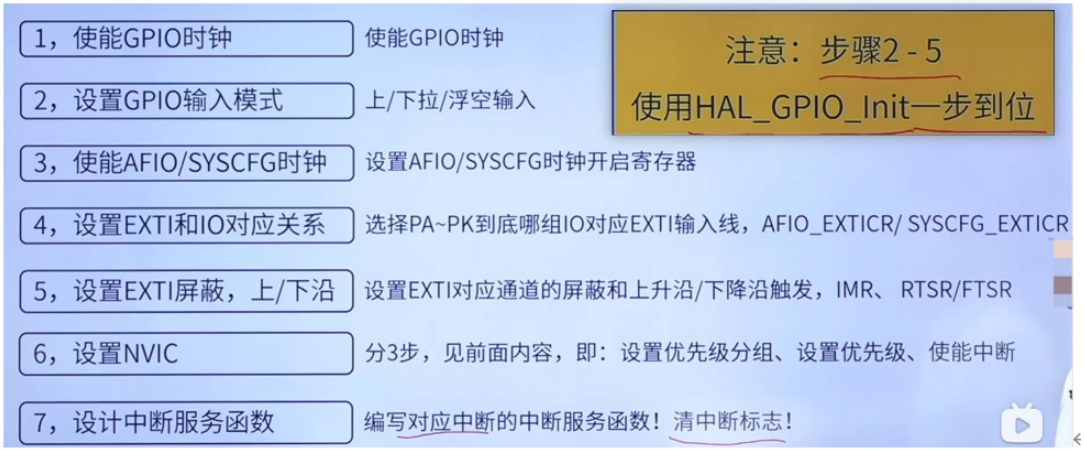
STM32F4xx 中文参考手册.pdf 8.2.4

3.4 中断的使用

3.4.1 如何使用中断

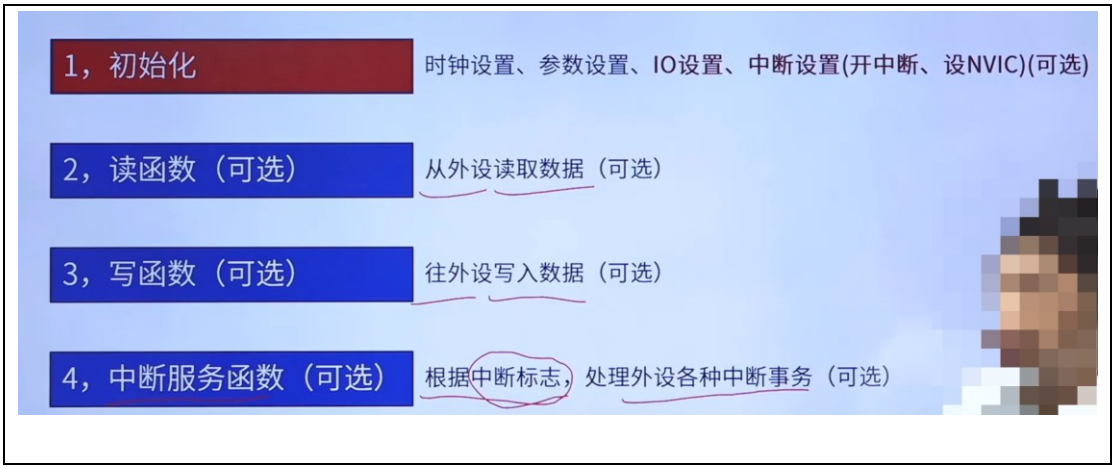


3.4.1.1 STM32 EXTI 的配置步骤（GPIO 外部中断）

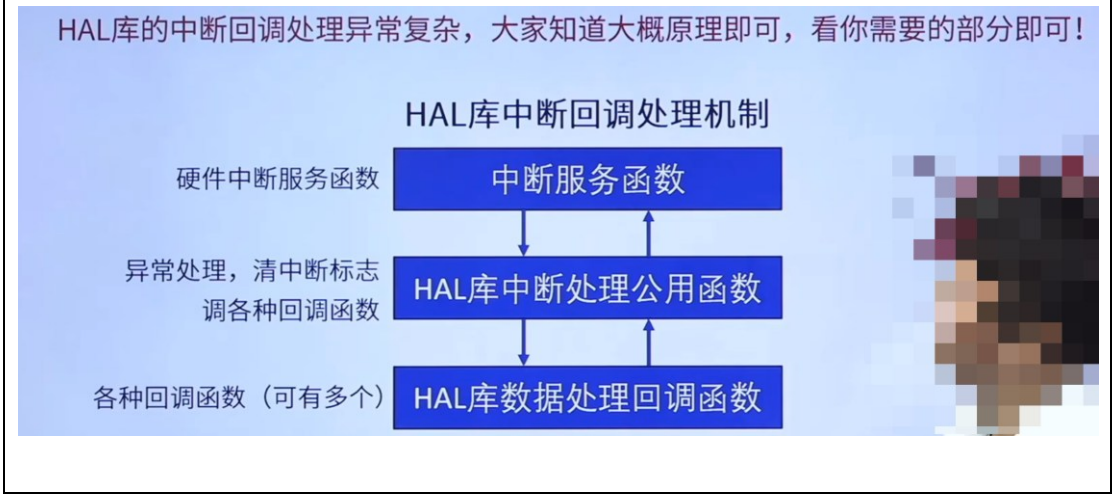


3.4.1.2 STM32 EXTI 的 HAL 库设置步骤（GPIO 外部中断）

3.4.2 通用外设驱动模型（四步法）



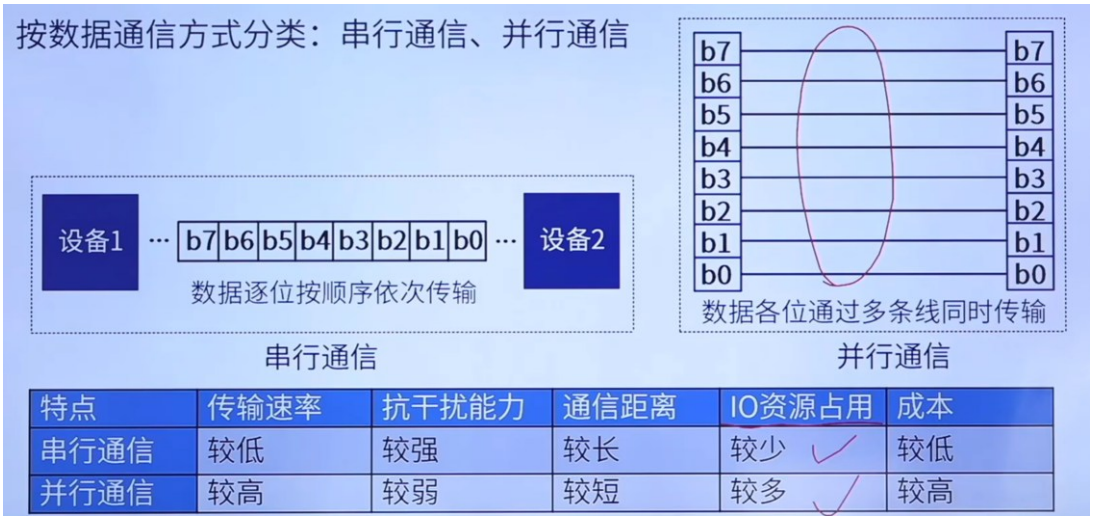
3.4.3 HAL 库中断回调处理机制介绍



4 串口

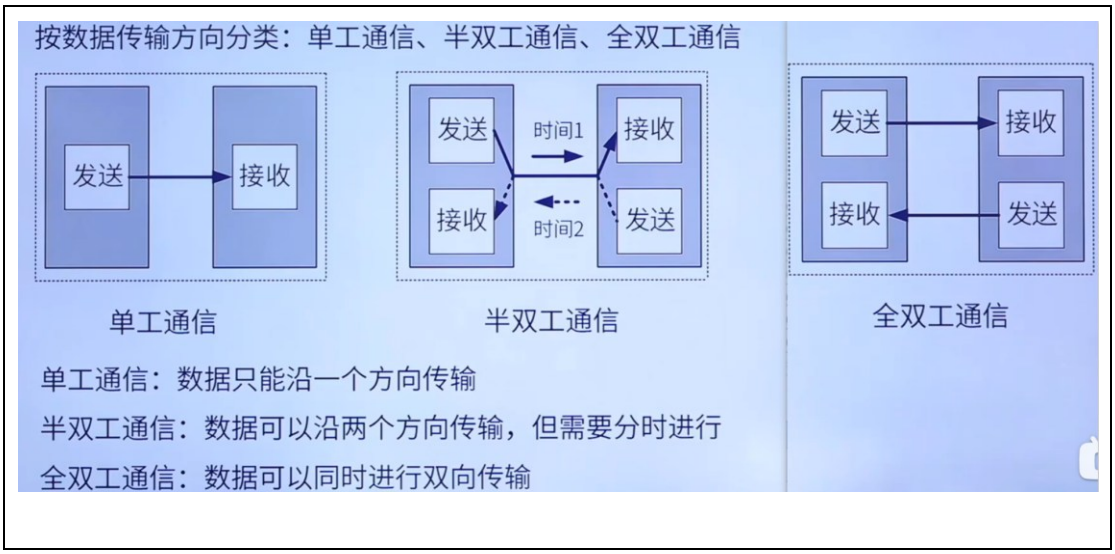
4.1 数据通行的基础概念

4.1.1 串行/并行通信

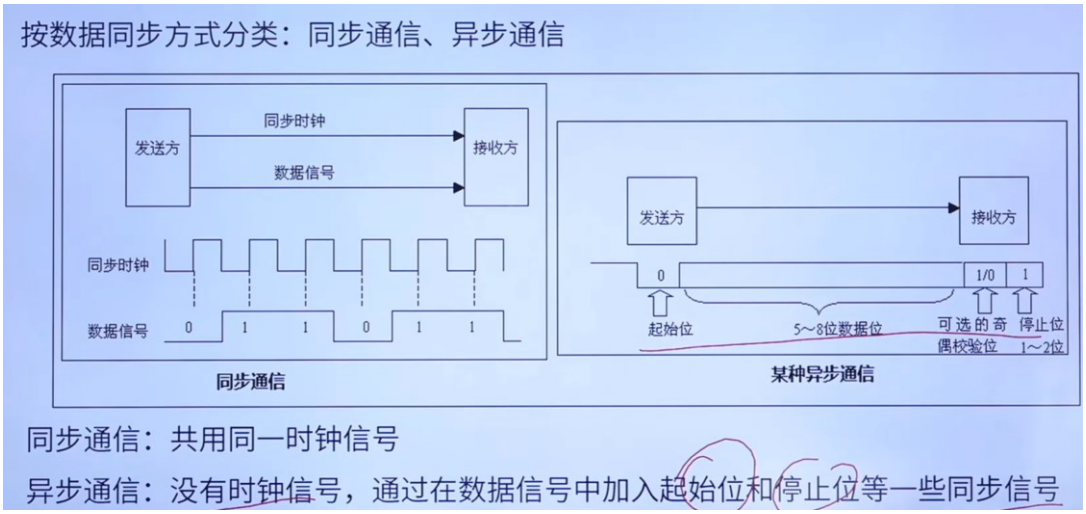


特点	传输速率	抗干扰能力	通信距离	IO资源占用	成本
串行通信	较低	较强	较长	较少	较低
并行通信	较高	较弱	较短	较多	较高

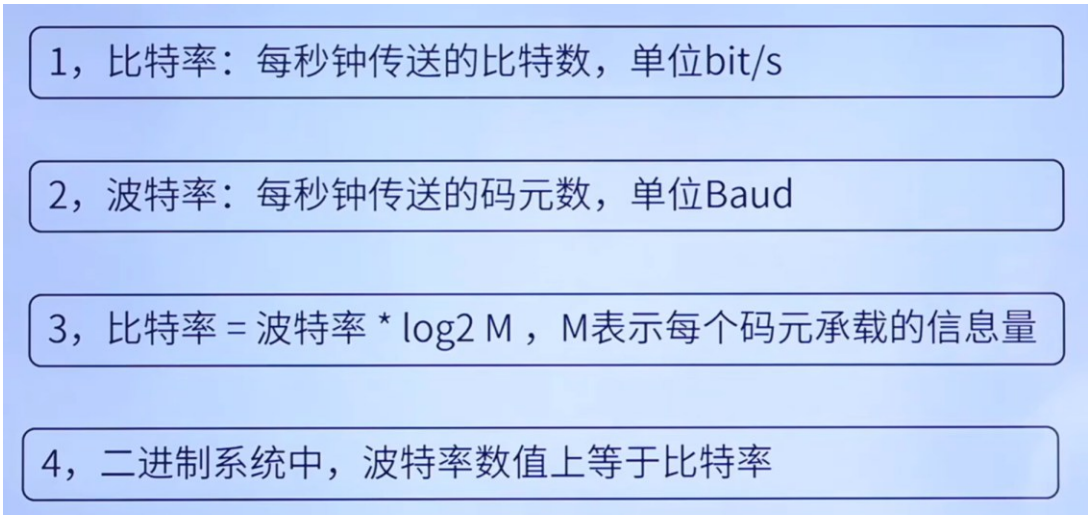
4.1.2 单工/半双工/全双工



4.1.3 同步/异步通信



4.1.4 波特率



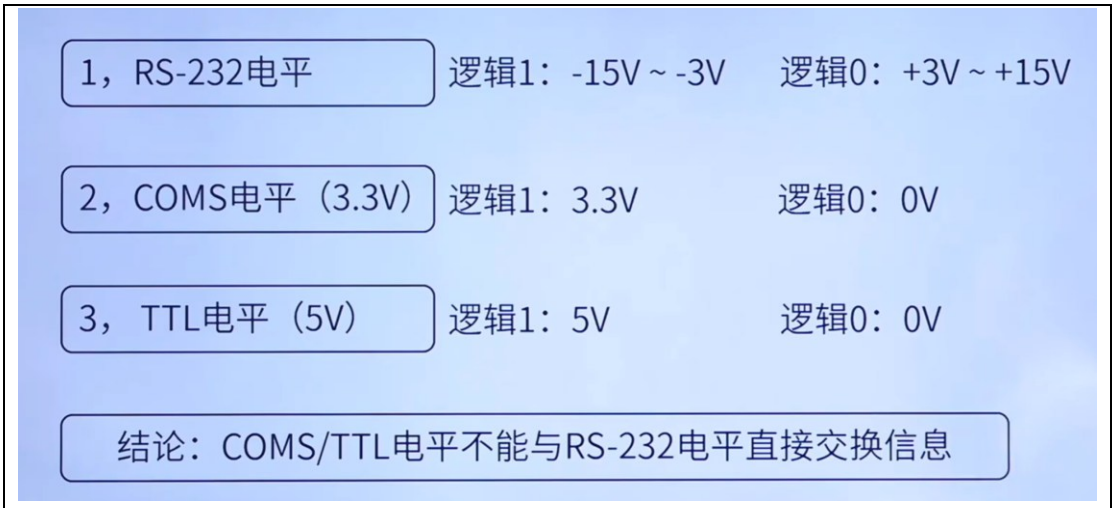
4.1.5 常见的串口通信接口

通信接口	接口引脚	数据同步方式	数据传输方向
UART (通用异步收发器)	TXD:发送端 RXD:接收端 GND:公共地	异步通信	全双工
1-wire	DQ:发送/接收端	异步通信	半双工
IIC	SCL:同步时钟 SDA:数据输入/输出端	同步通信	半双工
SPI	SCK:同步时钟 MISO:主机输入，从机输出 MOSI:主机输出，从机输入 CS:片选信号	同步通信	全双工

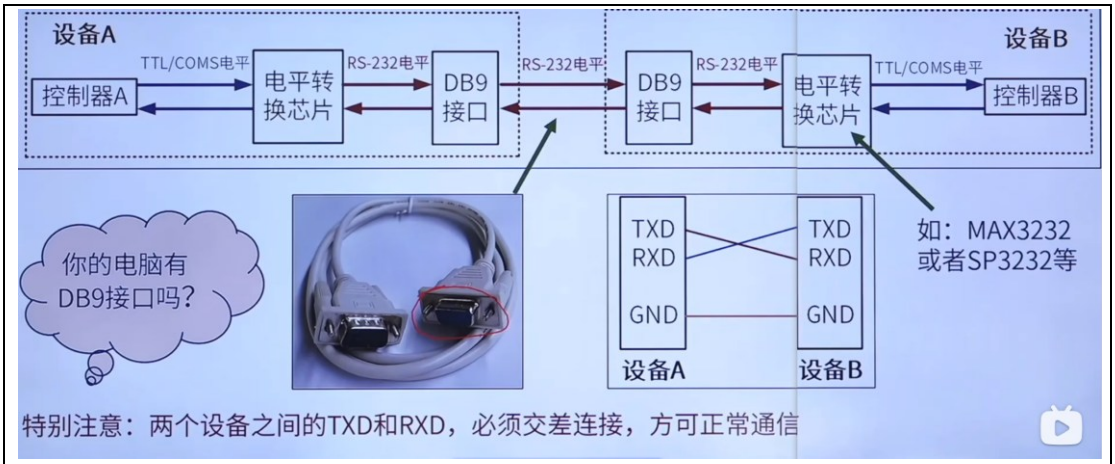
4.2 串口

4.2.1 什么是串口

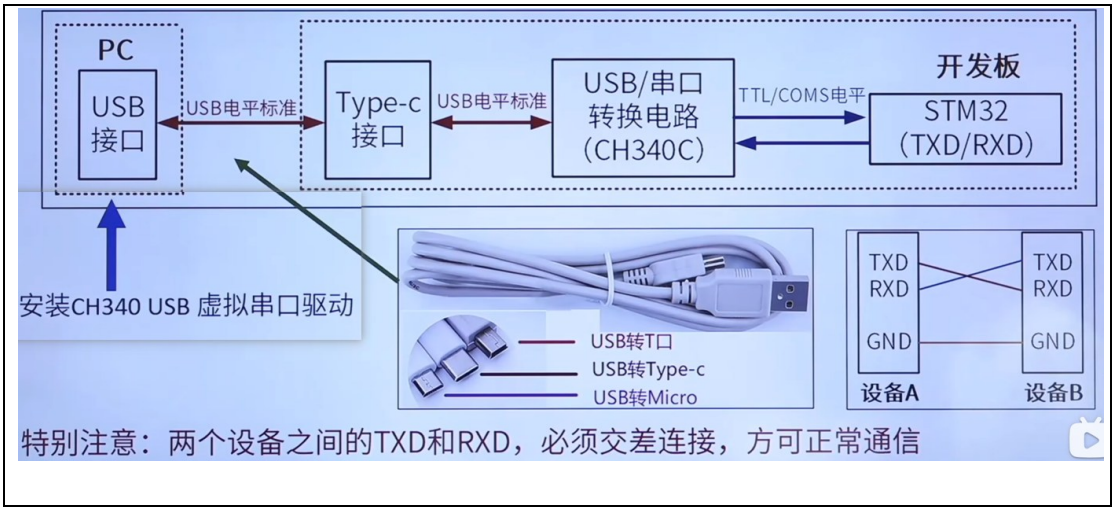
4.2.2 RS-232 电平与 COMS/TTL 电平对比



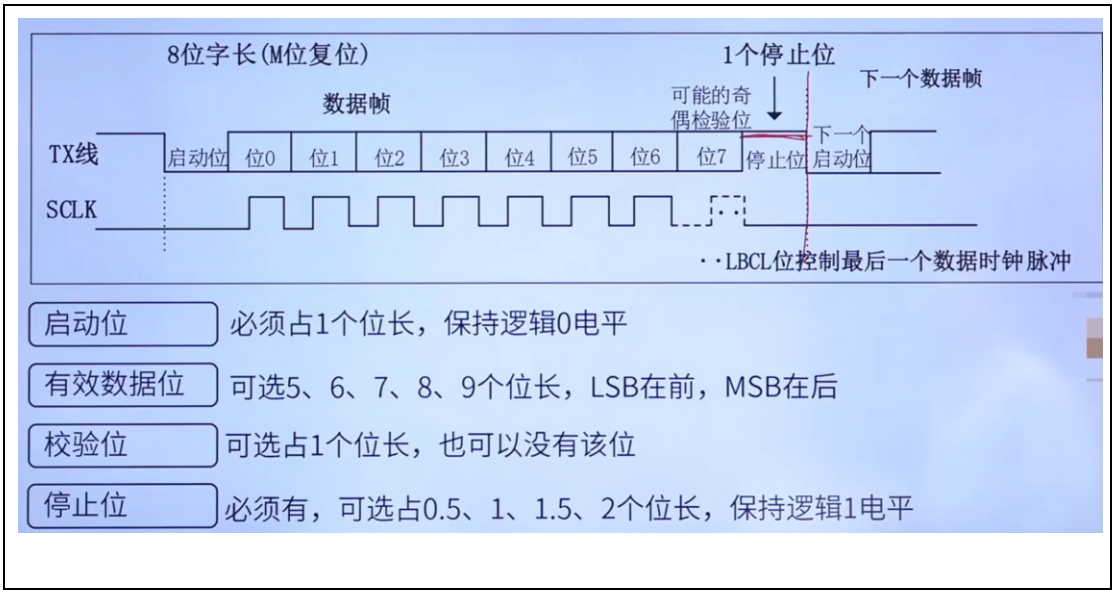
4.2.3 设备间的 RS-232 通信示意图



4.2.4 STM32 串口与电脑 USB 口通信示意图

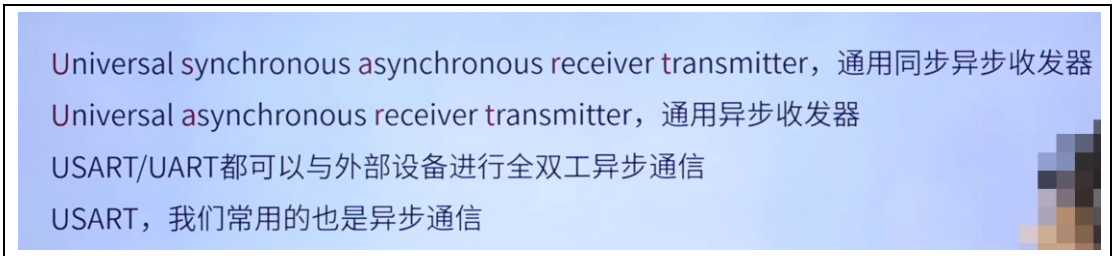


4.2.5 RS-232 异步通信协议

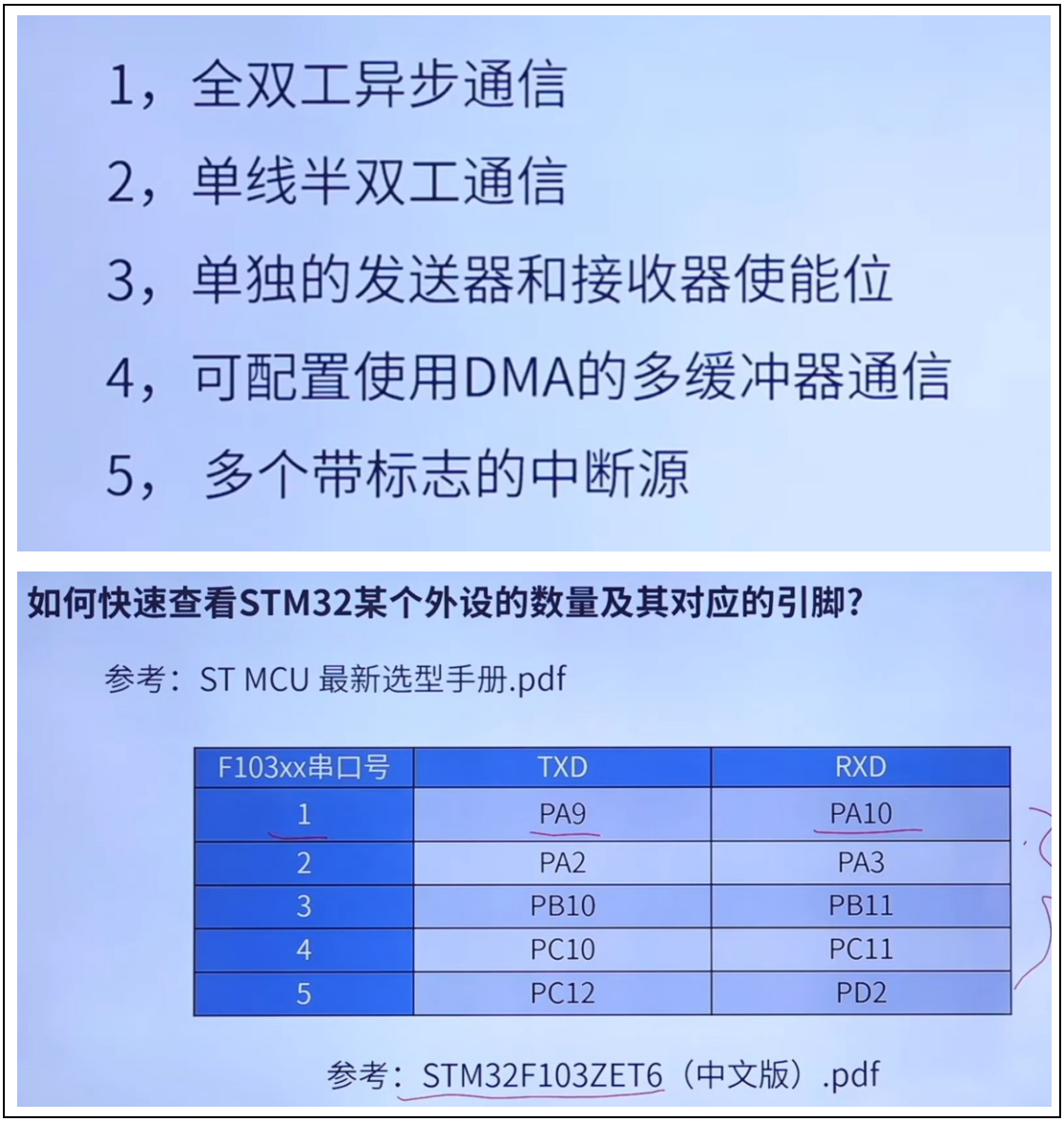


4.3 STM32 的 USART

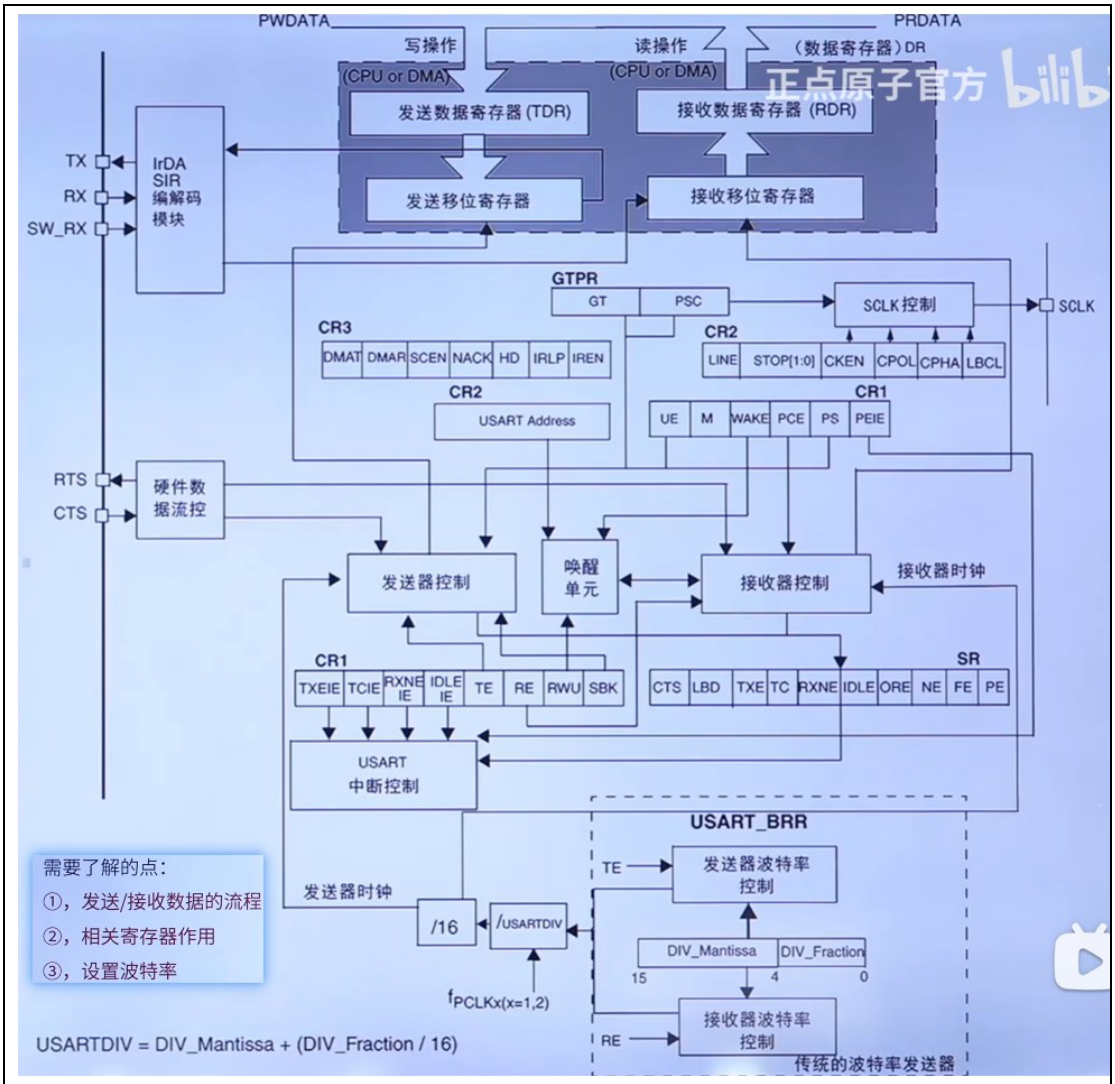
4.3.1 STM32 的 USART 简介



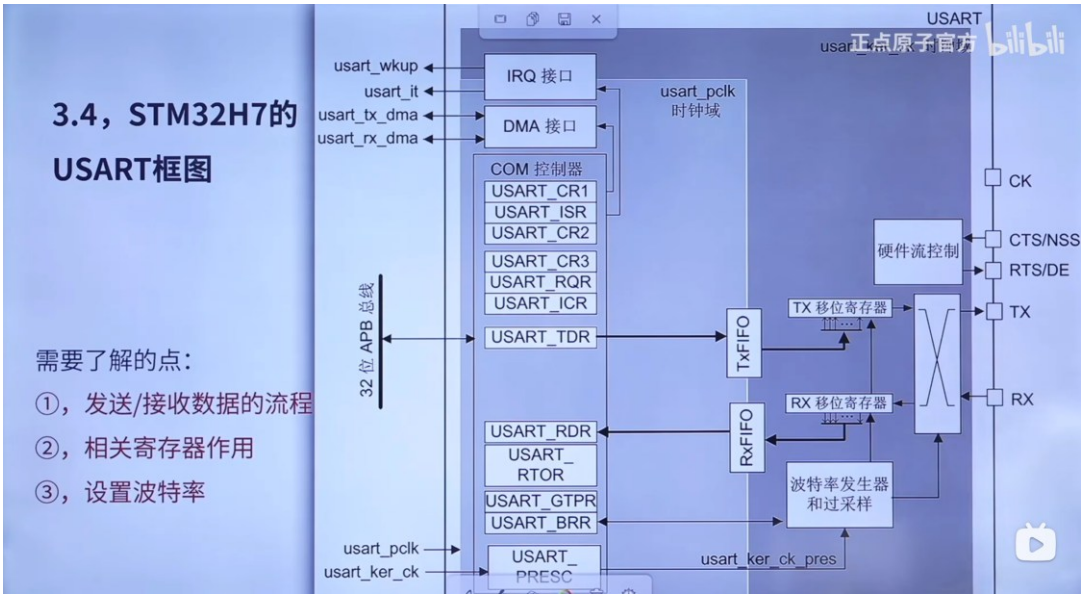
4.3.2 STM32 的 USART 主要特征



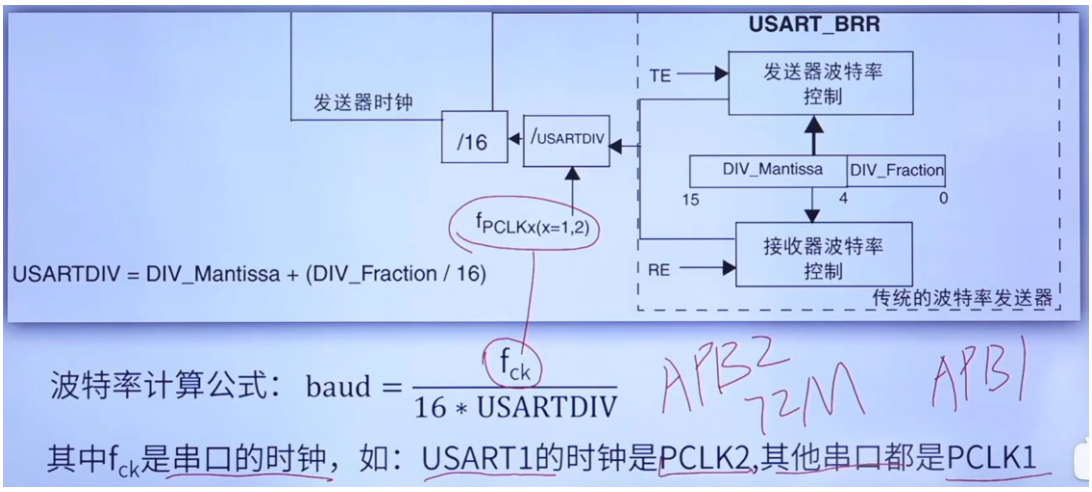
4.3.3 STM32F1/F4/F7 的 USART 框图



4.3.4 STM32H7 的 USART 框图

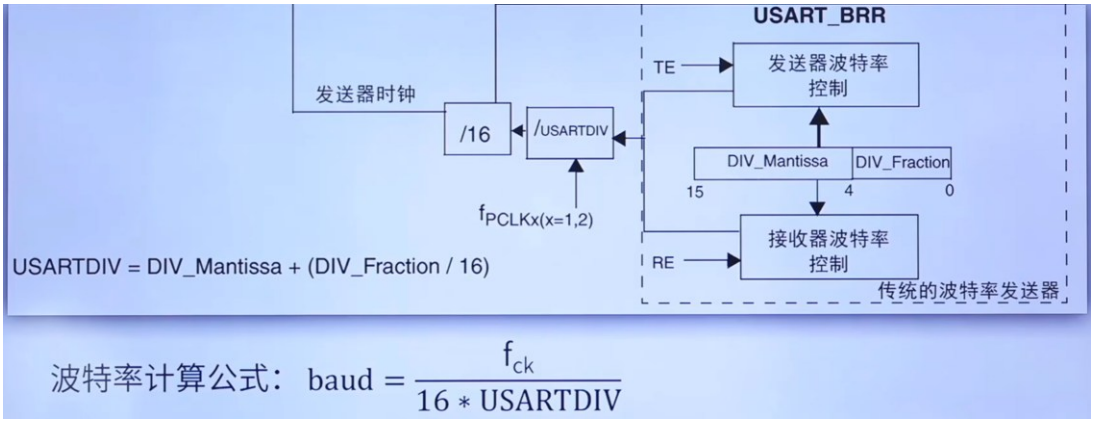


4.3.5 STM32F1/F4/F7/H7 的 USART 框图简化版



4.4 设置波特率

4.4.1 设置 USART 波特率（F1）



波特率计算公式： $baud = \frac{f_{ck}}{16 * USARTDIV}$

波特比率寄存器（BRR）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:16	保留位，硬件强制为0
位15:4	**DIV_Mantissa[11:0]**：USARTDIV的整数部分 这12位定义了USART分频器除法因子(USARTDIV)的整数部分。
位3:0	**DIV_Fraction[3:0]**：USARTDIV的小数部分 这4位定义了USART分频器除法因子(USARTDIV)的小数部分。

把USARTDIV的整数部分写入位[15:4]， USARTDIV的小数部分写入[3:0]

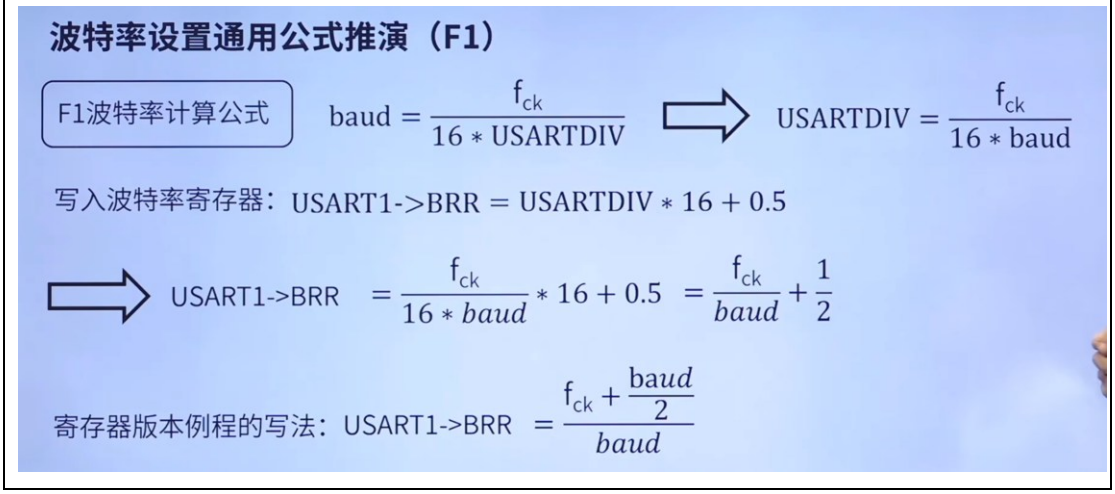
如何使用寄存器操作的方式设置波特率 - USART1为例

F1波特率计算公式 $baud = \frac{f_{ck}}{16 * USARTDIV}$

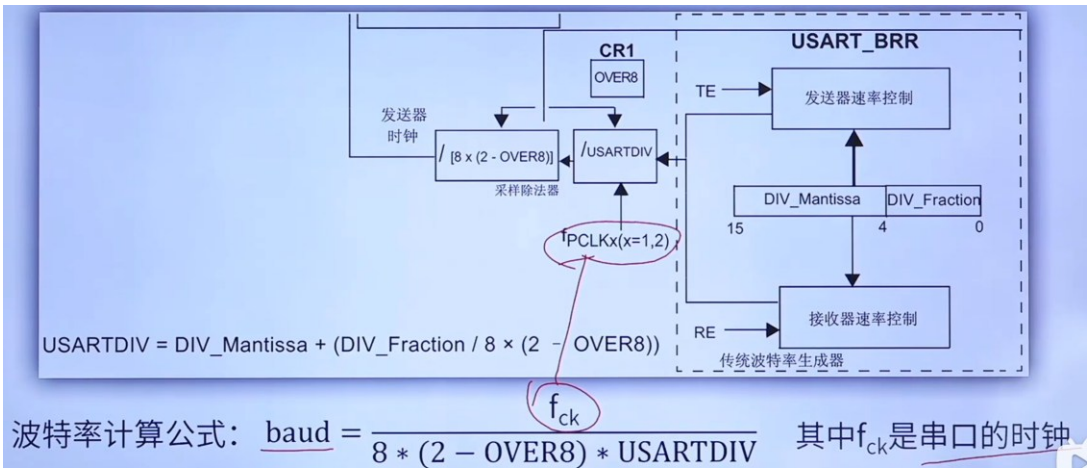
波特率为115200为例： $115200 = \frac{72000000}{16 * USARTDIV} \Rightarrow USARTDIV = 39.0625$

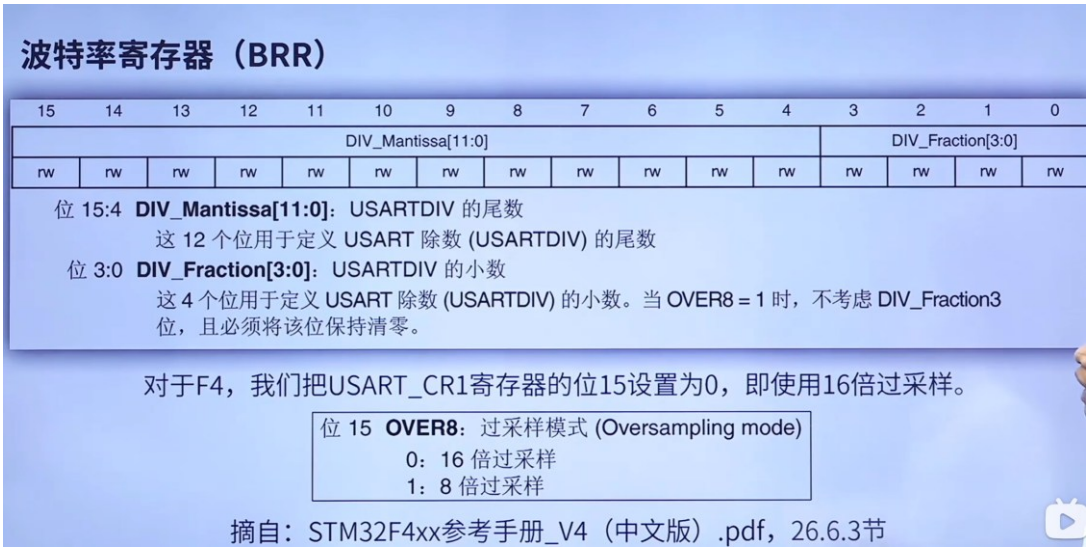
怎么把USARTDIV的值写入USART_BRR寄存器?

```
uint16_t mantissa;
uint16_t fraction;
mantissa=39;
fraction=0.0625*16+0.5=0x01;       /* USARTDIV = DIV_Mantissa + (DIV_Fraction/16) */
USART1->BRR = (mantissa << 4) + fraction;
```

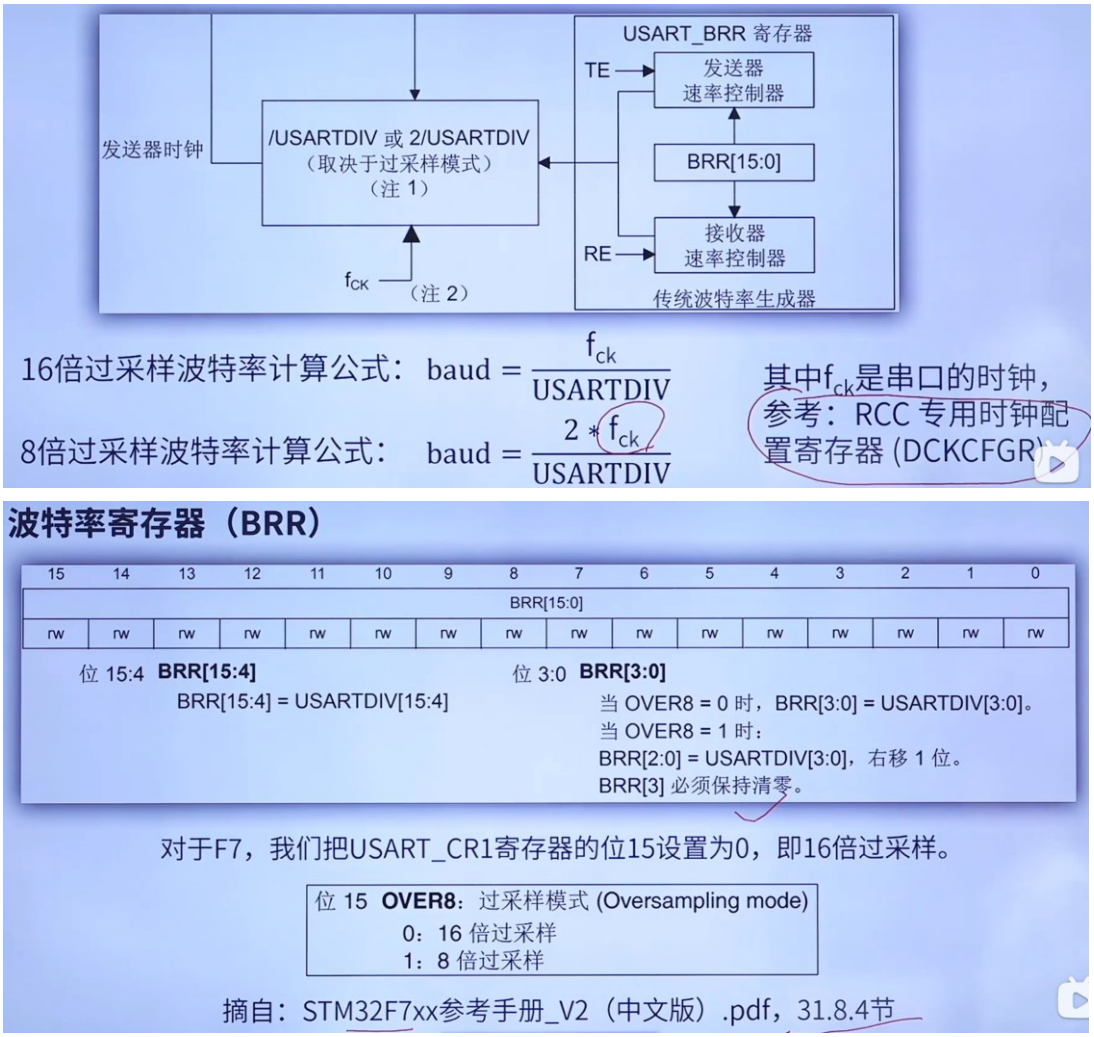


4.4.2 设置 USART 波特率（F4）

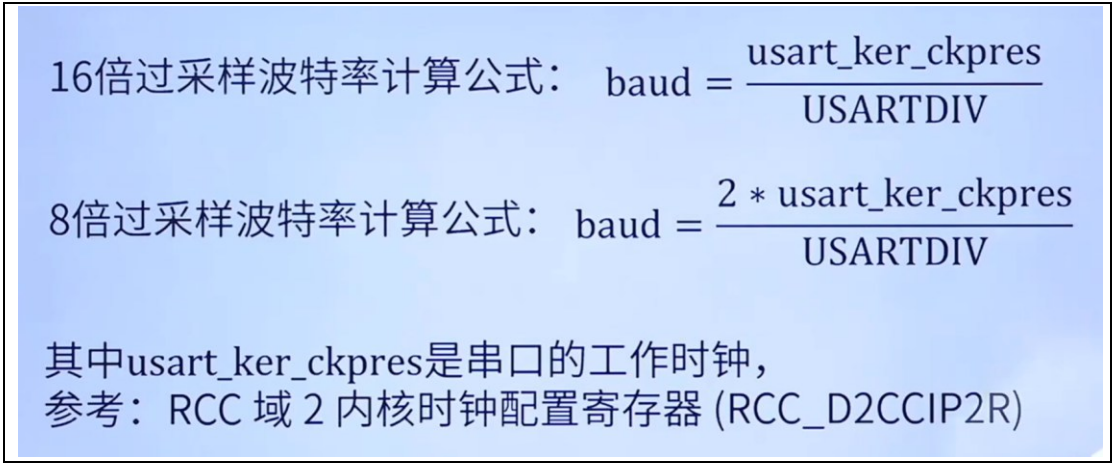




4.4.3 设置 USART 波特率（F7）



4.4.4 设置 USART 波特率（H7）



波特率寄存器（BRR）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BRR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:4 **BRR[15:4]**
BRR[15:4] = USARTDIV[15:4]

位 3:0 **BRR[3:0]**
当 OVER8 = 0 时，BRR[3:0] = USARTDIV[3:0]。
当 OVER8 = 1 时：
BRR[2:0] = USARTDIV[3:0]，右移 1 位。
BRR[3] 必须保持清零。

对于H7，我们把USART_CR1寄存器的位15设置为0，即16倍过采样。

位 15 **OVER8**：过采样模式 (Oversampling mode)
0：16 倍过采样
1：8 倍过采样

摘自：STM32H7xx参考手册_V1（中文版）.pdf，48.7.4节

4.5 USART 寄存器介绍（F1）

控制寄存器1（CR1）

1 该寄存器需要完成的配置：
位13：使能USART
位12：配置8个数据位
位10：禁止检验控制
位5：使能接收缓冲区非空中断
位3：使能发送
位2：使能接收

2 请参考：STM32F10xxx参考手册_V10（中文版）.pdf，25.6.4节

控制寄存器2（CR2）

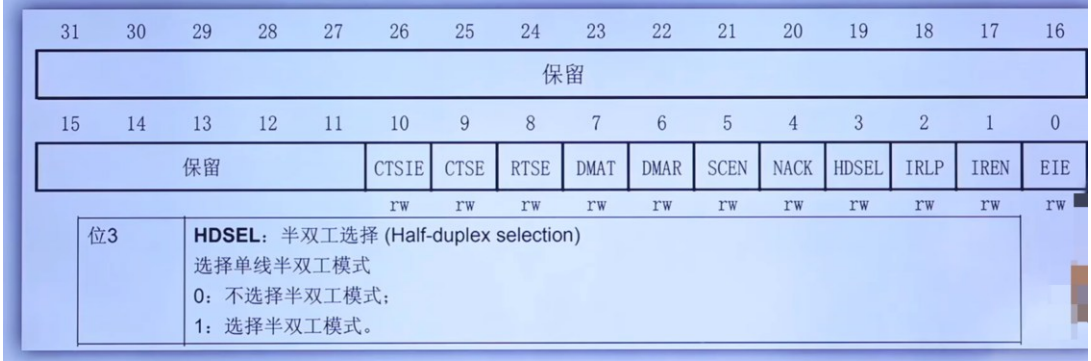
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留	LINEN	STOP[1:0]		CLKEN	CPOL	CPHA	LBCL	保留	LBDIE	LBDL	保留	ADD[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw		rw	rw	rw	rw

位13:12	**STOP**：停止位 (STOP bits) 这2位用来设置停止位的位数 00：1个停止位； 01：0.5个停止位； 10：2个停止位； 11：1.5个停止位； 注：UART4和UART5不能用0.5停止位和1.5停止位。

该寄存器需要完成的配置：配置1个停止位

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，25.6.5节

控制寄存器3（CR3）



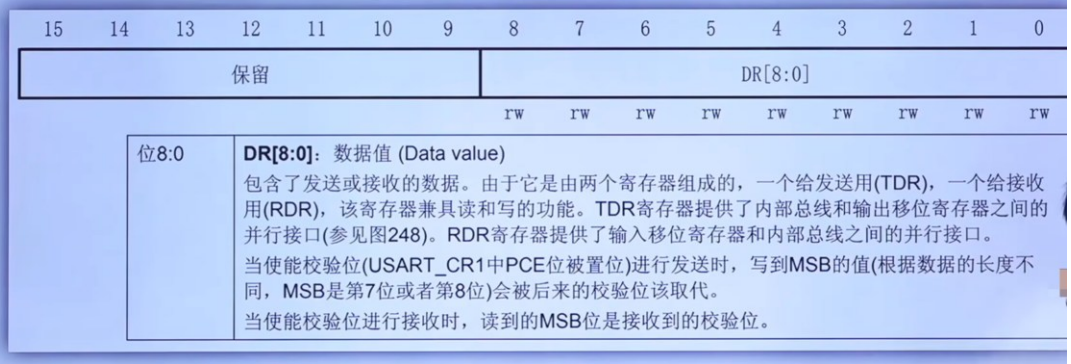
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留					CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位3	**HDSEL**：半双工选择 (Half-duplex selection) 选择单线半双工模式 0：不选择半双工模式； 1：选择半双工模式。

该寄存器需要完成的配置：配置不选择半双工模式

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，25.6.6节

数据寄存器（DR）



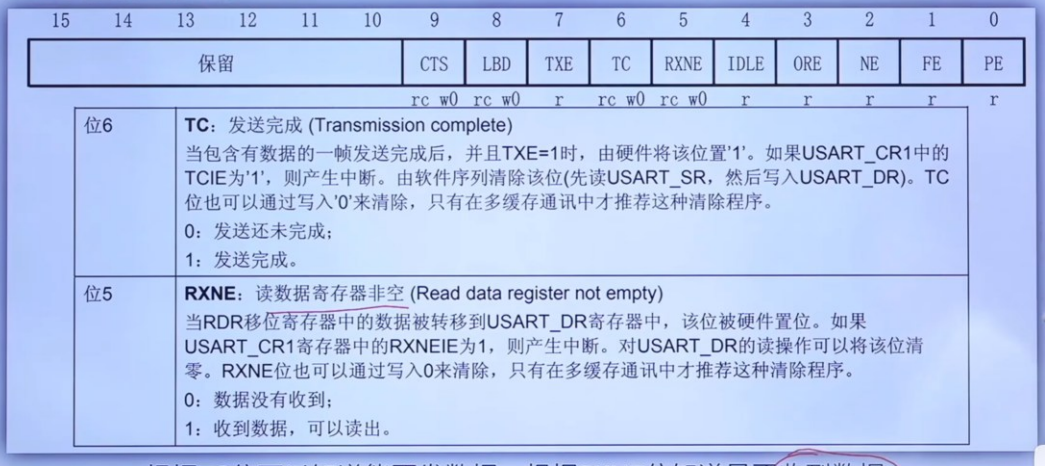
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留							DR[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw

位8:0	**DR[8:0]**：数据值 (Data value) 包含了发送或接收的数据。由于它是由两个寄存器组成的，一个给发送用(TDR)，一个给接收用(RDR)，该寄存器兼具读和写的功能。TDR寄存器提供了内部总线和输出移位寄存器之间的并行接口(参见图248)。RDR寄存器提供了输入移位寄存器和内部总线之间的并行接口。 当使能校验位(USART_CR1中PCE位被置位)进行发送时，写到MSB的值(根据数据的长度不同，MSB是第7位或者第8位)会被后来的校验位该取代。 当使能校验位进行接收时，读到的MSB位是接收到的校验位。

设置好控制和波特率寄存器后，往该寄存器写入数据即可发送，接收数据则读该寄存器

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，25.6.2节

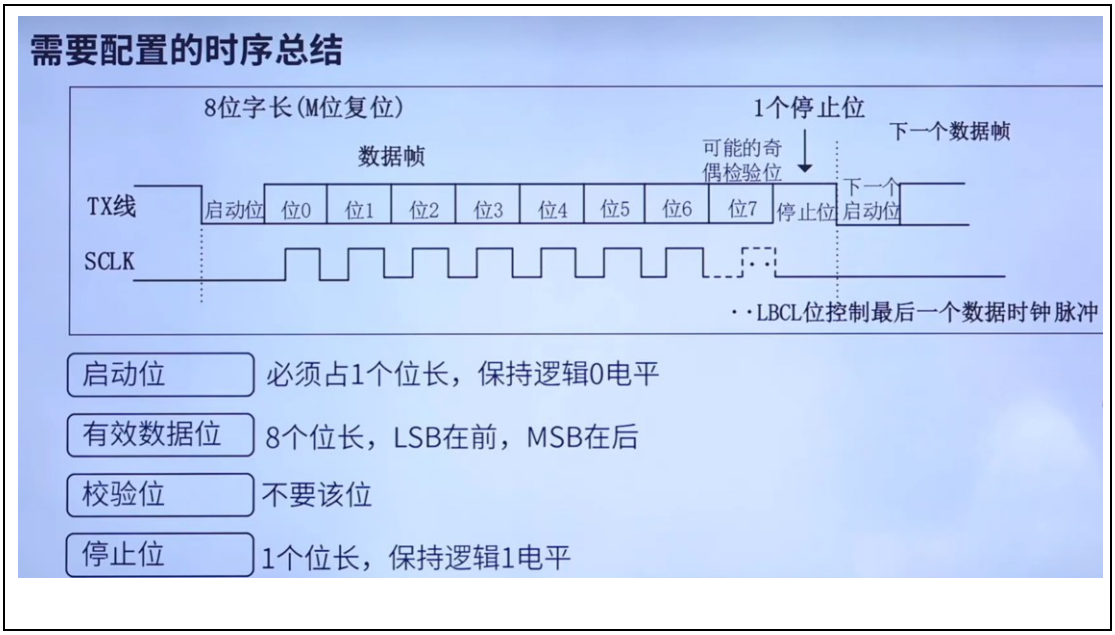
状态寄存器（SR）



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

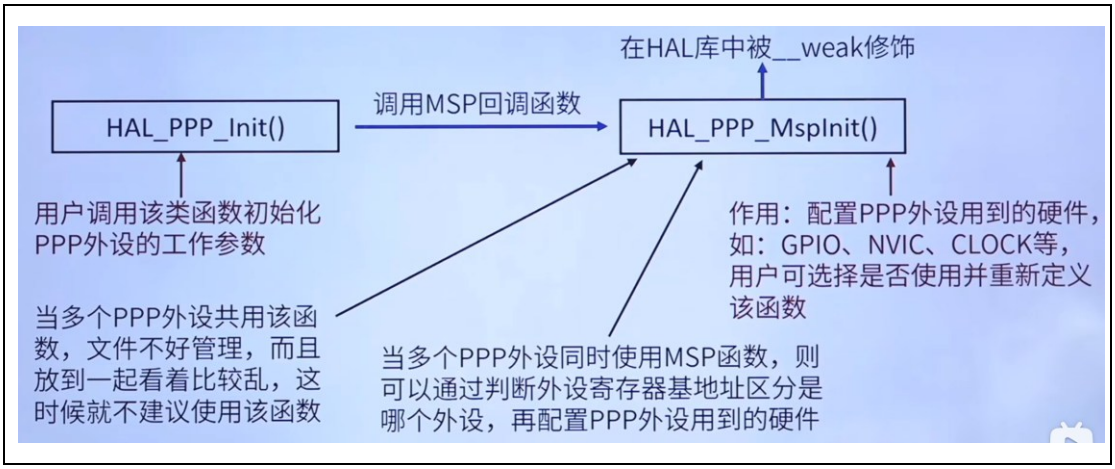
位6	**TC**：发送完成 (Transmission complete) 当包含有数据的一帧发送完成后，并且TXE=1时，由硬件将该位置'1'。如果USART_CR1中的TCIE为'1'，则产生中断。由软件序列清除该位(先读USART_SR，然后写入USART_DR)。TC位也可以通过写入'0'来清除，只有在多缓存通讯中才推荐这种清除程序。 0：发送还未完成； 1：发送完成。
位5	**RXNE**：读数据寄存器非空 (Read data register not empty) 当RDR移位寄存器中的数据被转移到USART_DR寄存器中，该位被硬件置位。如果USART_CR1寄存器中的RXNEIE为1，则产生中断。对USART_DR的读操作可以将该位清零。RXNE位也可以通过写入0来清除，只有在多缓存通讯中才推荐这种清除程序。 0：数据没有收到； 1：收到数据，可以读出。

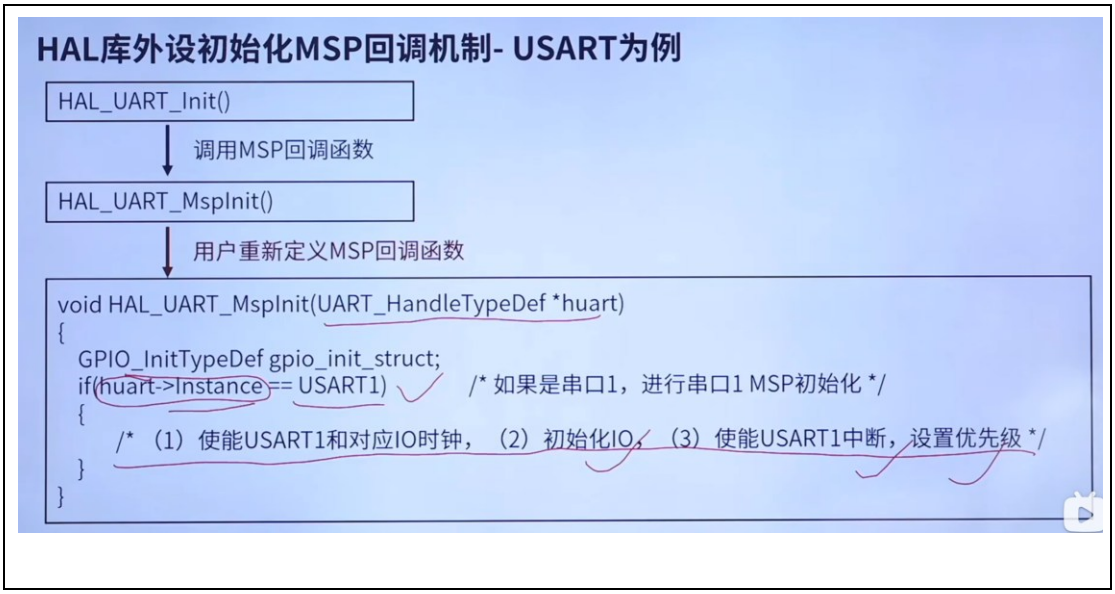
根据TC位可以知道能否发数据，根据RXNE位知道是否收到数据



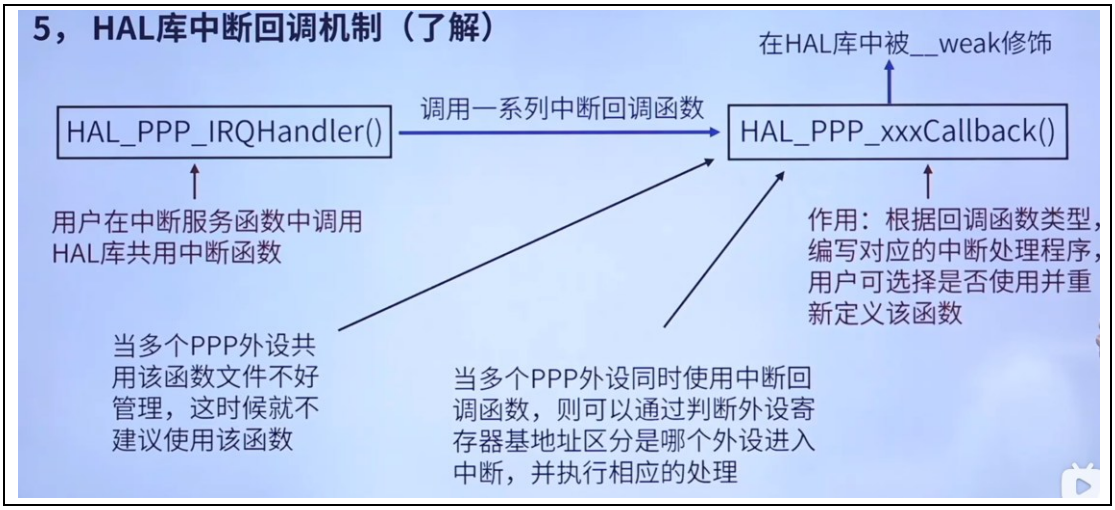
4.6 HAL 库回调机制

4.6.1 HAL 库外设初始化 MSP 回调机制





4.6.2 HAL 库中断回调机制



HAL库中断回调机制 - USART为例（F1）

USARTx_IRQHandler()或者UARTx_IRQHandler()

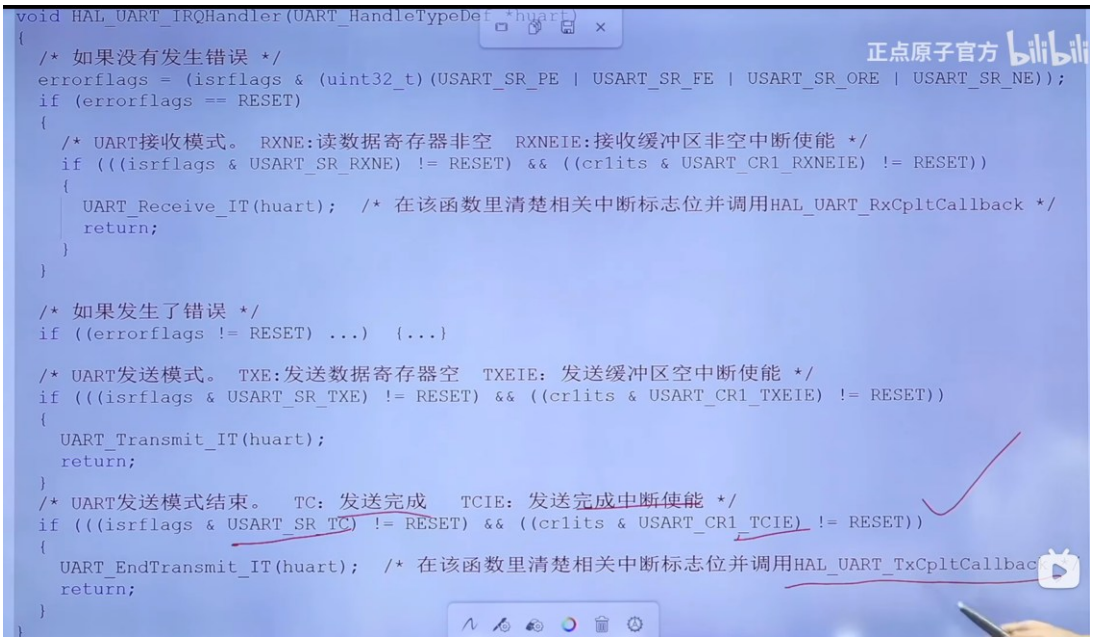
↓ 用户调用HAL库中断共用处理函数

HAL_USART_IRQHandler()或者HAL_UART_IRQHandler()

↓ HAL库自己调用中断回调函数

UART中断回调函数：

```
HAL_UART_TxCpltCallback()                 /* 发送完成回调函数 */
HAL_UART_TxHalfCpltCallback()             /* 半发送完成回调函数 */
HAL_UART_RxCpltCallback()                 /* 接收完成回调函数 */
HAL_UART_RxHalfCpltCallback()             /* 半接收完成回调函数 */
HAL_UART_ErrorCallback()                  /* UART错误回调函数 */
HAL_UART_AbortCpltCallback()              /* UART中止回调函数 */
HAL_UART_AbortTransmitCpltCallback()      /* UART发送中止回调函数 */
HAL_UART_AbortReceiveCpltCallback()       /* UART接收中止回调函数 */
```




```
void HAL_UART_IRQHandler(UART_HandleTypeDef *huart)
{
  /* 如果没有发生错误 */
  errorflags = (isrflags & (uint32_t)(USART_SR_PE | USART_SR_FE | USART_SR_ORE | USART_SR_NE));
  if (errorflags == RESET)
  {
    /* UART接收模式。 RXNE:读数据寄存器非空  RXNEIE:接收缓冲区非空中断使能 */
    if (((isrflags & USART_SR_RXNE) != RESET) && ((cr1its & USART_CR1_RXNEIE) != RESET))
    {
      UART_Receive_IT(huart);  /* 在该函数里清楚相关中断标志位并调用HAL_UART_RxCpltCallback */
      return;
    }
  }

  /* 如果发生了错误 */
  if ((errorflags != RESET) ...)  {...}

  /* UART发送模式。 TXE:发送数据寄存器空  TXEIE: 发送缓冲区空中断使能 */
  if (((isrflags & USART_SR_TXE) != RESET) && ((cr1its & USART_CR1_TXEIE) != RESET))
  {
    UART_Transmit_IT(huart);
    return;
  }
  /* UART发送模式结束。  TC: 发送完成  TCIE: 发送完成中断使能 */
  if (((isrflags & USART_SR_TC) != RESET) && ((cr1its & USART_CR1_TCIE) != RESET))
  {
    UART_EndTransmit_IT(huart);  /* 在该函数里清楚相关中断标志位并调用HAL_UART_TxCpltCallbac
    return;
  }
}
```

4.7 USART/UART 异步通信配置步骤

1，配置串口工作参数	HAL_UART_Init()
2，串口底层初始化	HAL_UART_MspInit()　配置GPIO、NVIC、CLOCK等
3，开启串口异步接收中断	HAL_UART_Receive_IT()
4，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
5，编写中断服务函数	USARTx_IRQHandler()、 UARTx_IRQHandler()
6，串口数据发送	USART_DR， HAL_UART_Transmit()

4.8 HAL 库相关函数介绍

1，HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart)

关键结构体（F1）：

```
typedef struct
{   uint32_t BaudRate;          /* 波特率 */
    uint32_t WordLength;        /* 字长 */
    uint32_t StopBits;          /* 停止位 */
    uint32_t Parity;            /* 奇偶校验位 */
    uint32_t Mode;              /* UART 模式 */
    uint32_t HwFlowCtl;         /* 硬件流设置 */
    uint32_t OverSampling;      /* 过采样设置 */
}UART_InitTypeDef
```

参考：STM32xxxx 开发指南.pdf

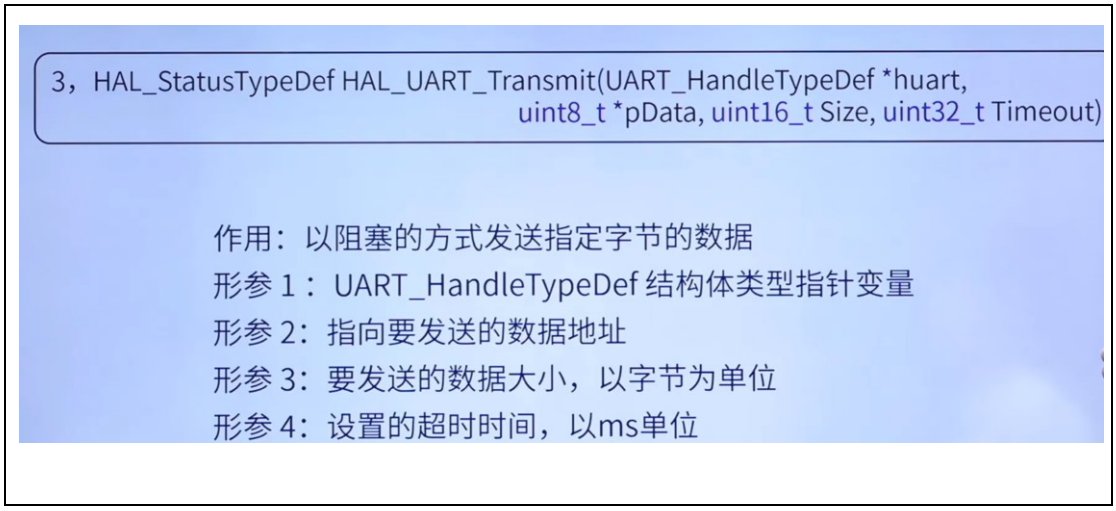
2，HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size)

作用：以中断的方式接收指定字节的数据

形参 1 是 UART_HandleTypeDef 结构体类型指针变量

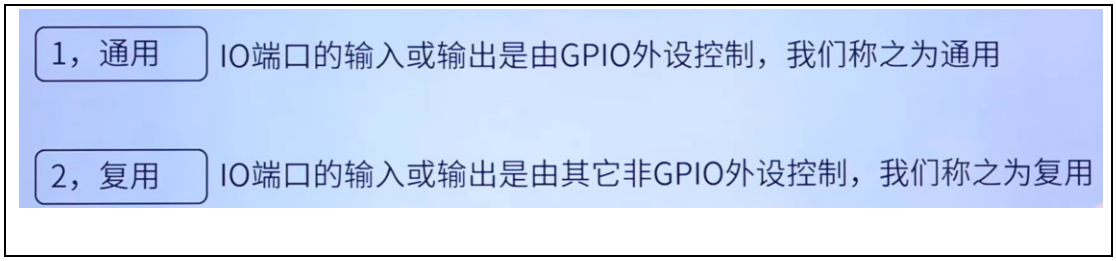
形参 2 是指向接收数据缓冲区

形参 3 是要接收的数据大小，以字节为单位

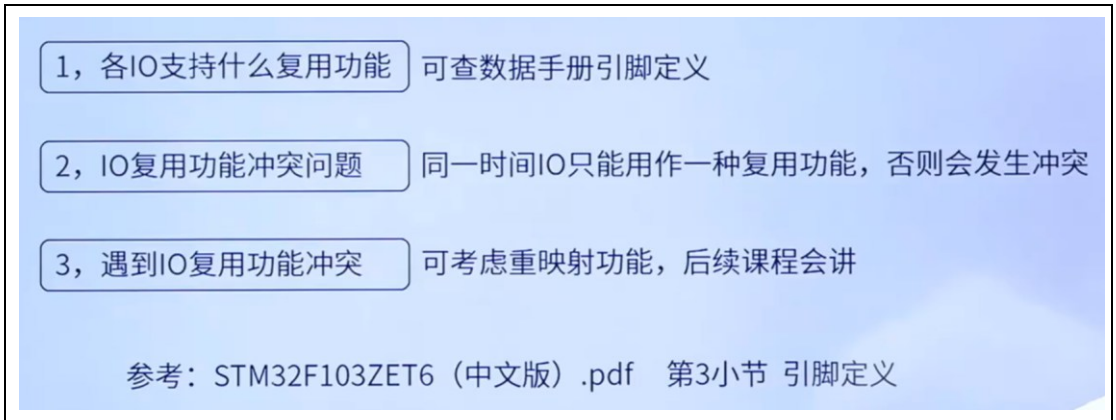


4.9 IO 引脚复用功能

4.9.1 何为复用



4.9.2 STM32F1`的 IO 引脚复用



4.9.3 STM32F4/F7/H7 的 IO 引脚复用

为了解决F1系列存在的IO复用功能冲突问题，F4往后的系列都加入了复用器

复用器特点：

1、每个 IO 引脚都有一个复用器

2、复用器采用 16 路复用功能输入（AF0 到 AF15）

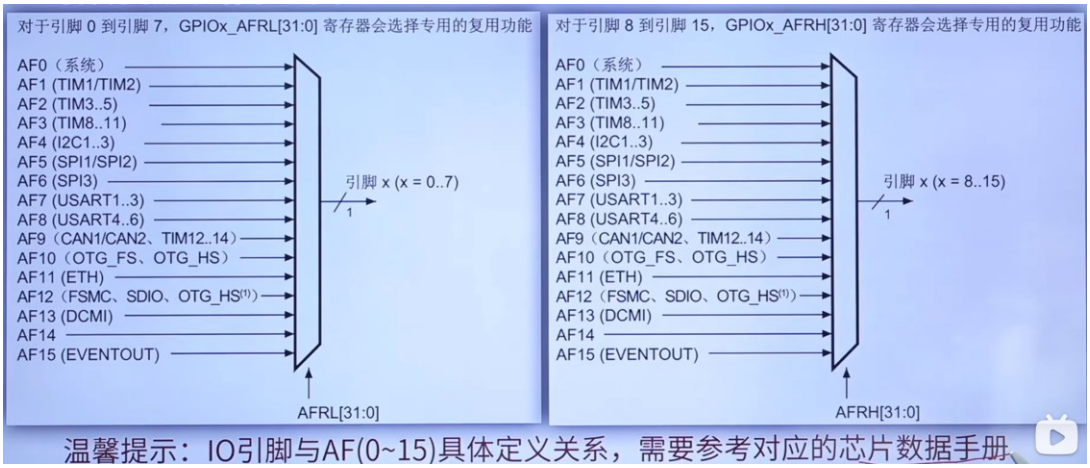
3、复用器一次仅允许一个外设的复用功能 (AF) 连接到 IO 引脚

4、通过GPIOx_AFRL和GPIOx_AFRH寄存器进行配置

温馨提示：复位完成后，所有 IO 都会连接到系统的复用功能 0 (AF0)

参考：STM32F4xx参考手册_V4（中文版）.pdf，7.3.2节

4.9.4 IO 引脚复用映射示意图



温馨提示：IO引脚与AF(0~15)具体定义关系，需要参考对应的芯片数据手册

4.9.5 GPIO 复用功能低位寄存器（AFRL）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **AFRHy**：端口 x 位 y 的复用功能选择 (Alternate function selection for port x bit y) (y = 8.0.15)
这些位通过软件写入，用于配置复用功能 I/O。
AFRHy 选择：

0000：AF0	1000：AF8
0001：AF1	1001：AF9
0010：AF2	1010：AF10
0011：AF3	1011：AF11
0100：AF4	1100：AF12
0101：AF5	1101：AF13
0110：AF6	1110：AF14
0111：AF7	1111：AF15

4.9.6 GPIO 复用功能高位寄存器（AFRH）

GPIO 复用功能高位寄存器(AFRH)

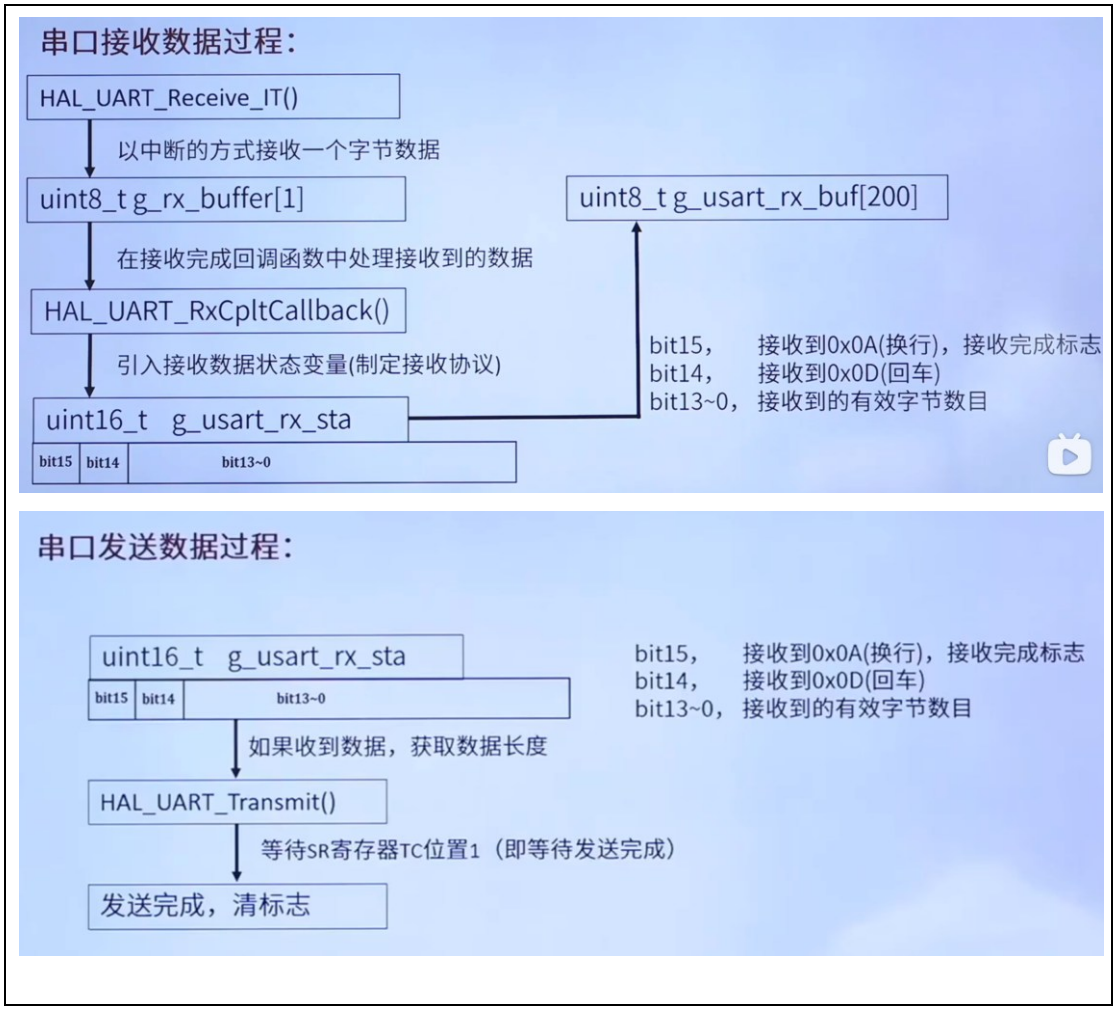
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **AFRHy**：端口 x 位 y 的复用功能选择 (Alternate function selection for port x bit y) (y = 8.0.15)
这些位通过软件写入，用于配置复用功能 I/O。
AFRHy 选择：

0000：AF0	1000：AF8
0001：AF1	1001：AF9
0010：AF2	1010：AF10
0011：AF3	1011：AF11
0100：AF4	1100：AF12
0101：AF5	1101：AF13
0110：AF6	1110：AF14
0111：AF7	1111：AF15

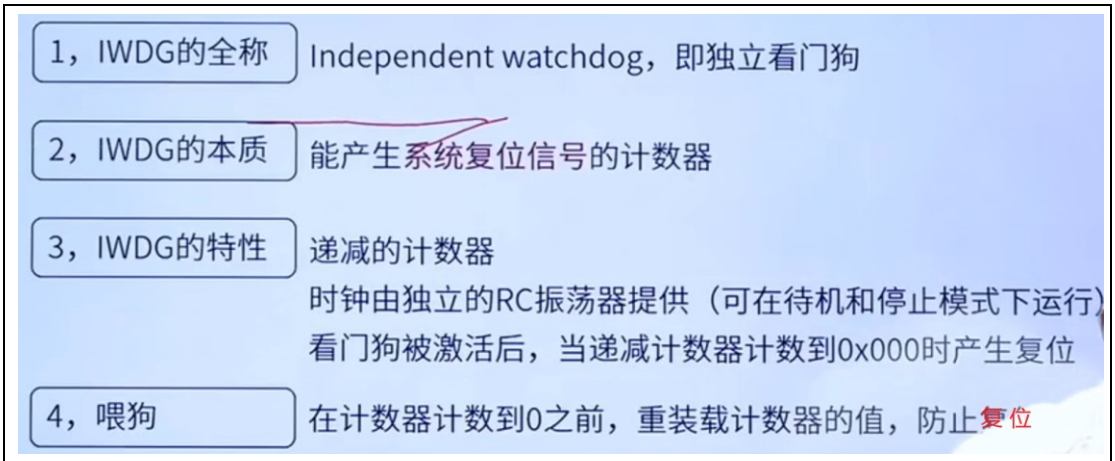
摘自：STM32F4xx参考手册_V4（中文版）.pdf，7.4.10节

4.9.7 串口源码解读

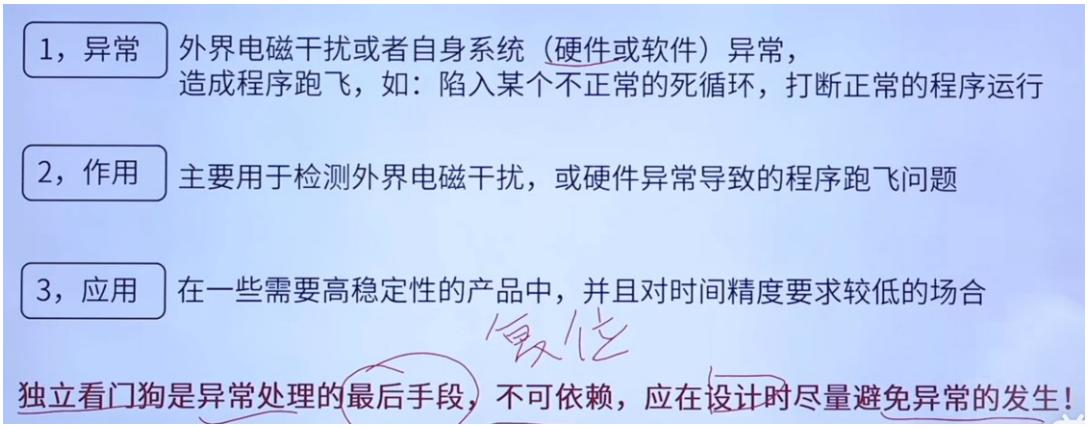

串口接收数据过程：
HAL_UART_Receive_IT()
以中断的方式接收一个字节数据
uint8_t g_rx_buffer[1]
在接收完成回调函数中处理接收到的数据
HAL_UART_RxCpltCallback()
引入接收数据状态变量(制定接收协议)
uint16_t g_usart_rx_sta
bit15
bit14
bit13~0
uint8_t g_usart_rx_buf[200]
bit15， 接收到0x0A(换行)，接收完成标志
bit14， 接收到0x0D(回车)
bit13~0， 接收到的有效字节数目
串口发送数据过程：
uint16_t g_usart_rx_sta
bit15
bit14
bit13~0
bit15， 接收到0x0A(换行)，接收完成标志
bit14， 接收到0x0D(回车)
bit13~0， 接收到的有效字节数目
如果收到数据，获取数据长度
HAL_UART_Transmit()
等待SR寄存器TC位置1（即等待发送完成）
发送完成，清标志

5 IWDG

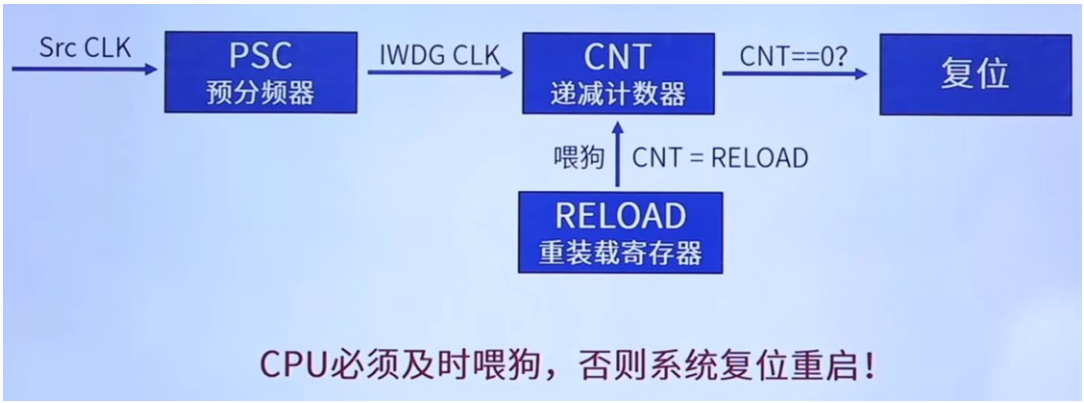
5.1 IWDG 简介



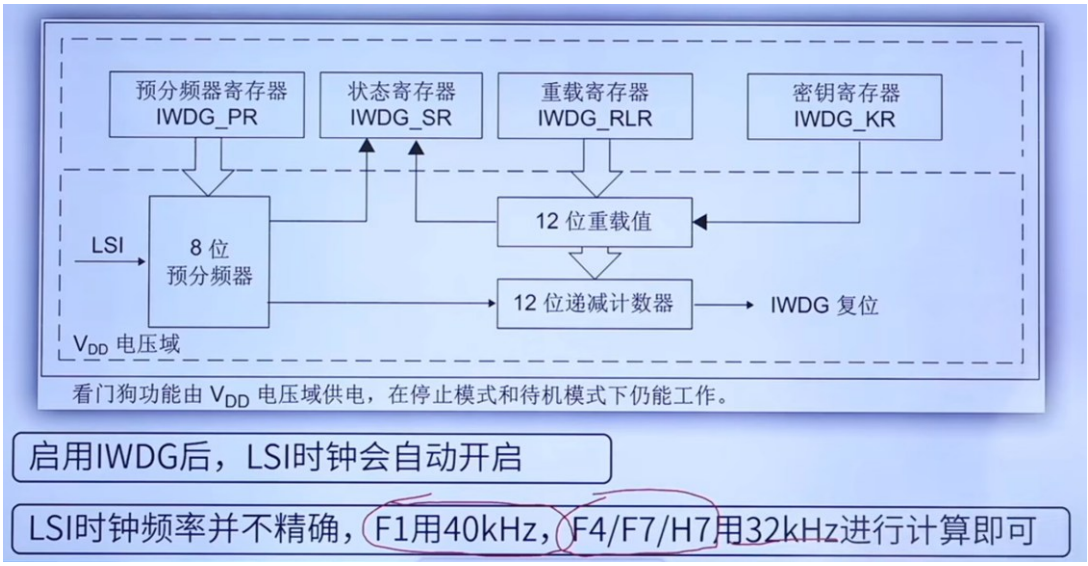
5.2 IWDG 有什么作用



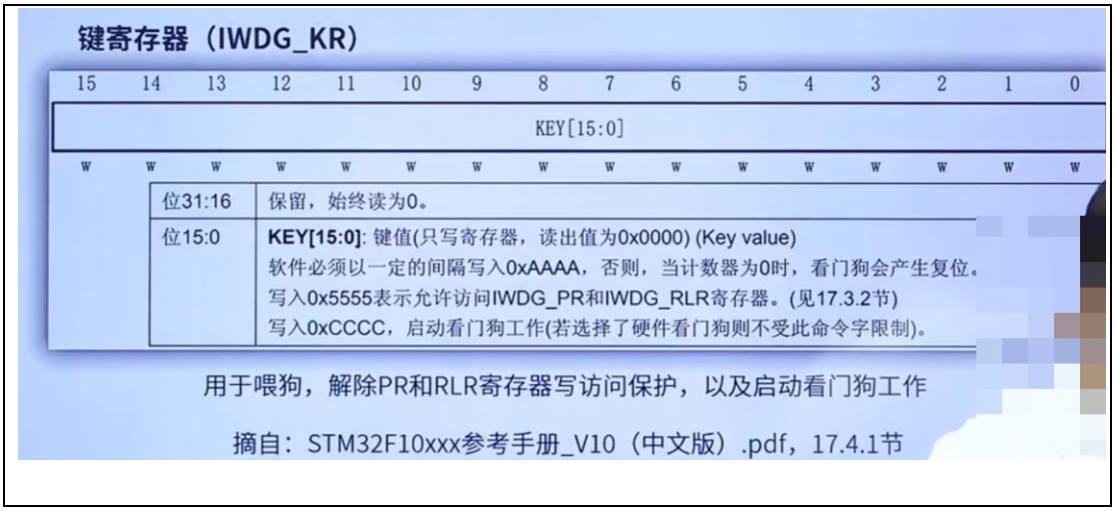
5.3 IWDG 工作原理



5.4 IWDG 框架



5.5 IWDG 寄存器



键寄存器（IWDG_KR）

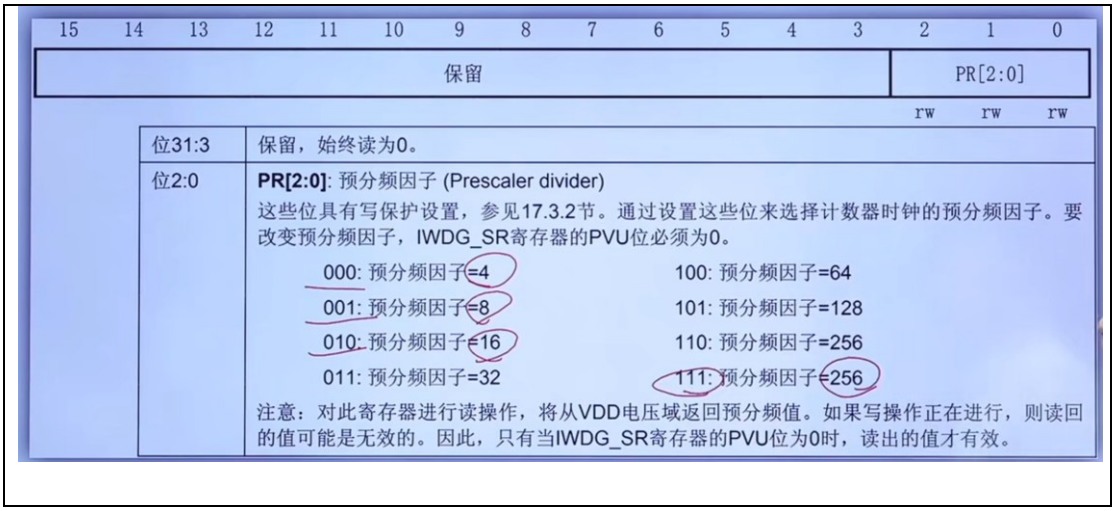
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位31:16	保留，始终读为0。
位15:0	**KEY[15:0]**: 键值(只写寄存器，读出值为0x0000) (Key value) 软件必须以一定的间隔写入0xAAAA，否则，当计数器为0时，看门狗会产生复位。 写入0x5555表示允许访问IWDG_PR和IWDG_RLR寄存器。(见17.3.2节) 写入0xCCCC，启动看门狗工作(若选择了硬件看门狗则不受此命令字限制)。

用于喂狗，解除PR和RLR寄存器写访问保护，以及启动看门狗工作

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，17.4.1节

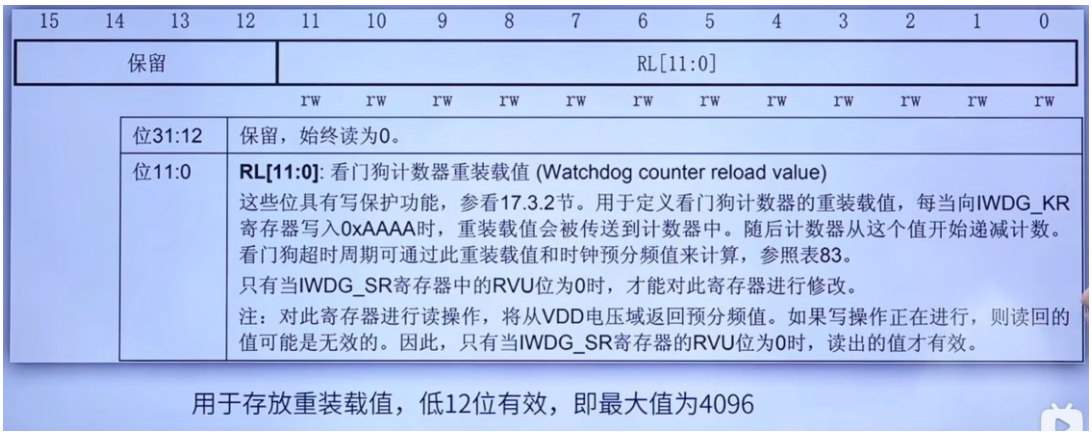
5.6 预分频寄存器（IWDG_PR）



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留													PR[2:0]		
													rw	rw	rw

位31:3	保留，始终读为0。
位2:0	**PR[2:0]**: 预分频因子 (Prescaler divider) 这些位具有写保护设置，参见17.3.2节。通过设置这些位来选择计数器时钟的预分频因子。要改变预分频因子，IWDG_SR寄存器的PVU位必须为0。 000: 预分频因子=4　　100: 预分频因子=64 001: 预分频因子=8　　101: 预分频因子=128 010: 预分频因子=16　　110: 预分频因子=256 011: 预分频因子=32　　111: 预分频因子=256 注意：对此寄存器进行读操作，将从VDD电压域返回预分频值。如果写操作正在进行，则读回的值可能是无效的。因此，只有当IWDG_SR寄存器的PVU位为0时，读出的值才有效。

5.7 重装载寄存器（IWDG_RLR）

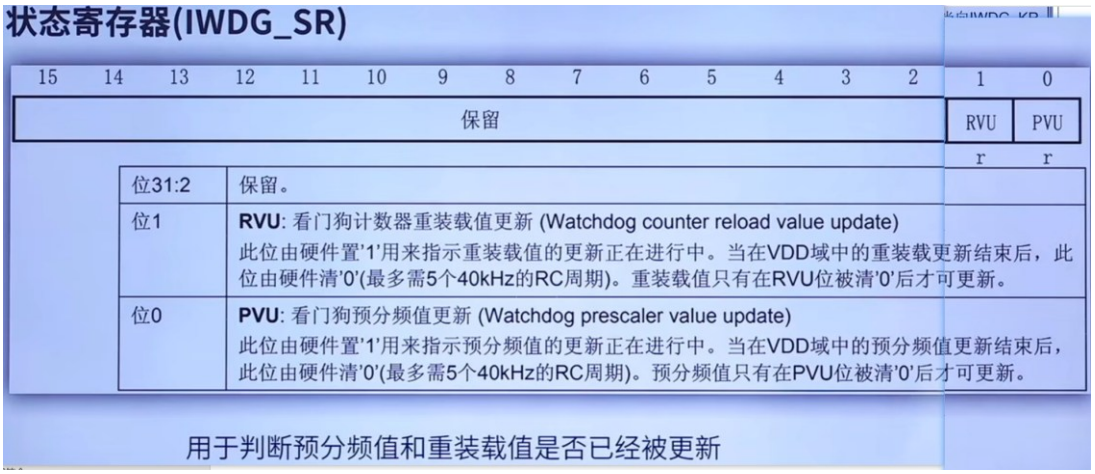


15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留				RL[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位31:12	保留，始终读为0。
位11:0	**RL[11:0]**: 看门狗计数器重装载值 (Watchdog counter reload value) 这些位具有写保护功能，参看17.3.2节。用于定义看门狗计数器的重装载值，每当向IWDG_KR寄存器写入0xAAAA时，重装载值会被传送到计数器中。随后计数器从这个值开始递减计数。看门狗超时周期可通过此重装载值和时钟预分频值来计算，参照表83。 只有当IWDG_SR寄存器中的RVU位为0时，才能对此寄存器进行修改。 注：对此寄存器进行读操作，将从VDD电压域返回预分频值。如果写操作正在进行，则读回的值可能是无效的。因此，只有当IWDG_SR寄存器的RVU位为0时，读出的值才有效。

用于存放重装载值，低12位有效，即最大值为4096

5.8 状态寄存器（IWDG_SR）



状态寄存器(IWDG_SR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留														RVU	PVU
														r	r

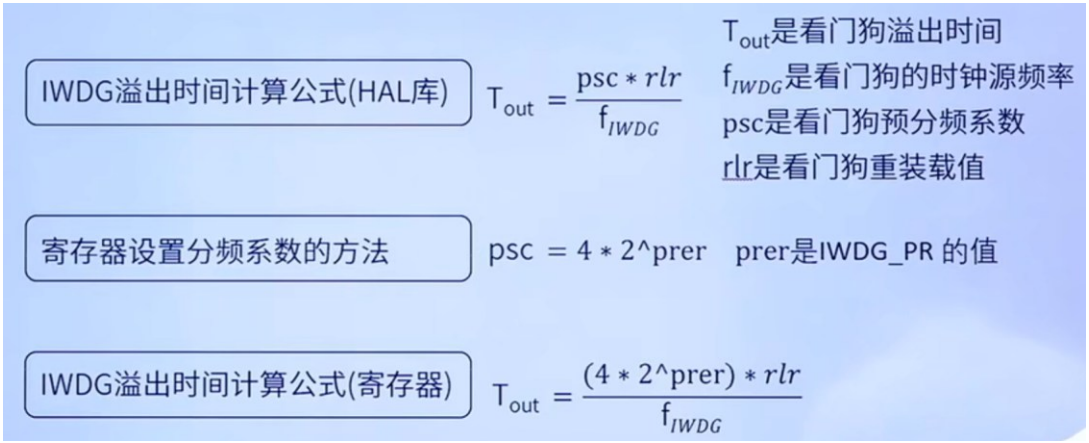
位31:2	保留。
位1	**RVU**: 看门狗计数器重装载值更新 (Watchdog counter reload value update) 此位由硬件置'1'用来指示重装载值的更新正在进行中。当在VDD域中的重装载更新结束后，此位由硬件清'0'(最多需5个40kHz的RC周期)。重装载值只有在RVU位被清'0'后才可更新。
位0	**PVU**: 看门狗预分频值更新 (Watchdog prescaler value update) 此位由硬件置'1'用来指示预分频值的更新正在进行中。当在VDD域中的预分频值更新结束后，此位由硬件清'0'(最多需5个40kHz的RC周期)。预分频值只有在PVU位被清'0'后才可更新。

用于判断预分频值和重装载值是否已经被更新

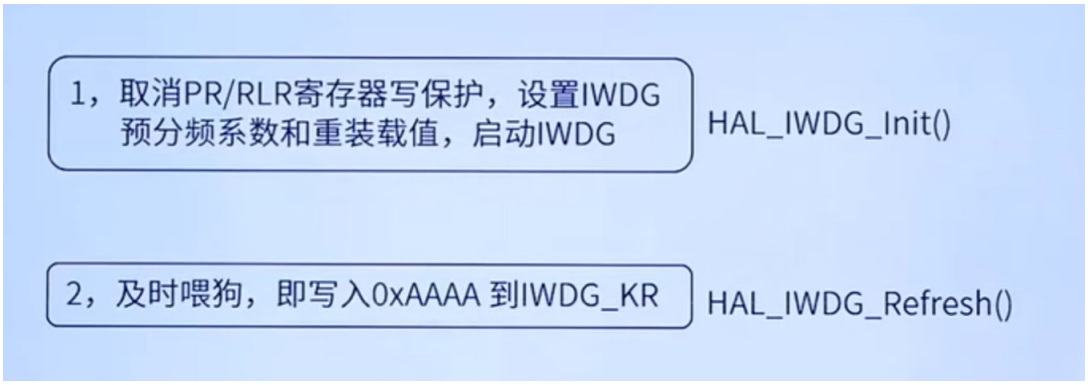
5.9 寄存器配置操作步骤

1. 通过在键寄存器 (IWDG_KR) 中写入 0xCCCC 来使能 IWDG。
2. 通过在键寄存器 (IWDG_KR) 中写入 0x5555 来使能寄存器访问。
3. 通过将预分频器寄存器 (IWDG_PR) 编程为 0~7 中的数值来配置预分频器。
4. 对重载寄存器 (IWDG_RLR) 进行写操作。
5. 等待寄存器更新 (IWDG_SR = 0x0000 0000)。
6. 刷新计数器值为 IWDG_RLR 的值 (IWDG_KR = 0xAAAA)。

5.10 IWDG 溢出时间计算

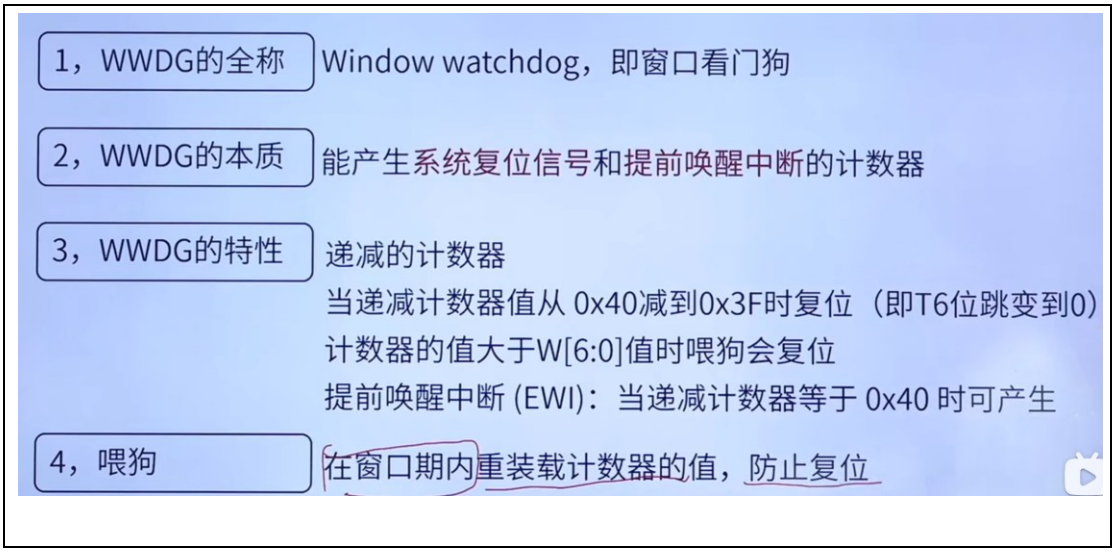


5.11 IWDG 配置步骤

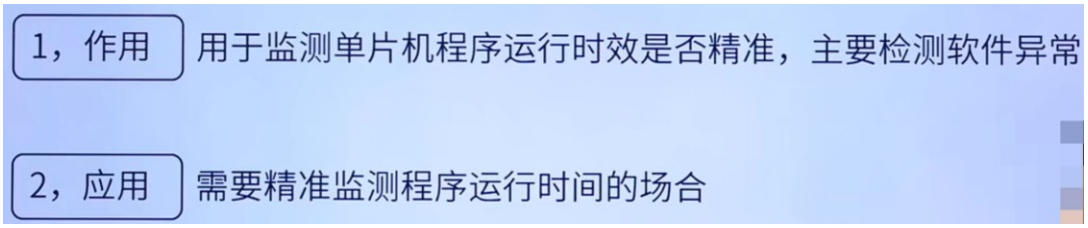


6 WWDG

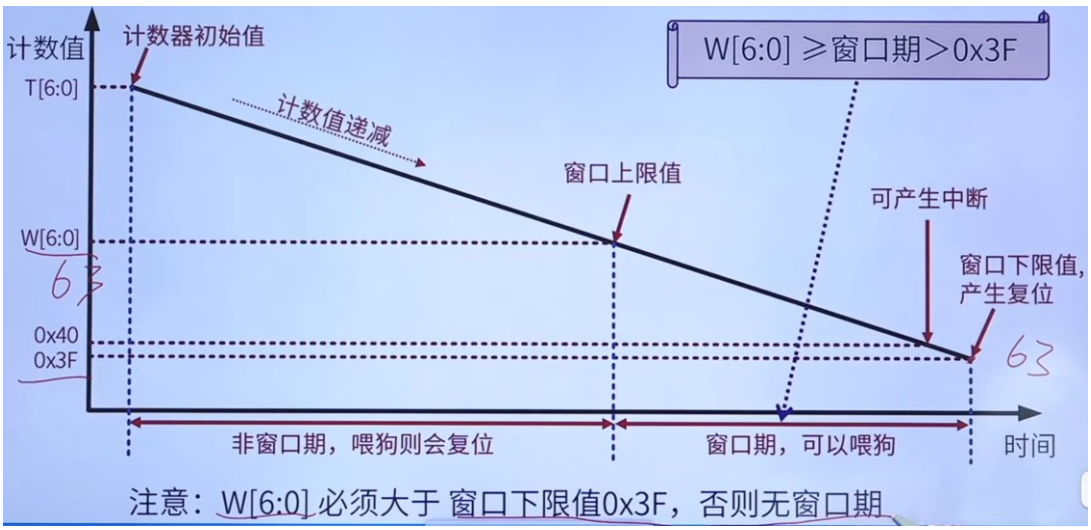
6.1 WWDG 简介



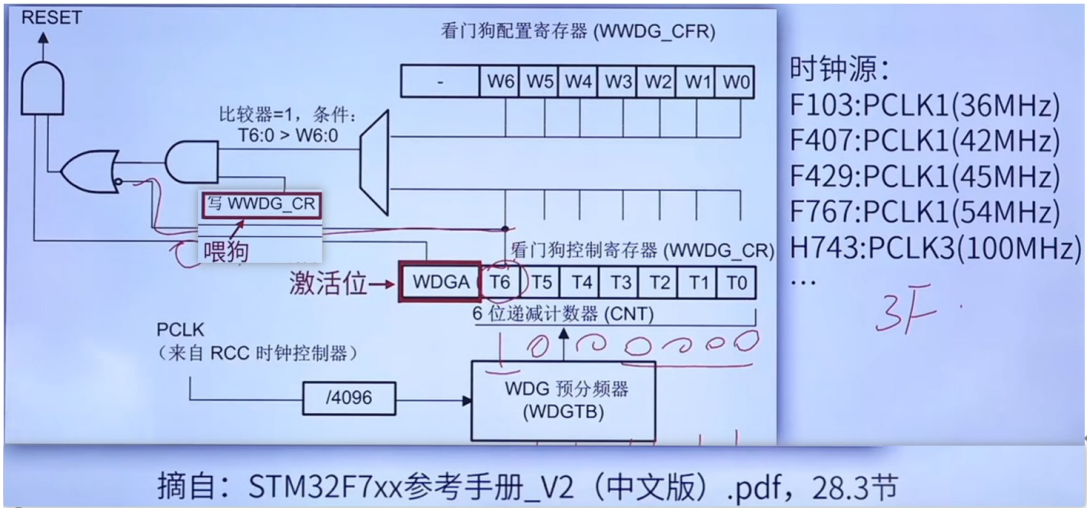
6.2 WWDG 有什么作用



6.3 WWDG 工作原理



6.4 WWDG 框图



6.5 WWDG 寄存器

控制寄存器 (WWDG_CR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGA	T[6:0]						
								rs	rw						

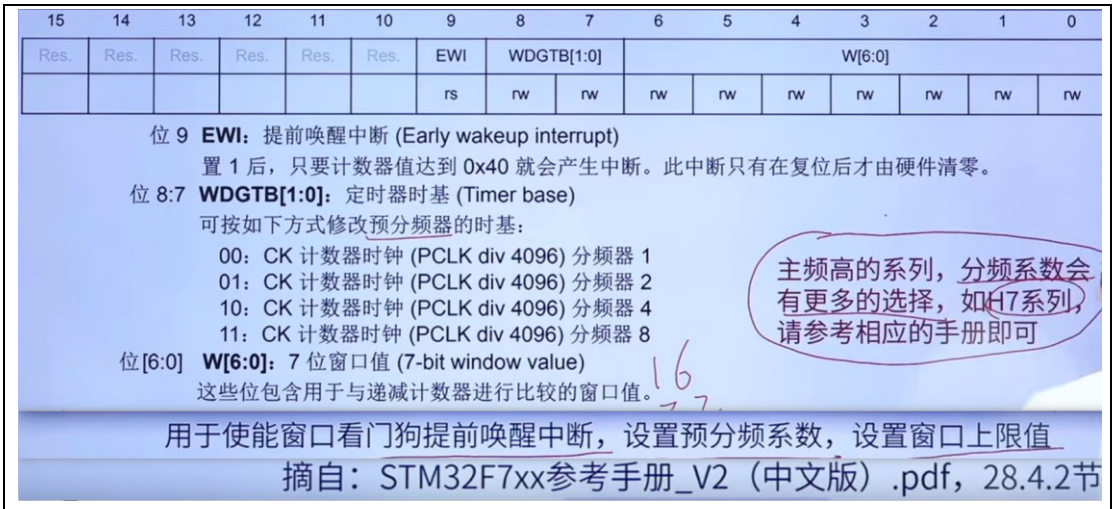
位 7 **WDGA**：激活位 (Activation bit)
此位由软件置 1，只有复位后才由硬件清零。当 WDGA = 1 时，看门狗可产生复位。
0：禁止看门狗
1：使能看门狗

位 6:0 **T[6:0]**：7 位计数器（MSB 到 LSB）
这些位用来存储看门狗计数器的值。它每隔 $(4096 \times 2^{WDGTB[1:0]})$ PCLK 个周期递减一次。当它从 0x40 滚动到 0x3F（T6 清零）时会产生复位。

用于使能窗口看门狗工作，以及重装载计数器值（即喂狗）

摘自：STM32F7xx参考手册_V2（中文版）.pdf，28.4.1节

6.6 配置寄存器（WWDG_CFR）



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	EWI	WDGTB[1:0]		W[6:0]						
						rs	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 9 **EWI**：提前唤醒中断 (Early wakeup interrupt)
置 1 后，只要计数器值达到 0x40 就会产生中断。此中断只有在复位后才由硬件清零。

位 8:7 **WDGTB[1:0]**：定时器时基 (Timer base)
可按如下方式修改预分频器的时基：
00：CK 计数器时钟 (PCLK div 4096) 分频器 1
01：CK 计数器时钟 (PCLK div 4096) 分频器 2
10：CK 计数器时钟 (PCLK div 4096) 分频器 4
11：CK 计数器时钟 (PCLK div 4096) 分频器 8

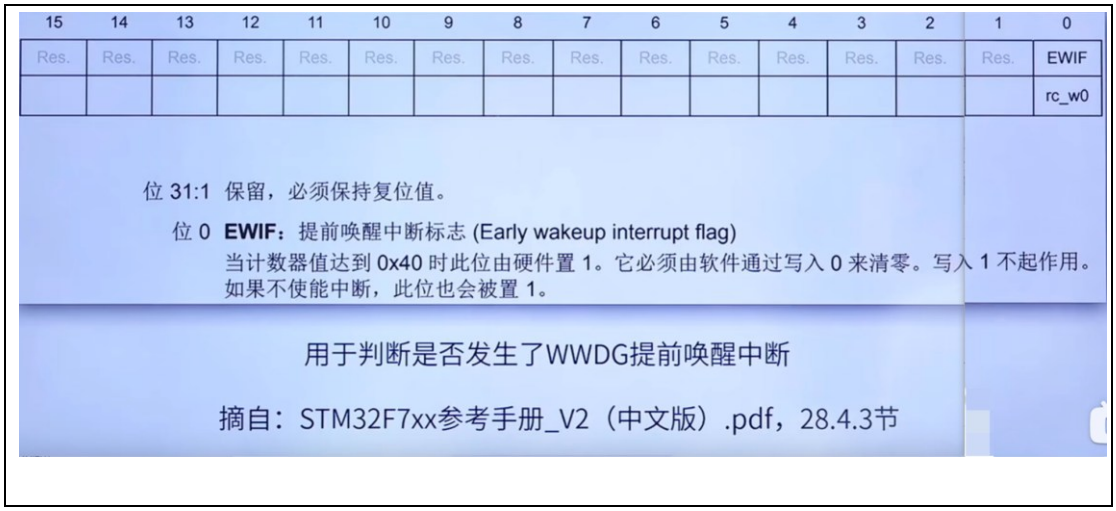
位[6:0] **W[6:0]**：7 位窗口值 (7-bit window value)
这些位包含用于与递减计数器进行比较的窗口值。

主频高的系列，分频系数会有更多的选择，如H7系列，请参考相应的手册即可

用于使能窗口看门狗提前唤醒中断，设置预分频系数，设置窗口上限值

摘自：STM32F7xx参考手册_V2（中文版）.pdf，28.4.2节

6.7 状态寄存器



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EWIF
															rc_w0

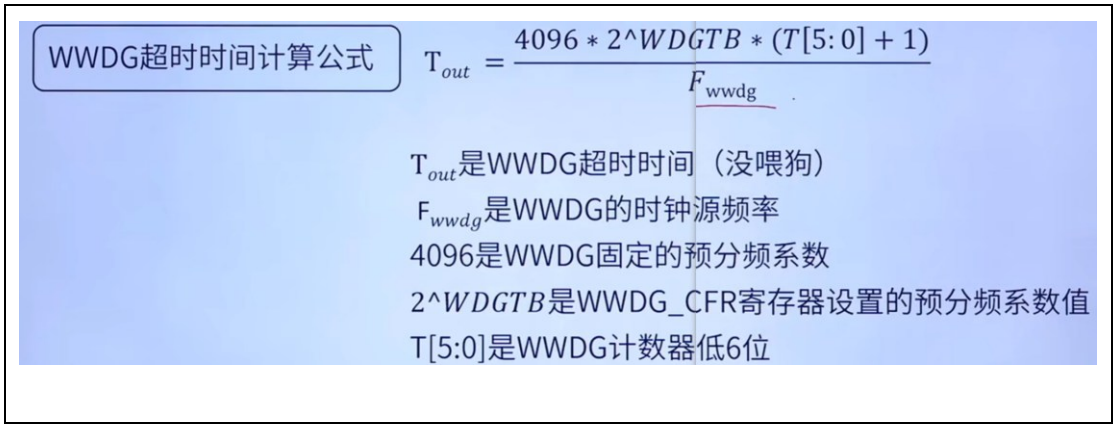
位 31:1 保留，必须保持复位值。

位 0 **EWIF**：提前唤醒中断标志 (Early wakeup interrupt flag)
当计数器值达到 0x40 时此位由硬件置 1。它必须由软件通过写入 0 来清零。写入 1 不起作用。
如果不使能中断，此位也会被置 1。

用于判断是否发生了WWDG提前唤醒中断

摘自：STM32F7xx参考手册_V2（中文版）.pdf，28.4.3节

6.8 WWDG 超时时间计算



WWDG超时时间计算公式 $T_{out} = \frac{4096 * 2^{WDGTB} * (T[5:0] + 1)}{F_{wwdg}}$

T_{out}是WWDG超时时间（没喂狗）

F_{wwdg}是WWDG的时钟源频率

4096是WWDG固定的预分频系数

2^{WDGTB}是WWDG_CFR寄存器设置的预分频系数值

T[5:0]是WWDG计数器低6位

6.9 WWDG 配置步骤

1，WWDG工作参数初始化	HAL_WWDG_Init()
2，WWDG Msp初始化	HAL_WWDG_MspInit() 配置NVIC、CLOCK等
3，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
4，编写中断服务函数	WWDG_IRQHandler() →HAL_WWDG_IRQHandler
5，重定义提前唤醒回调函数	HAL_WWDG_EarlyWakeupCallback()
6，在窗口期内喂狗	HAL_WWDG_Refresh()

6.9.1 相关 HAL 库函数介绍

函数	主要寄存器	主要功能
HAL_WWDG_Init	WWDG_CR/WWDG_CFR	使能WWDG，设置预分频系数和窗口值等
HAL_WWDG_Refresh	WWDG_CR	重装载计数器，即喂狗

```
typedef struct
{
  WWDG_TypeDef *Instance; /* WWDG 寄存器基地址 */
  WWDG_InitTypeDef Init;    /* WWDG 初始化参数 */
}WWDG_HandleTypeDef;
```

```
typedef struct
{
  uint32_t Prescaler; /* 预分频系数 */
  uint32_t Window;    /* 窗口值 */
  uint32_t Counter;   /* 计数器值 */
  uint32_t EWIMode;   /* 提前唤醒中断使能 */
}WWDG_InitTypeDef;
```

6.10 IWDG 和 WWDG 的主要区别

对比点	独立看门狗	窗口看门狗
时钟源	LSI(40KHz或32KHz)	PCLK1或PCLK3
复位条件	递减计数到0	计数值大于W[6:0]值喂狗或减到0x3F
中断	没有中断	计数值减到0x40可产生中断
递减计数器位数	12位（最大计数范围：4096~0）	7位（最大计数范围：127~63）
应用场合	防止程序跑飞，死循环，死机	检测程序时效，防止软件异常

7 TIMER

7.1 定时器概述

7.1.1 软件定时原理

使用纯软件（CPU死等）的方式实现定时（延时）功能

```
void delay_us(uint32_t us)
{
   us *= 72;
   while(us--);
}
```

缺点：

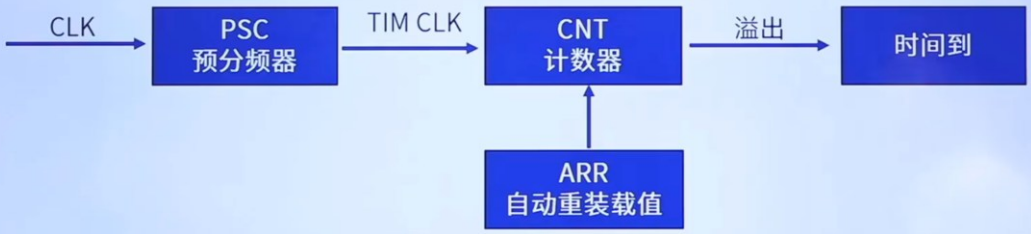
1，延时不精准

2，CPU死等

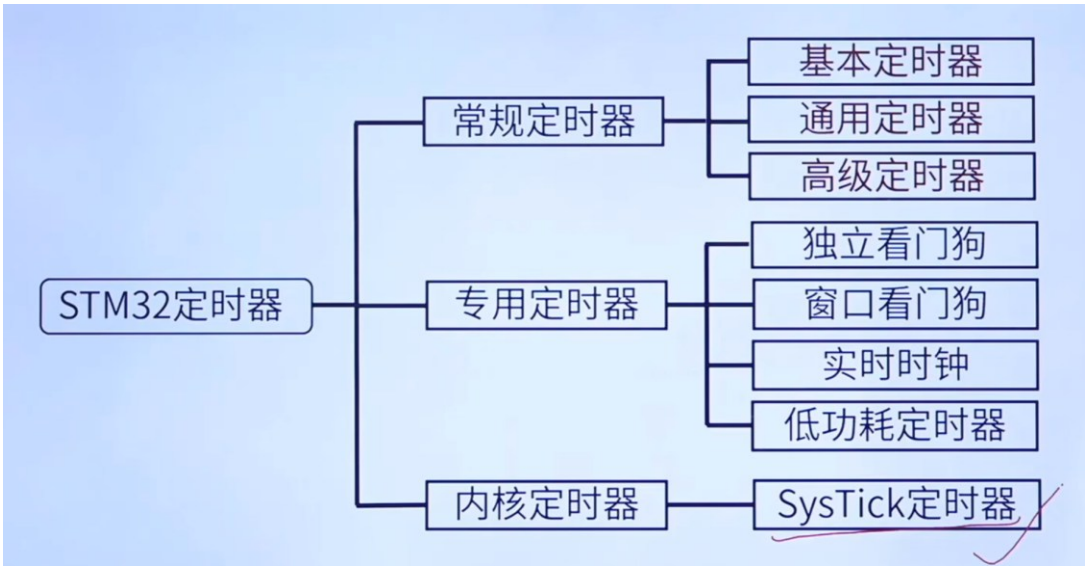
7.1.2 定时器定时原理

使用精准的时基，通过硬件的方式，实现定时功能

定时器核心就是计数器



7.1.3 STM32 定时器分类



7.1.4 STM32 定时器特性表

预分频系数是 1~65536

F1

定时器类型	定时器	计数器位数	计数模式	预分频系数（整数）	产生DMA请求	捕获/比较通道	互补输出
基本定时器	TIM6 TIM7	16	递增	1~65535	可以	0	无
通用定时器	TIM2 TIM3 TIM4 TIM5	16	递增、递减、中央对齐	1~65535	可以	4	无
高级定时器	TIM1 TIM8	16	递增、递减、中央对齐	1~65535	可以	4	有

H7

定时器类型	定时器	计数器位数	计数模式	预分频系数（整数）	产生DMA请求	捕获/比较通道	互补输出
基本定时器	TIM6 TIM7	16	递增	1~65535	可以	0	无
通用定时器	TIM2 TIM5	32	递增、递减、中心对齐	1~65535	可以	4	无
	TIM3 TIM4	16	递增、递减、中心对齐	1~65535	可以	4	无
	TIM12	16	递增	1~65535	不可以	2	无
	TIM13 TIM14	16	递增	1~65535	不可以	1	无
	TIM15	16	递增	1~65535	可以	2	有
	TIM16 TIM17	16	递增	1~65535	可以	1	有
高级定时器	TIM1 TIM8	16	递增、递减、中心对齐	1~65535	可以	4	有

7.1.5 STM32 基本，通用，高级定时器功能整体的区别

定时器类型	主要功能
基本定时器	没有输入输出通道，常用作时基，即定时功能
通用定时器	具有多路独立通道，可用于输入捕获/输出比较，也可用作时基
高级定时器	除具备通用定时器所有功能外，还具备带死区控制的互补信号输出、刹车输入等功能（可用于电机控制、数字电源设计等）

7.2 基本定时器

7.2.1 基本计时器简介

1，基本定时器　TIM6/TIM7

2，主要特性

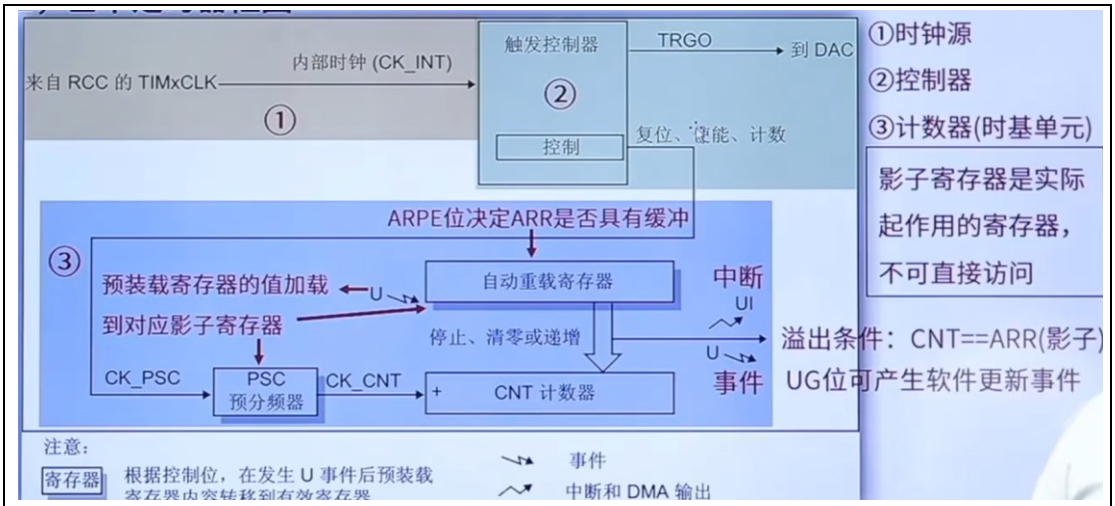
16位递增计数器（计数值：0~65535）

16位预分频器（分频系数：1~65536）

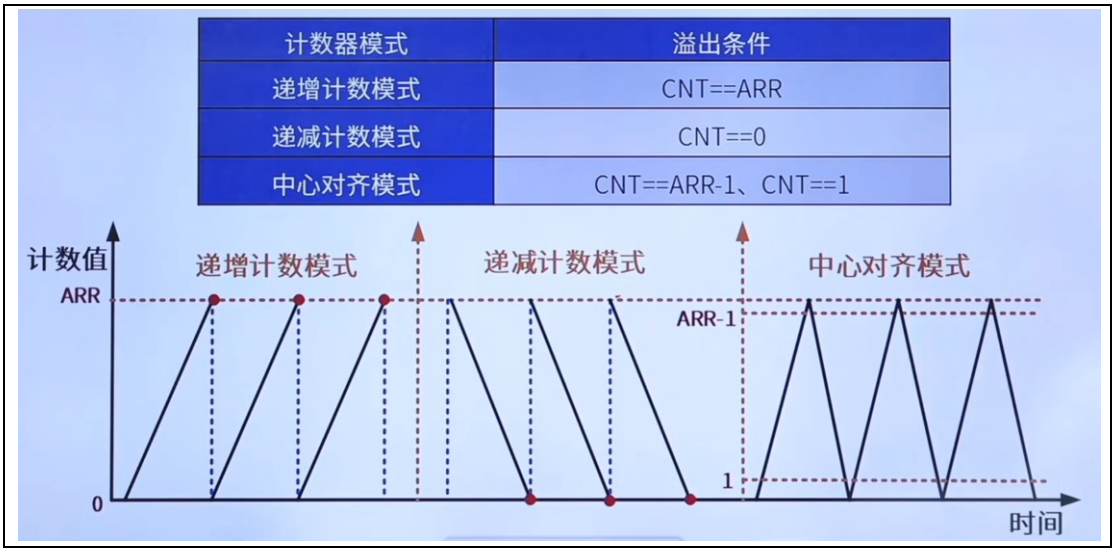
可用于触发DAC

在更新事件（计数器溢出）时，可产生中断/DMA请求

7.2.2 基本计时器框图



7.2.3 定时器计数模式及溢出条件

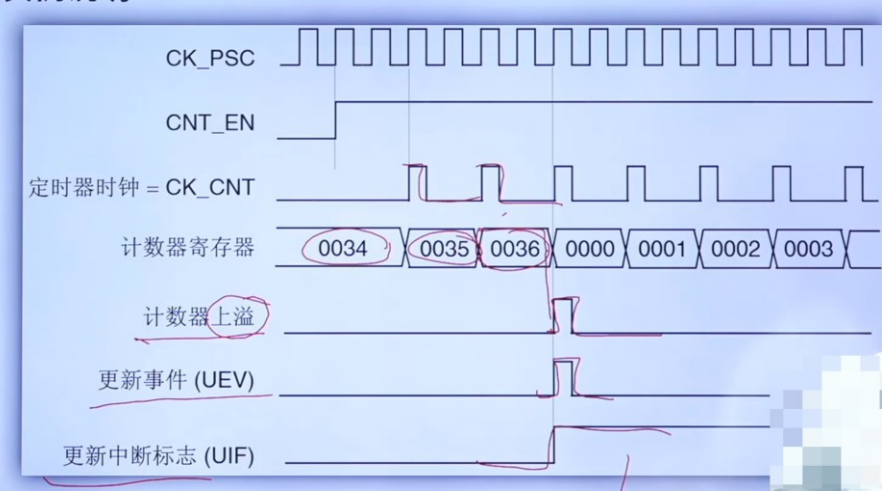


计数器模式	溢出条件
递增计数模式	CNT==ARR
递减计数模式	CNT==0
中心对齐模式	CNT==ARR-1、CNT==1

递增计数模式实例说明

PSC = 1

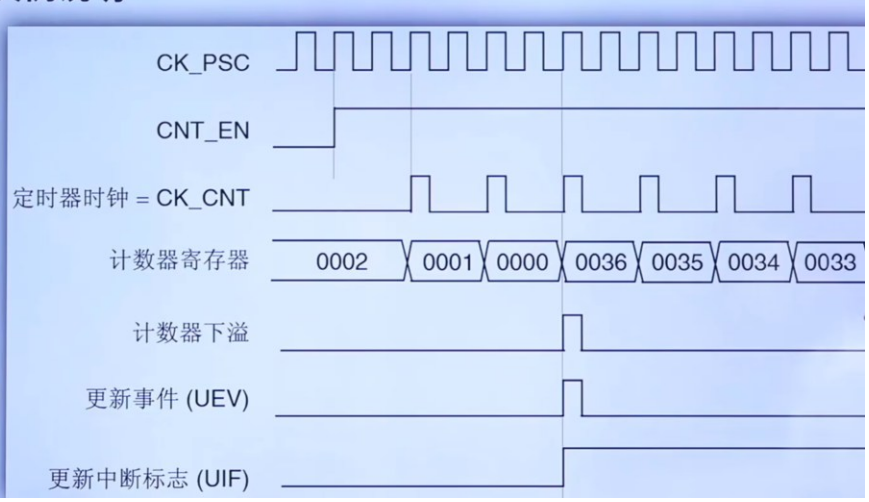
ARR = 36



递减计数模式实例说明

PSC = 1

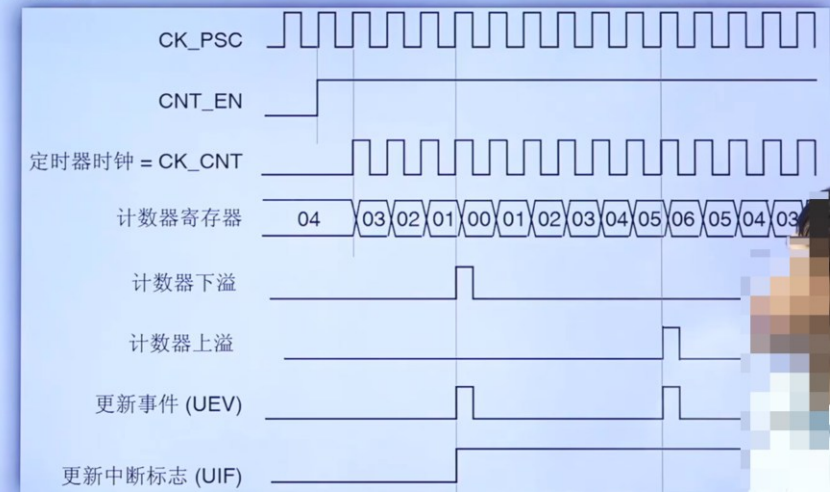
ARR = 36



中心对齐模式实例说明

PSC = 0

ARR = 6



7.2.4 定时器中断实验相关寄存器（F1）

TIM6 和TIM7 控制寄存器 1(TIMx_CR1)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留								ARPE	保留			OPM	URS	UDIS	CEN
res								rw	res			rw	rw	rw	rw

位7	**ARPE**：自动重装载预装载使能 (Auto-reload preload enable) 0：TIMx_ARR寄存器没有缓冲 1：TIMx_ARR寄存器具有缓冲
位0	**CEN**：计数器使能 (Counter enable) 0：关闭计数器 1：使能计数器 注：门控模式只能在软件已经设置了CEN位时有效，而触发模式可以自动地由硬件设置CEN位。 在单脉冲模式下，当产生更新事件时CEN被自动清除。

用于设置ARR寄存器是否具有缓冲，使能/关闭计数器

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，15.4.1节

TIM6 和TIM7 DMA/中断使能寄存器(TIMx_DIER)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留							UDE	保留							UIE
res							rw	res							rw

位15:9	保留，始终读为0。
位8	**UDE**：更新DMA请求使能 (Update DMA request enable) 0：禁止更新DMA请求 1：使能更新DMA请求
位0	**UIE**：更新中断使能 (Update interrupt enable) 0：禁止更新中断 1：使能更新中断

用于使能更新中断

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，15.4.3节

TIM6 和TIM7 状态寄存器(TIMx_SR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留															UIF
res															rc w0

位15:1	保留，始终读为0。
位0	**UIF**：更新中断标志 (Update interrupt flag) 硬件在更新中断时设置该位，它由软件清除。 0：没有产生更新。 1：产生了更新中断。下述情况下由硬件设置该位： – 计数器产生上溢或下溢并且TIMx_CR1中的UDIS=0； – 如果TIMx_CR1中的URS=0并且UDIS=0，当使用TIMx_EGR寄存器的UG位重新初始化计数器CNT时。

用于判断是否发生了更新中断，由硬件置1，软件清零

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，15.4.4节

TIM6 和TIM7 计数器(TIMx_CNT)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw															

位15:0	**CNT[15:0]**：计数器数值 (Counter value)

计数器实时数值，可用于设置计时器初始值，范围：0~65535

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，15.4.6节

TIM6 和TIM7 预分频器(TIMx_PSC)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw															

位15:0	**PSC[15:0]**：预分频器数值 (Prescaler value) 计数器的时钟频率CK_CNT等于f_{CK_PSC}/(PSC[15:0]+1)。 在每一次更新事件时，PSC的数值被传送到实际的预分频寄存器中。

用于设置预分频系数，范围：0~65535，实际预分频系数等于PSC+1

摘自：STM32F10xxx参考手册_V10（中文版）.pdf，15.4.7节

TIM6 和TIM7 自动重装载寄存器(TIMx_ARR)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw															

位15:0	**ARR[15:0]**：自动重装载数值 (Prescaler value) ARR的数值将传送到实际的自动重装载寄存器中。 关于ARR的更新和作用，详见15.3.1。 如果自动重装载数值为0，则计数器停止。

用于设置自动重装载值，范围：0~65535

7.2.5 定时器溢出时间计算方法

定时器溢出时间计算公式

$$T_{out} = \frac{(ARR + 1) * (PSC + 1)}{F_t}$$

T_{out}是定时器溢出时间

F_t是定时器的时钟源频率

ARR是自动重装载寄存器的值

PSC是预分频器寄存器的值

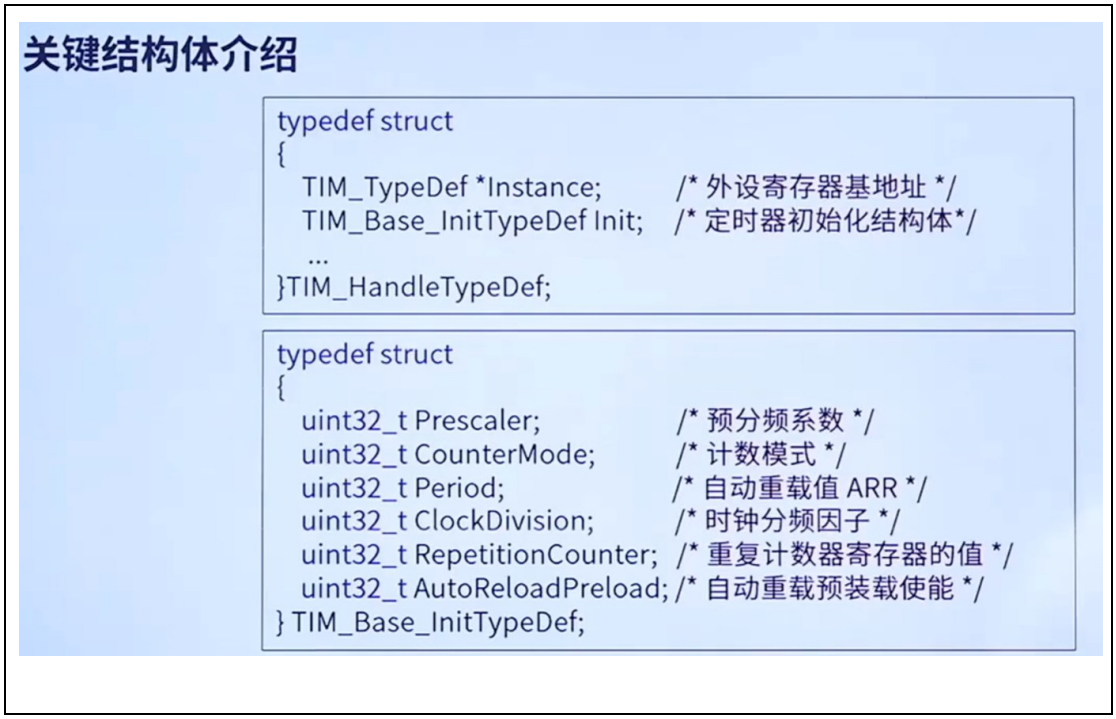
7.2.6 定时器中断实验配置步骤

1，配置定时器基础工作参数	HAL_TIM_Base_Init()
2，定时器基础MSP初始化	HAL_TIM_Base_MspInit()　配置NVIC、CLOCK等
3，使能更新中断并启动计数器	HAL_TIM_Base_Start_IT()
4，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
5，编写中断服务函数	TIMx_IRQHandler()等 → HAL_TIM_IRQHandler()
6，编写定时器更新中断回调函数	HAL_TIM_PeriodElapsedCallback()

7.2.6.1 相关 HAL 库函数介绍

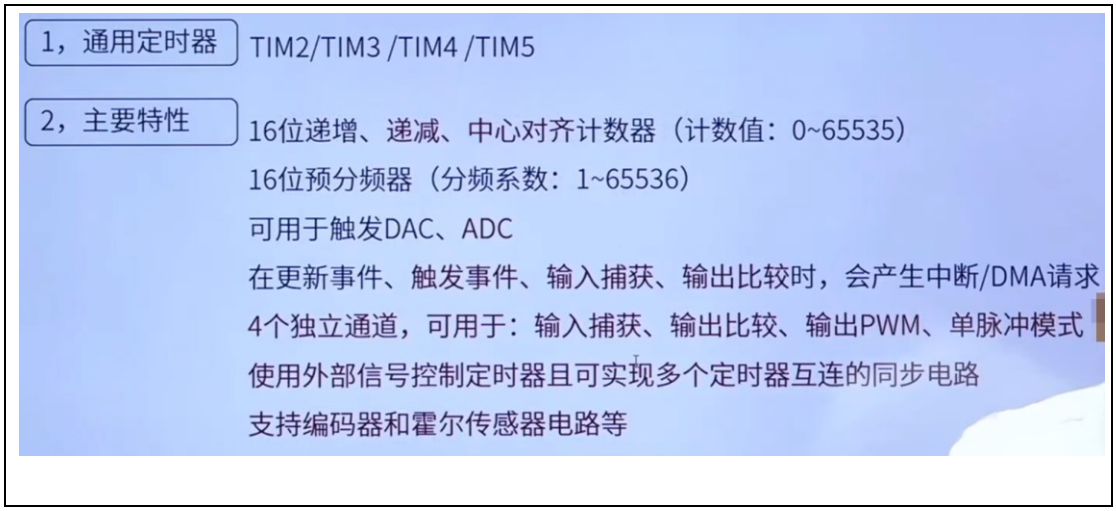
函数	主要寄存器	主要功能
HAL_TIM_Base_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_Base_MspInit()	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_Base_Start_IT()	DIER、CR1	使能更新中断并启动计数器
HAL_TIM_IRQHandler()	SR	定时器中断处理公用函数，处理各种中断
HAL_TIM_PeriodElapsedCallback()	无	定时器更新中断回调函数，由用户重定义

7.2.6.2 关键结构体介绍

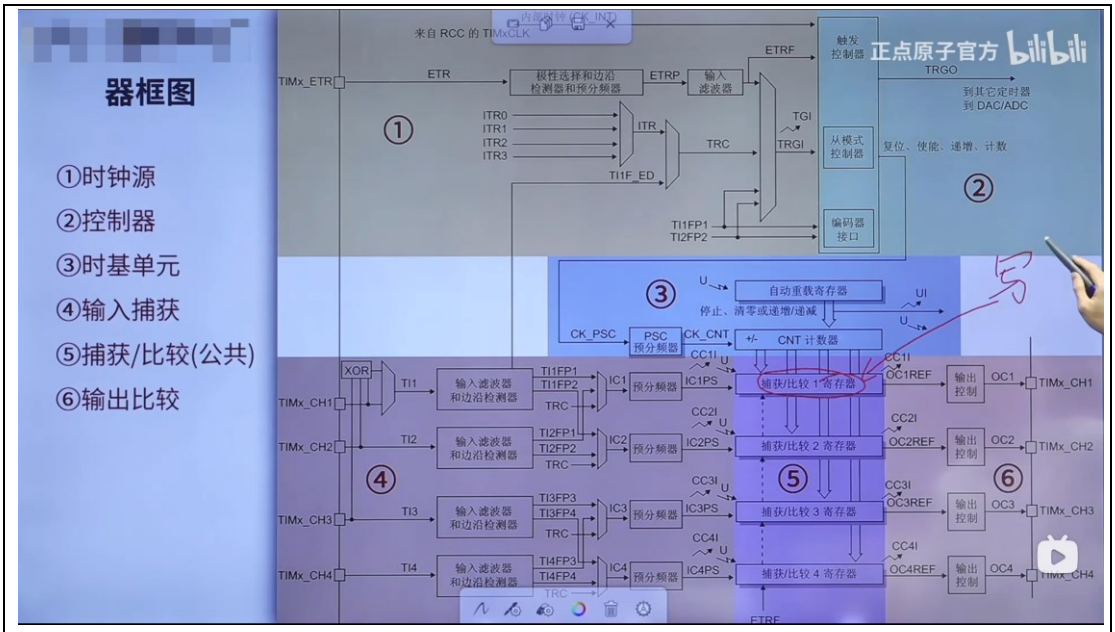


7.3 通用定时器

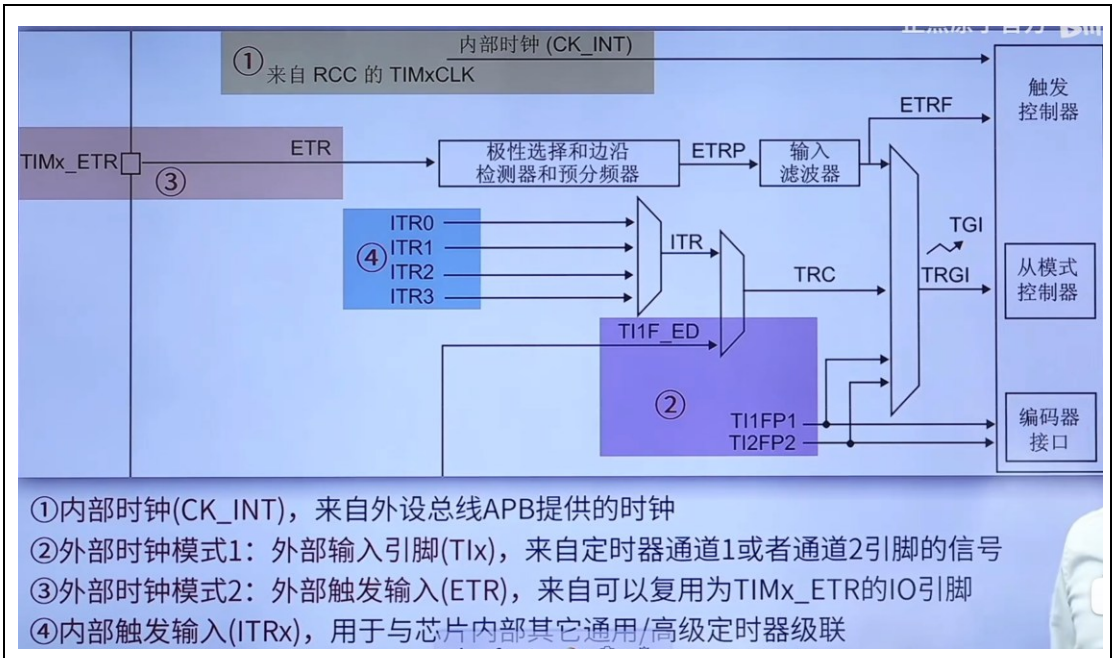
7.3.1 通用定时器简介（F1）



7.3.2 通用定时器框图



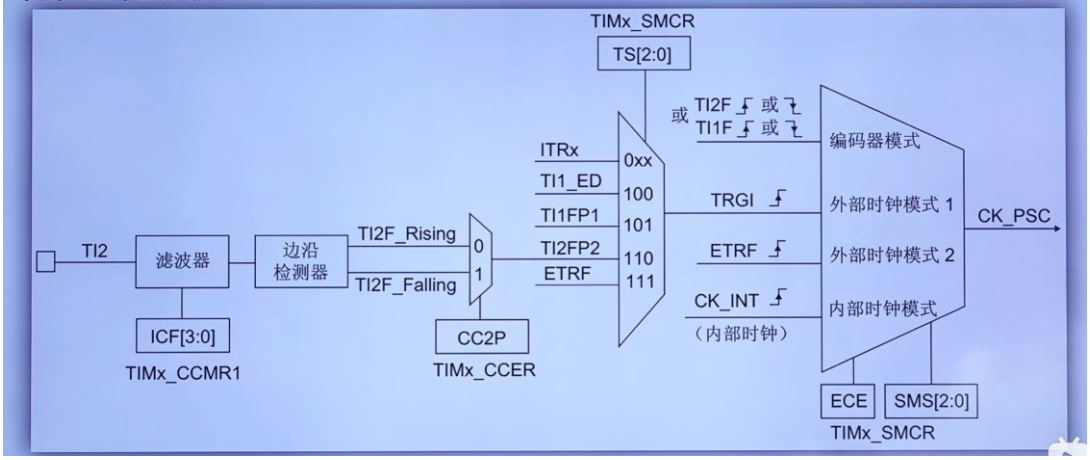
7.3.3 计数器时钟源



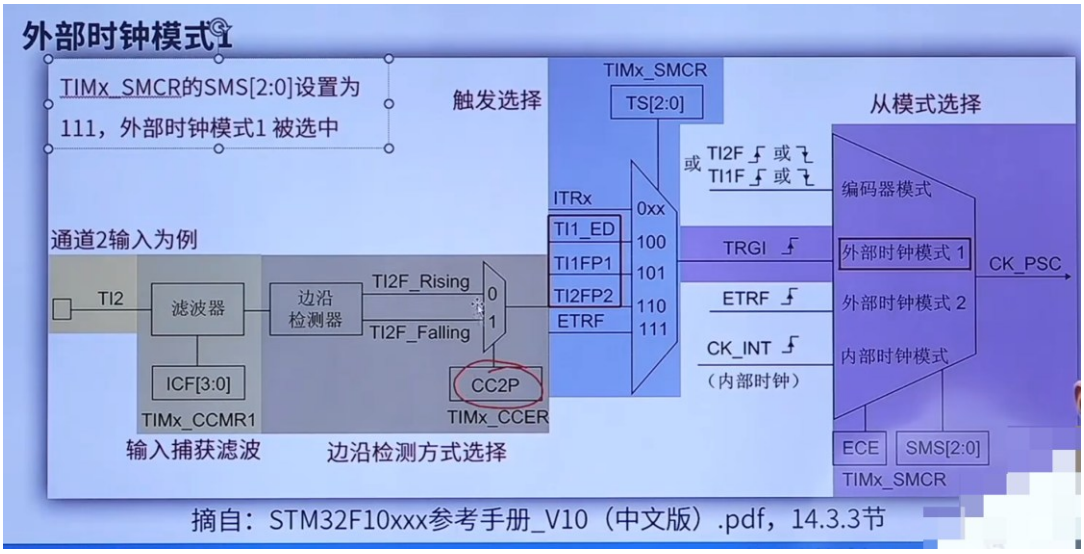
计数器时钟源寄存器设置方法（F1为例）

计数器时钟选择类型	设置方法
内部时钟(CK_INT)	设置TIMx_SMCR的SMS=000
外部时钟模式1：外部输入引脚(TIx)	设置TIMx_SMCR的SMS=111
外部时钟模式2：外部触发输入(ETR)	设置TIMx_SMCR的ECE=1
内部触发输入(ITRx)	设置可参考STM32F10xxx参考手册_V10（中文版）.pdf，14.3.15节

外部时钟模式1

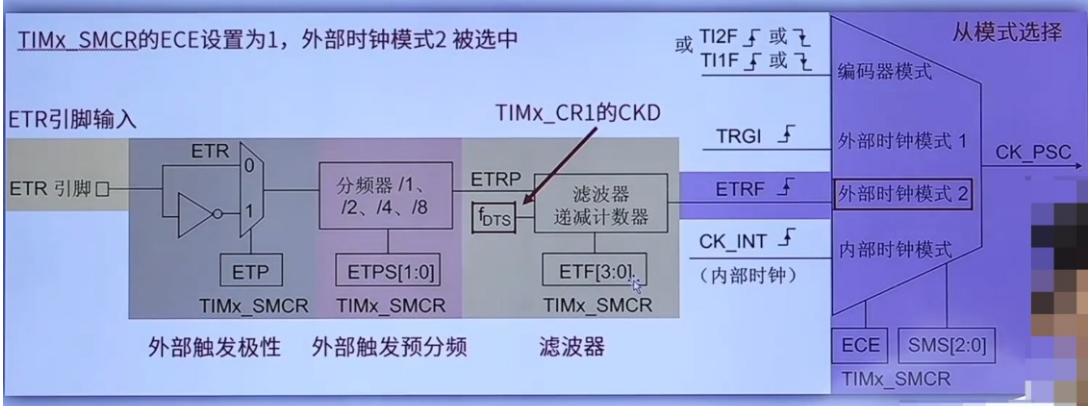


外部时钟模式1



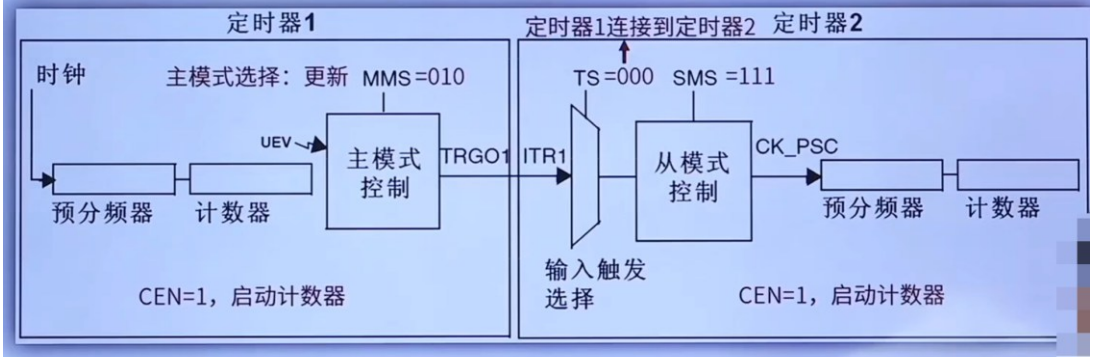
摘自：STM32F10xxx参考手册_V10（中文版）.pdf，14.3.3节

外部时钟模式2



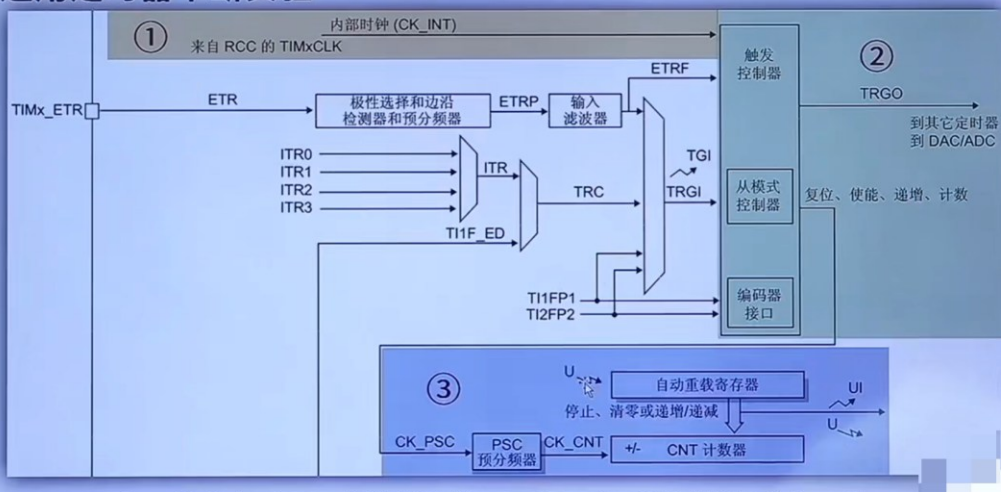
摘自：STM32F10xxx参考手册_V10（中文版）.pdf，14.3.3节

使用一个定时器作为另一个定时器的预分频器（F1为例）



摘自：STM32F10xxx参考手册_V10（中文版）.pdf，14.3.15节

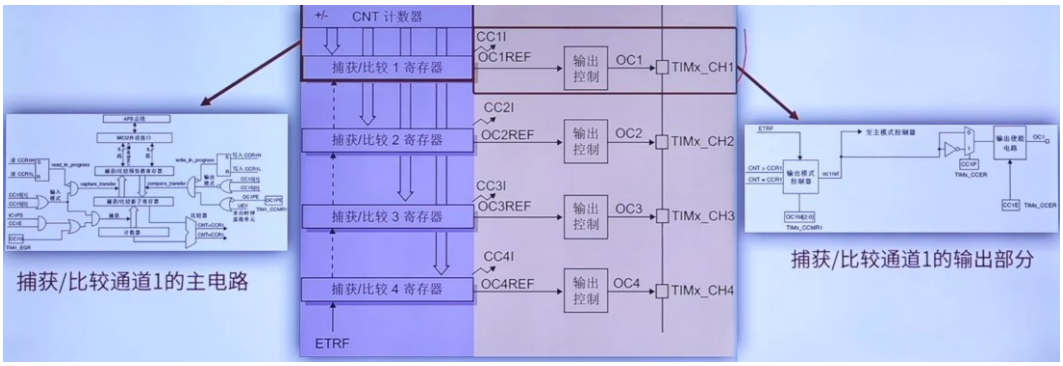
解读通用定时器中断实验

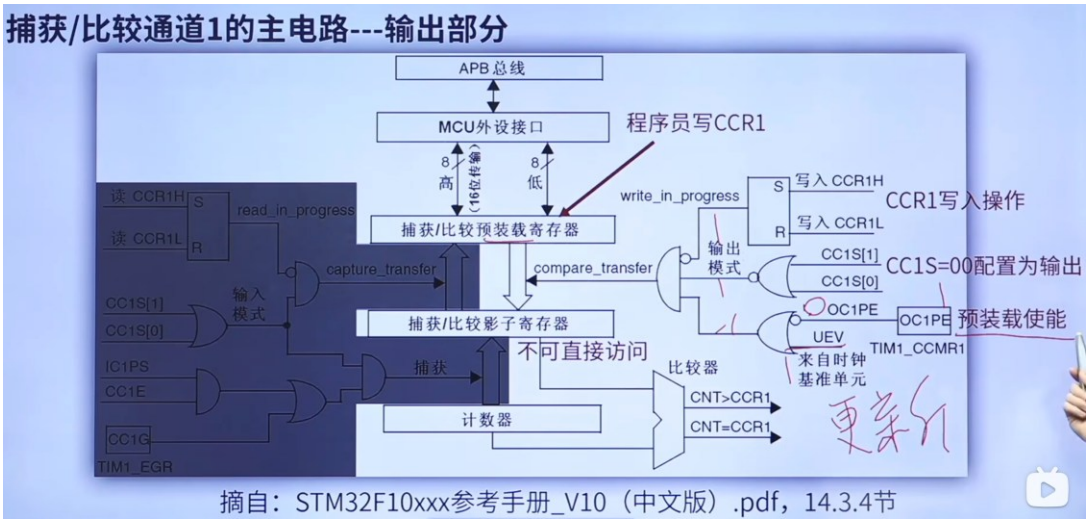


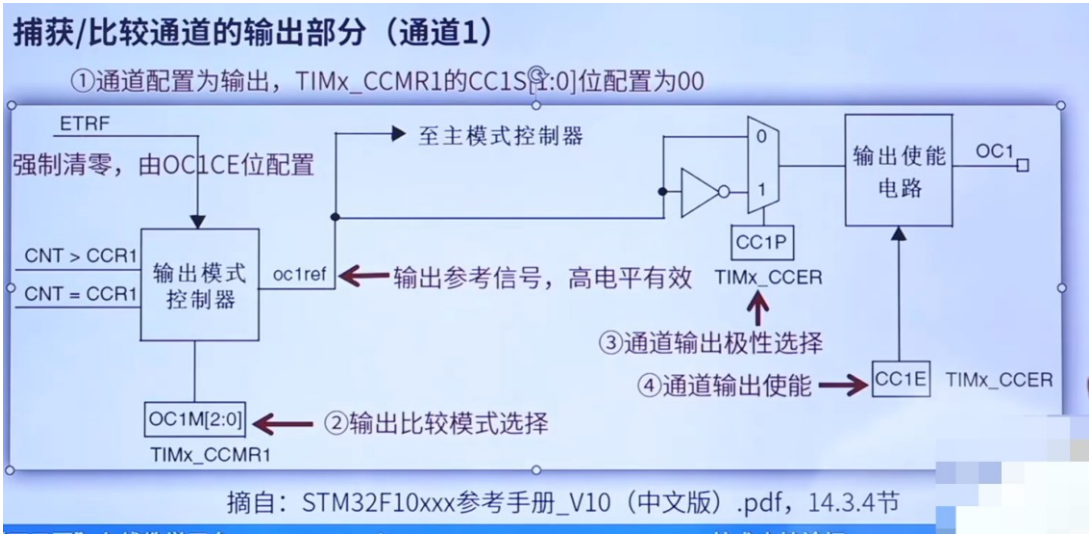
不同点：基本定时器只能递增计数，而通用定时器计数模式有三种

7.3.4 通用定时器 PWM 输出实验

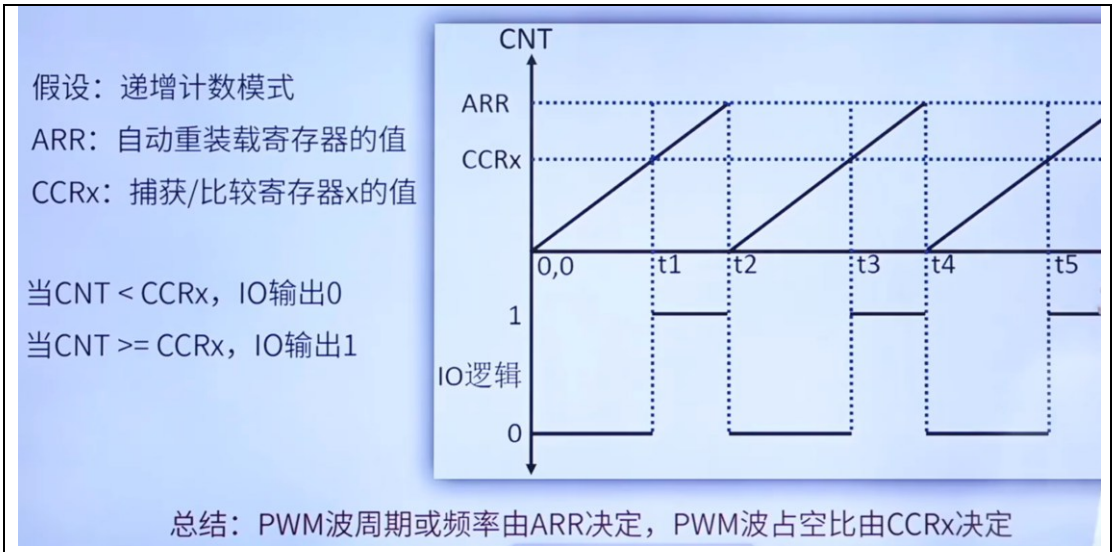
7.3.4.1 通用定时器输出比较部分框图介绍



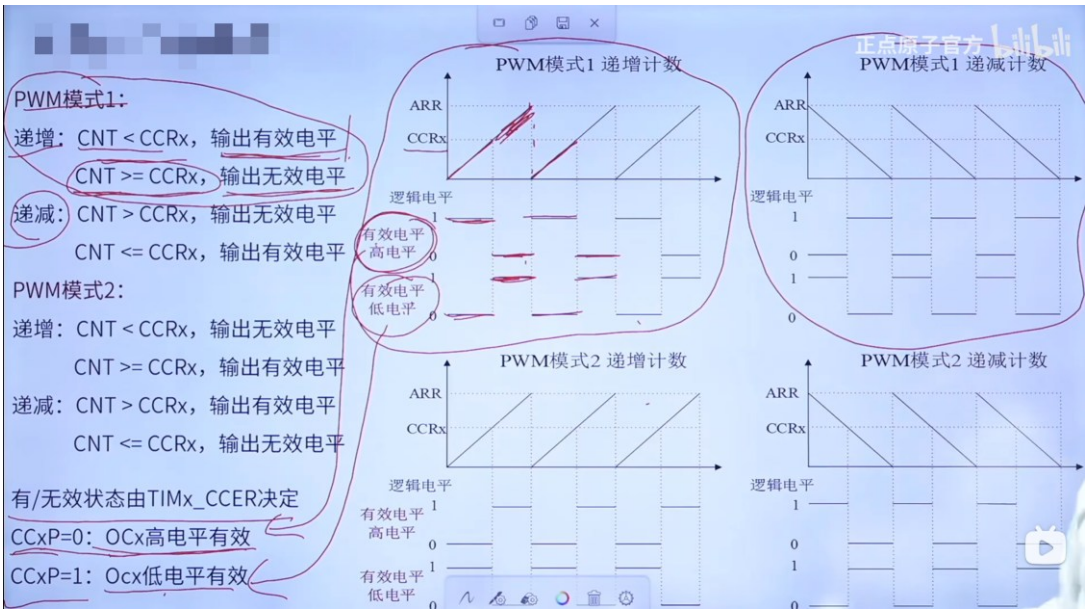




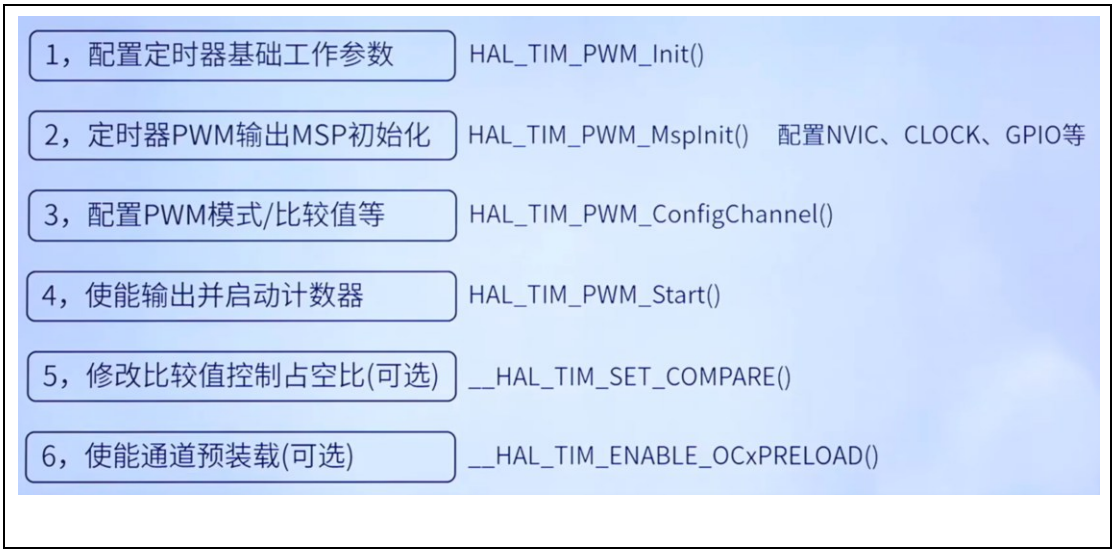
7.3.4.2 通用定时器输出 PWM 原理



7.3.4.3 PWM 模式



7.3.4.4 通用定时器 PWM 输出实验配置步骤

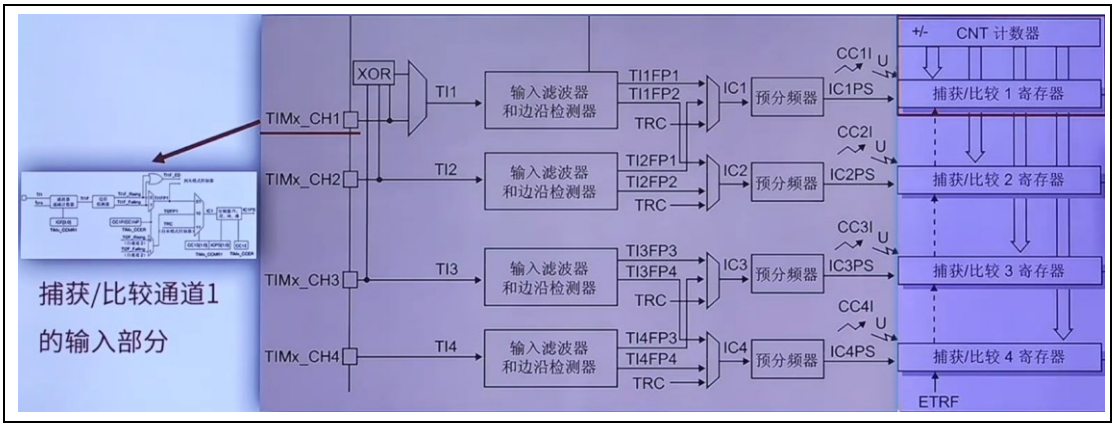


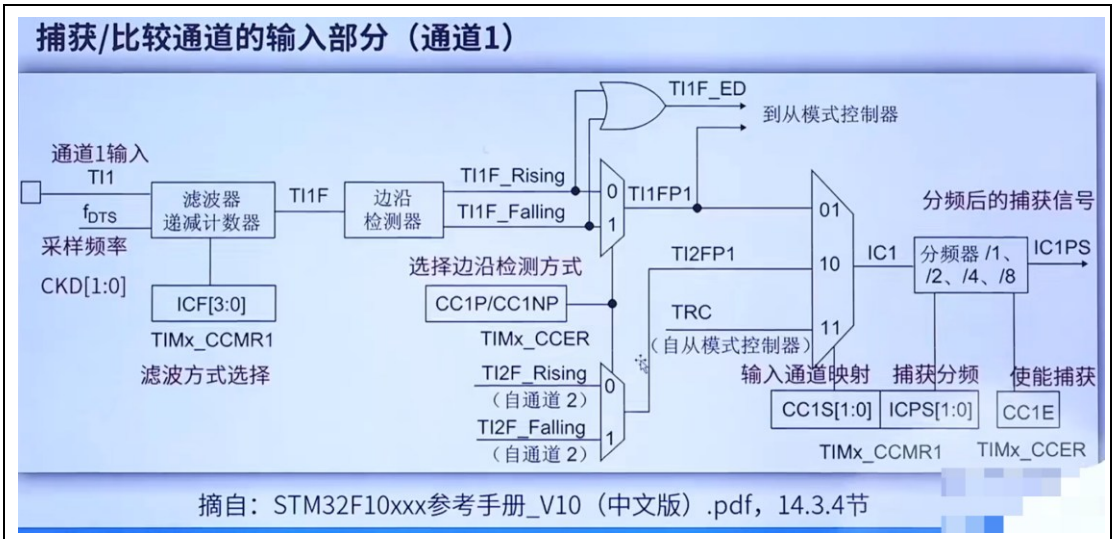
7.3.4.4.1 相关 HAL 库函数介绍

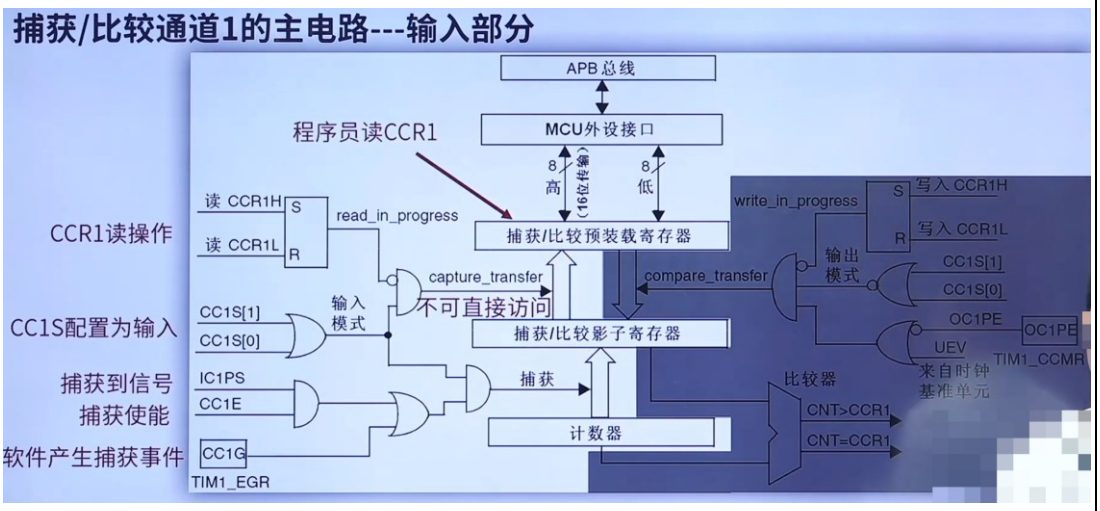
7.3.4.4.2 关键结构体介绍

```
typedef struct
{
  uint32_t OCMode;        /* 输出比较模式选择 */
  uint32_t Pulse;         /* 设置比较值 */
  uint32_t OCPolarity;    /* 设置输出比较极性 */
  uint32_t OCNPolarity;   /* 设置互补输出比较极性 */
  uint32_t OCFastMode;    /* 使能或失能输出比较快速模式 */
  uint32_t OCIdleState;   /* 空闲状态下OC1输出 */
  uint32_t OCNIdleState;  /* 空闲状态下OC1N输出 */
} TIM_OC_InitTypeDef;
```

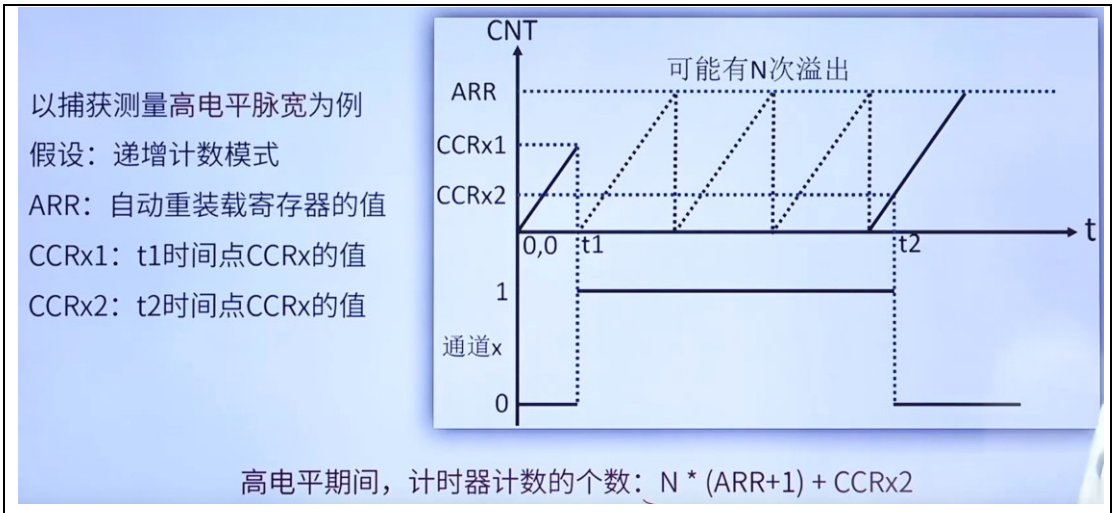
7.3.5 通用定时器输入捕获部分框图介绍







7.3.6 通用定时器输入捕获脉宽测量原理



7.3.7 通用定时器输入捕获实验配置步骤

1，配置定时器基础工作参数	HAL_TIM_IC_Init()
2，定时器输入捕获MSP初始化	HAL_TIM_IC_MspInit()　配置NVIC、CLOCK、GPIO等
3，配置输入通道映射、捕获边沿等	HAL_TIM_IC_ConfigChannel()
4，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
5，使能定时器更新中断	__HAL_TIM_ENABLE_IT()
6，使能捕获、捕获中断及计数器	HAL_TIM_IC_Start_IT()
7，编写中断服务函数	TIMx_IRQHandler()等 → HAL_TIM_IRQHandler()
8，编写更新中断和捕获回调函数	HAL_TIM_PeriodElapsedCallback() HAL_TIM_IC_CaptureCallback()

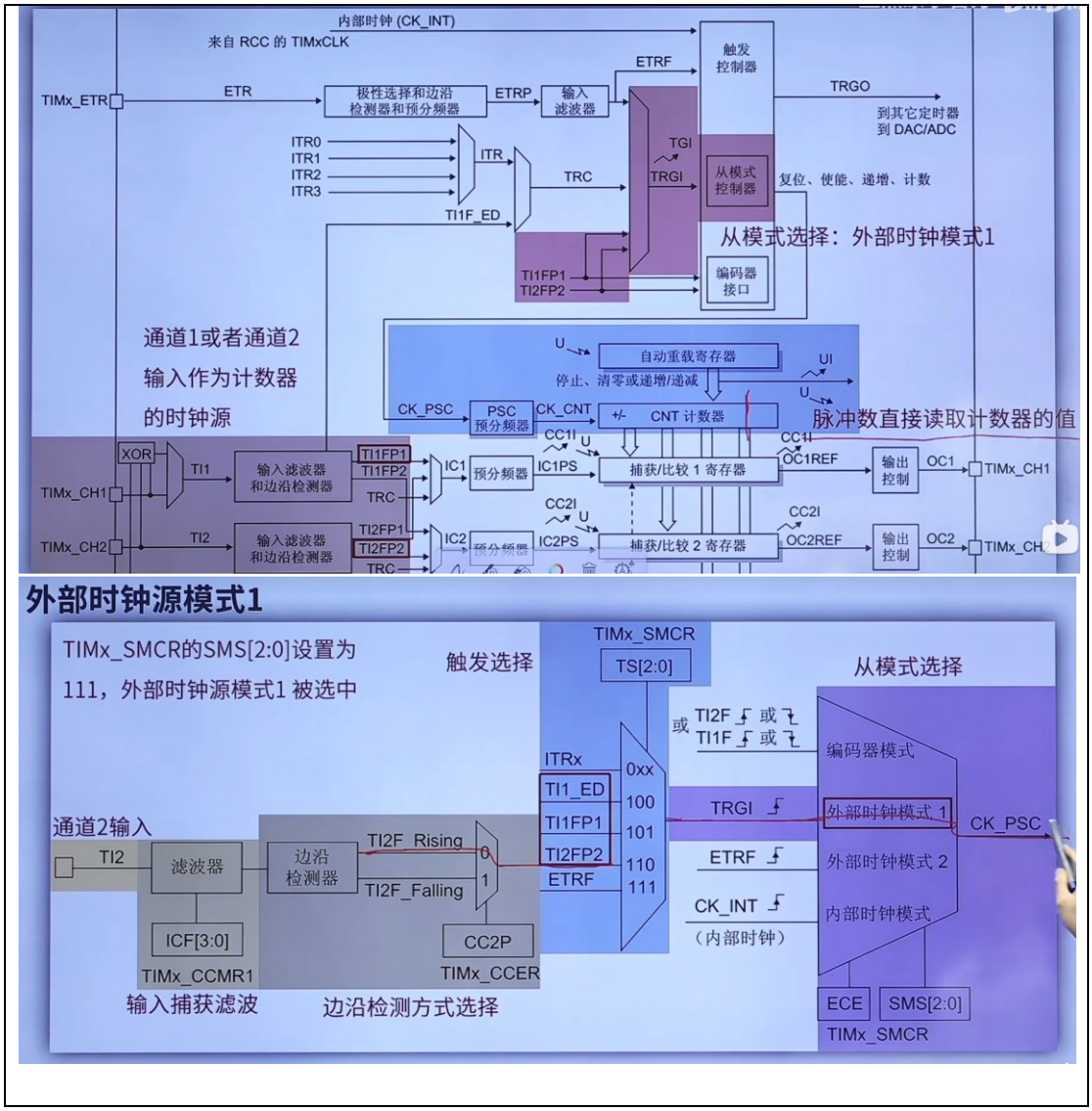
相关HAL库函数介绍

函数	主要寄存器	主要功能
HAL_TIM_IC_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_IC_MspInit()	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_IC_ConfigChannel()	CCMRx、CCER	配置通道映射、捕获边沿、分频、滤波等
__HAL_TIM_ENABLE_IT()	DIER	使能更新中断等
HAL_TIM_IC_Start_IT()	CCER、DIER、CR1	使能输入捕获、捕获中断并启动计数器
HAL_TIM_IRQHandler()	SR	定时器中断处理公用函数，处理各种中断
HAL_TIM_PeriodElapsedCallback()	无	定时器更新中断回调函数，由用户重定义
HAL_TIM_IC_CaptureCallback()	无	定时器输入捕获回调函数，由用户重定义

关键结构体介绍

```
typedef struct
{
  uint32_t ICPolarity;   /* 输入捕获触发方式选择，比如上升、下降捕获 */
  uint32_t ICSelection; /* 输入捕获选择，用于设置映射关系 */
  uint32_t ICPrescaler; /* 输入捕获分频系数 */
  uint32_t ICFilter;     /* 输入捕获滤波器设置 */
} TIM_IC_InitTypeDef;
```

7.3.8 通用定时器脉冲计数实验原理



7.3.9 通用定时器脉冲计数实验配置步骤

相关HAL库函数介绍

函数	主要寄存器	主要功能
HAL_TIM_IC_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_IC_MspInit()	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_SlaveConfigSynchro()	SMCR、CCMRx、CCER	配置定时器从模式、触发选择、分频、滤波等
HAL_TIM_IC_Start()	CCER、CR1	使能输入捕获、启动计数器
__HAL_TIM_GET_COUNTER()	CNT	获取计数器当前值
__HAL_TIM_SET_COUNTER()	CNT	设置计数器的值

关键结构体介绍

```
typedef struct
{
  uint32_t SlaveMode;          /* 从模式选择 */
  uint32_t InputTrigger;       /* 输入触发源选择 */
  uint32_t TriggerPolarity;    /* 输入触发极性 */
  uint32_t TriggerPrescaler;   /* 输入触发预分频 */
  uint32_t TriggerFilter;      /* 输入滤波器设置 */
} TIM_SlaveConfigTypeDef;
```

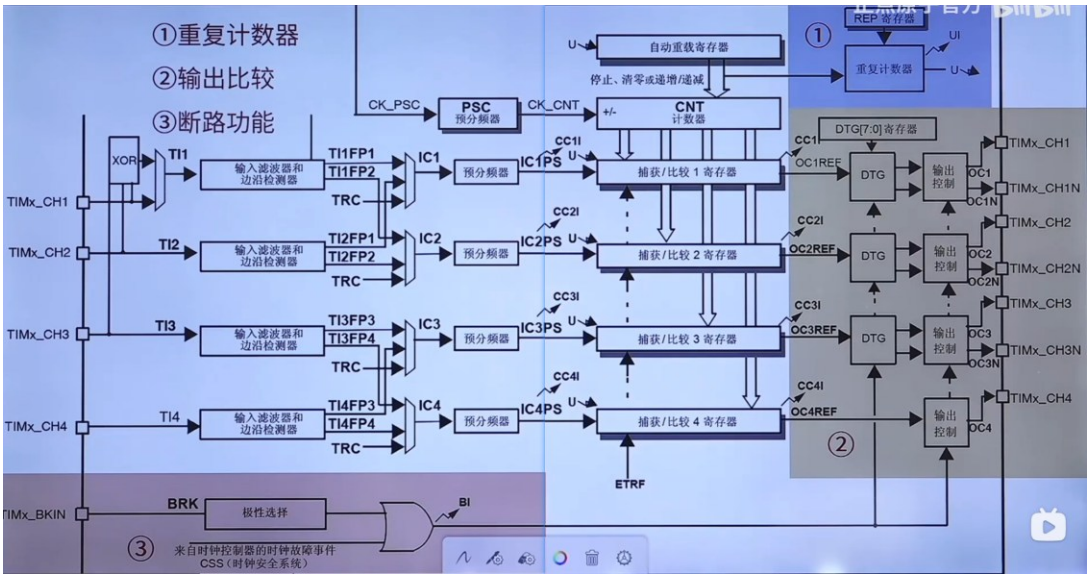
7.4 高级定时器

7.4.1 高级定时器简介（F1 ）

1，高级定时器　TIM1/TIM8

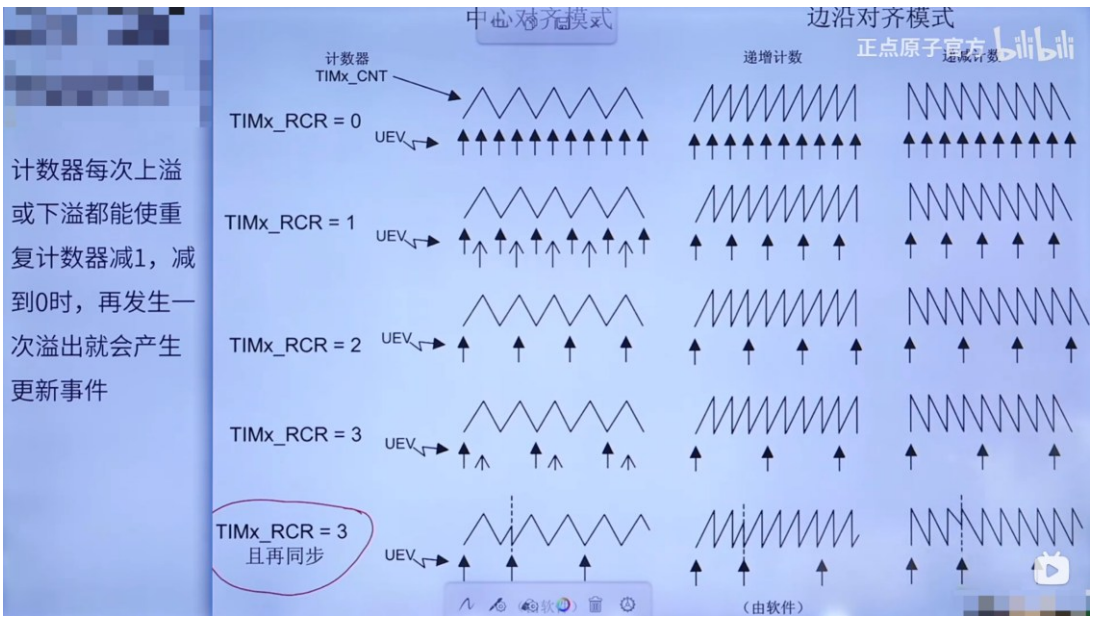
2，主要特性
- 16位递增、递减、中心对齐计数器（计数值：0~65535）
- 16位预分频器（分频系数：1~65536）
- 可用于触发DAC、ADC
- 在更新事件、触发事件、输入捕获、输出比较时，会产生中断/DMA请求
- 4个独立通道，可用于：输入捕获、输出比较、输出PWM、单脉冲模式
- 使用外部信号控制定时器且可实现多个定时器互连的同步电路
- 支持编码器和霍尔传感器电路等
- 重复计数器
- 死区时间带可编程的互补输出
- 断路输入，用于将定时器的输出信号置于用户可选的安全配置中

7.4.2 高级定时器框图



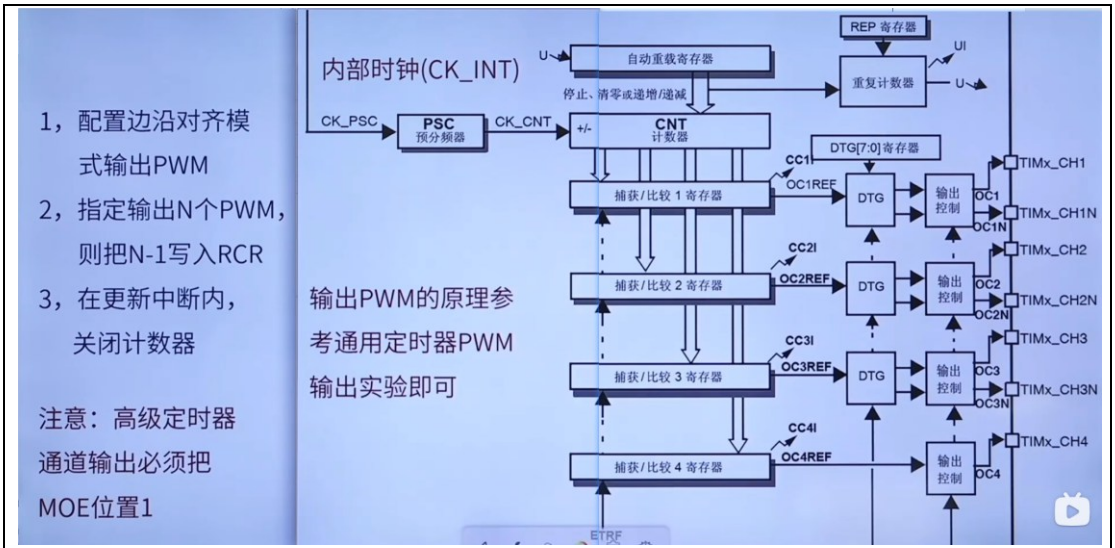
7.4.3 高级定时器输出指定个数 PWM 实验

7.4.3.1 重复计数器特性



：如果设置 RCR 为 N，更新事件将在 N+1 次溢出时发生

7.4.3.2 高级定时器输出指定个数 PWM 实验原理



7.4.3.3 高级定时器输出指定个数 PWM 实验配置步骤

1，配置定时器基础工作参数	HAL_TIM_PWM_Init()
2，定时器PWM输出MSP初始化	HAL_TIM_PWM_MspInit() 配置NVIC、CLOCK、GPIO等
3，配置PWM模式/比较值等	HAL_TIM_PWM_ConfigChannel()
4，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
5，使能定时器更新中断	__HAL_TIM_ENABLE_IT()
6，使能输出、主输出、计数器	HAL_TIM_PWM_Start()
7，编写中断服务函数	TIMx_IRQHandler()等 → HAL_TIM_IRQHandler()
8，编写更新中断回调函数	HAL_TIM_PeriodElapsedCallback()

相关HAL库函数介绍

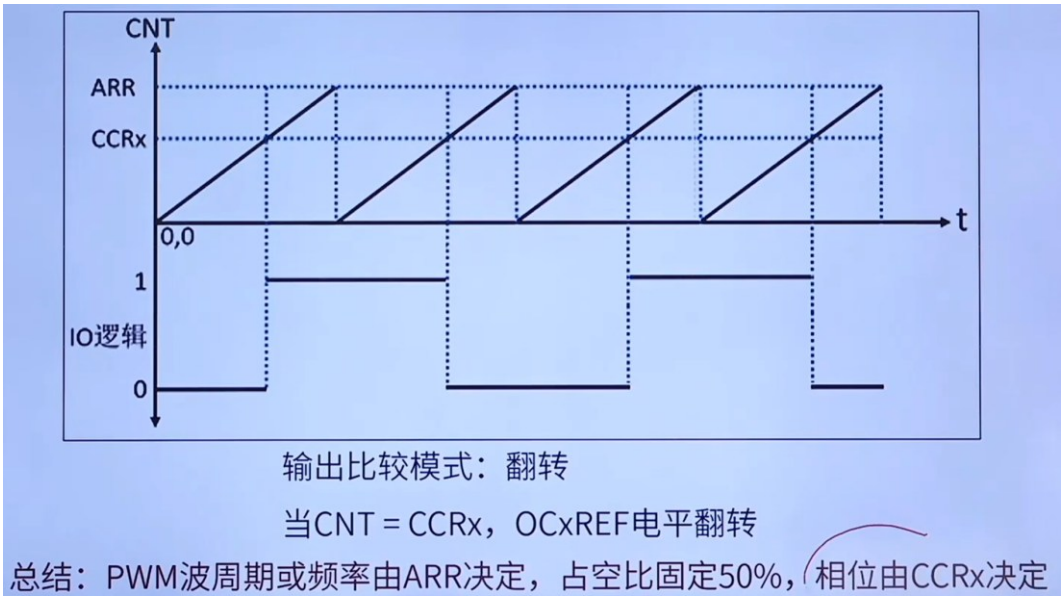
函数	主要寄存器	主要功能
HAL_TIM_PWM_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_PWM_MspInit()	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_PWM_ConfigChannel()	CCMRx、CCRx、CCER	配置PWM模式、比较值、输出极性等
__HAL_TIM_ENABLE_IT()	CCER	使能更新中断等
HAL_TIM_PWM_Start()	CCER、CR1	使能输出、主输出、启动计数器
HAL_TIM_IRQHandler()	SR	定时器中断处理公用函数，处理各种中断
HAL_TIM_PeriodElapsedCallback()	无	定时器更新中断回调函数，由用户重定义
HAL_TIM_GenerateEvent()	EGR	通过软件产生事件
__HAL_TIM_ENABLE()	CR1	启动计数器

关键结构体介绍

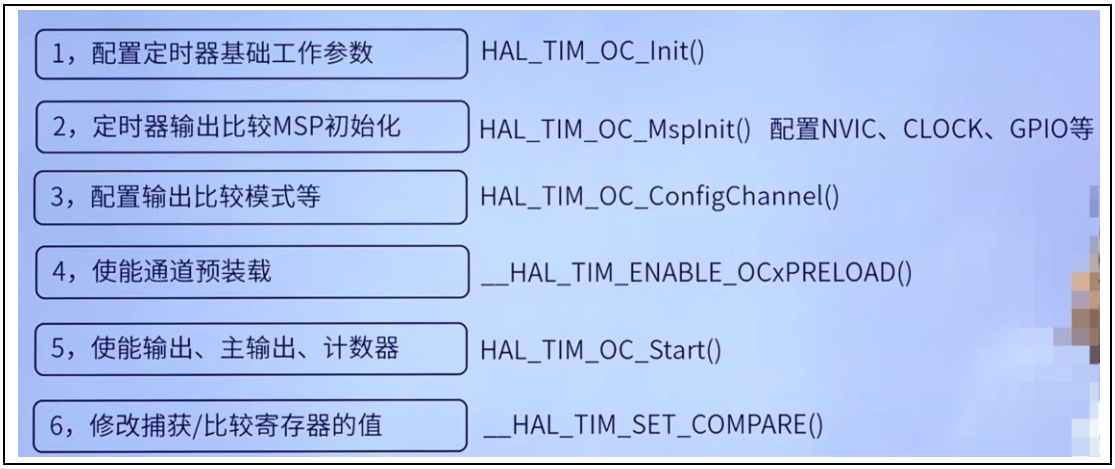
```
typedef struct
{
  uint32_t OCMode;        /* 输出比较模式选择 */
  uint32_t Pulse;         /* 设置比较值 */
  uint32_t OCPolarity;    /* 设置输出比较极性 */
  uint32_t OCNPolarity;   /* 设置互补输出比较极性 */
  uint32_t OCFastMode;    /* 使能或失能输出比较快速模式 */
  uint32_t OCIdleState;   /* 空闲状态下OC1输出 */
  uint32_t OCNIdleState;  /* 空闲状态下OC1N输出 */
} TIM_OC_InitTypeDef;
```

7.4.4 高级定时器输出比较模式实验

7.4.4.1 高级定时器输出比较模式实验原理



7.4.4.2 高级定时器输出比较实验配置步骤



相关HAL库函数介绍

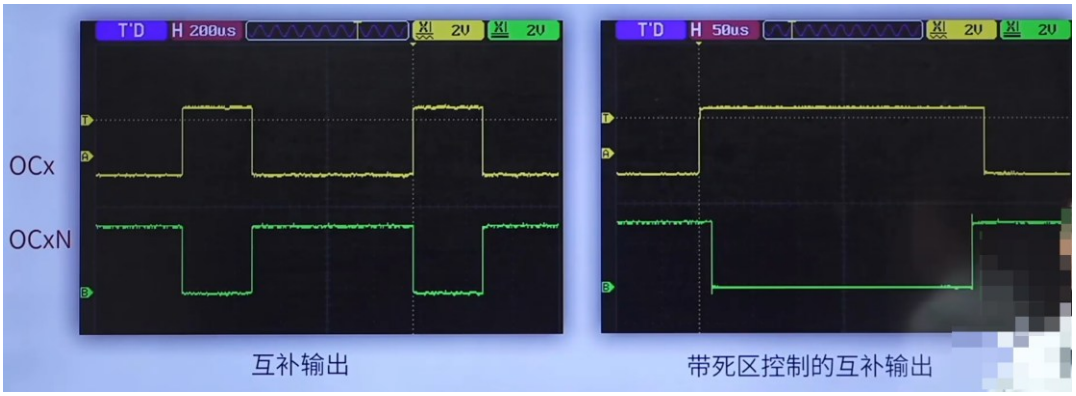
函数	主要寄存器	主要功能
HAL_TIM_OC_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_OC_MspInit	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_OC_ConfigChannel()	CCMRx、CCRx、CCER	设置输出比较模式、比较值、输出极性等
__HAL_TIM_ENABLE_OCxPRELOAD()	CCMRx	使能通道预装载
HAL_TIM_OC_Start()	CR1、CCER、BDTR	使能输出比较、主输出、启动计数器
__HAL_TIM_SET_COMPARE()	CCRx	修改捕获/比较寄存器的值

关键结构体介绍

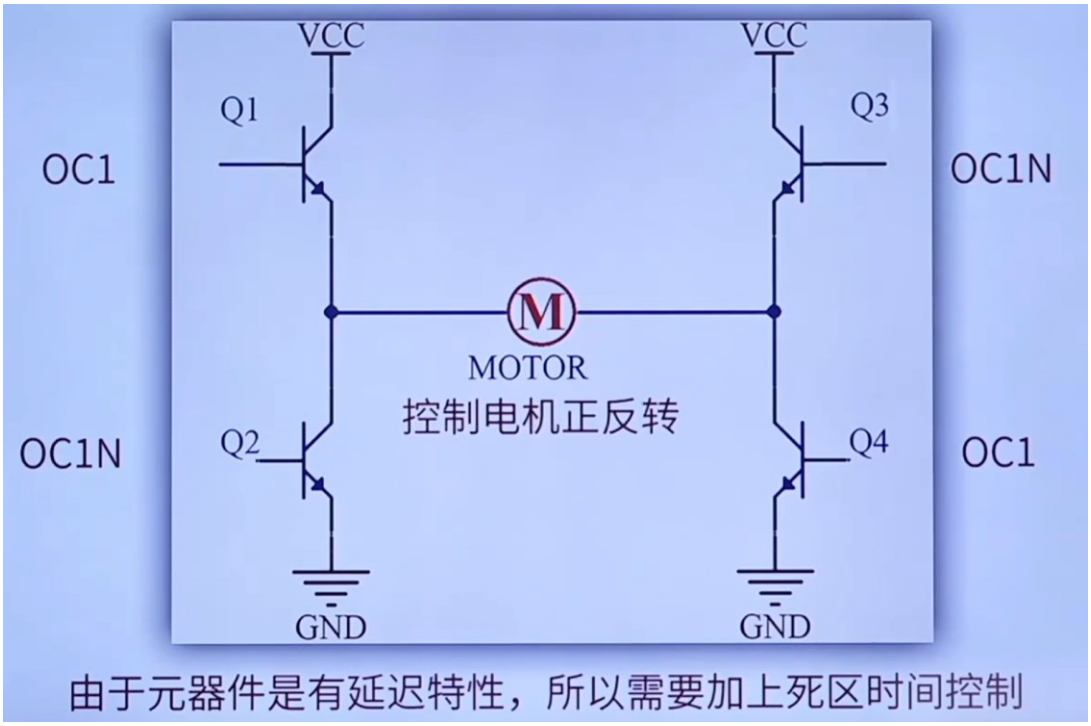
```
typedef struct
{
  uint32_t OCMode;        /* 输出比较模式选择 */
  uint32_t Pulse;         /* 设置比较值 */
  uint32_t OCPolarity;    /* 设置输出比较极性 */
  uint32_t OCNPolarity;   /* 设置互补输出比较极性 */
  uint32_t OCFastMode;    /* 使能或失能输出比较快速模式 */
  uint32_t OCIdleState;   /* 空闲状态下OC1输出 */
  uint32_t OCNIdleState;  /* 空闲状态下OC1N输出 */
} TIM_OC_InitTypeDef;
```

7.4.5 高级定时器互补输出带死区控制实验

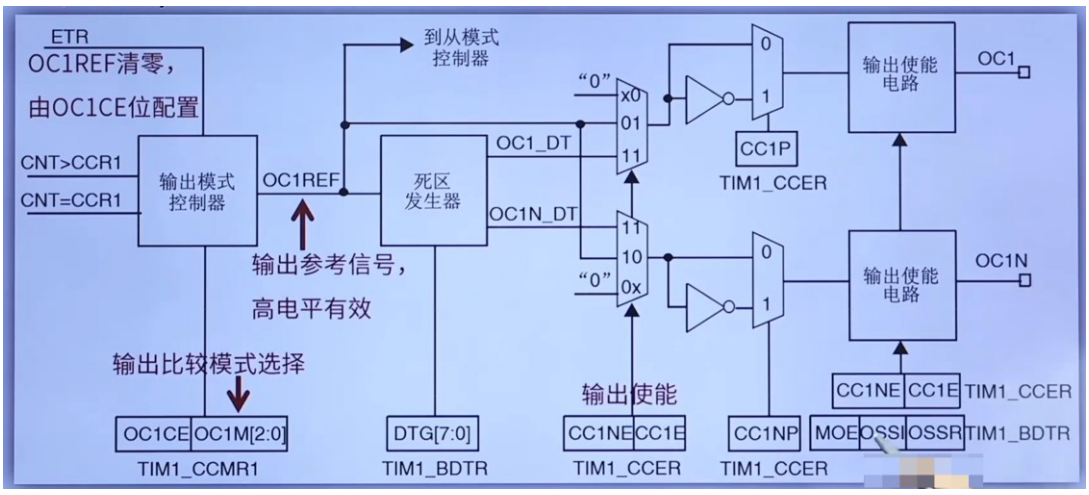
7.4.5.1 互补输出，还带死区控制，什么意思?



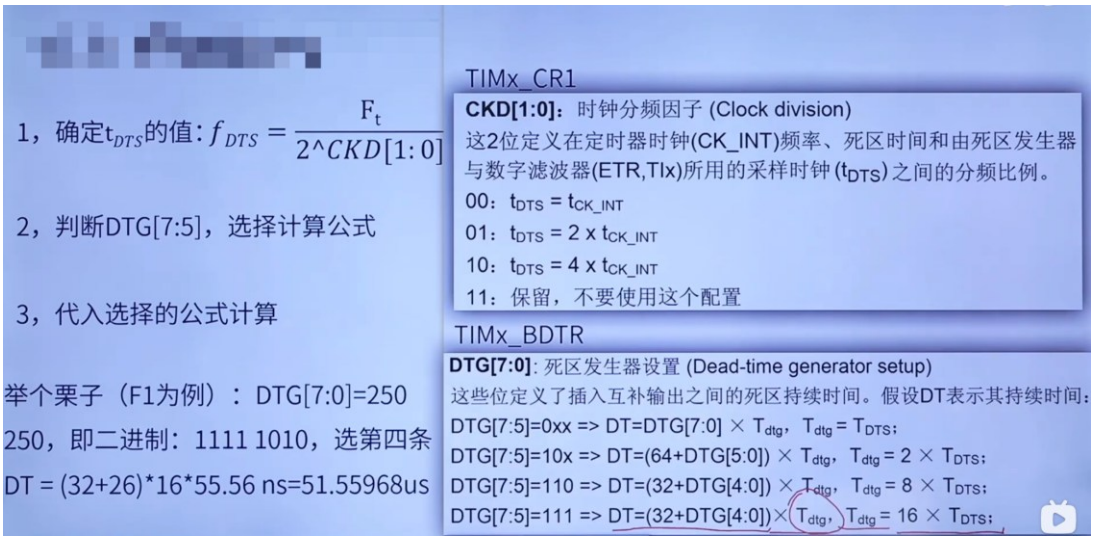
7.4.5.2 带死区控制的互补输出应用之 H 桥



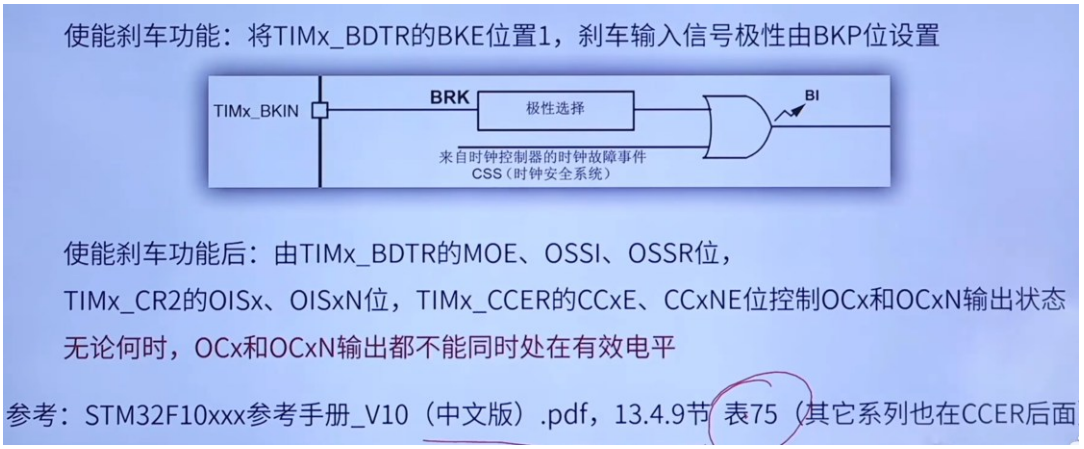
7.4.5.3 捕获/比较通道的输出部分（通道 1 至 3）

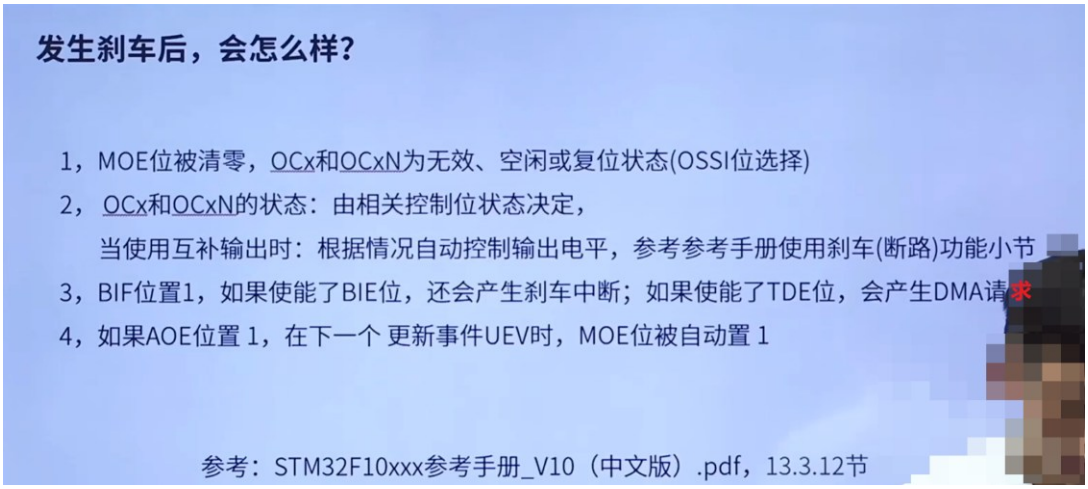


7.4.5.4 死区时间计算

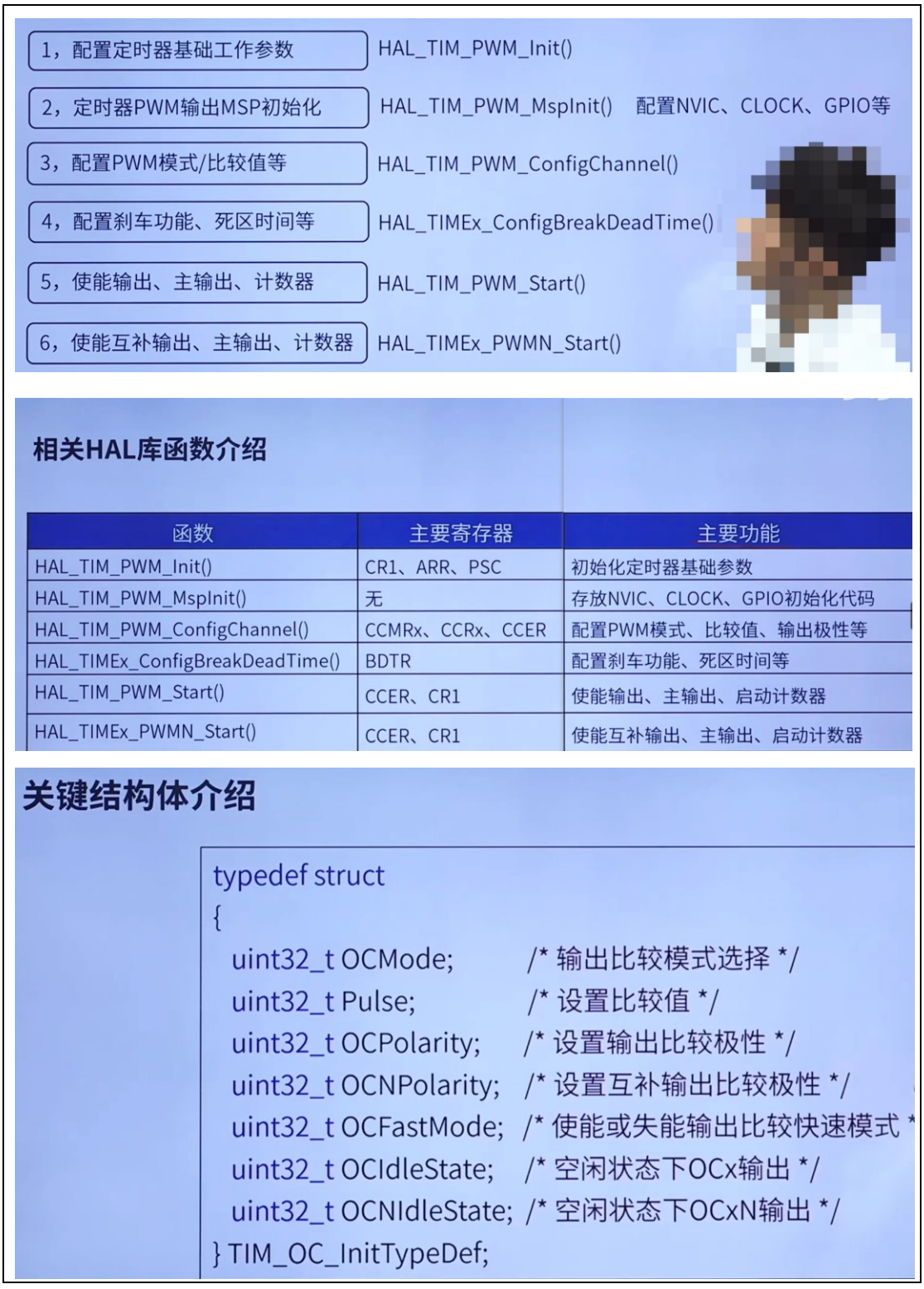


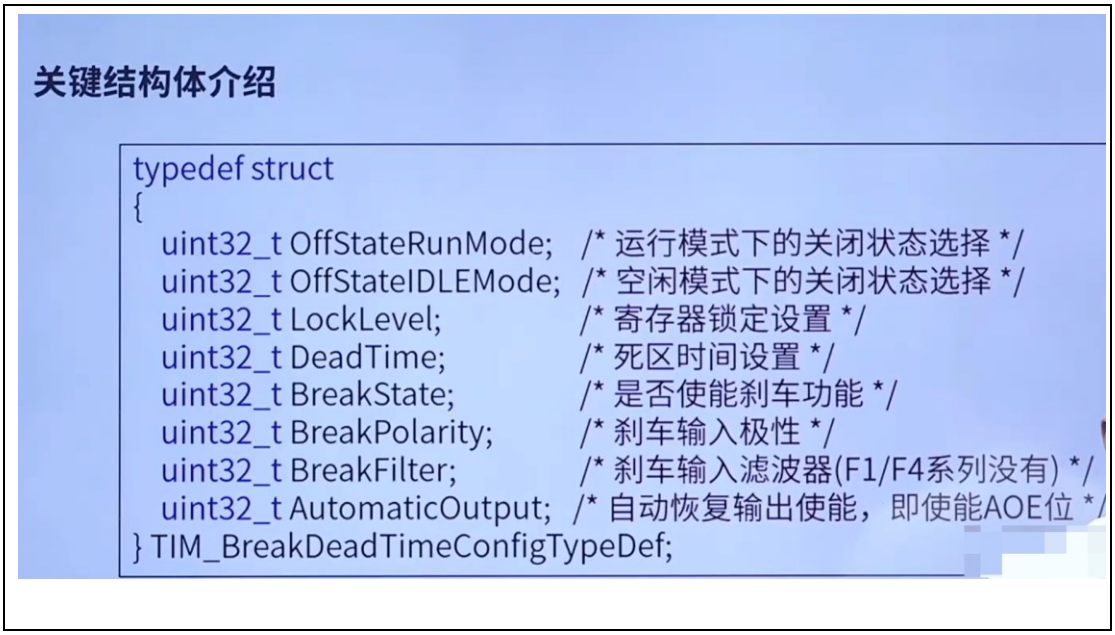
7.4.5.5 刹车（断路）功能





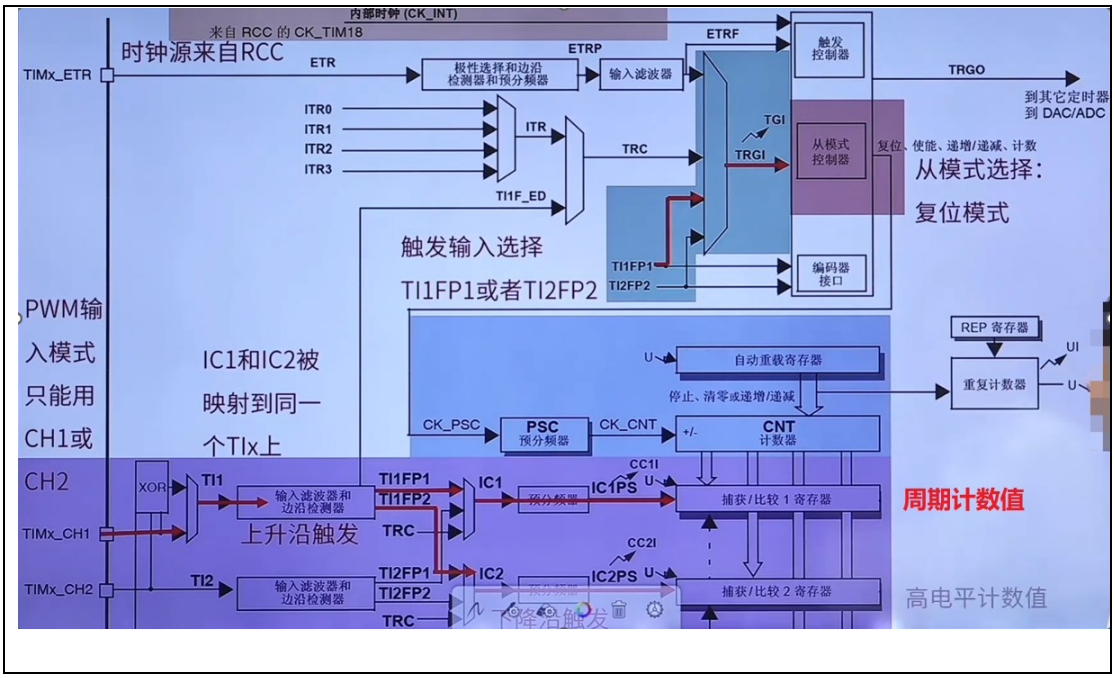
7.4.5.6 高级定时器互补输出带死区控制实验配置步骤



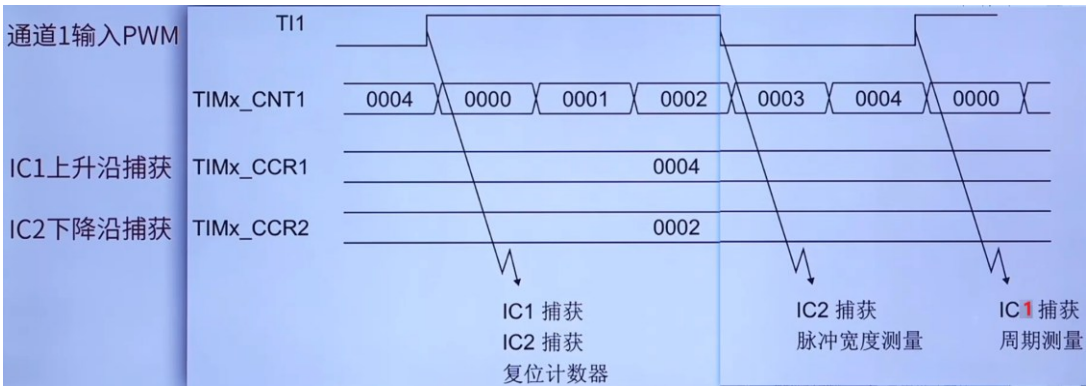


7.4.6 高级定时器 PWM 输入模式实验

7.4.6.1 PWM 输入模式工作原理



7.4.6.2 PWM 输入模式时序

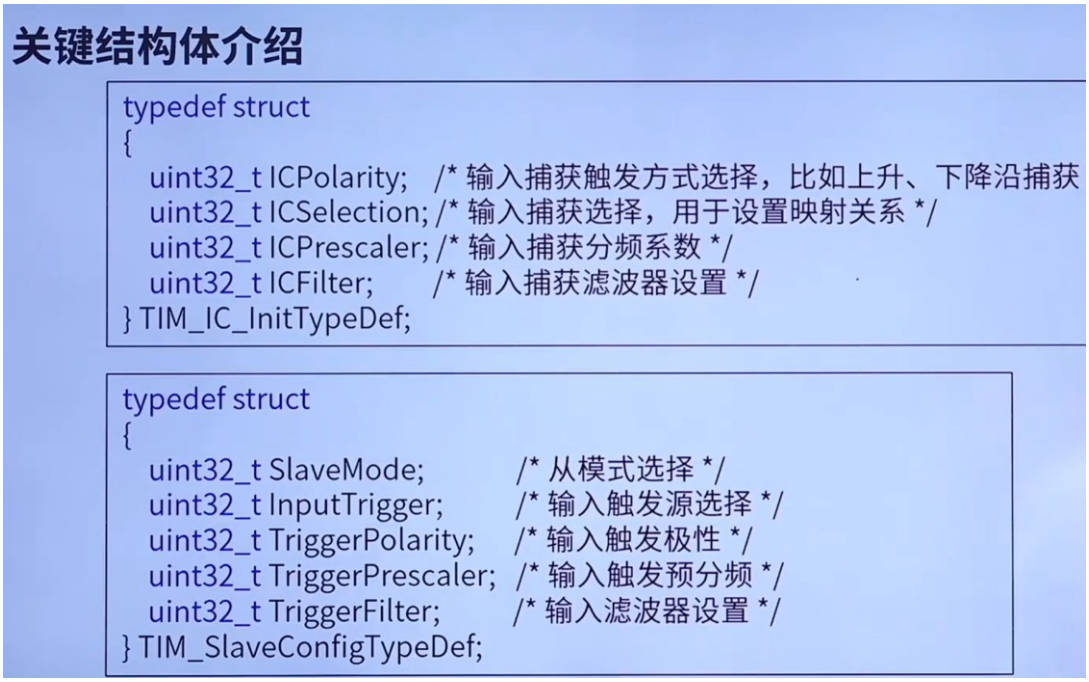


7.4.6.3 高级定时器 PWM 输入模式实验配置步骤

1，配置定时器基础工作参数	HAL_TIM_IC_Init()
2，定时器捕获输入MSP初始化	HAL_TIM_IC_MspInit()　配置NVIC、CLOCK、GPIO等
3，配置IC1/2映射、捕获边沿等	HAL_TIM_IC_ConfigChannel()
4，配置从模式，触发源等	HAL_TIM_SlaveConfigSynchro()
5，设置优先级，使能中断	HAL_NVIC_SetPriority()、 HAL_NVIC_EnableIRQ()
6，使能捕获、捕获中断及计数器	HAL_TIM_IC_Start_IT()、 HAL_TIM_IC_Start()
7，编写中断服务函数	TIMx_IRQHandler()等 → HAL_TIM_IRQHandler()
8，编写输入捕获回调函数	HAL_TIM_IC_CaptureCallback()

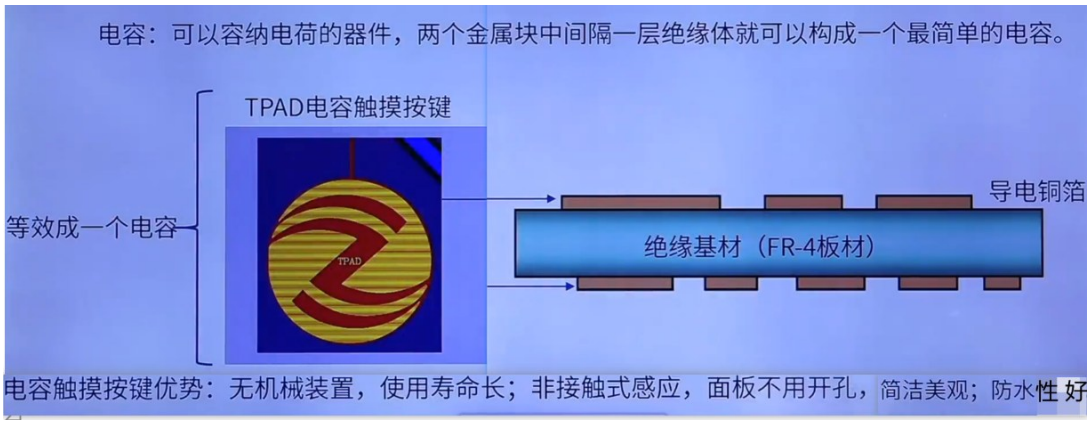
相关HAL库函数介绍

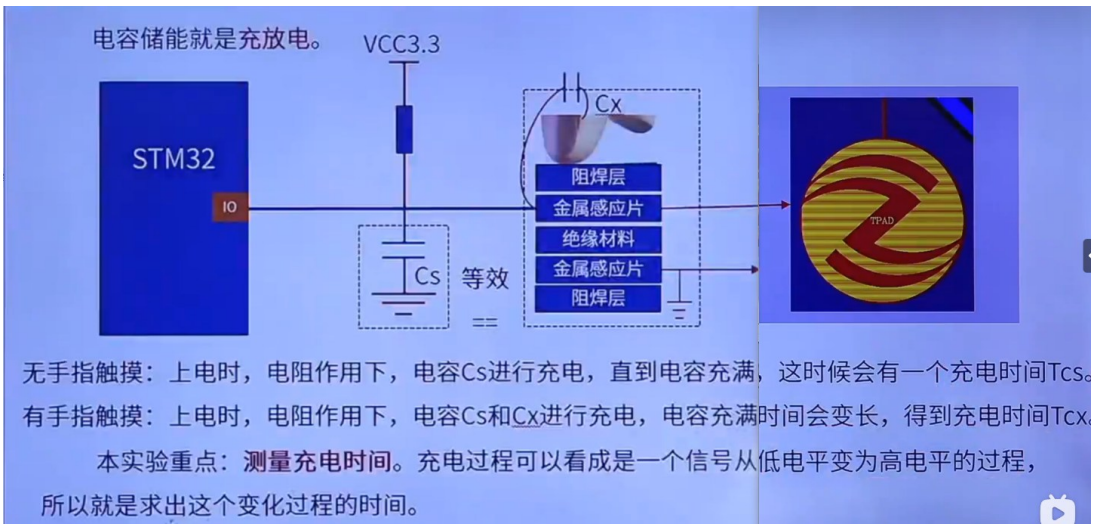
函数	主要寄存器	主要功能
HAL_TIM_IC_Init()	CR1、ARR、PSC	初始化定时器基础参数
HAL_TIM_IC_MspInit()	无	存放NVIC、CLOCK、GPIO初始化代码
HAL_TIM_IC_ConfigChannel()	CCMRx、CCER	配置通道映射、捕获边沿、分频、滤波等
HAL_TIM_SlaveConfigSynchro()	SMCR、CCER	配置从模式、触发源、触发边沿等
HAL_TIM_IC_Start_IT()	CCER、DIER、CR1	使能输入捕获、捕获中断并启动计数器
HAL_TIM_IRQHandler()	SR	定时器中断处理公用函数，处理各种中断
HAL_TIM_IC_CaptureCallback()	无	定时器输入捕获回调函数，由用户重定义



8 电容触摸按键实验

8.1 电容触摸按键原理介绍

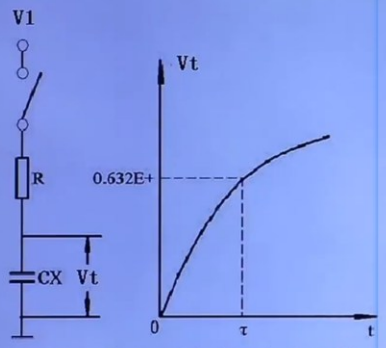




无手指触摸：上电时，电阻作用下，电容Cs进行充电，直到电容充满，这时候会有一个充电时间Tcs。

有手指触摸：上电时，电阻作用下，电容Cs和Cx进行充电，电容充满时间会变长，得到充电时间Tcx。

本实验重点：**测量充电时间**。充电过程可以看成是一个信号从低电平变为高电平的过程，所以就是求出这个变化过程的时间。



充电时电容两端电压变化曲线

RC电路充放电公式：

$$Vt = V0 + (V1 - V0) * (e^{-\frac{t}{RC}})$$

V0为电容上的初始电压值

V1为电容最终可充到或放到的电压值

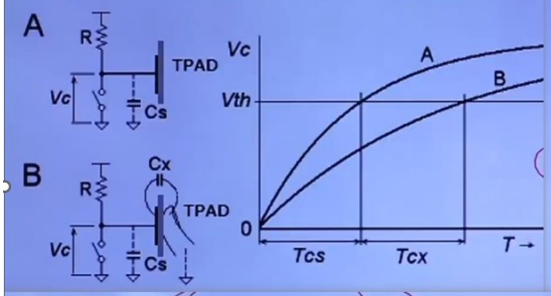
Vt为t时刻电容上的电压值

如果V0为0，即从0V开始充电，公式简化为：

$$Vt = V1 * (e^{-\frac{t}{RC}})$$

结论：同样条件下，电容值C跟时间值t成正比关系，电容越大，充电到达某个临界值时间越长。

同样条件下，电容值C跟时间值t成正比关系，电容越大，充电到达某个临界值时间越长。



R：外接电容充放电电阻

Cs：TPAD和PCB间的杂散电容

Cx：手指按下时，手指和TPAD之间的电容

开关：电容放电开关，由STM32 IO口代替

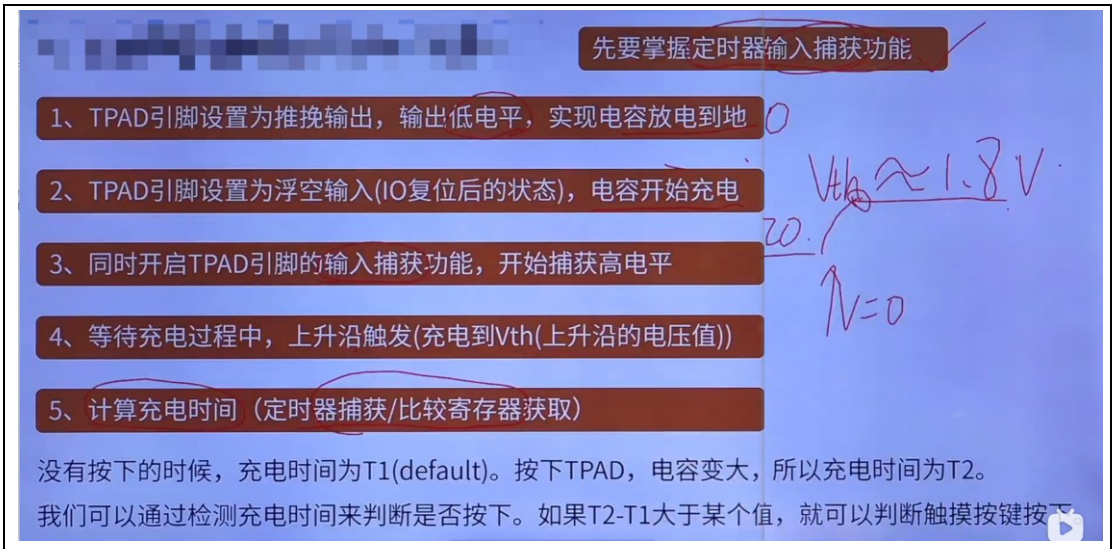
Vth：STM32认为高电平的最低电压值(1.8V)

可以利用定时器输入捕获功能计算充电时间。

注意：电容的初始状态要为0（即一开始要放电）

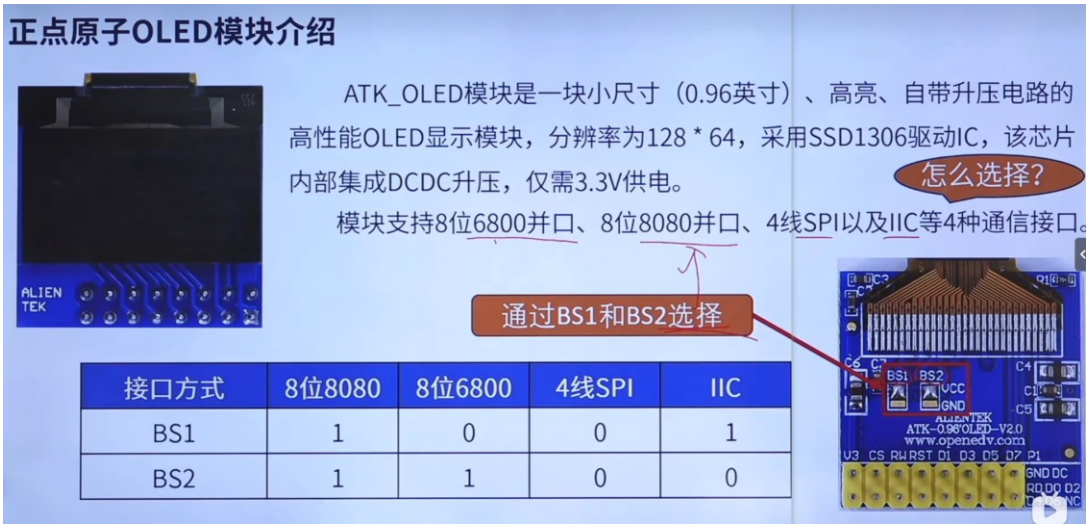
先测量的到无触摸时充电时间Tcs作为比较基准，然后定时循环测量充电时间T与Tcs作比较，如果超过一定阈值就认为有手指触摸

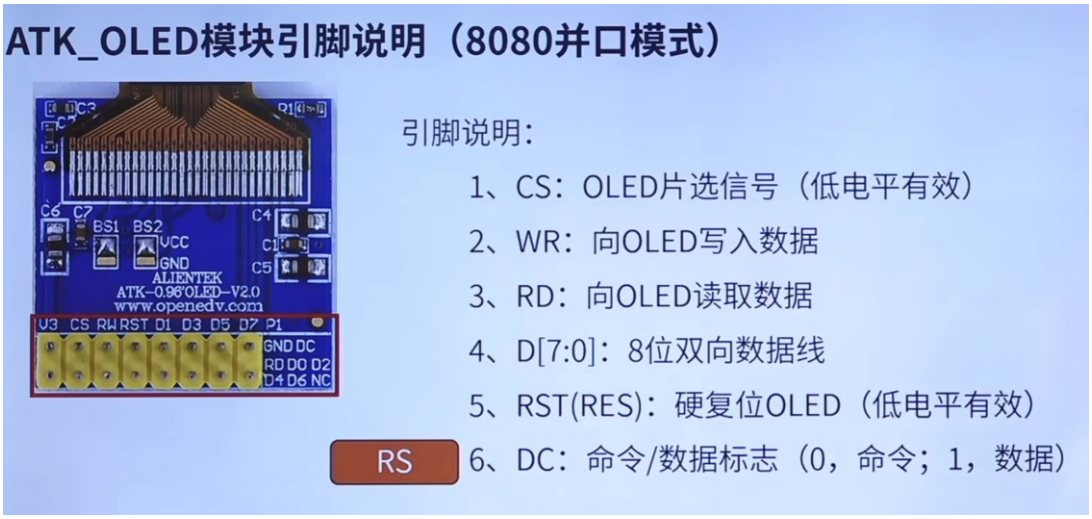
8.2 检测电容触摸按键过程



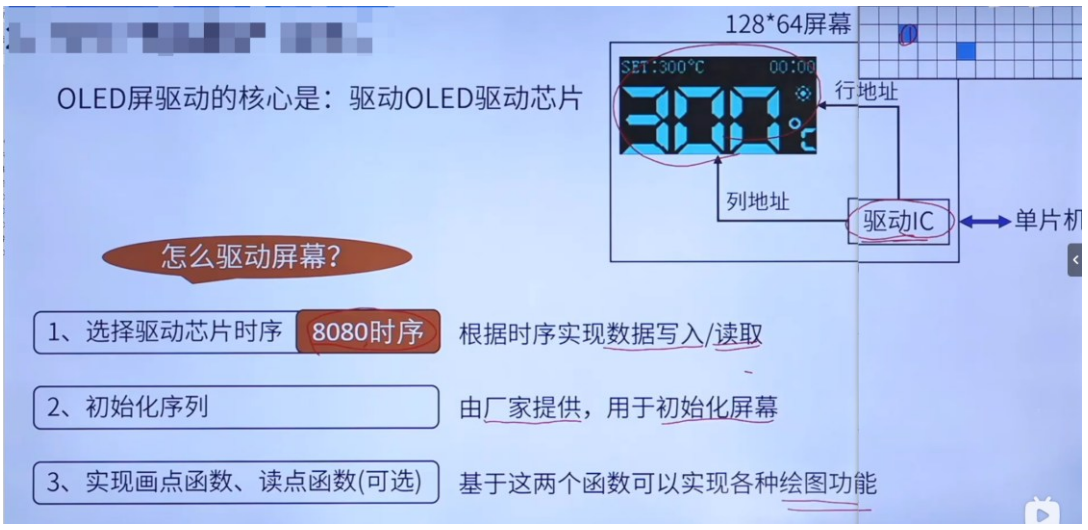
9 OLED 实验

9.1 OLED 显示屏介绍


正点原子OLED模块介绍
ATK_OLED模块是一块小尺寸（0.96英寸）、高亮、自带升压电路的高性能OLED显示模块，分辨率为128 * 64，采用SSD1306驱动IC，该芯片内部集成DCDC升压，仅需3.3V供电。
模块支持8位6800并口、8位8080并口、4线SPI以及IIC等4种通信接口。
怎么选择？
通过BS1和BS2选择
接口方式 | 8位8080 | 8位6800 | 4线SPI | IIC
BS1 | 1 | 0 | 0 | 1
BS2 | 1 | 1 | 0 | 0


ATK_OLED模块引脚说明（8080并口模式）
引脚说明：
1、CS：OLED片选信号（低电平有效）
2、WR：向OLED写入数据
3、RD：向OLED读取数据
4、D[7:0]：8位双向数据线
5、RST(RES)：硬复位OLED（低电平有效）
RS
6、DC：命令/数据标志（0，命令；1，数据）

9.2 OLED 驱动原理


OLED屏驱动的核心是：驱动OLED驱动芯片
128*64屏幕
行地址
列地址
驱动IC
单片机
怎么驱动屏幕？
1、选择驱动芯片时序
8080时序
根据时序实现数据写入/读取
2、初始化序列
由厂家提供，用于初始化屏幕
3、实现画点函数、读点函数(可选)
基于这两个函数可以实现各种绘图功能

SSD1306工作时序(8080时序)

并口总线时序，常用于屏幕驱动IC的访问，由Intel提出，也叫英特尔总线

8080并口读/写的过程：

1、设置DC为高(数据)/低(命令) — 根据写入/读取的数据类型

2、拉低片选 — 选中SSD1306

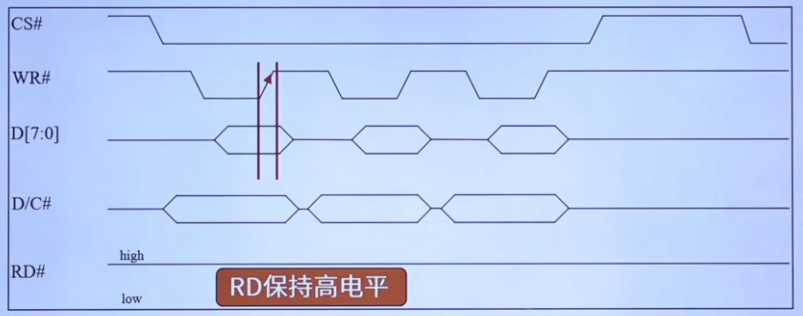
3、设置RD/WR为低 — 根据是读数据还是写数据

4、拉高RD/WR，出现上升沿 — 在RD的上升沿，使数据锁存到数据线D[7:0]上；在WR的上升沿，使数据写入到SSD1306里

读和写：写命令、读命令、写数据、读数据

SSD1306工作时序(8080时序)--写时序

注意:在WR上升沿，使数据写入到SSD1306里



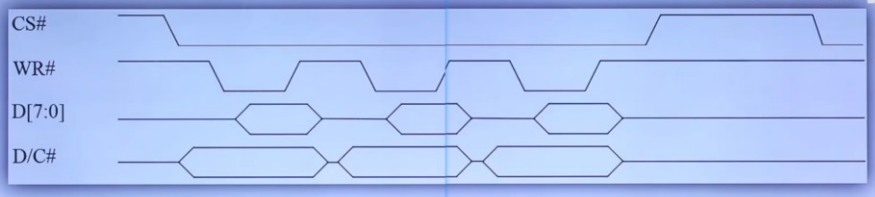
释放DC引脚

释放CS

数据类型(高:数据;低:命令) → 设置DC引脚电平 → 选中SSD1306 → CS引脚拉低 → 写数据 → WR拉低 → 数据线 → 准备数据 → 数据写到SSD130 → WR拉高

SSD1306写时序

正点原子官方 bilibili

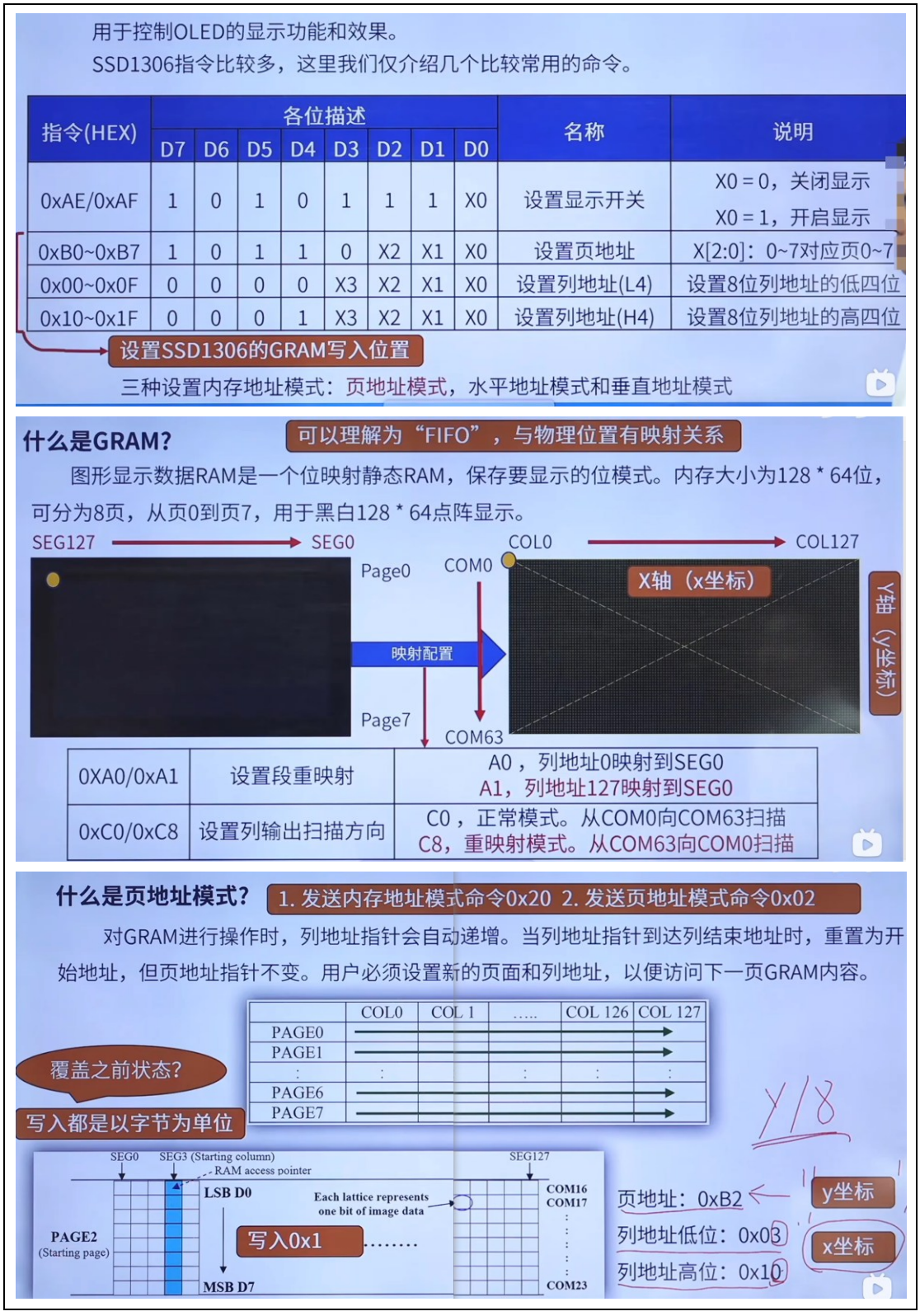


D[7:0]用到的是PC0~7

```
void oled_data_out(uint8_t data)
{
    GPIOC->ODR = (GPIOC->ODR & 0XFF00) | (data & 0X00FF);
}
```

```
void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    OLED_RS(cmd);      /* 数据类型，由传参决定 */
    OLED_CS(0);        /* 拉低片选线，选中SSD1306 */
    OLED_WR(0);        /* 拉低WR线，准备数据 */
    oled_data_out(data); /* WR低电平期间，准备数据 */
    OLED_WR(1);        /* 在WR上升沿，数据发出 */
    OLED_CS(1);        /* 取消片选 */
    OLED_RS(1);        /* 释放RS线，恢复默认 */
}
```

9.3 OLED 驱动芯片简介



指令(HEX)	各位描述 D7	D6	D5	D4	D3	D2	D1	D0	名称	说明
0xAE/0xAF	1	0	1	0	1	1	1	X0	设置显示开关	X0 = 0，关闭显示 X0 = 1，开启显示
0xB0~0xB7	1	0	1	1	0	X2	X1	X0	设置页地址	X[2:0]：0~7对应页0~7
0x00~0x0F	0	0	0	0	X3	X2	X1	X0	设置列地址(L4)	设置8位列地址的低四位
0x10~0x1F	0	0	0	1	X3	X2	X1	X0	设置列地址(H4)	设置8位列地址的高四位

解决显示覆盖问题?

画点前，必须要把设置的点所在字节的每一位弄清楚当前状态（0/1）

能读模式　读→改→写　慢

SPI和IIC不支持

建立一个OLED的GRAM

单片机内部需要128*8个字节，每次修改的时候，只是修改单片机上的GRAM（实际上就是SRAM），在修改完之后，一次性把单片机内部的GRAM写入到OLED的GRAM。

注意：SRAM很小的的单片机，内存不够，推荐还是使用并口模式，节省内存

9.4 字符显示原理

什么是ASCII字符?

基于拉丁字母的一套电脑编码系统，共128个字符：33个控制字符 和 95个显示字符

95个显示字符如下：

!"#$%&'()*+,-

./0123456789:;<=>?@ABCDEFGH

IJKLMNOPQRSTUVWXYZ[\]^_`a

bcdefghijklmnopqrstuvwxyz{|}~

ASCII 显示字符码表

值	符号	值	符号	值	符号
32	空格	44	,	65 ~ 90	A ~ Z
33	!	45	-	91	[
34	"	46	.	92	\
35	#	47	/	93	]
36	$	48 ~ 57	0 ~ 9	94	^
37	%	58	:	95	_
38	&	59	;	96	`
39	'	60	<	97~122	a ~ z
40	(	61	=	123	{
41	)	62	>	124	|
42	*	63	?	125	}
43	+	64	@	126	~

什么是内码（编码）？

计算机使用的二进制字符编码，就叫内码，简称编码。字符包含各种文字！

字符集	编码长度	说明
ASCII	1个字节	拉丁字母编码，仅128个编码，最简单
GB2312	2个字节	简体中文字符编码，包含约6000多汉字编码
GBK	2个字节	对GB2312的扩充，支持繁体中文，约2W多汉字编码
BIG5	2个字节	繁体中文字符编码，在台湾、香港用的多
UNICODE	一般2个字节	国际标准编码，支持各国文字

此代码可以输出字符内码：　printf("%x\r\n",'A');

参考资料：www.cnblogs.com/gavin-num1/p/5170247.html

如何将字符显示在OLED上？

1，显示字符，必须先有其点阵数据，点阵数据的集合，叫做字库

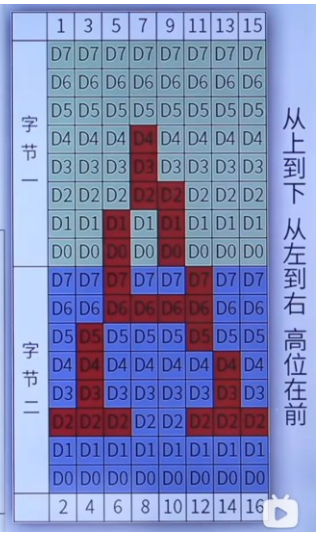
2，单片机根据点阵数据按取模方向进行描点还原，就能显示字符

3，ASCII字符宽度 = 汉字宽度的一半

16*16大小，字符A的点阵数据数组：

```
uint8_t oled_ascii_1608[]=
{
  0x00,0x04,0x00,0x3C,0x03,0xC4,0x1C,0x40,
  0x07,0x40,0x00,0xE4,0x00,0x1C,0x00,0x04
};
```

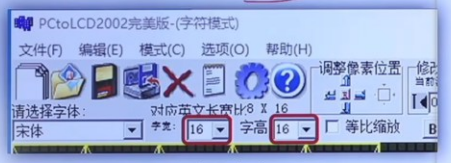
第一个字节：0x00
第二个字节：0x04
第三个字节：0x00
第四个字节：0x3C
第五个字节：0x03
第六个字节：0xC4
第七个字节：0x1C
第八个字节：0x40



ASCII字库制作（16*16）

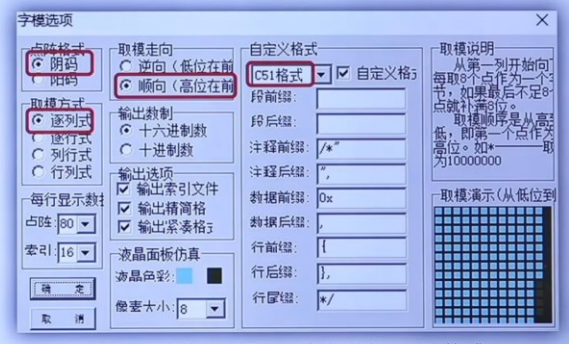
使用PCtoLCD2002.exe软件制作ASCII字库！

第1步，设置字体大小



16*16大小

第2步，设置字模选项

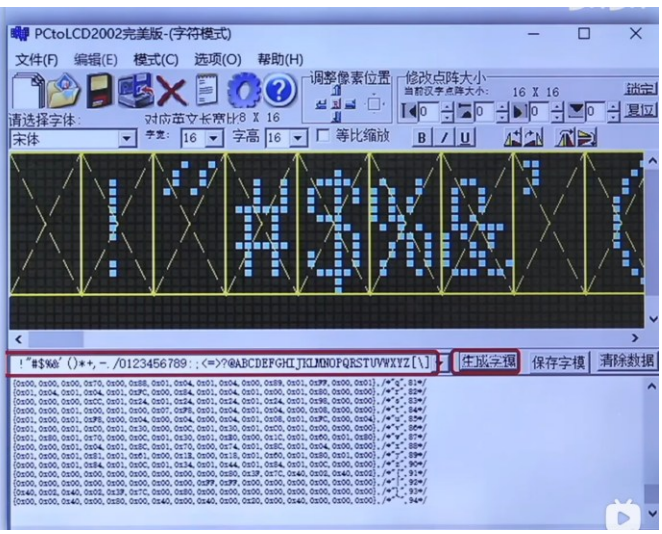


阴码，逐列式，顺向，十六进制，C51格式

第3步，输入ASCII字符集（95个）

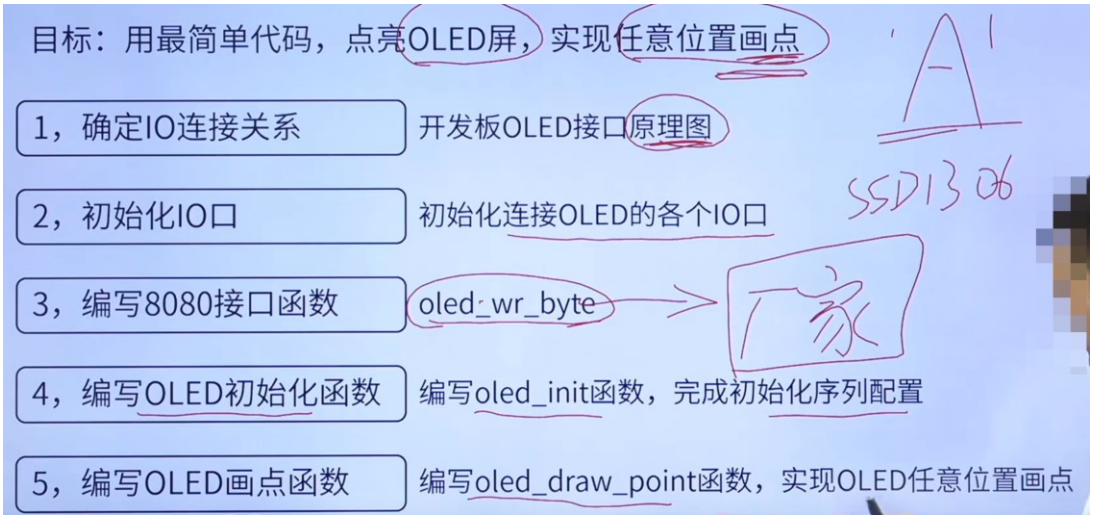
第4步，生成字模

```
{0x00,0x00,.....,0x00,0x00},  /* " ", 0 */
{0x00,0x00,.....,0x00,0x00},  /* "!", 1*/
{0x00,0x00,.....,0x00,0x00},  /* """, 2 */
{0x00,0x00,.....,0x00,0x00},  /* "#", 3 */
{0x00,0x00,.....,0x00,0x00},  /* "$", 4 */
{0x0F,0x00,.....,0x00,0x00},  /* "%", 5 */
                ......
{0x00,0x00,.....,0x40,0x02},  /* "{", 91 */
{0x00,0x00,.....,0x00,0x00},  /* "|", 92 */
{0x40,0x02,.....,0x00,0x00},  / *"}", 93 */
{0x00,0x00,.....,0x00,0x00},  /* "~", 94 */
```



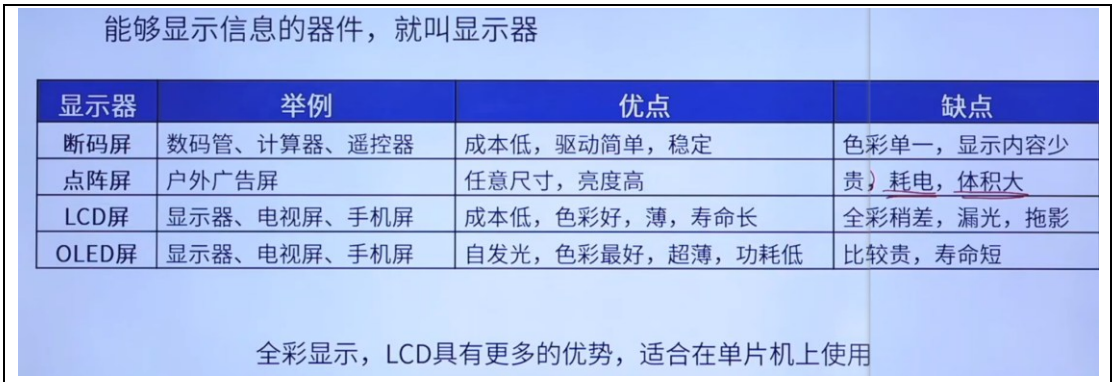
9.5 OLED 基本驱动步骤

9.5.1 OLED 基本驱动实现



10 LCD 实验

10.1 显示器分类



显示器	举例	优点	缺点
断码屏	数码管、计算器、遥控器	成本低，驱动简单，稳定	色彩单一，显示内容少
点阵屏	户外广告屏	任意尺寸，亮度高	贵，耗电，体积大
LCD屏	显示器、电视屏、手机屏	成本低，色彩好，薄，寿命长	全彩稍差，漏光，拖影
OLED屏	显示器、电视屏、手机屏	自发光，色彩最好，超薄，功耗低	比较贵，寿命短

10.2 LCD 简介

Liquid Crystal Display，即液晶显示器，由：玻璃基板、背光、驱动IC等组成

全彩LCD，是一种全彩显示屏（RGB565、RGB888），可以显示各种颜色

1，低成本	低至几块钱的价格
2，高解析度	可高达500ppi的解析度，显示细腻
3，高对比度	可高达1000：1的对比度，色彩清晰艳丽
4，响应速度快	可高达1ms响应速度，显示效果好

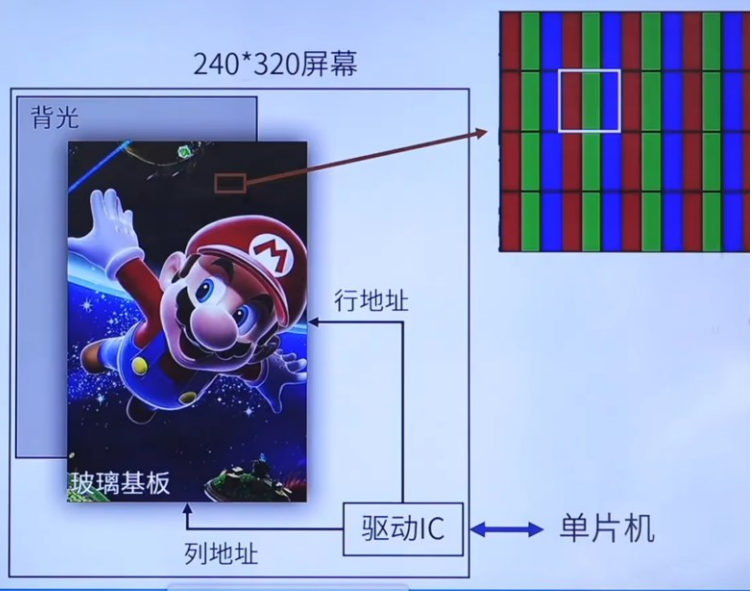
LCD原理：https://zhuanlan.zhihu.com/p/133306648

LCD基本组成

1，玻璃基板

2，背光

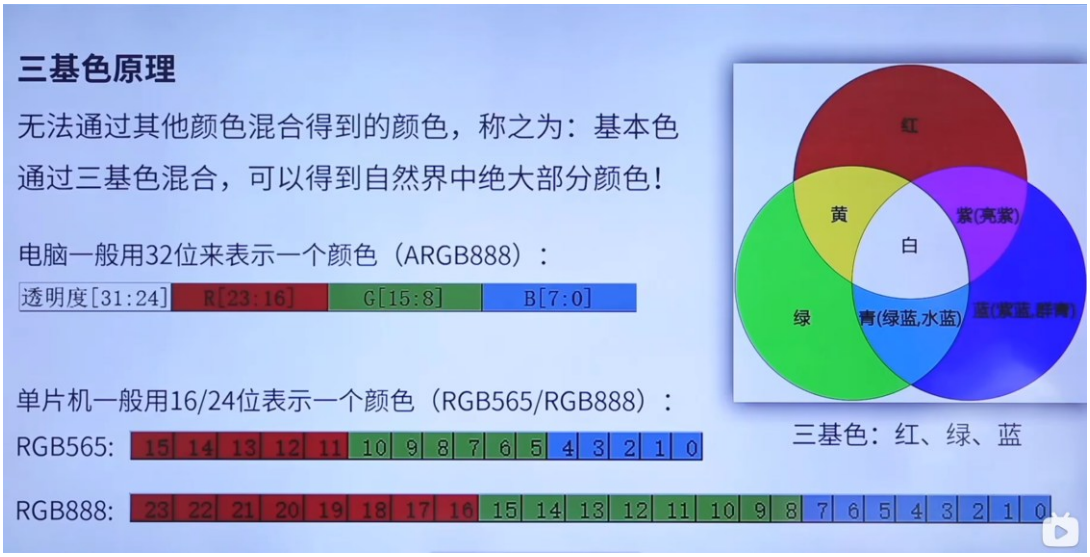
3，驱动IC



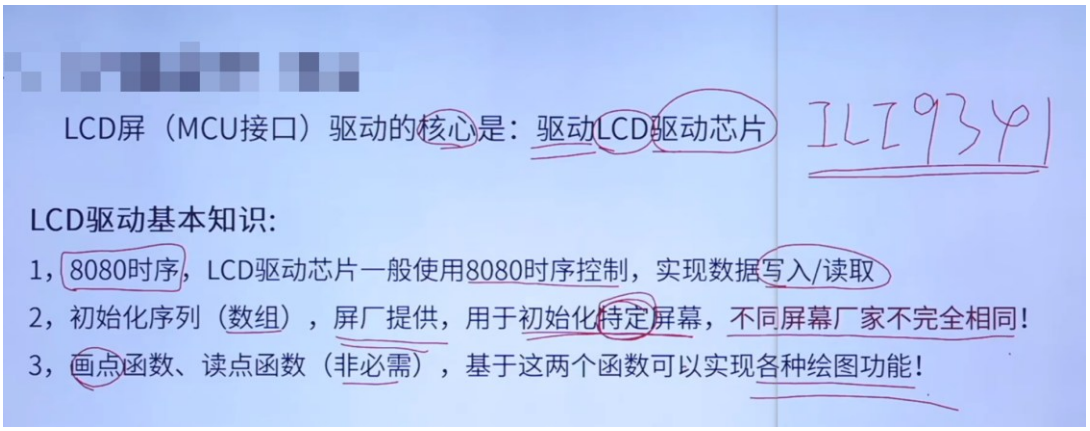
LCD接口分类

接口	分辨率	特性
MCU	≤800*480	带SRAM，无需频繁刷新，无需大内存，驱动简单
RGB	≤1280*800	不带SRAM，需要实时刷新，需要大内存，驱动稍微复杂
MIPI	4K	不带SRAM，支持分辨率高，省电，大部分手机屏用此接口

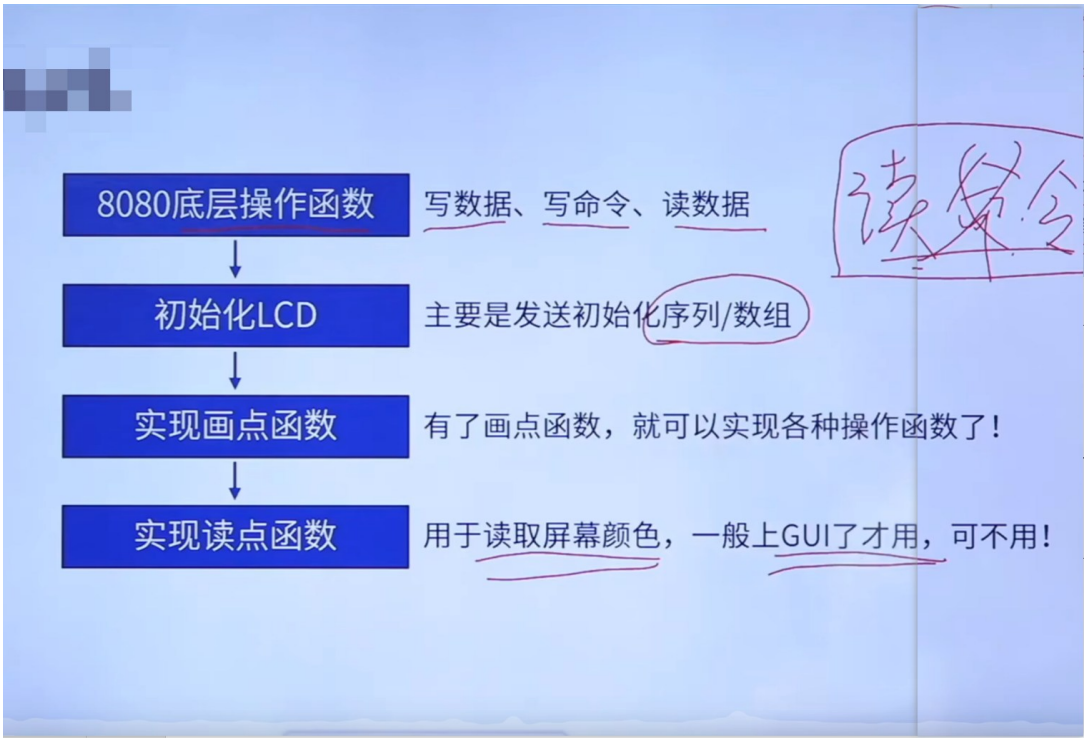
MCU屏接口由于自带SRAM，驱动简单，大部分单片机都能驱动！


三基色原理
无法通过其他颜色混合得到的颜色，称之为：基本色
通过三基色混合，可以得到自然界中绝大部分颜色！
电脑一般用32位来表示一个颜色（ARGB888）：
透明度[31:24]
R[23:16]
G[15:8]
B[7:0]
单片机一般用16/24位表示一个颜色（RGB565/RGB888）：
RGB565:
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
RGB888:
23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
红
黄
白
紫(亮紫)
绿
青(绿蓝,水蓝)
蓝(紫蓝,群青)
三基色：红、绿、蓝

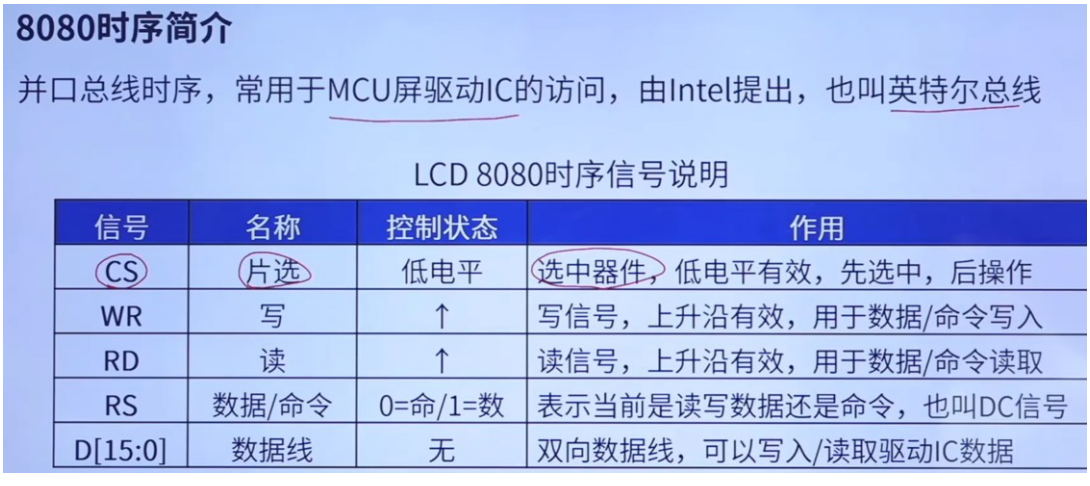
10.3 LCD 驱动原理


LCD屏（MCU接口）驱动的核心是：驱动LCD驱动芯片
ILI9341
LCD驱动基本知识:
1，8080时序，LCD驱动芯片一般使用8080时序控制，实现数据写入/读取
2，初始化序列（数组），屏厂提供，用于初始化特定屏幕，不同屏幕厂家不完全相同！
3，画点函数、读点函数（非必需），基于这两个函数可以实现各种绘图功能！

10.4 LCD 驱动的一般过程



10.5 8080 时序简介



8080时序简介

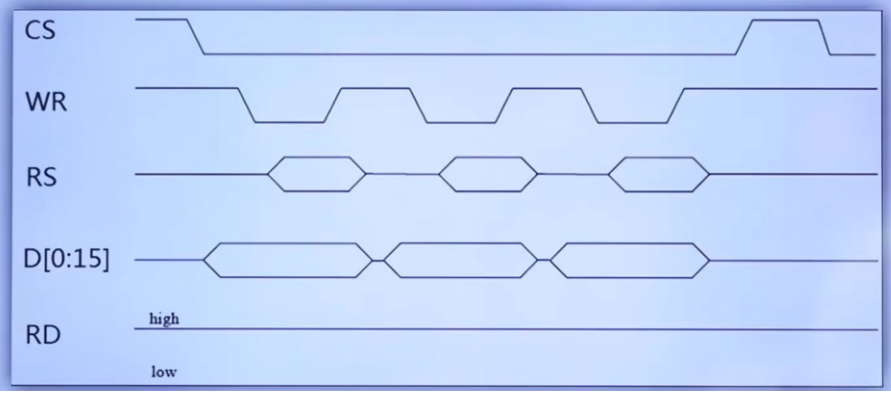
并口总线时序，常用于MCU屏驱动IC的访问，由Intel提出，也叫英特尔总线

LCD 8080时序信号说明

信号	名称	控制状态	作用
CS	片选	低电平	选中器件，低电平有效，先选中，后操作
WR	写	↑	写信号，上升沿有效，用于数据/命令写入
RD	读	↑	读信号，上升沿有效，用于数据/命令读取
RS	数据/命令	0=命/1=数	表示当前是读写数据还是命令，也叫DC信号
D[15:0]	数据线	无	双向数据线，可以写入/读取驱动IC数据

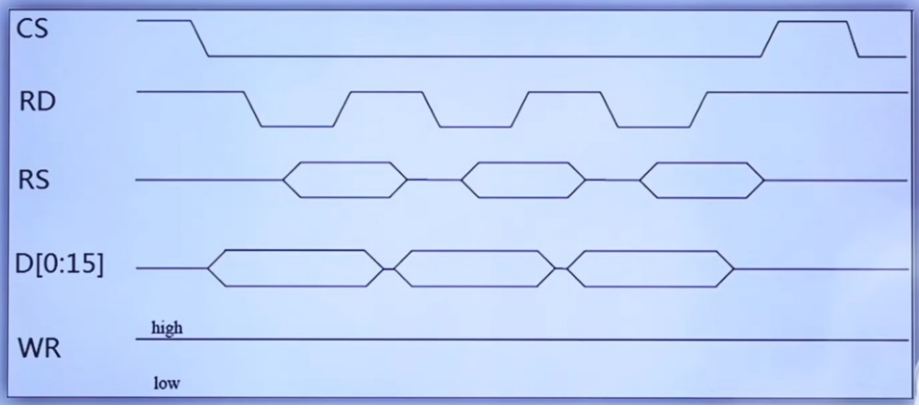
8080写时序

数据（RS=1）/命令（RS=0）在WR的上升沿，写入LCD驱动IC，RD保持高电平



8080读时序

数据（RS=1）/命令（RS=0）在RD的上升沿，读取到MCU，WR保持高电平



8080读写简化代码

```
void lcd_wr_data(uint16_t data)
{
    LCD_RS(1);              /* 操作数据 */
    LCD_CS(0);              /* 选中 */
    LCD_DATA_OUT(data);     /* 数据 */
    LCD_WR(0);              /* WR低电平 */
    LCD_WR(1);              /* WR高电平 */
    LCD_CS(1);              /* 释放片选 */
}
```

```
uint16_t lcd_rd_data(void)
{
    uint16_t ram;           /* 定义变量 */
    LCD_RS(1);              /* 操作数据 */
    LCD_CS(0);              /* 选中 */
    LCD_RD(0);              /* RD低电平 */
    ram = LCD_DATA_IN;      /* 读取数据 */
    LCD_RD(1);              /* RD高电平 */
    LCD_CS(1);              /* 释放片选 */
    return ram;             /* 返回读数 */
}
```

10.6 LCD 驱动芯片简介

用于控制LCD的各种显示功能和效果，整体功能较复杂。常见型号：ILI9341/ST7789等

一般我们只需要：6条指令即可完成对LCD的基本使用（以9341为例）

指令(HEX)	名称	作用
0XD3	读ID	用于读取LCD控制器的ID，区分型号用
0X36	访问控制	设置GRAM读写方向，控制显示方向
0X2A	列地址	一般用于设置X坐标
0X2B	页地址	一般用于设置Y坐标
0X2C	写GRAM	用于往LCD写GRAM数据
0X2E	读GRAM	用于读取LCD的GRAM数据

读ID指令（0XD3）

读取LCD控制器型号，通过型号可以执行不同LCD初始化，以兼容不同LCD

顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	1	1	0	1	0	0	1	1	D3H
参数1	1	↑	1	XX	X	X	X	X	X	X	X	X	X
参数2	1	↑	1	XX	0	0	0	0	0	0	0	0	00H
参数3	1	↑	1	XX	1	0	0	1	0	0	1	1	93H
参数4	1	↑	1	XX	0	1	0	0	0	0	0	1	41H

访问控制指令（0X36）

实现GRAM读写方向控制，即：控制GRAM自增方向，从而控制显示方向

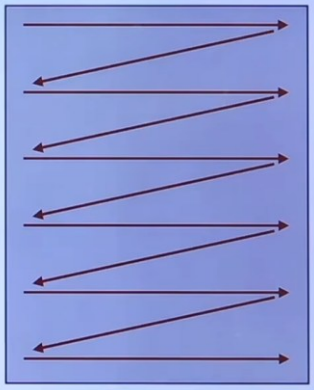
顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	1	0	1	1	0	36H
参数1	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	XXH

MX,MY,MV：共同控制GRAM自增方向（扫描方向）

BGR位：可以控制RGB、BGR顺序

MX、MY、MV扫描方向控制关系

控制位			效果
MY	MX	MV	LCD扫描方向（GRAM自增方式）
0	0	0	从左到右, 从上到下
1	0	0	从左到右, 从下到上
0	1	0	从右到左, 从上到下
1	1	0	从右到左, 从下到上
0	0	1	从上到下, 从左到右
0	1	1	从上到下, 从右到左
1	0	1	从下到上, 从左到右
1	1	1	从下到上, 从右到左



从左到右，从上到下

X坐标设置指令（0X2A）

列地址设置指令，一般用于设置X坐标

顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2AH
参数1	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	SC
参数2	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
参数3	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	EC
参数4	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	

SC：起始坐标

EC：结束坐标

设置关系：0≤SC≤EC≤239（LCD像素宽度）

Y坐标设置指令（0X2B）

页地址设置指令，一般用于设置Y坐标

顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	1	2BH
参数1	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SP
参数2	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	
参数3	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	EP
参数4	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0	

SP：起始坐标

EP：结束坐标

设置关系：0≤SP≤EP≤319（LCD像素高度）

写GRAM指令（0X2C）

发送该指令后，数据线变成16位，可以开始写入GRAM数据，支持地址自增

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	1	0	0	2CH
参数1	1	1	↑	D1[15：0]									XX
……	1	1	↑	D2[15：0]									XX
参数N	1	1	↑	Dn[15：0]									XX

每次写入1个像素点的颜色值（RGB565），地址自增方向由MX/MY/MV控制

无需重新设置坐标，可实现连续写入，大大提高写入速度

读GRAM指令（0X2E）

发送该指令后，数据线变成16位，可以开始读取GRAM数据，支持地址自增

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D11	D10~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX		0	0	1	0	1	1	1	0	2EH
参数1	1	↑	1	XX										dummy
参数2	1	↑	1	R1[4:0]	XX	G1[5:0]						XX		R1G1
参数3	1	↑	1	B1[4:0]	XX	R2[4:0]					XX			B1R2
参数4	1	↑	1	G2[5:0]	XX	B2[4:0]					XX			G2B2
参数5	1	↑	1	R3[4:0]	XX	G3[5:0]						XX		R3G3
参数N	1	↑	1	按以上规律输出										

读1个点的颜色，要读3次

1，dummy

2，R1G1

3，B1R2

```
uint16_t r, g, b;
r = lcd_rd_data();
g = lcd_rd_data();
b = lcd_rd_data();
```

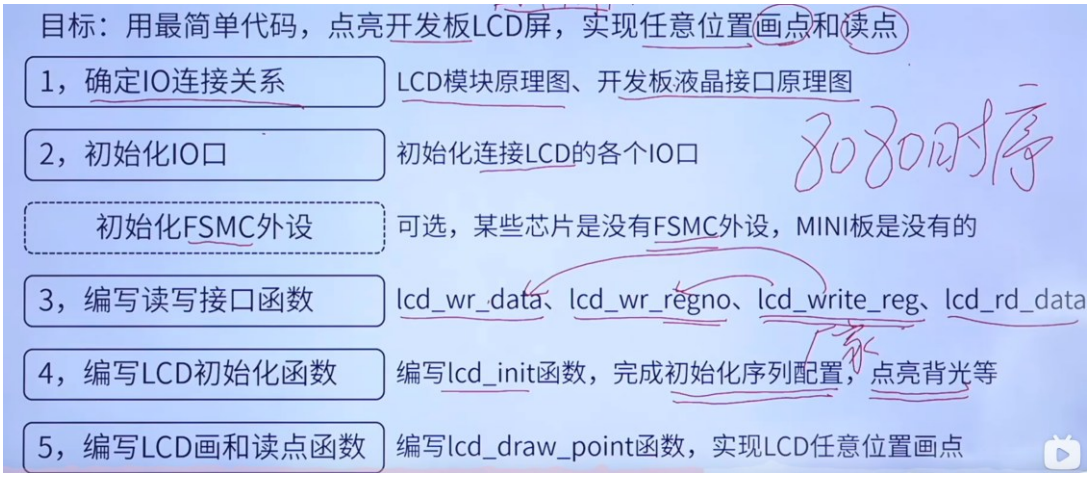
```
g = r & 0XFF;
return (((r >> 11) << 11) | ((g >> 2) << 5) | (b >> 11));
```

读点函数代码(精简)

```
uint16_t lcd_rd_data(void)
{
    uint16_t ram;           /* 定义变量 */
    DATA_IN_MODE();         /* 设置数据输入 */
    LCD_RS(1);              /* 操作数据 */
    LCD_CS(0);              /* 选中 */
    LCD_RD(0);              /* RD低电平 */
    ram = LCD_DATA_IN;      /* 读取数据 */
    LCD_RD(1);              /* RD高电平 */
    LCD_CS(1);              /* 释放片选 */
    DATA_OUT_MODE();        /* 设置数据输出 */
    return ram;             /* 返回读数 */
}
```

```
uint16_t lcd_read_point (uint16_t x, uint16_t y)
{
    uint16_t r = 0, g = 0, b = 0;   /* 定义变量 */
    lcd_set_cursor(x, y);           /* 设置坐标 */
    lcd_wr_regno(0X2E);             /* 发读点命令 */
    r = lcd_rd_data();              /* 假读 */
    r = lcd_rd_data();              /* 读rg */
    b = lcd_rd_data();              /* 读b */
    g = r & 0XFF;                   /* 得到g值 */
    return (((r >> 11) << 11) | ((g >> 2) << 5) | (b >> 11));
}
```

10.7 LCD 基本驱动步骤

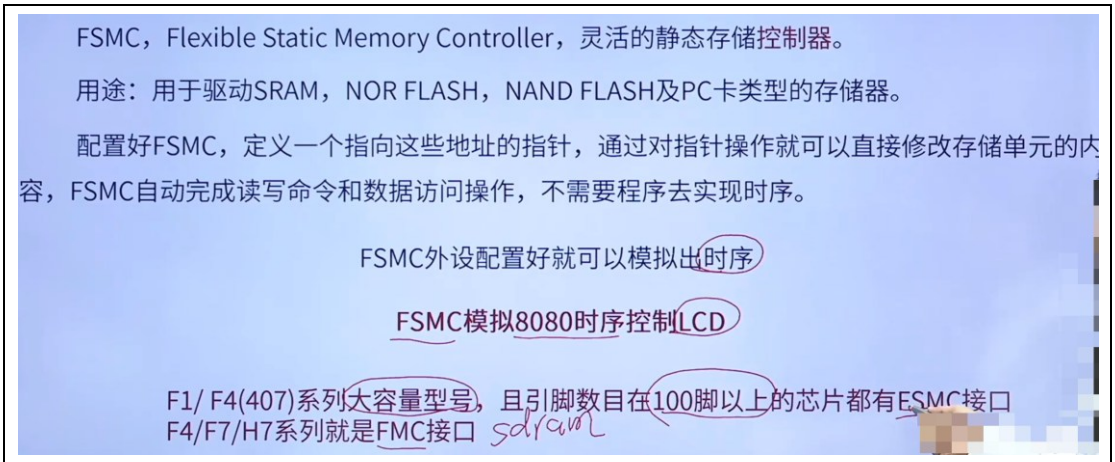


LCD控制结构体简介

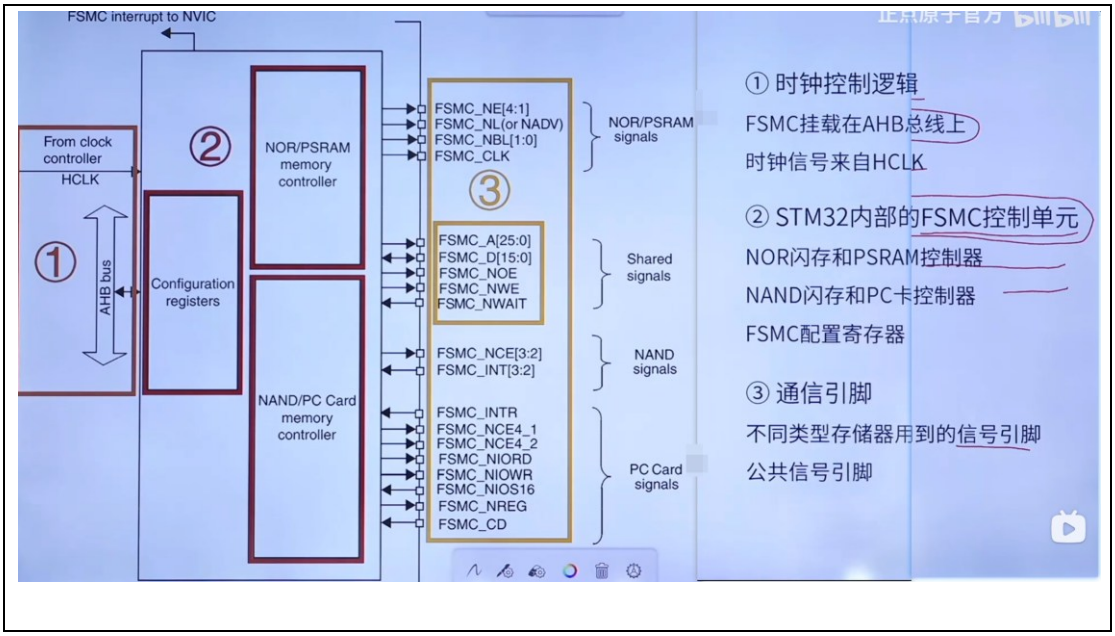
```
/* LCD重要参数集 */
typedef struct
{
  uint16_t  width;      /* LCD 宽度 */
  uint16_t  height;     /* LCD 高度 */
  uint16_t  id;         /* LCD ID */
  uint8_t   dir;        /* 横屏还是竖屏控制：0，竖屏；1，横屏 */
  uint16_t  wramcmd;    /* 开始写gram指令 */
  uint16_t  setxcmd;    /* 设置x坐标指令 */
  uint16_t  setycmd;    /* 设置y坐标指令 */
}_lcd_dev;
```

10.8 FSMC 介绍

10.8.1 FSMC 简介



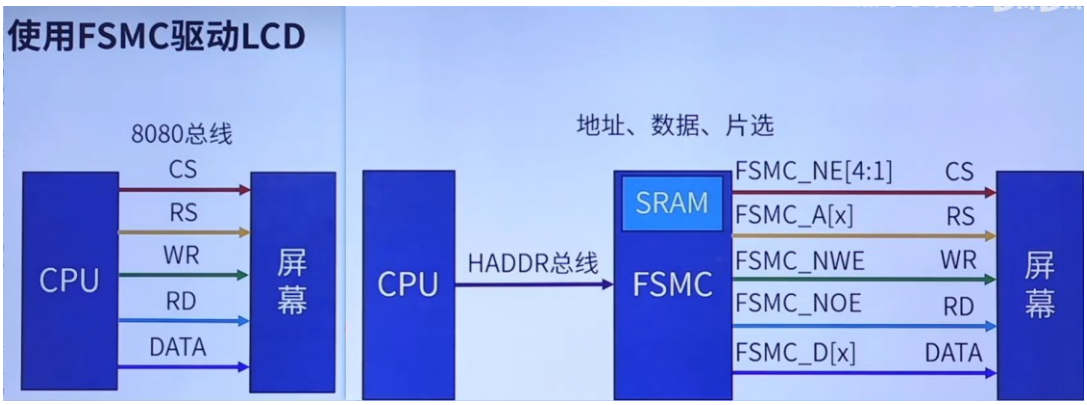
10.8.2 FSMC 框图介绍



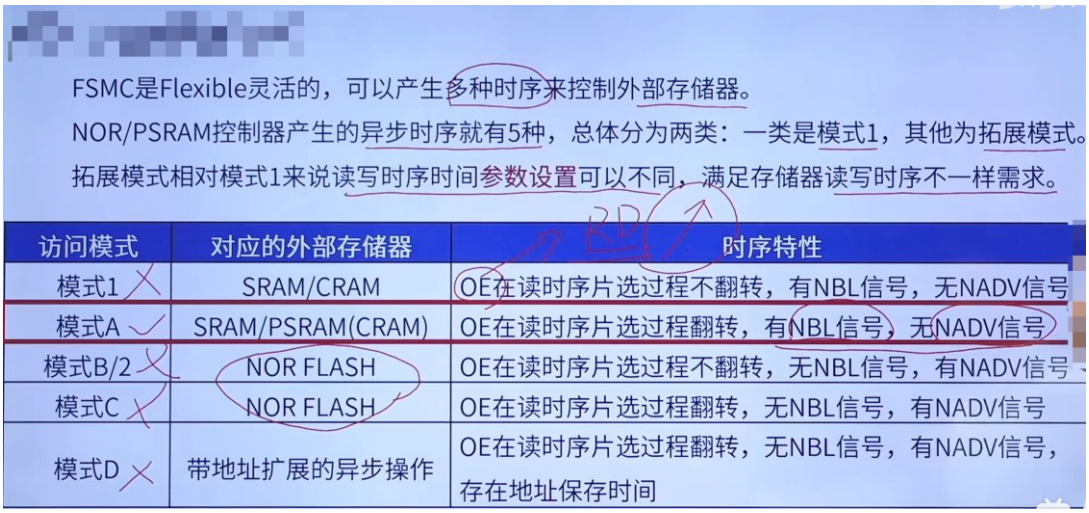
10.8.3 FSMC 通信引脚介绍

FSMC信号	信号方向	功能
FSMC_NE[x]	输出	片选引脚，x=1…4，每个对应不同的内存块
FSMC_CLK	输出	时钟（同步突发模式使用）
FSMC_A[25:0]	输出	地址总线
FSMC_D[15:0]	输出/输入	双向数据总线
FSMC_NOE	输出	输出使能（“N”表明低电平有效信号）
FSMC_NWE	输出	写使能
FSMC_NWAIT	输入	NOR闪存要求FSMC等待的信号
FSMC_NADV	输出	地址、数据线复用时作锁存信号

10.8.4 使用 FSMC 驱动 LCD



10.8.5 FSMC 时序介绍



FSMC是Flexible灵活的，可以产生多种时序来控制外部存储器。

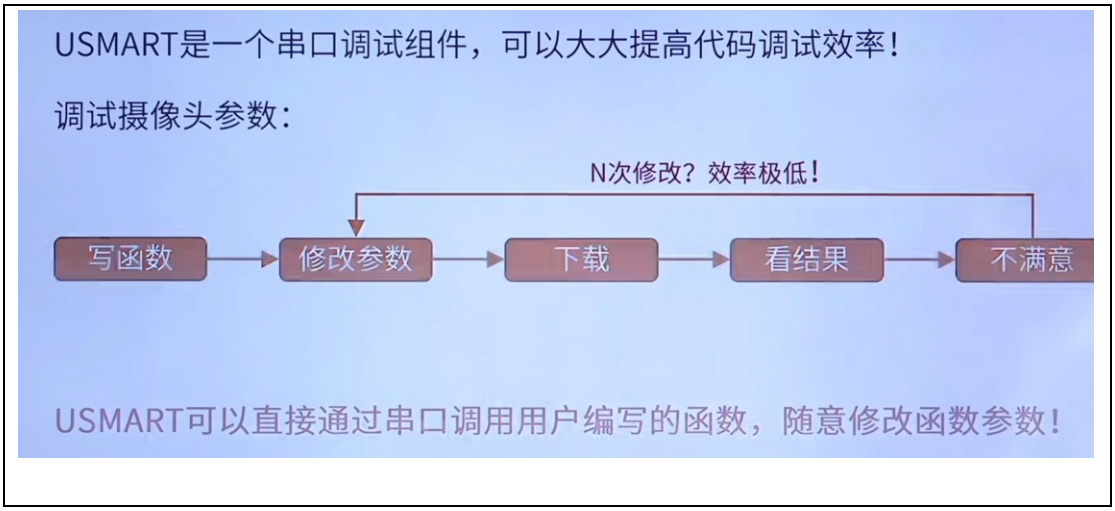
NOR/PSRAM控制器产生的异步时序就有5种，总体分为两类：一类是模式1，其他为拓展模式。

拓展模式相对模式1来说读写时序时间参数设置可以不同，满足存储器读写时序不一样需求。

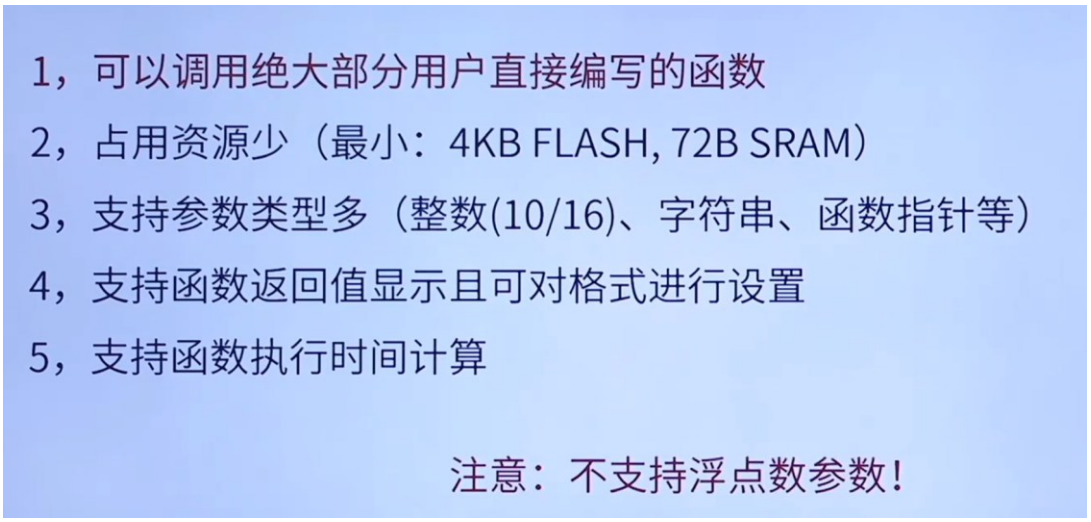
访问模式	对应的外部存储器	时序特性
模式1	SRAM/CRAM	OE在读时序片选过程不翻转，有NBL信号，无NADV信号
模式A	SRAM/PSRAM(CRAM)	OE在读时序片选过程翻转，有NBL信号，无NADV信号
模式B/2	NOR FLASH	OE在读时序片选过程不翻转，无NBL信号，有NADV信号
模式C	NOR FLASH	OE在读时序片选过程翻转，无NBL信号，有NADV信号
模式D	带地址扩展的异步操作	OE在读时序片选过程翻转，无NBL信号，有NADV信号，存在地址保存时间

11 USMART

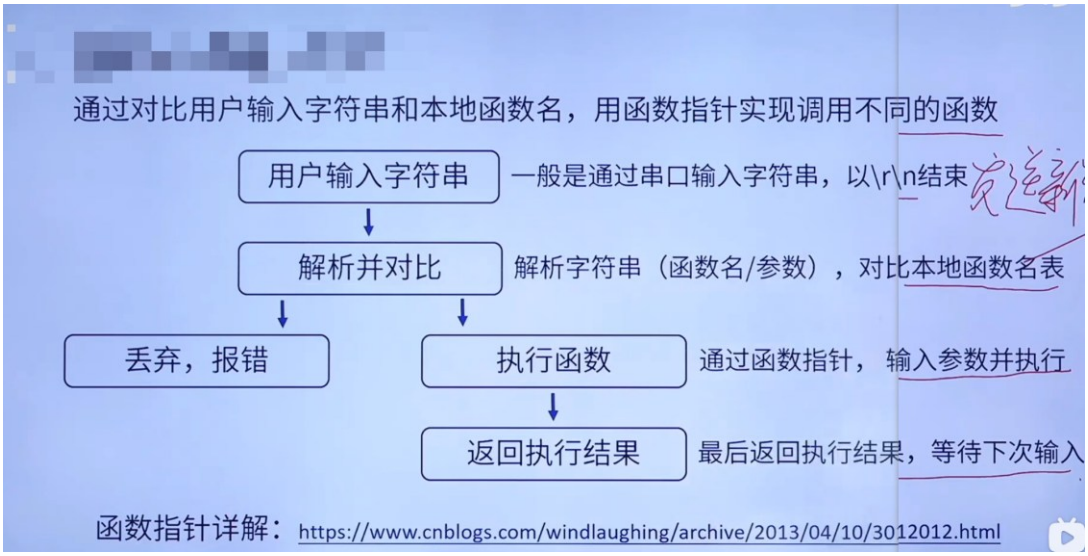
11.1 USMART 简介



11.2 USMART 主要特点



11.3 USMART 原理



11.4 USMART 组成

文件	功能描述
usmart .c/h	核心文件，用于处理命令以及对外部交互
usmart_config.c	函数管理文件，用于添加用户需要USMART管理的函数
usmart_port.c/h	移植文件，用于USMART移植
usmart_str.c/h	字符串处理文件，用于字符串转换、参数获取等

修改：usmart_port.c/.h，即可完成移植

修改：usmart_config.c，即可添加自己想要调用的函数

11.5 USMART 扫描函数

void usmart_scan(void);

扫描函数，解析用户输入字符串数据，进而实现函数调用和组件各个控制功能

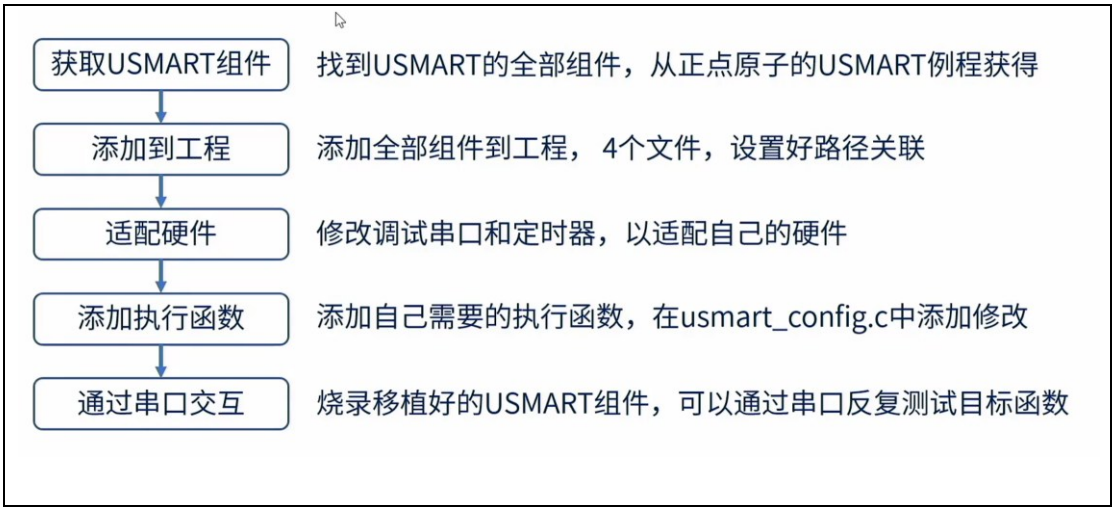
usmart_scan调用以下获取用户输入字符串数据函数:

```
char *usmart_get_input_string(void)
{
  uint8_t len;
  char *pbuf = 0;
  if (g_usart_rx_sta & 0x8000)       /* 串口接收完成? */
  {
    len = g_usart_rx_sta & 0x3fff;  /* 得到此次接收到的数据长度 */
    g_usart_rx_buf[len] = '\0';      /* 在末尾加入结束符. */
    pbuf = (char*)g_usart_rx_buf;
    g_usart_rx_sta = 0;              /* 开启下一次接收 */
  }
  return pbuf;
}
```

g_usart_rx_sta：串口的接收缓冲标记

g_usart_rx_sta	
bit15	接收完成标志
bit14	接收到 0X0D 标志
bit13~0	接收到的有效字节个数

11.6 USMART 移植



11.7 USMART 使用

USMART有7个系统命令：

？：获取帮助信息

help：获取帮助信息

list：可用的函数列表

id：可用函数的ID列表

hex：参数16进制显示，后跟空格+数字即执行进制转换

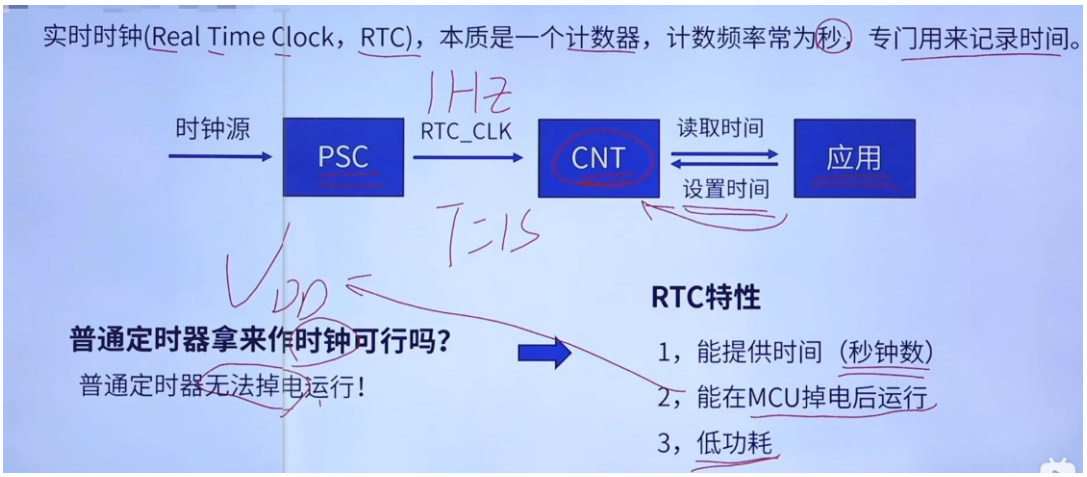
dec：参数10进制显示，后跟空格+数字即执行进制转换

runtime 1，开启函数运行计时；

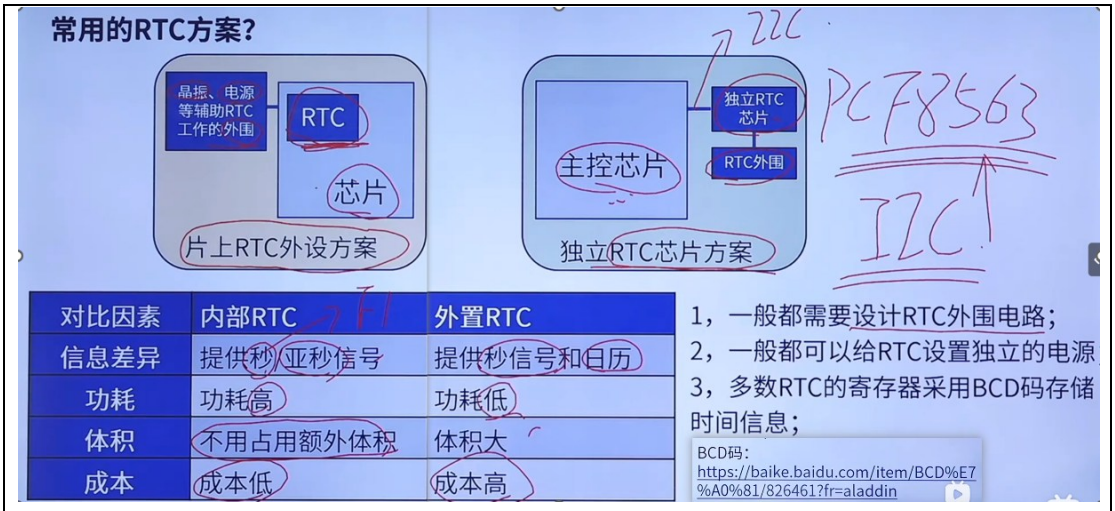
0，关闭函数运行计时；

12 RTC

12.1 RTC 简介

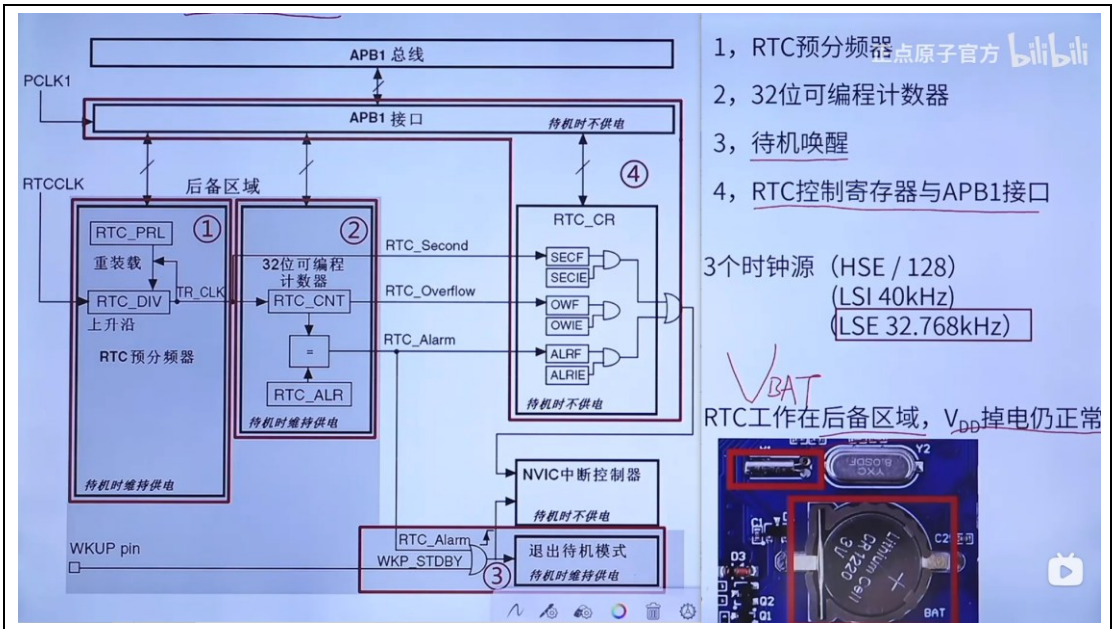


12.2 常用的 RTC 方案



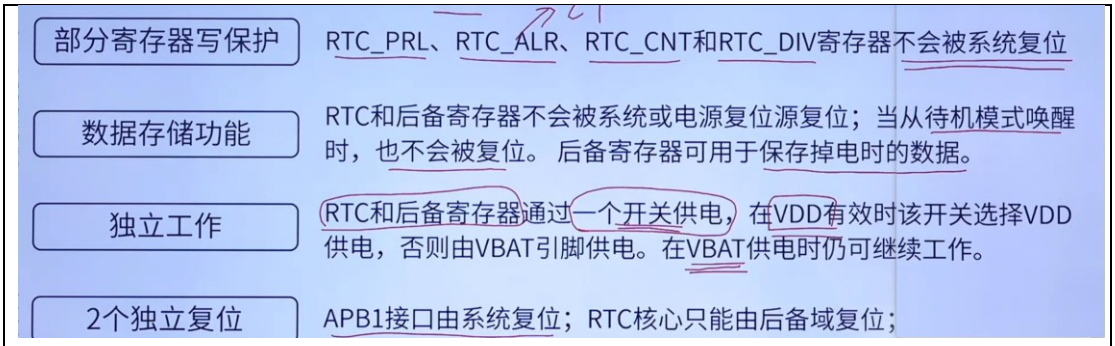
12.3 F1

12.3.1 STM32 F1 RTC 框图介绍

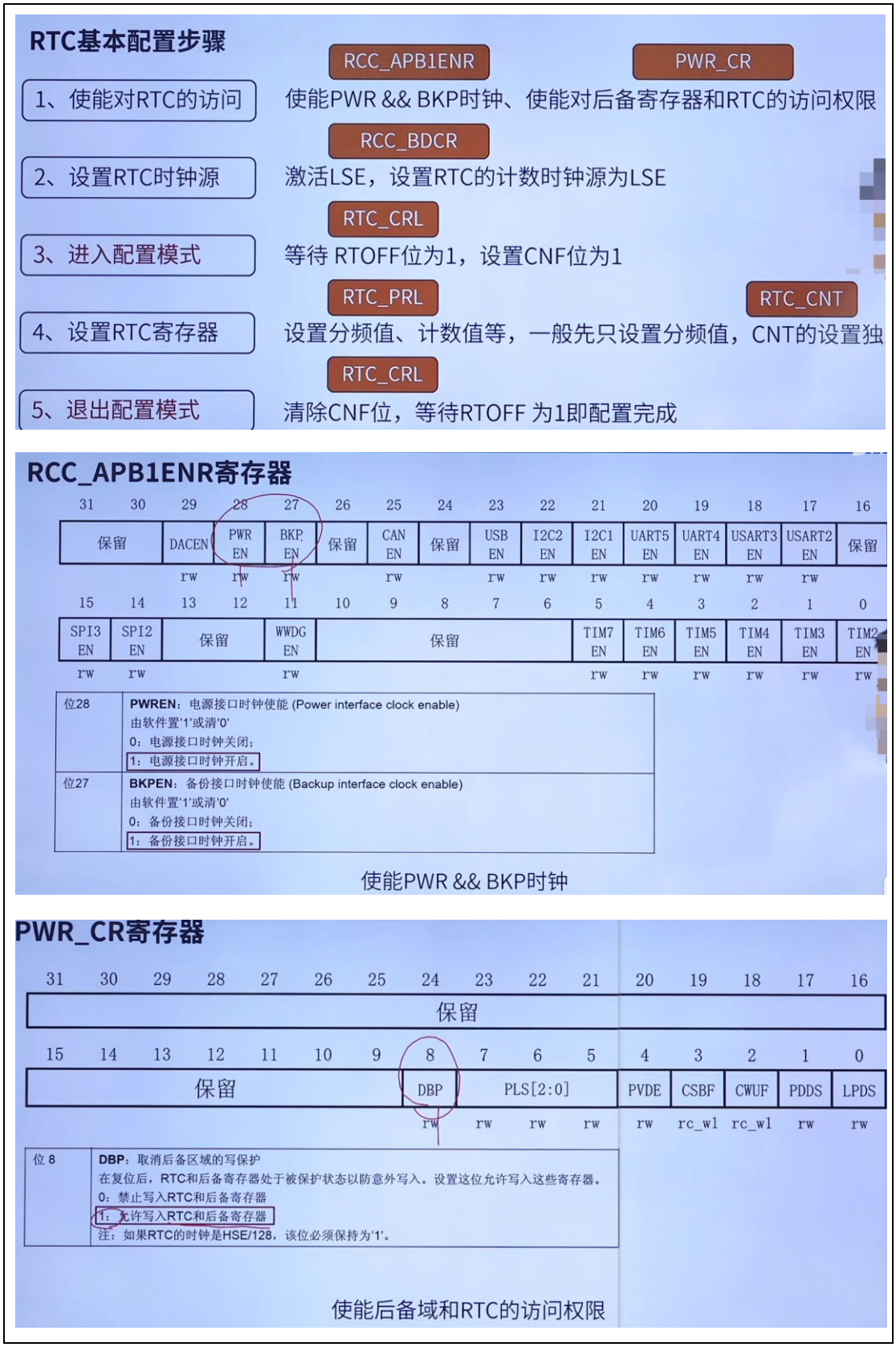


12.3.2 RTC 相关寄存器介绍

12.3.2.1 后备寄存器和 RTC 寄存器特性（F1）



12.3.2.2 F1 RTC 相关寄存器介绍



RCC_BDCR寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															BDRST
															rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC EN	保留					RTCSEL[1:0]		保留					LSE BYP	LSE RDY	LSEON
rw						rw	rw						rw	r	rw

位15	**RTCEN**：RTC时钟使能 (RTC clock enable) 由软件置'1'或清'0' 0：RTC时钟关闭； 1：RTC时钟开启。
位9:8	**RTCSEL[1:0]**：RTC时钟源选择 (RTC clock source selection) 由软件设置来选择RTC时钟源。一旦RTC时钟源被选定，直到下次后备域被复位，它不能在被改变。可通过设置BDRST位来清除。 00：无时钟； 01：LSE振荡器作为RTC时钟； 10：LSI振荡器作为RTC时钟； 11：HSE振荡器在128分频后作为RTC时钟。
位0	**LSEON**：外部低速振荡器使能 (External low-speed oscillator enable) 由软件置'1'或清'0' 0：外部32kHz振荡器关闭； 1：外部32kHz振荡器开启。

通过RCC_BDCR配置：
1，开启RTC时钟
2，开启LSE时钟；
3，选择RTC计数时钟源；

RTC_CRL寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留										RTOFF	CNF	RSF	OWF	ALRF	SECF
										r	rw	rc w0	rc w0	rc w0	rc w0

位5	**RTOFF**：RTC操作关闭 (RTC operation OFF) RTC模块利用这位来指示对其寄存器进行的最后一次操作的状态，指示操作是否完成。若此位为'0'，则表示无法对任何的RTC寄存器进行写操作。此位为只读位。 0：上一次对RTC寄存器的写操作仍在进行； 1：上一次对RTC寄存器的写操作已经完成。
位4	**CNF**：配置标志 (Configuration flag) 此位必须由软件置'1'以进入配置模式，从而允许向RTC_CNT、RTC_ALR或RTC_PRL寄存器写入数据。只有当此位在被置'1'并重新由软件清'0'后，才会执行写操作。 0：退出配置模式(开始更新RTC寄存器)； 1：进入配置模式。

RTC进入配置模式，关注的位

RTC_CRH寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留													OWIE	ALRIE	SECIE
													rw	rw	rw

复位值为0，置1可开启相关中断

RTC_PRLH寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留												PRL[19:16]			
												w	w	w	w

位3:0	**PRL[19:16]**：RTC预分频装载值高位 (RTC prescaler reload value high) 根据以下公式，这些位用来定义计数器的时钟频率： $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$ 注：不推荐使用0值，否则无法正确的产生RTC中断和标志位。

使计数器每计数一次就是1秒，即需要32768分频。

RTC_PRLL寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRL[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位15:0	**PRL[15:0]**：RTC预分频装载值低位 根据以下公式，这些位用来定义计数器的时钟频率： $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$

RTC_PRLH寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留												PRL[19:16]			
												w	w	w	w

位3:0	**PRL[19:16]**：RTC预分频装载值高位 (RTC prescaler reload value high) 根据以下公式，这些位用来定义计数器的时钟频率： $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$ 注：不推荐使用0值，否则无法正确的产生RTC中断和标志位。

RTC_PRLL寄存器

使计数器每计数一次就是1秒，即需要32768分频。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRL[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位15:0	**PRL[15:0]**：RTC预分频装载值低位 根据以下公式，这些位用来定义计数器的时钟频率： $f_{TR_CLK} = f_{RTCCLK}/(PRL[19:0]+1)$

与RTC_PRL相似的寄存器RTC_DIV可以在不停止分频计数器的工作，获得预分频计数器的当前值

RTC_CNTH寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_CNT[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位15:0	**RTC_CNT[31:16]**：RTC计数器高位 (RTC counter high) 可通过读RTC_CNTH寄存器来获得RTC计数器当前值的高位部分。要对此寄存器进行写操作前，必须先进入配置模式(参见16.3.4节)。

注意：进入配置模式才能对寄存器进行操作

RTC_CNTL寄存器

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTC_CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位15:0	**RTC_CNT[15:0]**：RTC计数器低位。 可通过读RTC_CNTL寄存器来获得RTC计数器当前值的低位部分。要对此寄存器进行写操作，必须先进入配置模式(参见16.3.4节)。

与RTC_CNT相似的寄存器还有RTC_ALR，这里就不做介绍。

12.3.3 RTC 相关 HAL 库驱动介绍

驱动函数	关联寄存器	功能描述
HAL_RTC_Init(...)	CRL/CRH/PRLH/PRLL	初始化RTC
HAL_RTC_MspInit(…)	初始化回调	使能RTC时钟
HAL_RCC_OscConfig(…)	RCC_CR/PWR_CR	开启LSE时钟源
HAL_RCCEx_PeriphCLKConfig(...)	RCC_BDCR	设置RTC时钟源为LSE
HAL_PWR_EnableBkUpAccess(...)	PWR_CR	使能备份域的访问权限
HAL_RTCEx_BKUPWrite/Read()	BKP_DRx	读/写备份域数据寄存器

以下是需要开启的时钟源：
__HAL_RCC_RTC_ENABLE()
__HAL_RCC_PWR_CLK_ENABLE()
__HAL_RCC_BKP_CLK_ENABLE()

12.3.4 RTC 基本驱动步骤（F1 ）

1、使能电源时钟并使能后备域访问
__HAL_RCC_PWR_CLK_ENABLE 使能电源时钟
__HAL_RCC_BKP_CLK_ENABLE 使能备份时钟
HAL_PWR_EnableBkUpAccess 使能备份访问

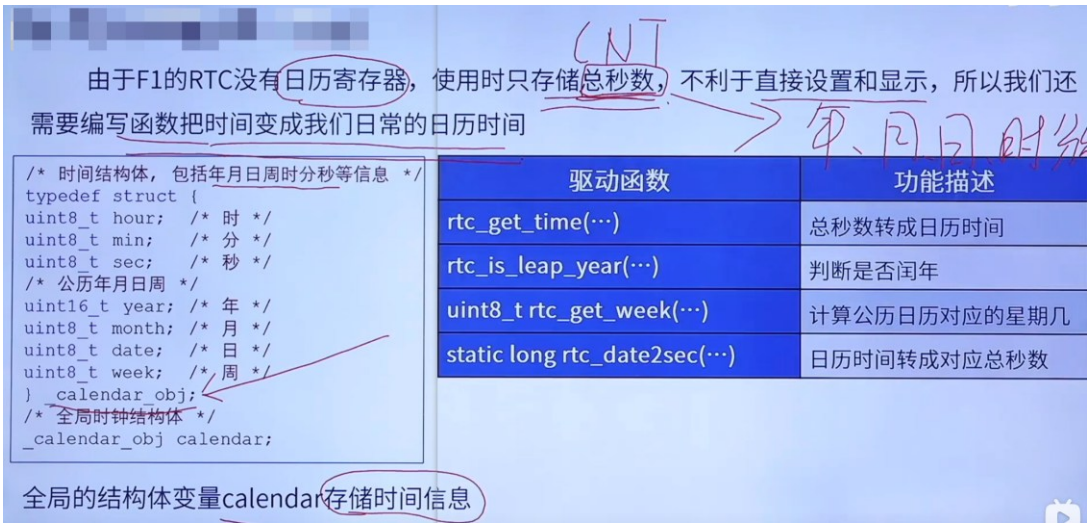
2、开启LSE/选择RTC时钟源/使能RTC时钟
HAL_RCC_OscConfig 开启LSE
HAL_RCCEx_PeriphCLKConfig 选择RTC时钟源
__HAL_RCC_RTC_ENABLE 使能RTC时钟

3、初始化RTC，设置分频值以及工作参数
HAL_RTC_Init 初始化RTC
HAL_RTC_MspInit 完成RTC底层初始化工作

4、设置RTC的日期和时间
操作寄存器方式实现rtc_set_time

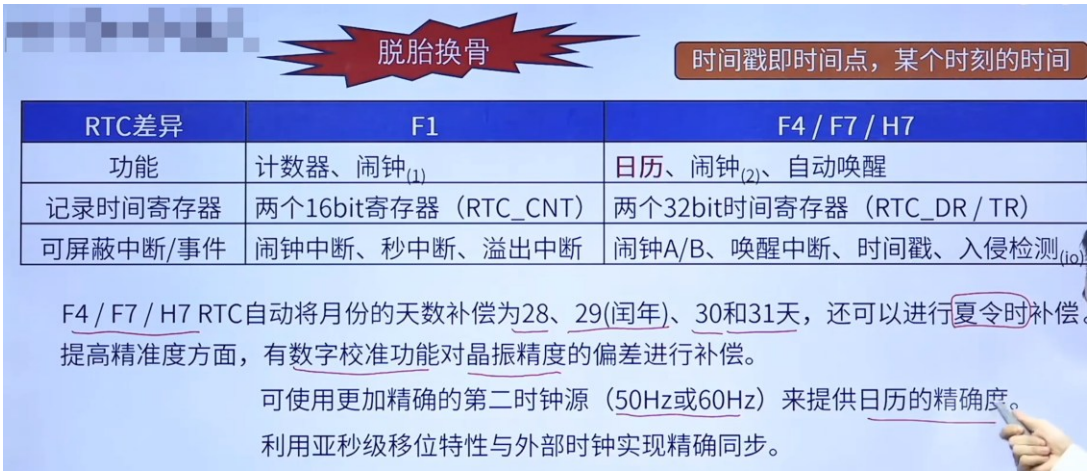
5、获取RTC当前日期和时间
定义rtc_get_time函数

12.3.5 时间设置和读取

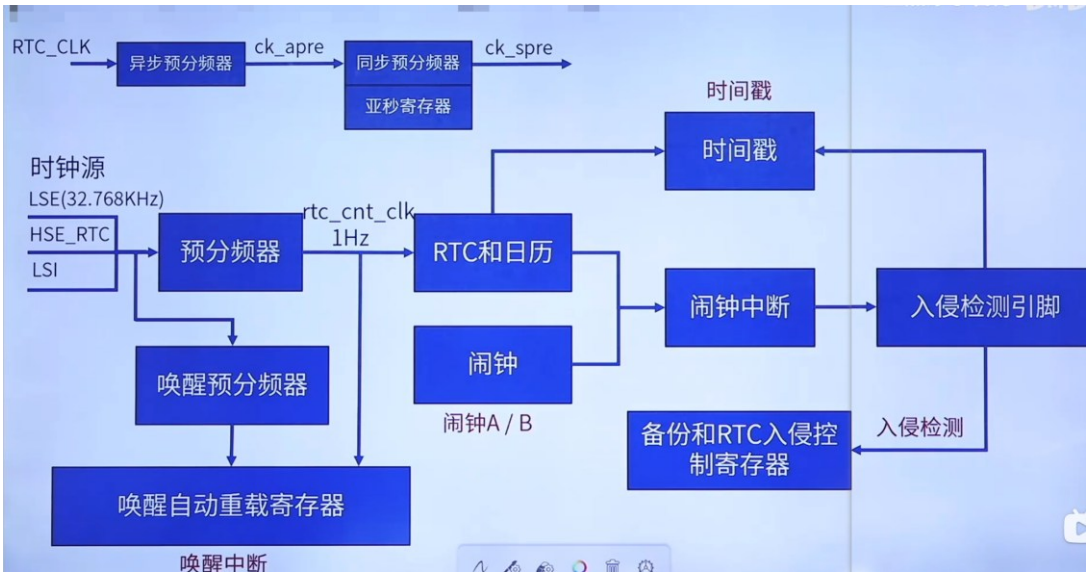


12.4 F4/F7/H7

12.4.1 特点



12.4.2 STM32 RTV 框图介绍



12.4.3 RTC 相关寄存器介绍

BCD码，二进码十进制数格式

寄存器	名称	作用
$RTC_TR_{(WP)}$	RTC时间寄存器	可以设置时间（PM / H / M / S）BCD格式
$RTC_DR_{(WP)}$	RTC日期寄存器	可以设置日期（Y / W / M /D）BCD格式
$RTC_CR_{(WP)}$	RTC控制寄存器	用于配置中断使能位
$RTC_ISR_{(WP)}$	RTC初始化和状态寄存器	用于查看当前RTC状态，中断标记
$RTC_PRER_{(WP)}$	RTC预分频寄存器	用于RTC分频
$RTC_WUTR_{(WP)}$	RTC唤醒定时器寄存器	用于设置自动唤醒重装载值，可用于设置唤醒周期
$RTC_ALRAMAR_{(WP)}$	RTC闹钟A寄存器	用于设置闹钟A的各种信息（D、W、H、M、S）
RTC_WPR	RTC写保护寄存器	对受写保护的寄存器进行解锁，依次写入0xCA、0x53
RTC_BKPxR	RTC备份寄存器	用于存储用户数据
$RCC_BDCR_{(WP)}$	备份区域控制寄存器	用于RTC时钟源选择及使能设置

RTC_TR寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									PM	HT[1:0]		HU[3:0]			
									rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	MNT[2:0]			MNU[3:0]				Reserved	ST[2:0]			SU[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

PM 0：AM或24小时制　1：PM

RTC_DR寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								YT[3:0]				YU[3:0]			
								rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDU[2:0]			MT	MU[3:0]				Reserved	DT[1:0]		DU[3:0]				
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

RTC_CR寄存器

用到闹钟和唤醒功能

夏令时相关

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	ITSE 内部事件时间戳使能	COE	OSEL[1:0]		POL	COSEL	BKP	SUB1H	ADD1H
							rw	rw	rw		rw	rw	rw	w	w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TSIE	WUTIE	ALRBIE	ALRAIE	TSE	WUTE	ALRBE	ALRAE	粗略数字校准使能	FMT	BYPSHAD	REFCKON	TSEDGE	WUCKSEL[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

WUTIE 唤醒定时器中断使能位（1）

ALRAIE 闹钟A中断使能位（1）

WUTE 唤醒定时器使能位（1）

ALRAE 闹钟A使能位（1）

FMT 小时格式选择位（0：24小时/天格式　1：AM/PM小时格式）　跟RTC_TR的PM位相关

唤醒时钟选择（10x：选择ck_spre时钟(通常为1Hz)）

RTC_ISR寄存器

闹钟和唤醒功能相关状态标志、初始化配置标志

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	内部时间戳标志 ITSF	RECALPF
														rc_w0	r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
入侵检测事件 TAMP3F	TAMP2F	TAMP1F	TSOVF	TSF	WUTF	ALRBF	ALRAF	INIT	INITF	RSF	INITS	SHPF	WUTWF	ALRBWF	ALRAWF
rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rw	r	rc_w0	r	r	r	r	r

WUTF 唤醒定时器标志

ALRAF 闹钟A标志

INIT 初始化模式（置1进入初始化模式）

INITF 初始化标志（查看是否可以对RTC寄存器进行操作：1允许 0不允许）

RSF 寄存器同步标志（日历寄存器和影子寄存器同步：1同步 0未同步）

WUTWF 唤醒定时器写标志（判断是否能对唤醒定时器操作：1允许 0不允许）

ALRAWF 闹钟A写标志（判断是否能对闹钟A操作：1允许 0不允许）

RTC_PRER寄存器

注意：在初始化模式（INITF=1）下，才可以配置

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PREDIV_A[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	PREDIV_S[14:0]														
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

PREDIV_A 异步预分频系数（7位）
fck_apre = fRTCCLK / (PREDIV_A + 1)

PREDIV_S 同步预分频系数（15位）
fck_spre = fck_apre / (PREDIV_S + 1)

为了获得1Hz频率同步预分频时钟

PREDIV_A = 127

PREDIV_S = 255

fck_spre = frtcclk / {(PREDIV_A + 1) * (PREDIV_S + 1)}

为了最大程度降低功耗，推荐将异步预分频器配置为较高的值

RTC_WUTR寄存器

注意：等待RTC_ISR的WUTWF为1，才可以配置

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WUT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

WUT 唤醒自动重载值

当使能唤醒定时器时（WUTE置1），每（WUT[15:0]+1）个ck_wut周期将WUTF标志置1一次。

唤醒中断时间 = (WUT[15:0] + 1) / ck_wut

唤醒中断时间设置为1秒

ck_wut = 1Hz　　WUT[15:0] = 0

RTC_ALRMAR寄存器

注意：在RTC_ISR的ALRAWF为1，才可以配置

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MSK4	WDSEL	DT[1:0]		DU[3:0]				MSK3	PM	HT[1:0]		HU[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MSK2	MNT[2:0]			MNU[3:0]				MSK1	ST[2:0]			SU[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

MSK4 闹钟A日期掩码（0：如果匹配上，则闹钟A置1　1：在闹钟A比较中，XX无关）

MSK3 闹钟A小时掩码

MSK2 闹钟A分钟掩码

MSK1 闹钟A秒掩码

闹钟格式：[星期/日期]-[时]-[分]-[秒]

假如MASK[星期/日期]，即星期和日期字段不关心，则闹钟每天都会响。

RTC_WPR寄存器

注意：写一个错误的关键词将再次激活RTC的寄存器写保护

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	KEY							
								w	w	w	w	w	w	w	w

KEY 写保护关键字，写入秘钥使能对RTC寄存器的写保护

备份域复位后，所有RTC寄存器均受写保护

1. 将“0xCA”写入RTC_WPR寄存器
2. 将“0x53”写入RTC_WPR寄存器

RTC_BKPxR寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BKP[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BKP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	w	rw	rw

寄存器组共有32个，每个32位，可以存储128个字节数据，需要电池来维持它的数据。

RCC_BDCR寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BDRST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCEN	Res.	Res.	Res.	Res.	Res.	RTCSEL[1:0]		Res.	LSECSSD	LSECSSON	LSEDRV[1:0]		LSEBYP	LSERDY	LSEON
rw									r	rs	rw		rw	r	rw

12.4.4 RTC 相关 HAL 库驱动介绍

日历功能

驱动函数	关联寄存器	功能描述
HAL_RTC_Init(...)	RTC_CR/PRER/WPR	初始化RTC
HAL_RTC_MspInit(…)	初始化回调	使能RTC时钟
HAL_RCC_OscConfig(…)	RCC_CR	开启LSE时钟源
HAL_RCCEx_PeriphCLKConfig(…)	RCC_BDCR	设置RTC时钟源为LSE
HAL_RTC_Set/GetTime(…)	RTC_TR	设置/读取时间（时/分/秒）
HAL_RTC_Set/GetDate(…)	RTC_DR	设置/读取日期（年/月/日/星期）
HAL_PWR_EnableBkUpAccess(…)	PWR_CR	使能备份域的访问权限
HAL_RTCEx_BKUPWrite/Read()	RTC_BKPxR	读/写备份域数据寄存器

以下是需要开启的时钟源：
__HAL_RCC_RTC_CLK_ENABLE()
__HAL_RCC_RTC_ENABLE()　使能RTC

RTC_InitTypeDef

```
typedef struct
{
    uint32_t HourFormat;        /* 小时格式 RTC_CR(FMT) */
    uint32_t AsynchPrediv;      /* 异步预分频系数 RTC_PRER(PREDIV_A) */
    uint32_t SynchPrediv;       /* 同步预分频系数 RTC_PRER(PREDIV_S) */
    uint32_t OutPut;            /* 连接到RTC_ALARM输出标志 RTC_CR(OSEL) */
    uint32_t OutPutPolarity;    /* 设置RTC_ALARM的输出极性 RTC_CR(POL) */
    uint32_t OutPutType;        /* 设置RTC_ALARM的输出类型为开漏输出还是推挽输出 */
} RTC_InitTypeDef;
```

```
HAL_RTC_Init(RTC_HandleTypeDef *hrtc);
HAL_RTC_MspInit(RTC_HandleTypeDef *hrtc);
```

RTC_TimeTypeDef

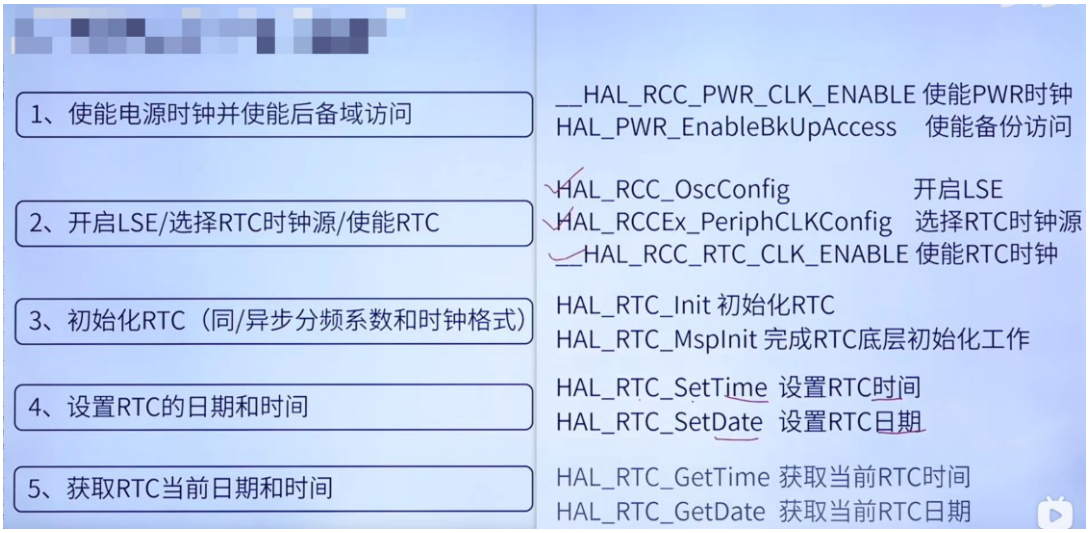
```
typedef struct
{
    uint8_t Hours;              /* 小时设置 */
    uint8_t Minutes;            /* 分钟设置 */
    uint8_t Seconds;            /* 秒设置 */
    uint8_t TimeFormat;         /* AM/PM设置 */
    uint32_t SubSeconds;        /* 亚秒设置 */
    uint32_t SecondFraction;    /* 亚秒预分频系数 */
    uint32_t DayLightSaving;    /* 夏令时日历时间设置*/
    uint32_t StoreOperation     /* 记录是否对夏令时修改*/
} RTC_TimeTypeDef;
```

RTC_DateTypeDef

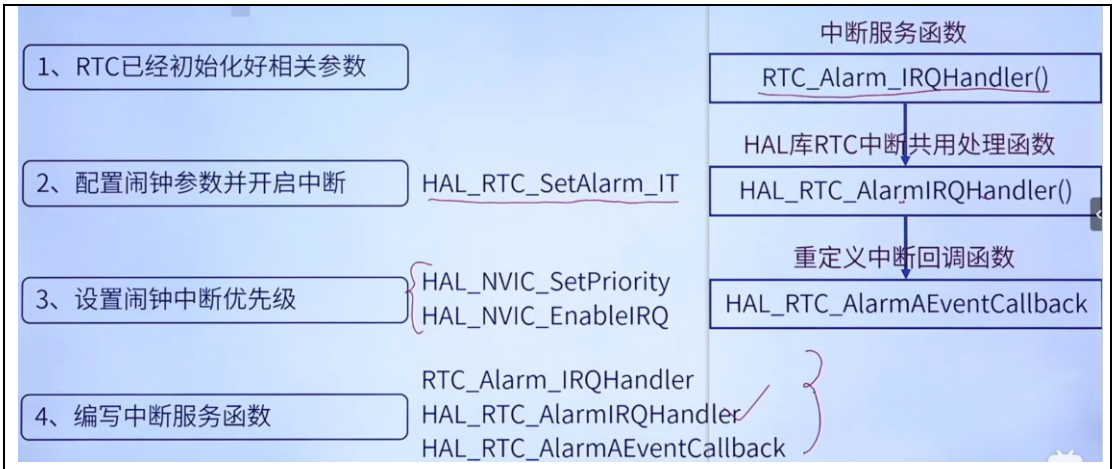
```
typedef struct
{
    uint8_t WeekDay;    /* 星期设置 */
    uint8_t Month;      /* 月份设置 */
    uint8_t Date;       /* 日期设置 */
    uint8_t Year;       /* 年份设置 */
} RTC_DateTypeDef;
```

```
HAL_RTC_Set/GetTime(RTC_HandleTypeDef *hrtc, RTC_TimeTypeDef *sTime, uint32_t Format);
```

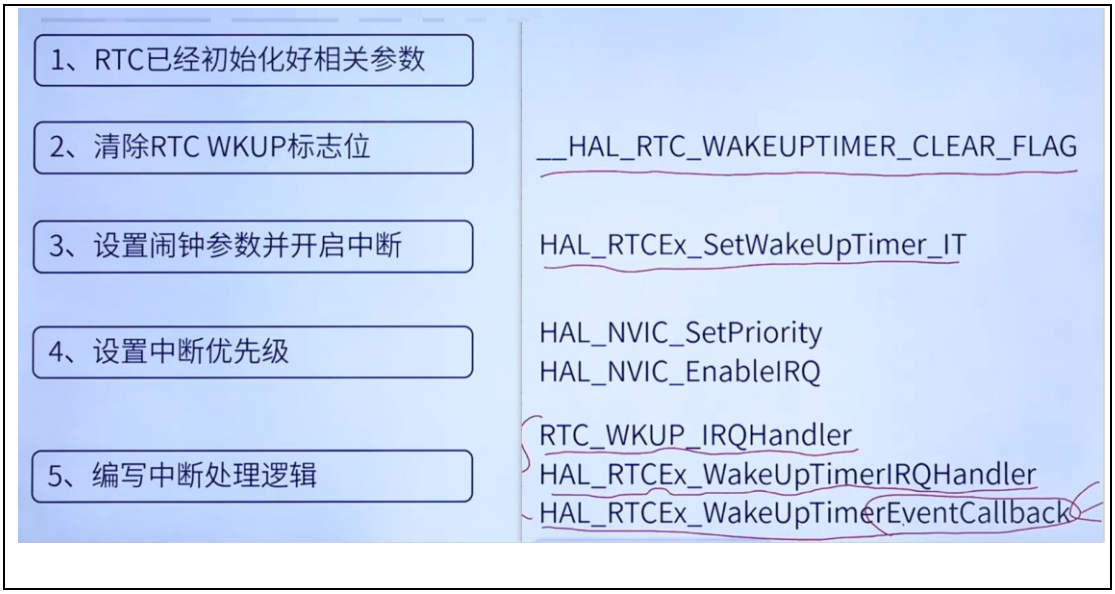
12.4.5 RTC 基本驱动步骤



12.4.6 RTC 闹钟配置一般步骤

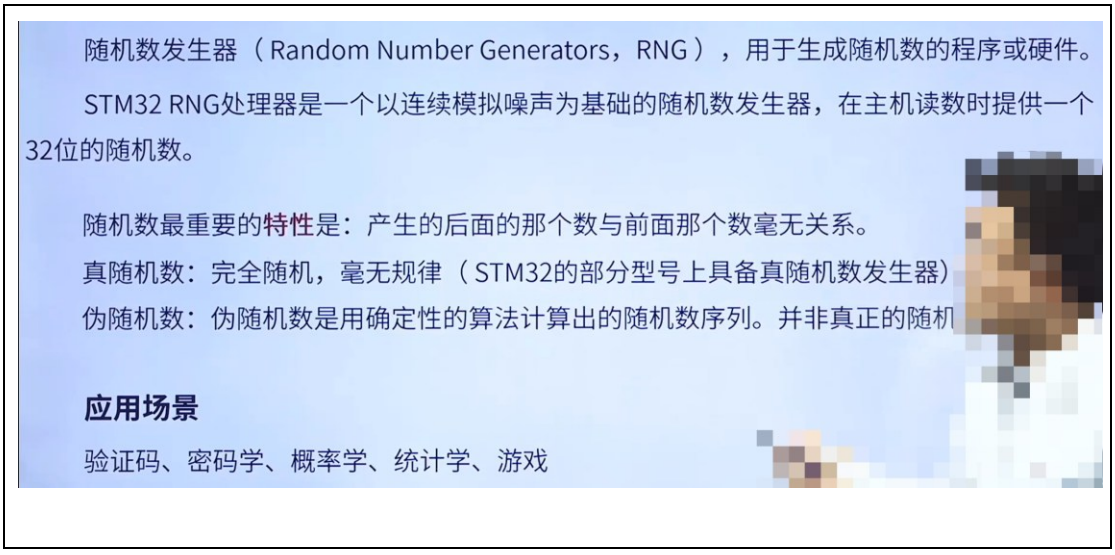


12.4.7 RTC 周期性自动唤醒配置一般步骤

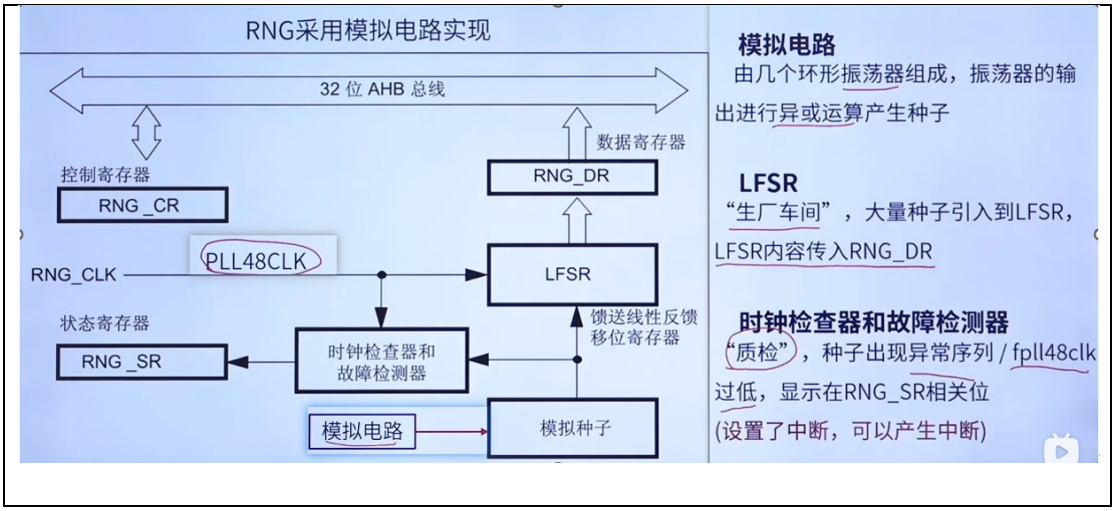


13 硬件随机数实验（RNG）

13.1 RNG 简介



13.2 RNG 框图介绍



13.3 RNG 相关寄存器介绍

寄存器	名称	作用
RNG_CR	RNG控制寄存器	用于使能随机数发生器和中断
RNG_SR	RNG状态寄存器	用于显示RNG当前的一些状态
RNG_DR	RNG数据寄存器	用于存储32位随机数值

RNG控制寄存器RNG_CR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												IE	RNGEN	Reserved	
												rw	rw		

位 3 **IE**：中断使能 (Interrupt enable)
0：禁止 RNG 中断。
1：使能 RNG 中断。只要 RNG_SR 寄存器中 DRDY=1 或 SEIS=1 或 CEIS=1，就会挂起中断。

位 2 **RNGEN**：随机数发生器使能 (Random number generator enable)
0：禁止随机数发生器。
1：使能随机数发生器。

使能随机数发生器，一般不用到中断

RNG状态寄存器RNG_SR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved									SEIS	CEIS	Reserved		SECS	CECS	DRDY
									rc_w0	rc_w0			r	r	r

位 0 **DRDY**：数据就绪 (Data ready)
0：RNG_DR 寄存器尚未有效，无可用随机数据
1：RNG_DR 寄存器包含有效随机数据
注意：如果 RNG_CR 寄存器中 IE = 1，则会挂起中断。
读取 RNG_DR 寄存器后，此位恢复到 0，直到计算出新的有效值。

主要关注的是DRDY位，表示RNG_DR寄存器包含的随机数数据是否有效

RNG数据寄存器RNG_DR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RNDATA															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RNDATA															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

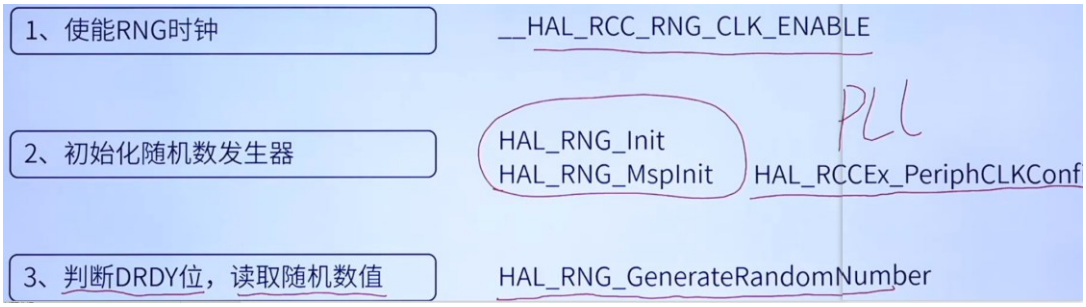
位 31:0 **RNDATA**：随机数据 (Random data)
32 位随机数据。

等待RNG_SR的DRDY位置位后，读取RNG_DR获取32位随机数值。

13.4 RNG 相关 HAL 库驱动介绍

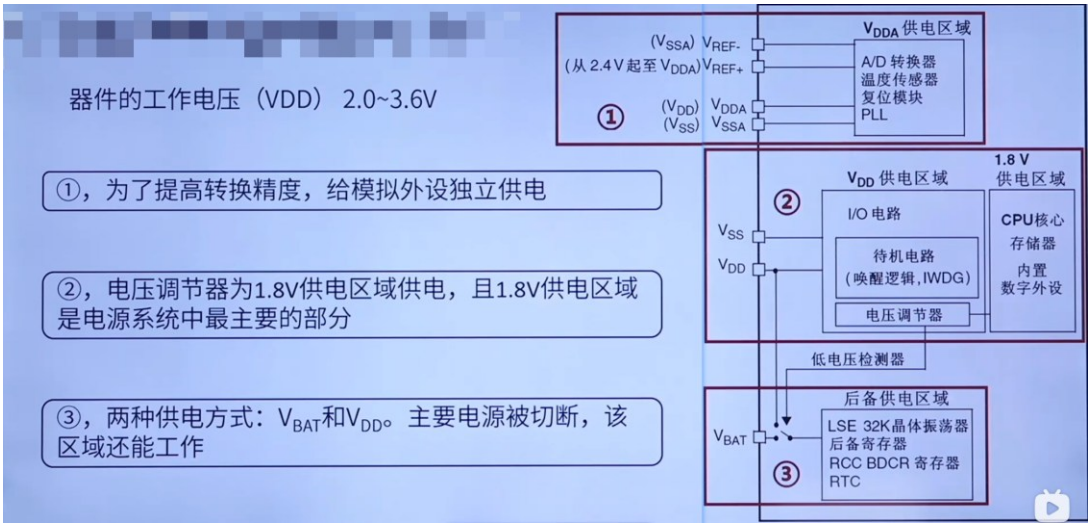
驱动函数	关联寄存器	功能描述
HAL_RNG_Init(…)	RNG_CR	初始化RNG
HAL_RNG_MspInit(…)	初始化回调	使能时钟、外设、选择时钟源
HAL_RCCEx_PeriphCLKConfig(...)	RCC_BDCR	设置RNG时钟源为PLL
HAL_RNG_GenerateRandomNumber(…)	RNG_DR	判断DRDY位并读取随机数
__HAL_RCC_RNG_CLK_ENABLE(…)	AHB2ENR	使能RNG时钟
__HAL_RNG_GET_FLAG(…)	RNG_SR	获取RNG相关标记

13.5 RNG 基本驱动步骤

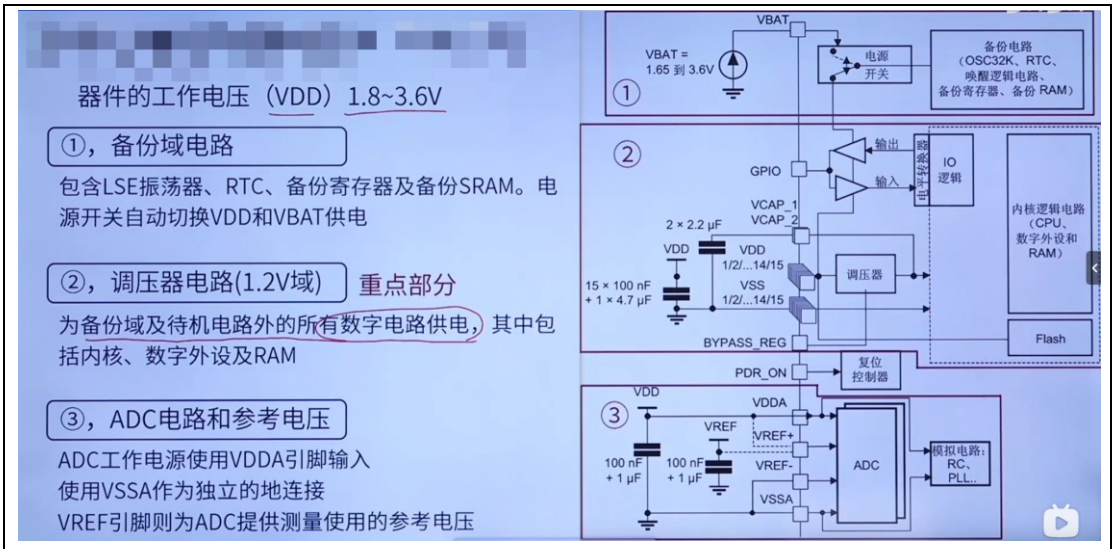


14 低功耗实验

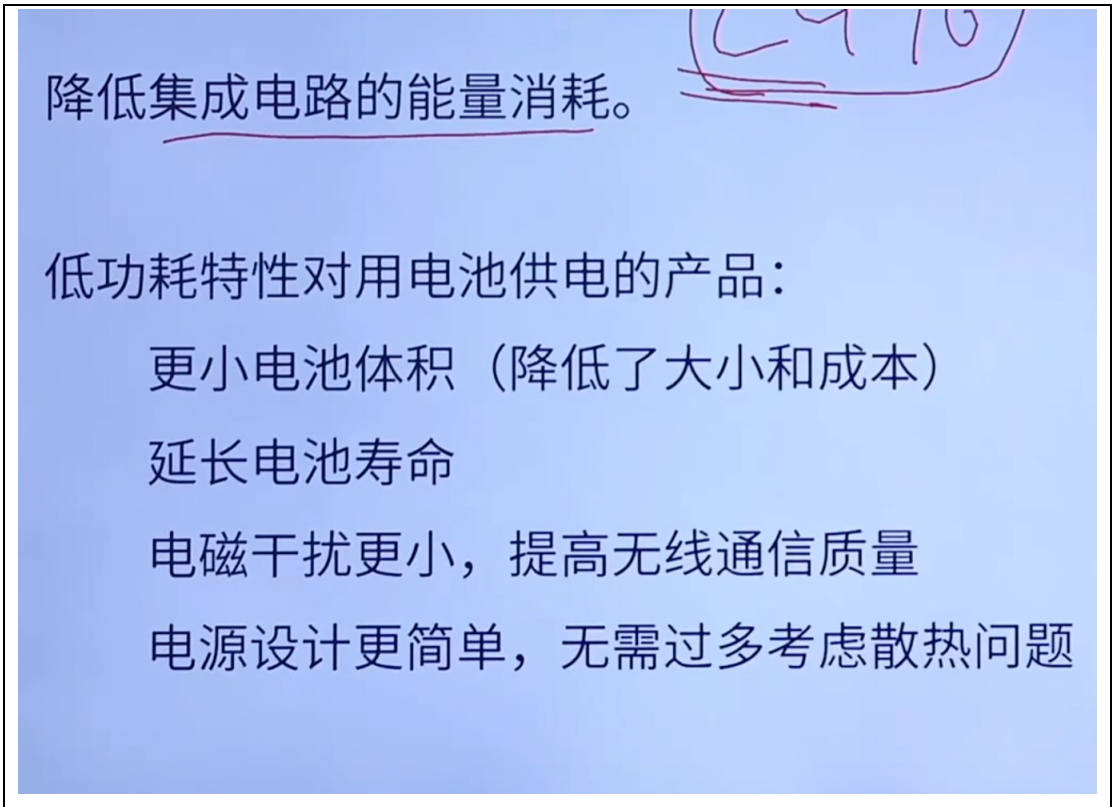
14.1 STM32 电源系统结构介绍（F1）



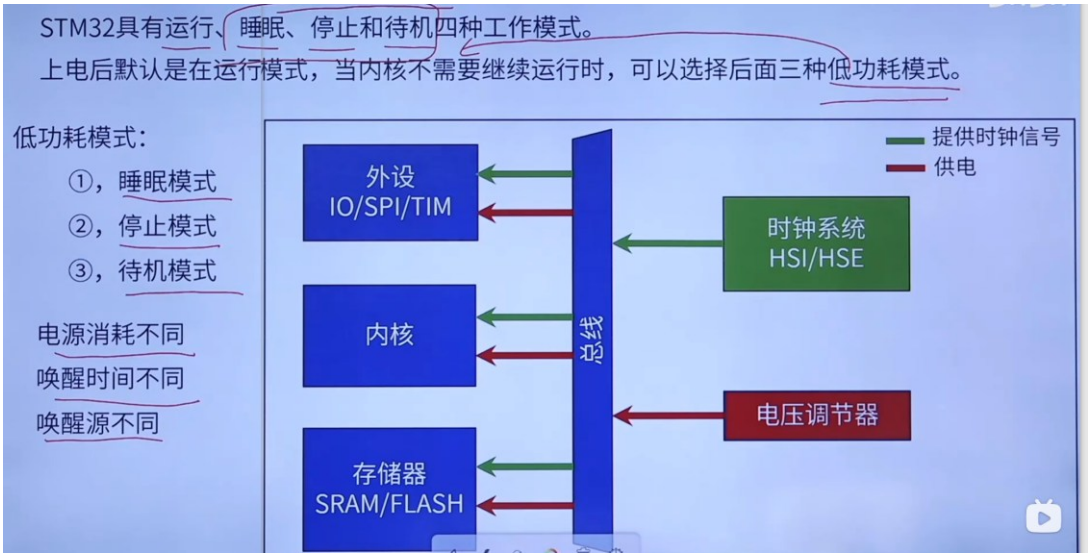
14.2 STM32 电源系统结构介绍（F4/F7）

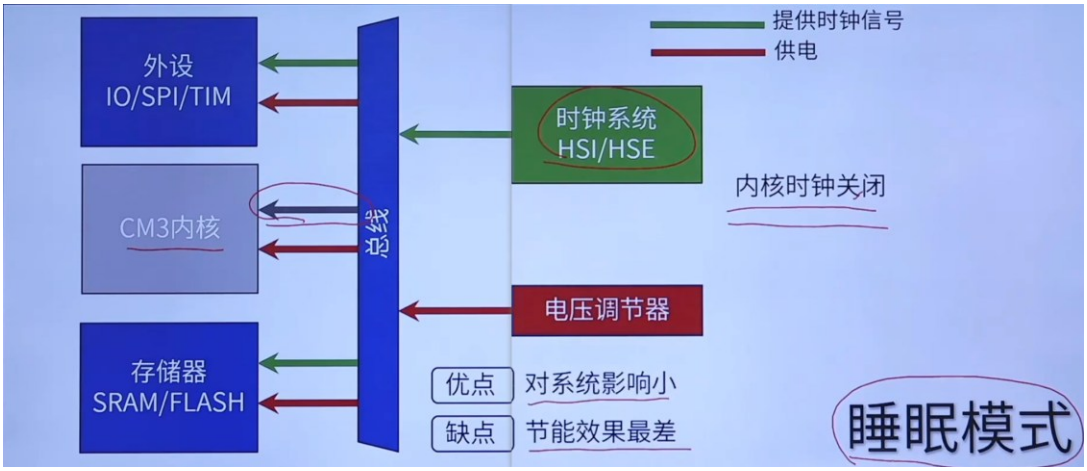


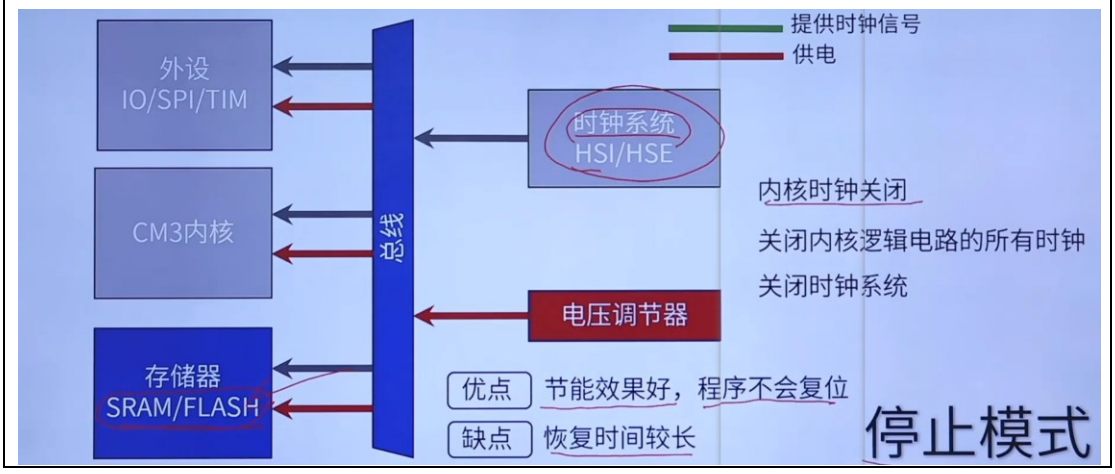
14.3 低功耗是什么？

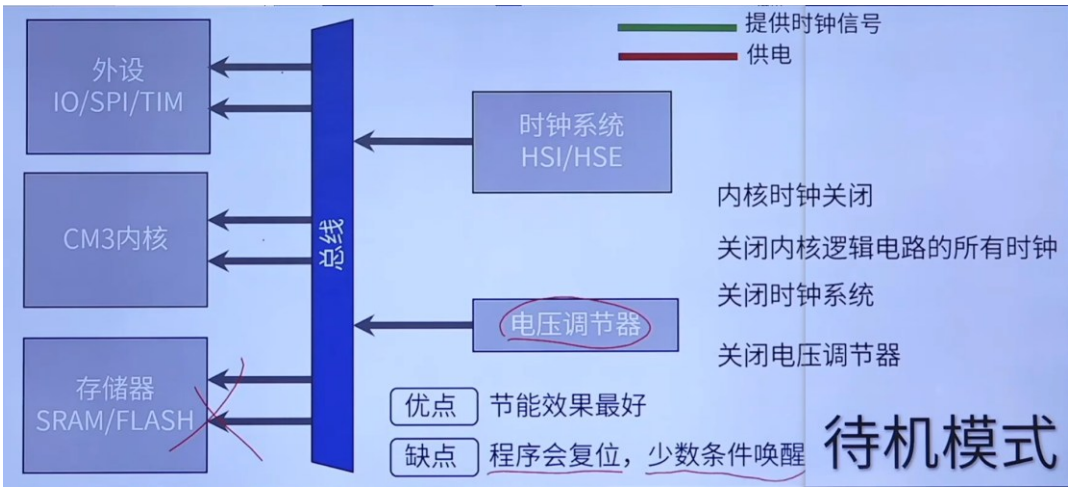


14.4 低功耗模式介绍（F1/F4/F7）









14.5 低功耗模式表（F1/F4/F7）

模式	进入	唤醒	对内核电路时钟影响	对VDD区域时钟的影响	电压调节器
睡眠(立即休眠或退出时休眠)	WFI	任意中断	CPU时钟关，对其他时钟或模拟时钟源无影响	无	开
	WFE	唤醒事件			
停止	PDDS和LPDS位+SLEEPDEEP位+WFI或WFE	任意外部中断（在外部中断寄存器中设置）	关闭所有内核电路时钟	HSI和HSE的振荡器关闭	开启或处于低功耗模式（依据电源控制寄存器PWR_CR的设定）
待机	PDDS位+SLEEPDEEP位+WFI或WFE	WKUP引脚的上升沿、RTC闹钟(唤醒/入侵/时间戳)事件、NRST引脚上的外部复位、IWDG复位			关

14.6 三种模式的功耗（F1）

同等条件下（T = 25°C，VDD = 3.3V，系统时钟72MHz）

模式	主要影响	唤醒时间	供应电流(典型值)
正常模式	所有外设正常工作	0	51mA
睡眠模式	CPU时钟关闭	1.8us	29.5mA
停止模式	1.8V区域时钟关闭, 电压调节器低功耗	5.4us	35uA
待机模式	1.8V区域时钟关闭, 电压调节器关闭	50us	3.8uA

数据来自于《STM32F103 芯片数据手册》5.3.5和5.3.7小节整理

14.7 低功耗相关寄存器介绍(F1)

寄存器	名称	作用
SCB_SCR	系统控制寄存器	选择休眠和深度休眠模式，用于其他低功耗特性的控制
PWR_CR	电源控制寄存器	可以设置低功耗相关（清除标记位、模式）
PWR_CSR	电源控制/状态寄存器	用于查看系统当前状态（待机/唤醒标志 使能唤醒引脚）

系统控制寄存器(SCB_SCR)

表 13.1　系统控制寄存器（地址：0xE000_ED10）

位段	名称	类型	复位值	描述
4	SEVONPEND	RW	-	发生异常悬起时请发送事件，用于在一个新的中断悬起时从 WFE 指令处唤醒。不管这个中断的优先级是否比当前的高，都唤醒。如果没有 WFE 导致睡眠，则下次使用 WFE 时将立即唤醒
3	保留	-	-	-
2	SLEEPDEEP	R/W	0	当进入睡眠模式时，使能外部的 SLEEPDEEP 信号，以允许停止系统时钟
1	SLEEPONEXIT	R/W	-	激活“SleepOnExit”功能
0	保留	-	-	-

进入停止模式 or 待机模式，SLEEPDEEP置1

来自： Cortex-M3权威指南(中文

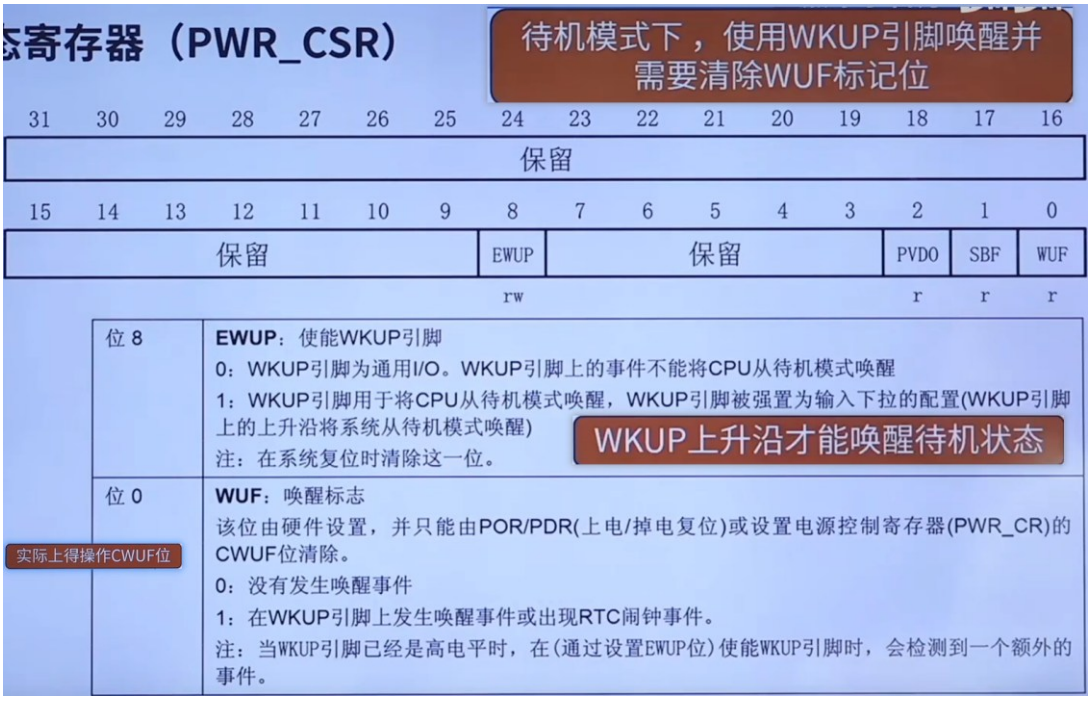
电源控制寄存器
PWR_CR

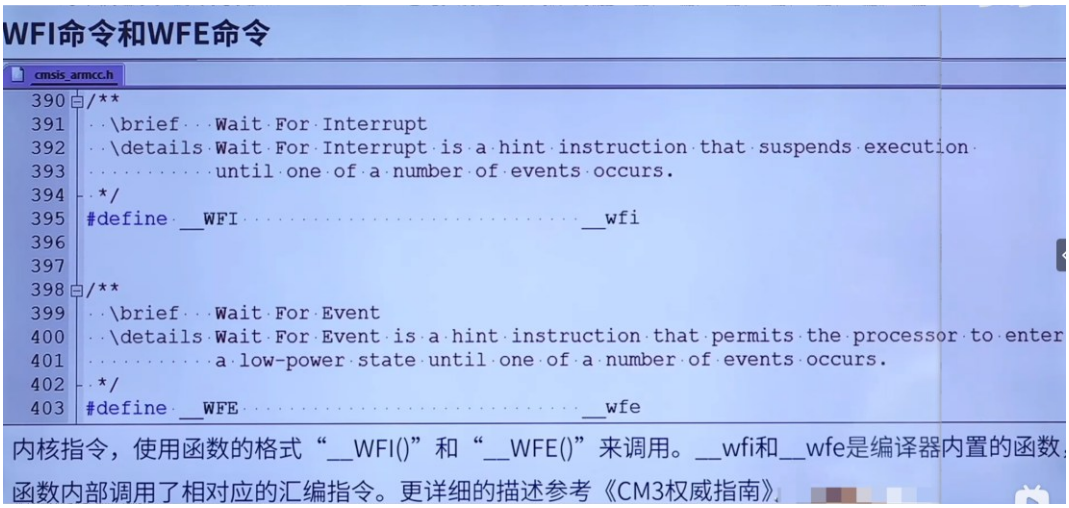
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留							DBP	PLS[2:0]			PVDE	CSBF	CWUF	PDDS	LPDS
							rw	rw	rw	rw	rw	rc_w1	rc_w1	rw	rw

停止模式
PDDS清0
LPDS选调节器模式

待机模式
PDDS置1
清除唤醒位 CWUF

位 3	**CSBF**：清除待机位 始终读出为0 0：无功效 1：清除SBF待机位(写)
位 2	**CWUF**：清除唤醒位 始终读出为0 0：无功效 1：2个系统时钟周期后清除WUF唤醒位(写)
位 1	**PDDS**：掉电深睡眠 与LPDS位协同操作 0：当CPU进入深睡眠时进入停机模式，调压器的状态由LPDS位控制。 1：CPU进入深睡眠时进入待机模式。
位 0	**LPDS**：深睡眠下的低功耗 PDDS=0时，与PDDS位协同操作 0：在停机模式下电压调压器开启 1：在停机模式下电压调压器处于低功耗模式


寄存器（PWR_CSR）
待机模式下，使用WKUP引脚唤醒并需要清除WUF标记位
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16
保留
15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
保留
EWUP
保留
PVDO
SBF
WUF
rw
r
r
r
位 8
EWUP：使能WKUP引脚
0：WKUP引脚为通用I/O。WKUP引脚上的事件不能将CPU从待机模式唤醒
1：WKUP引脚用于将CPU从待机模式唤醒，WKUP引脚被强置为输入下拉的配置(WKUP引脚上的上升沿将系统从待机模式唤醒)
注：在系统复位时清除这一位。
WKUP上升沿才能唤醒待机状态
位 0
WUF：唤醒标志
该位由硬件设置，并只能由POR/PDR(上电/掉电复位)或设置电源控制寄存器(PWR_CR)的CWUF位清除。
实际上得操作CWUF位
0：没有发生唤醒事件
1：在WKUP引脚上发生唤醒事件或出现RTC闹钟事件。
注：当WKUP引脚已经是高电平时，在(通过设置EWUP位)使能WKUP引脚时，会检测到一个额外的事件。


WFI命令和WFE命令
cmsis_armcc.h
390 /**
391 \brief Wait For Interrupt
392 \details Wait For Interrupt is a hint instruction that suspends execution
393 until one of a number of events occurs.
394 */
395 #define __WFI __wfi
396
397
398 /**
399 \brief Wait For Event
400 \details Wait For Event is a hint instruction that permits the processor to enter
401 a low-power state until one of a number of events occurs.
402 */
403 #define __WFE __wfe
内核指令，使用函数的格式“__WFI()”和“__WFE()”来调用。__wfi和__wfe是编译器内置的函数，
函数内部调用了相对应的汇编指令。更详细的描述参考《CM3权威指南》

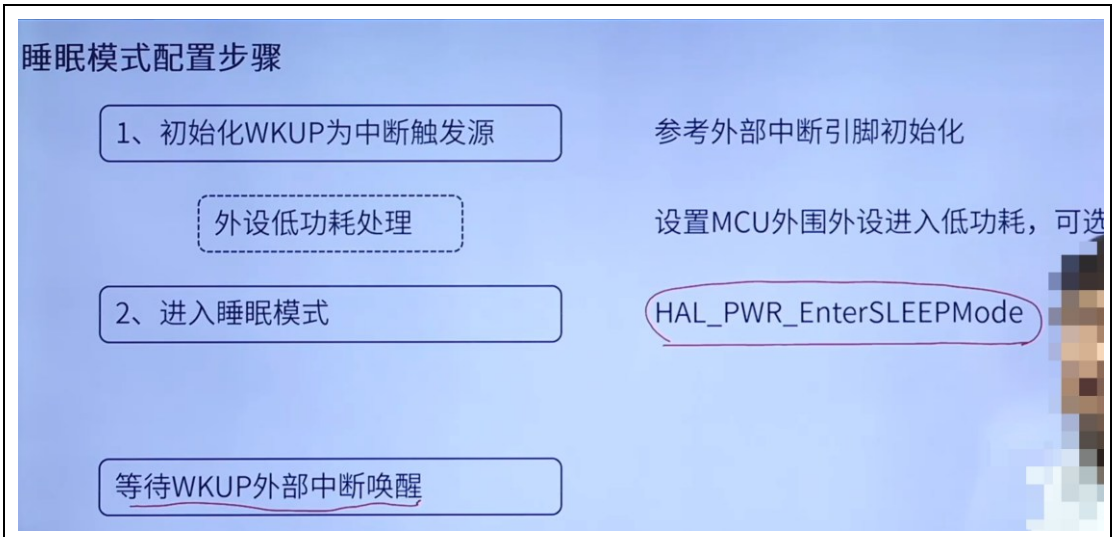
14.8 低功耗相关 HAL 库驱动介绍

驱动函数	关联寄存器	功能描述
HAL_PWR_EnterSLEEPMode(...)	SCB_SCR	进入睡眠模式
HAL_PWR_EnterSTOPMode(…)	PWR_CR/SCB_SCR	进入停止模式
HAL_PWR_EnterSTANDBYMode(…)	PWR_CR/SCB_SCR	进入待机模式
HAL_PWR_EnableWakeUpPin(…)	PWR_CSR	使能WKUP管脚唤醒功能
__HAL_PWR_CLEAR_FLAG(…)	PWR_CR	清除PWR的相关标记
__HAL_RCC_PWR_CLK_ENABLE(…)	RCC_APB1ENR	使能电源时钟

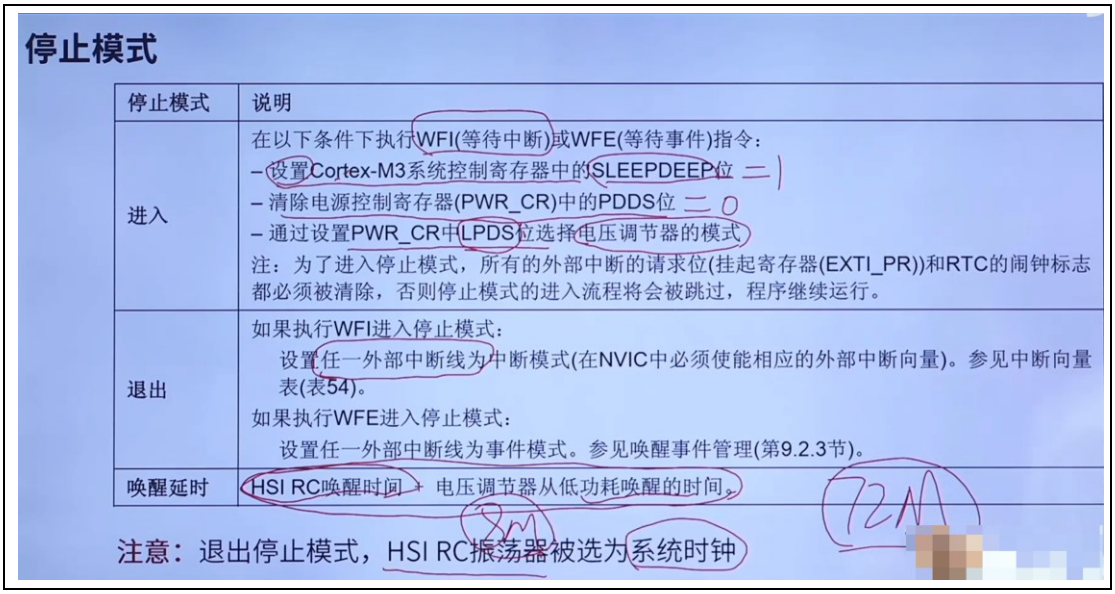
14.9 低功耗模式的使用步骤

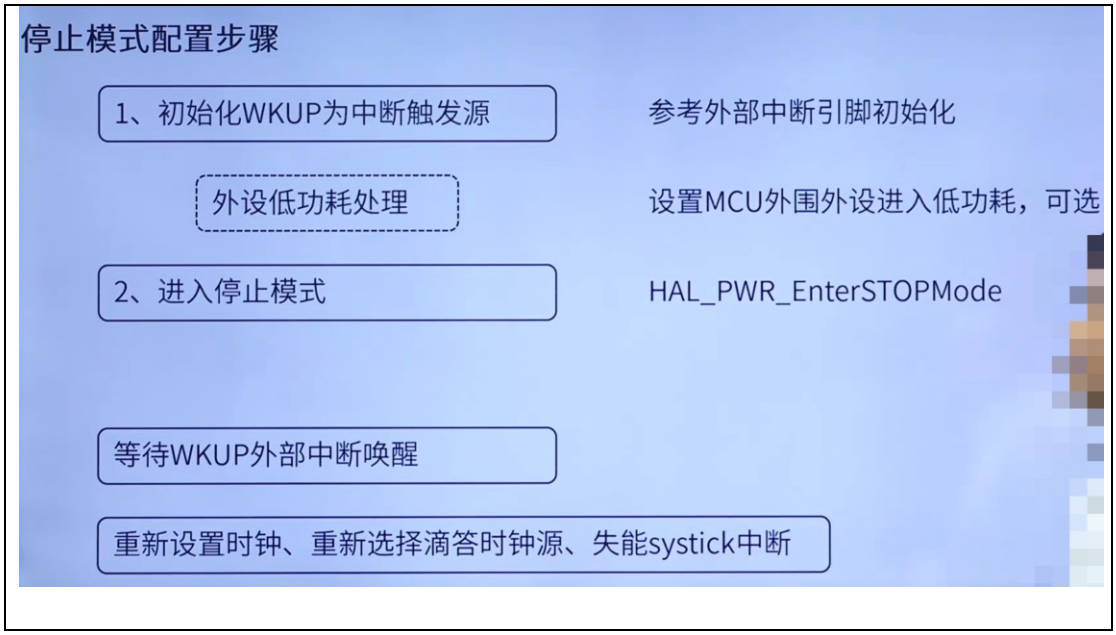
14.9.1 睡眠模式

睡眠模式	SLEEP-NOW模式	说明
	进入	在以下条件下执行WFI(等待中断)或WFE(等待事件)指令： - SLEEPDEEP = 0 和 - SLEEPONEXIT = 0 参考Cortex-M3系统控制寄存器。
	退出	如果执行WFI进入睡眠模式： 中断：参考中断向量表(表54) 如果执行WFE进入睡眠模式： 唤醒事件：参考唤醒事件管理(第9.2.3节)
	唤醒延时	无
	SLEEP-ON_EXIT模式	说明
	进入	在以下条件下执行WFI指令： - SLEEPDEEP = 0和 - SLEEPONEXIT = 1 参考Cortex™-M3系统控制寄存器
	退出	中断：参考中断向量表(表54)
	唤醒延时	

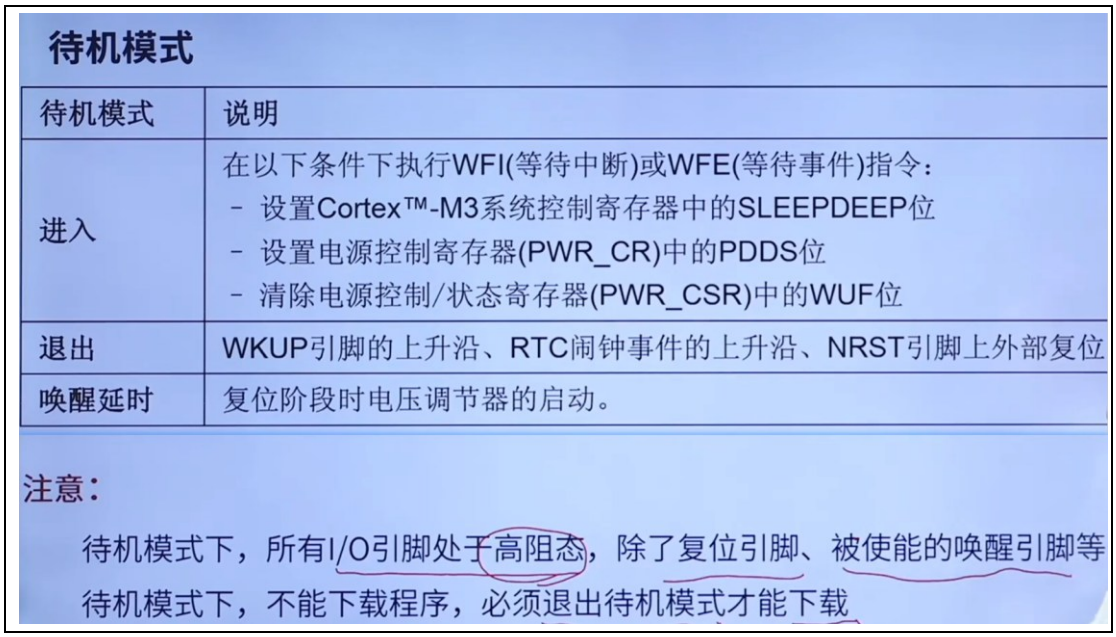

睡眠模式配置步骤
1、初始化WKUP为中断触发源
参考外部中断引脚初始化
外设低功耗处理
设置MCU外围外设进入低功耗，可选
2、进入睡眠模式
HAL_PWR_EnterSLEEPMode
等待WKUP外部中断唤醒

14.9.2 停止模式


停止模式
停止模式
说明
进入
在以下条件下执行WFI(等待中断)或WFE(等待事件)指令：
– 设置Cortex-M3系统控制寄存器中的SLEEPDEEP位
– 清除电源控制寄存器(PWR_CR)中的PDDS位
– 通过设置PWR_CR中LPDS位选择电压调节器的模式
注：为了进入停止模式，所有的外部中断的请求位(挂起寄存器(EXTI_PR))和RTC的闹钟标志都必须被清除，否则停止模式的进入流程将会被跳过，程序继续运行。
退出
如果执行WFI进入停止模式：
设置任一外部中断线为中断模式(在NVIC中必须使能相应的外部中断向量)。参见中断向量表(表54)。
如果执行WFE进入停止模式：
设置任一外部中断线为事件模式。参见唤醒事件管理(第9.2.3节)。
唤醒延时
HSI RC唤醒时间 + 电压调节器从低功耗唤醒的时间。
注意：退出停止模式，HSI RC振荡器被选为系统时钟



14.9.3 待机模式



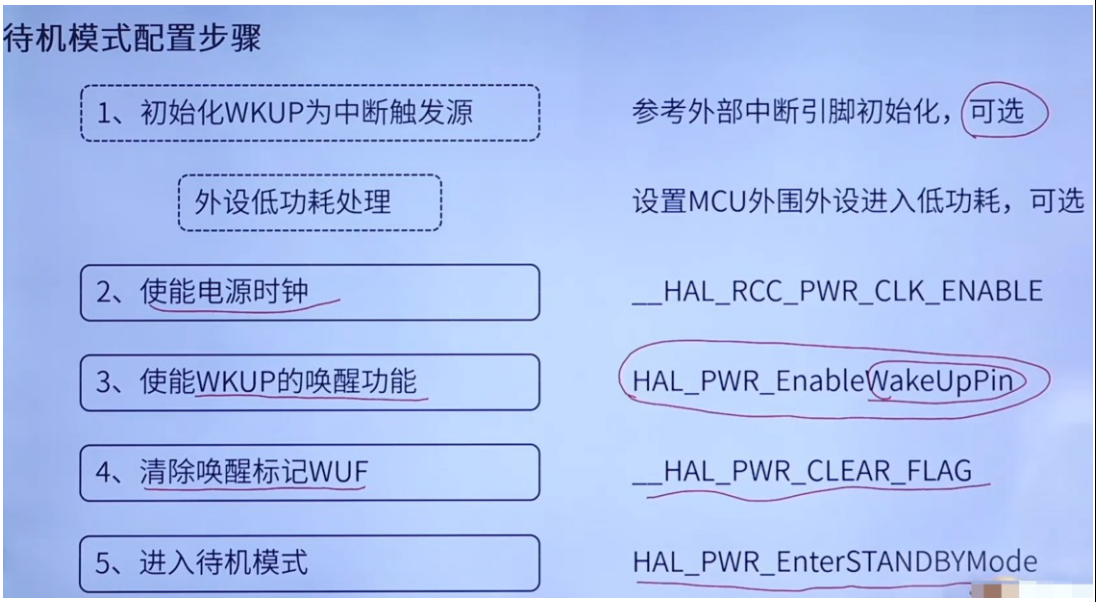
待机模式

待机模式	说明
进入	在以下条件下执行WFI(等待中断)或WFE(等待事件)指令： - 设置Cortex™-M3系统控制寄存器中的SLEEPDEEP位 - 设置电源控制寄存器(PWR_CR)中的PDDS位 - 清除电源控制/状态寄存器(PWR_CSR)中的WUF位
退出	WKUP引脚的上升沿、RTC闹钟事件的上升沿、NRST引脚上外部复位
唤醒延时	复位阶段时电压调节器的启动。

注意：

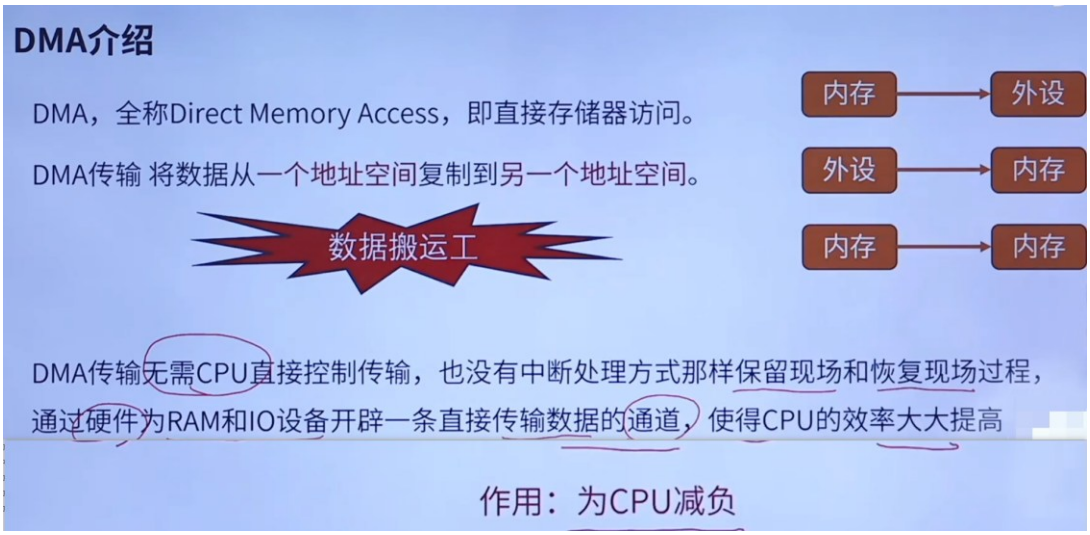
待机模式下，所有I/O引脚处于高阻态，除了复位引脚、被使能的唤醒引脚等

待机模式下，不能下载程序，必须退出待机模式才能下载



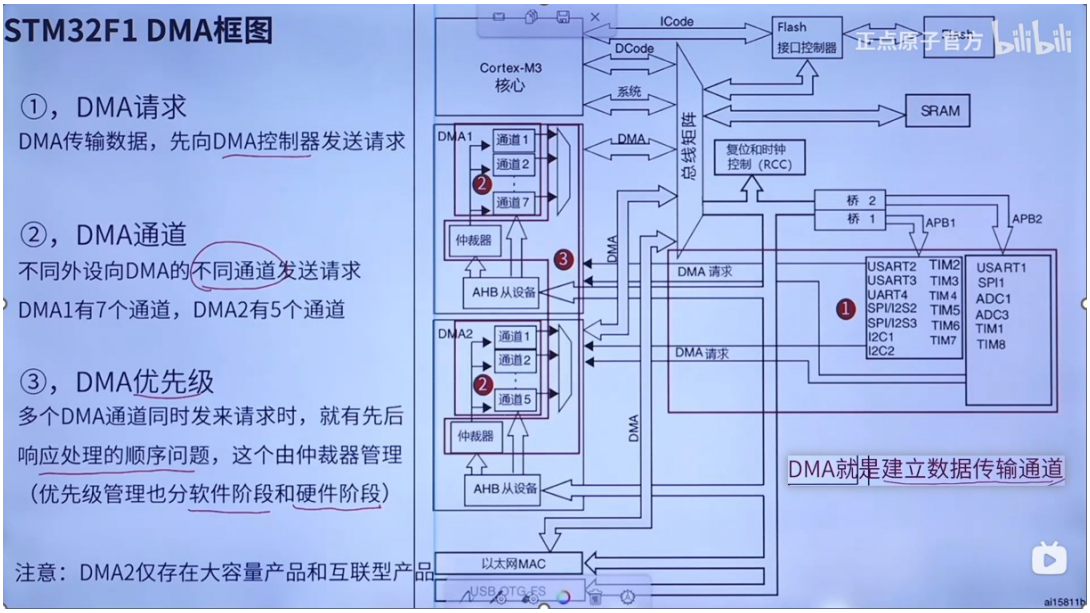
15 DMA 实验

15.1 DMA 介绍

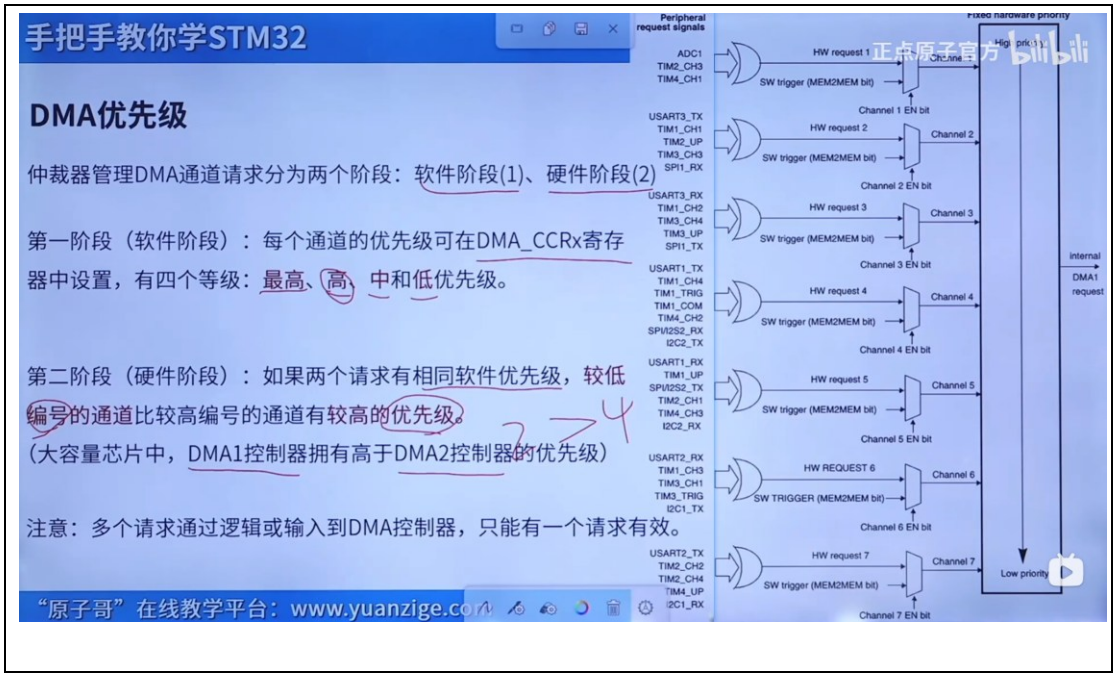


15.2 F1

15.2.1 DMA 结构框图介绍（F1）

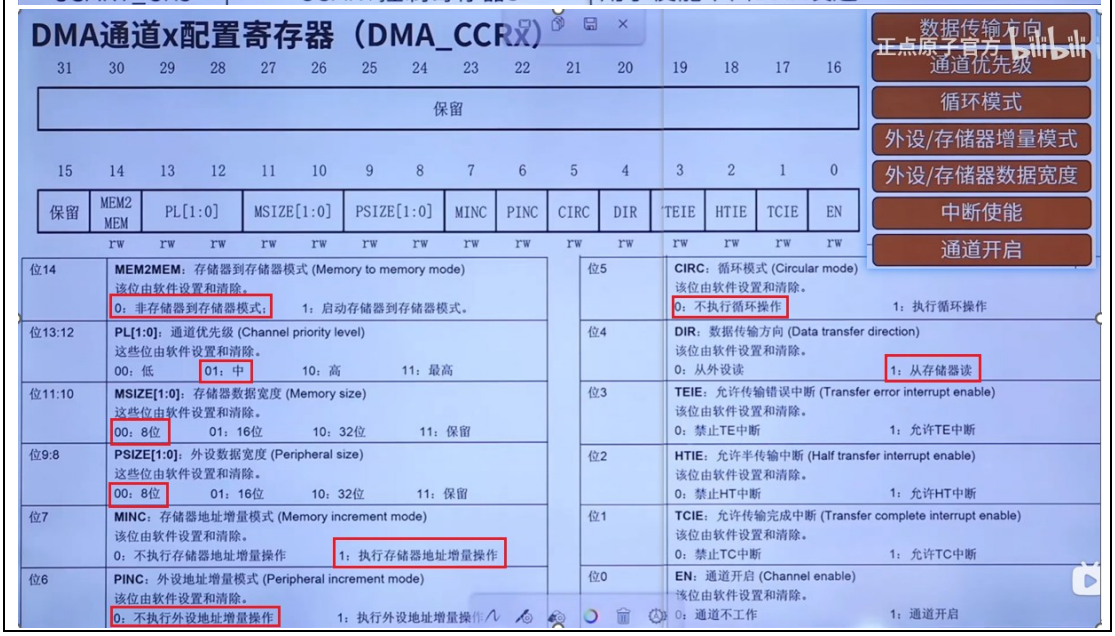


外设	通道1	通道2	通道3	通道4	通道5	通道6	通道7
ADC1	ADC1						
SPI/I²S		SPI1_RX	SPI1_TX	SPI/I2S2_RX	SPI/I2S2_TX		
USART		USART3_TX	USART3_RX	USART1_TX	USART1_RX	USART2_RX	USART2_TX
I²C				I2C2_TX	I2C2_RX	I2C1_TX	I2C1_RX
TIM1		TIM1_CH1	TIM1_CH2	TIM1_TX4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
TIM2	TIM2_CH3	TIM2_UP			TIM2_CH1		TIM2_CH2 TIM2_CH4
TIM3		TIM3_CH3	TIM3_CH4 TIM3_UP			TIM3_CH1 TIM3_TRIG	
TIM4	TIM4_CH1			TIM4_CH2	TIM4_CH3		TIM4_UP



15.2.2 DMA 相关寄存器介绍(F1)

寄存器	名称	作用
DMA_CCRx	DMA通道x配置寄存器	用于配置DMA（核心控制寄存器）
DMA_ISR	DMA中断状态寄存器	用于查询当前DMA传输状态
DMA_IFCR	DMA中断标志清除寄存器	用来清除DMA_ISR对应位
DMA_CNDTRx	DMA通道x传输数量寄存器	用于控制DMA通道x每次传输的数据量
DMA_CPARx	DMA通道x外设地址寄存器	用于存储STM32外设地址
DMA_CMARx	DMA通道x存储器地址寄存器	用于存放存储器的地址
USART_CR3	USART控制寄存器3	用于使能串口DMA发送



DMA中断状态寄存器（DMA_ISR）

查询当前DMA传输状态

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留				TEIF7	HTIF7	TCIF7	GIF7	TEIF6	HTIF6	TCIF6	GIF6	TEIF5	HTIF5	TCIF5	GIF5
				r	r	r	r	r	r	r	r	r	r	r	r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TEIF4	HTIF4	TCIF4	GIF4	TEIF3	HTIF3	TCIF3	GIF3	TEIF2	HTIF2	TCIF2	GIF2	TEIF1	HTIF1	TCIF1	GIF1
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位27，23，19，15，11，7，3	**TEIFx**：通道x的传输错误标志(x = 1 … 7) (Channel x transfer error flag) 硬件设置这些位。在DMA_IFCR寄存器的相应位写入'1'可以清除这里对应的标志位。 0：在通道x没有传输错误(TE)；　　1：在通道x发生了传输错误(TE)。
位26，22，18，14，10，6，2	**HTIFx**：通道x的半传输标志(x = 1 … 7) (Channel x half transfer flag) 硬件设置这些位。在DMA_IFCR寄存器的相应位写入'1'可以清除这里对应的标志位。 0：在通道x没有半传输事件(HT)；　　1：在通道x产生了半传输事件(HT)。
位25，21，17，13，9，5，1	**TCIFx**：通道x的传输完成标志(x = 1 … 7) (Channel x transfer complete flag) 硬件设置这些位。在DMA_IFCR寄存器的相应位写入'1'可以清除这里对应的标志位。 0：在通道x没有传输完成事件(TC)；　　1：在通道x产生了传输完成事件(TC)。
位24，20，16，12，8，4，0	**GIFx**：通道x的全局中断标志(x = 1 … 7) (Channel x global interrupt flag) 硬件设置这些位。在DMA_IFCR寄存器的相应位写入'1'可以清除这里对应的标志位。 0：在通道x没有TE、HT或TC事件；　　1：在通道x产生了TE、HT或TC事件。

传输出现错误

传输一半的数据

数据传输结束

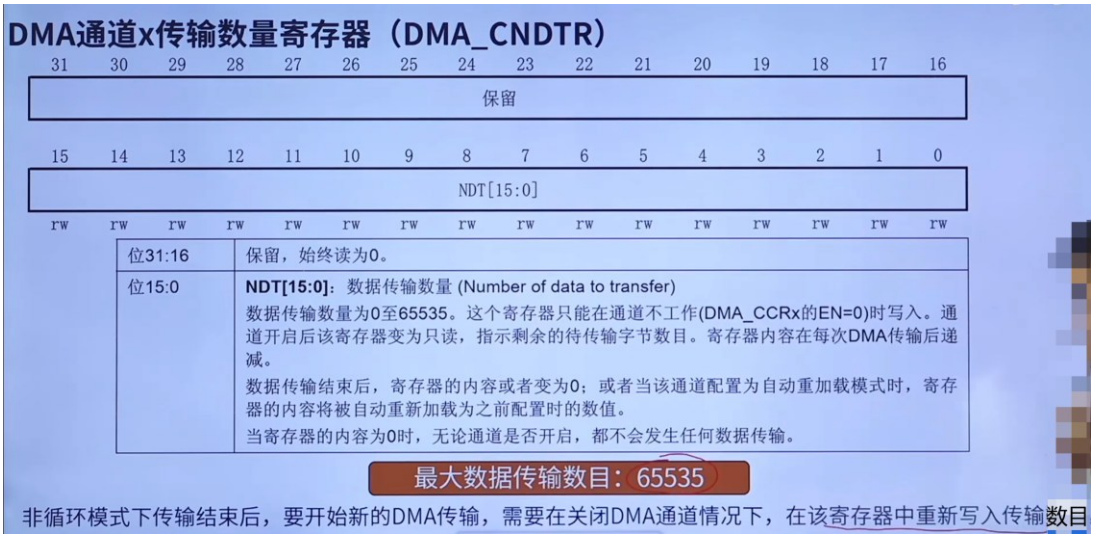
注意：当设置了允许中断时，将会产生中断

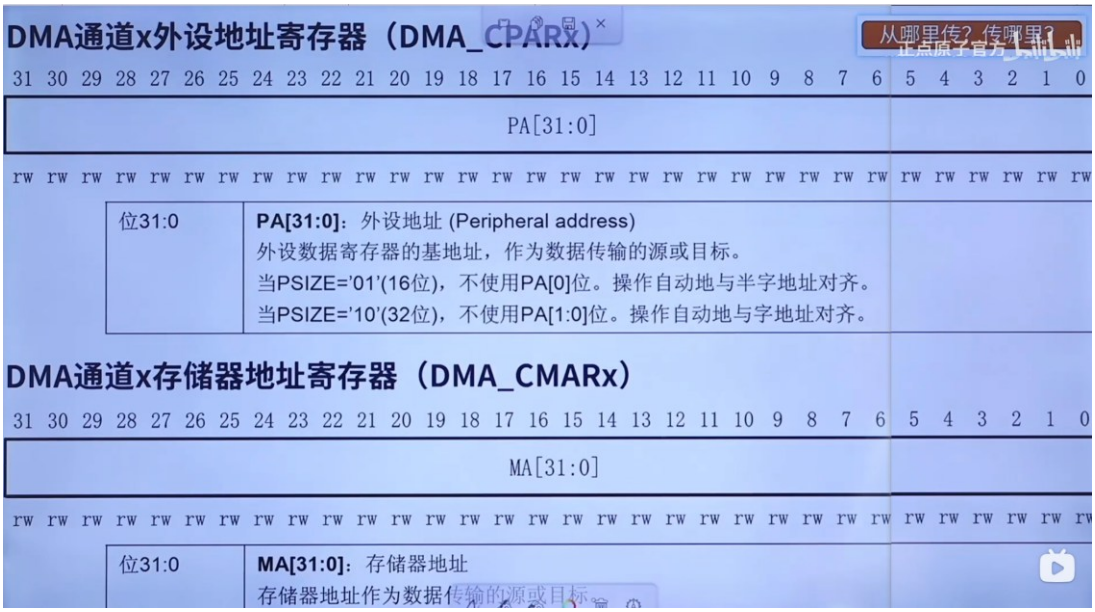
DMA中断标志清除寄存器（DMA_IFCR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
保留				CTEIF 7	CHTIF 7	CTCIF 7	CGIF 7	CTEIF 6	CHTIF 6	CTCIF 6	CGIF 6	CTEIF 5	CHTIF 5	CTCIF 5	CGIF 5
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CTEIF 4	CHTIF 4	CTCIF 4	CGIF 4	CTEIF 3	CHTIF 3	CTCIF 3	CGIF 3	CTEIF 2	CHTIF 2	CTCIF 2	CGIF 2	CTEIF 1	CHTIF 1	CTCIF 1	CGIF 1
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位27，23，19，15，11，7，3	**CTEIFx**：清除通道x的传输错误标志(x = 1 … 7) (Channel x transfer error clear) 这些位由软件设置和清除。 0：不起作用　　1：清除DMA_ISR寄存器中的对应TEIF标志。
位26，22，18，14，10，6，2	**CHTIFx**：清除通道x的半传输标志(x = 1 … 7) (Channel x half transfer clear) 这些位由软件设置和清除。 0：不起作用　　1：清除DMA_ISR寄存器中的对应HTIF标志。
位25，21，17，13，9，5，1	**CTCIFx**：清除通道x的传输完成标志(x = 1 … 7) (Channel x transfer complete clear) 这些位由软件设置和清除。 0：不起作用　　1：清除DMA_ISR寄存器中的对应TCIF标志。
位24，20，16，12，8，4，0	**CGIFx**：清除通道x的全局中断标志(x = 1 … 7) (Channel x global interrupt clear) 这些位由软件设置和清除。 0：不起作用　　1：清除DMA_ISR寄存器中的对应的GIF、TEIF、HTIF和TCIF标志。

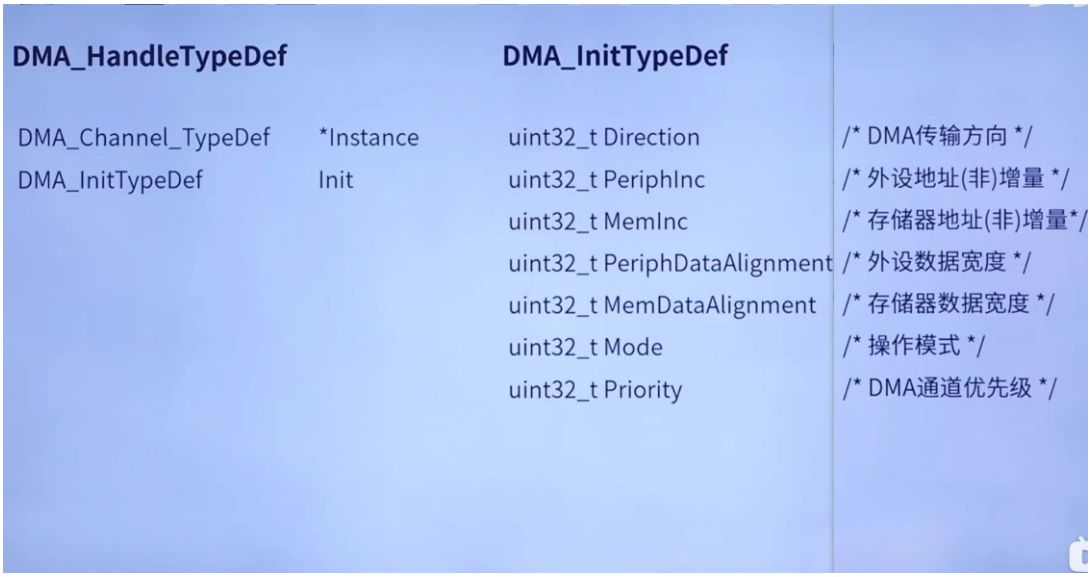




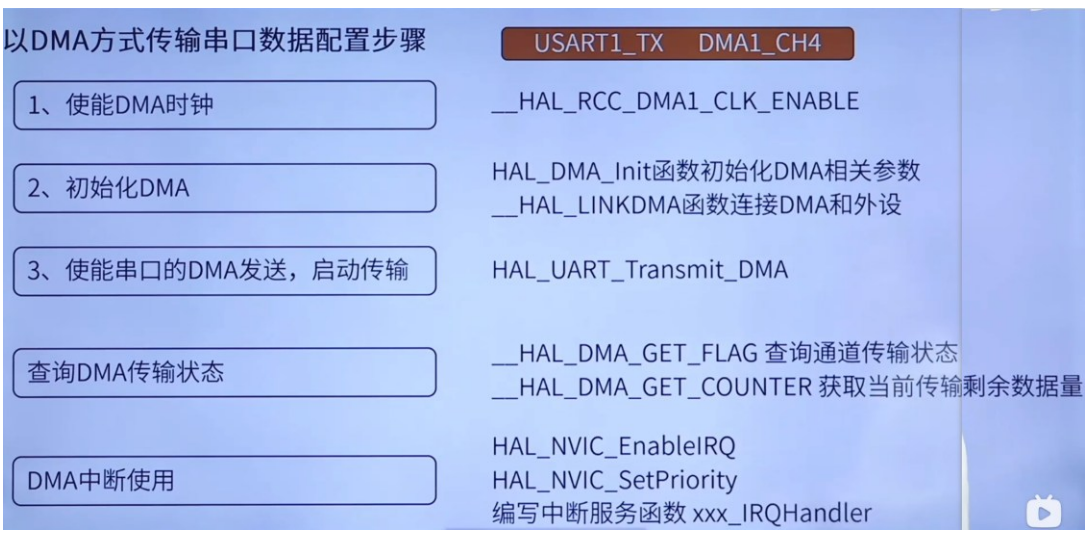
15.2.3 DMA 相关 HAL 库驱动介绍

驱动函数	关联寄存器	功能描述
__HAL_RCC_DMAx_CLK_ENABLE(…)	RCC_AHBENR	使能DMAx时钟
HAL_DMA_Init(…)	DMA_CCR	初始化DMA
HAL_DMA_Start_IT(…)	DMA_CCR/CPAR/CMAR/CNDTR	开始DMA传输
__HAL_LINKDMA(…)		用来连接DMA和外设句柄
HAL_UART_Transmit_DMA(…)	CCR/CPAR/CMAR/CNDTR/USART_CR3	使能DMA发送，启动传输
__HAL_DMA_GET_FLAG(…)	DMA_ISR	查询DMA传输通道的状态
__HAL_DMA_ENABLE(…)	DMA_CCR(EN)	使能DMA外设
__HAL_DMA_DISABLE(…)	DMA_CCR(EN)	失能DMA外设

DMA外设相关结构体：DMA_HandleTypeDef 和 DMA_InitTypeDef

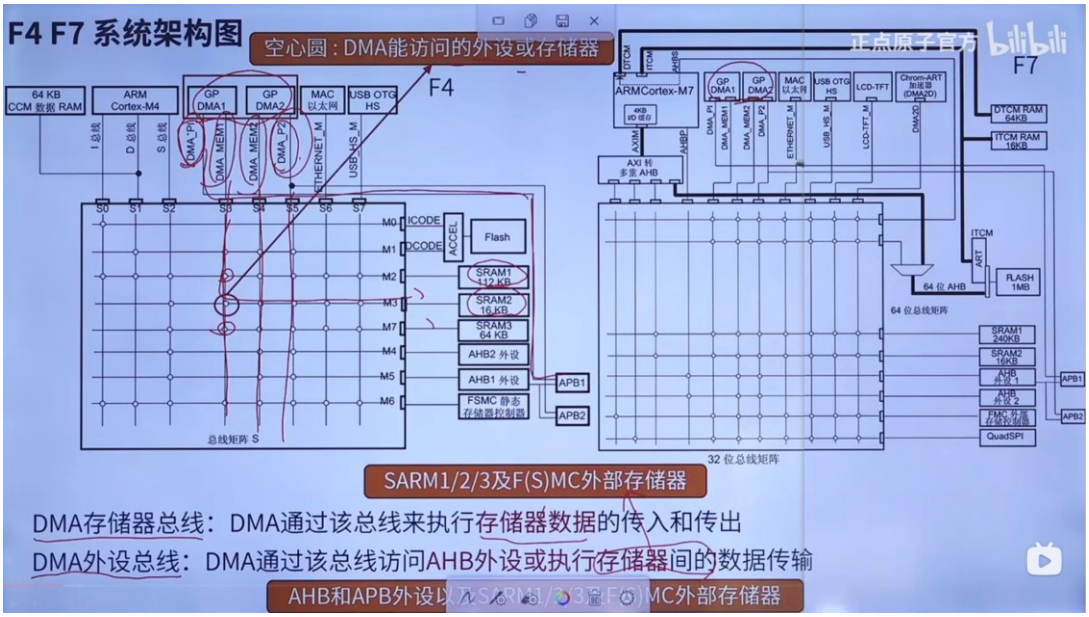


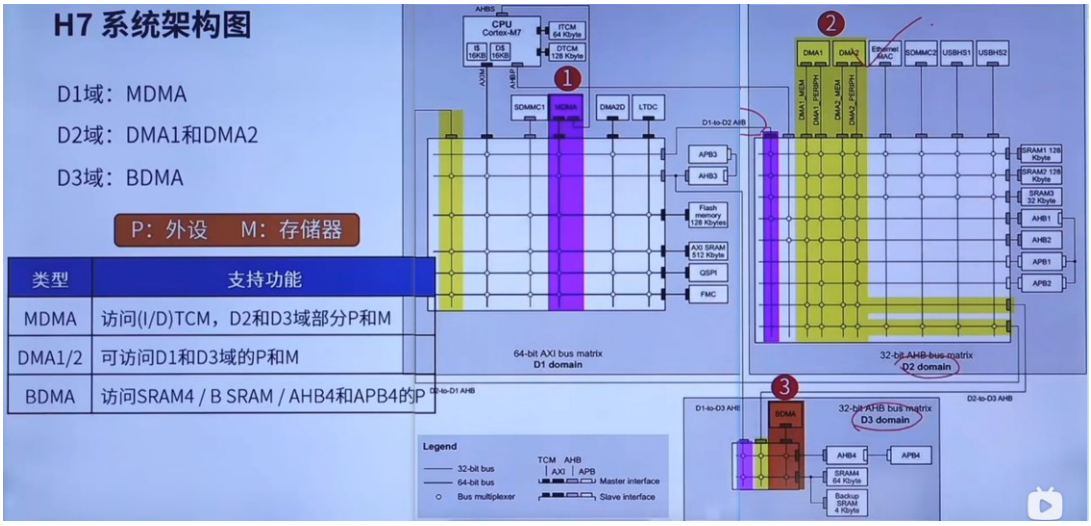
15.2.4 DMA 配置步骤



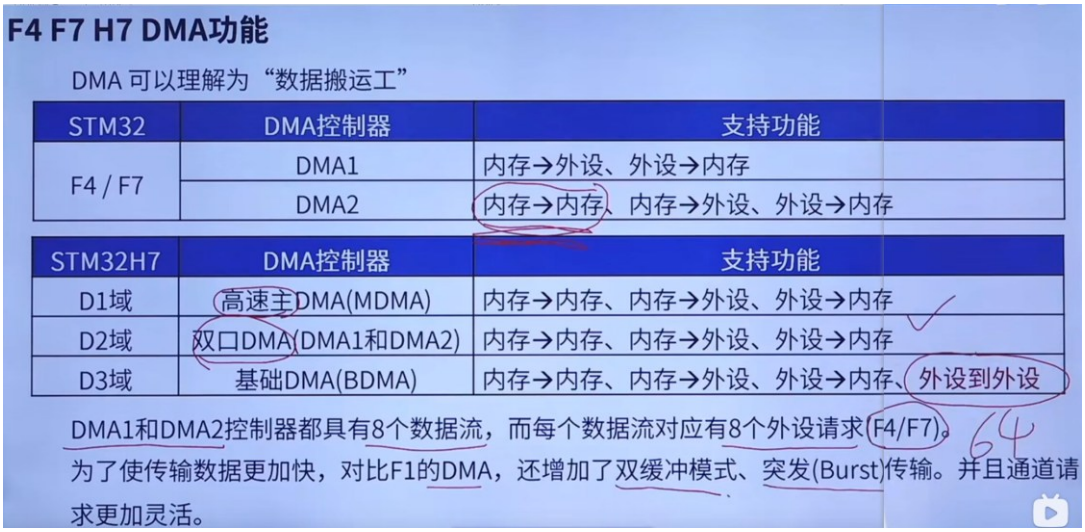
15.3 F4/F7/H7

15.3.1 F4/F7/H7 DMA 结构框图介绍





15.3.2 F4 H7 DMA 功能



F4 F7 H7 DMA功能

DMA 可以理解为“数据搬运工”

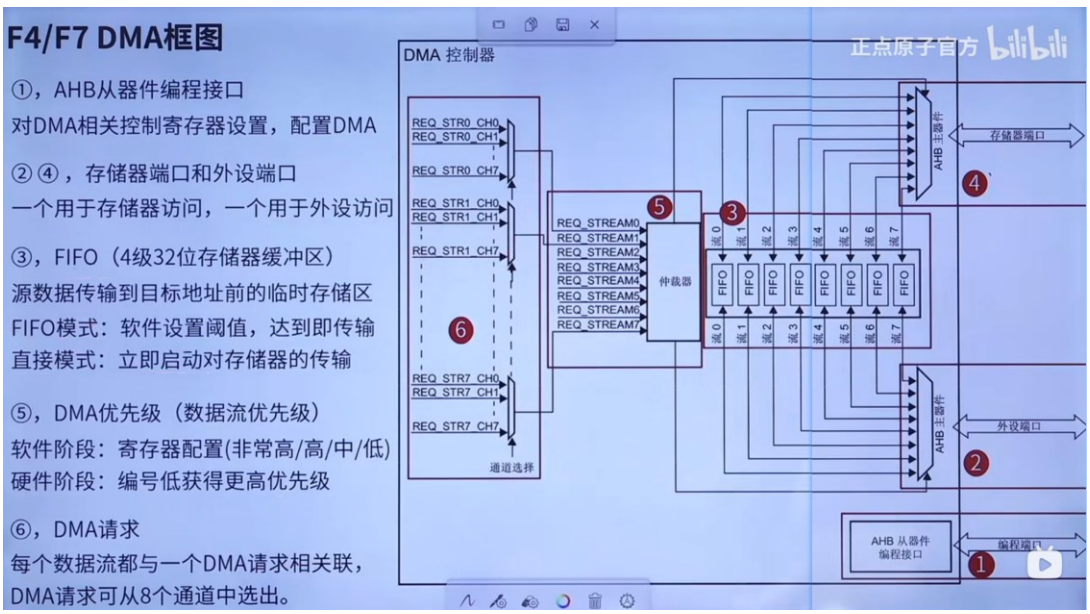
STM32	DMA控制器	支持功能
F4 / F7	DMA1	内存→外设、外设→内存
	DMA2	内存→内存、内存→外设、外设→内存

STM32H7	DMA控制器	支持功能
D1域	高速主DMA(MDMA)	内存→内存、内存→外设、外设→内存
D2域	双口DMA(DMA1和DMA2)	内存→内存、内存→外设、外设→内存
D3域	基础DMA(BDMA)	内存→内存、内存→外设、外设→内存、外设到外设

DMA1和DMA2控制器都具有8个数据流，而每个数据流对应有8个外设请求(F4/F7)。

为了使传输数据更加快，对比F1的DMA，还增加了双缓冲模式、突发(Burst)传输。并且通道请求更加灵活。

15.3.3 F4/F7 DMA 框图



F4/F7 DMA框图

①，AHB从器件编程接口
对DMA相关控制寄存器设置，配置DMA

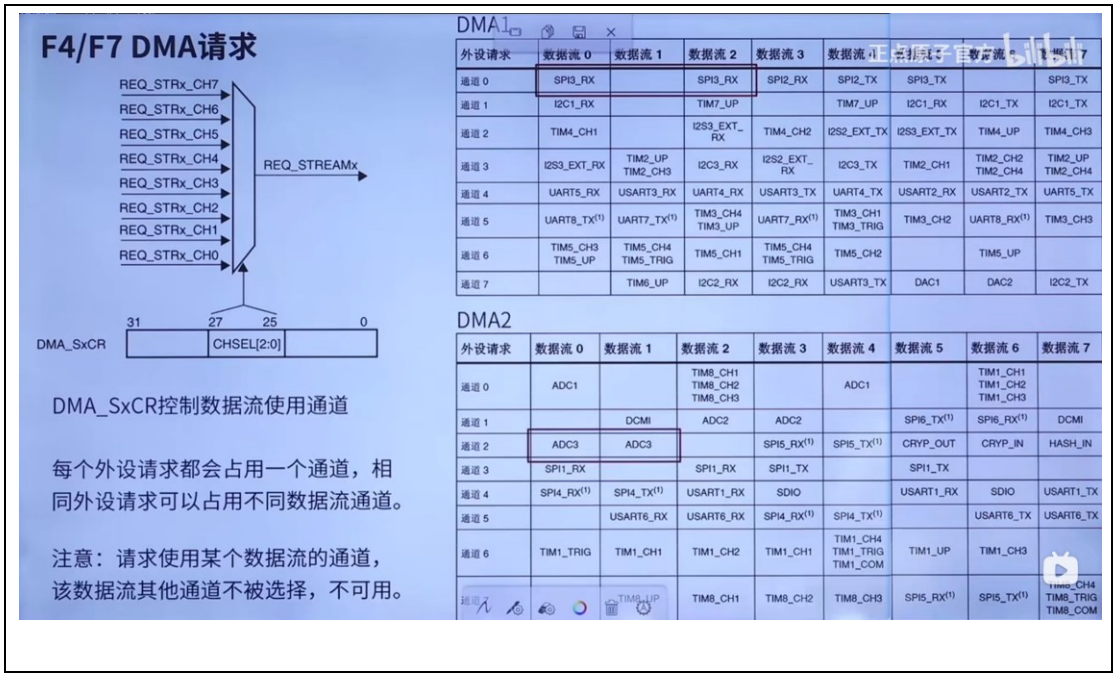
②④，存储器端口和外设端口
一个用于存储器访问，一个用于外设访问

③，FIFO（4级32位存储器缓冲区）
源数据传输到目标地址前的临时存储区
FIFO模式：软件设置阈值，达到即传输
直接模式：立即启动对存储器的传输

⑤，DMA优先级（数据流优先级）
软件阶段：寄存器配置(非常高/高/中/低)
硬件阶段：编号低获得更高优先级

⑥，DMA请求
每个数据流都与一个DMA请求相关联，
DMA请求可从8个通道中选出。

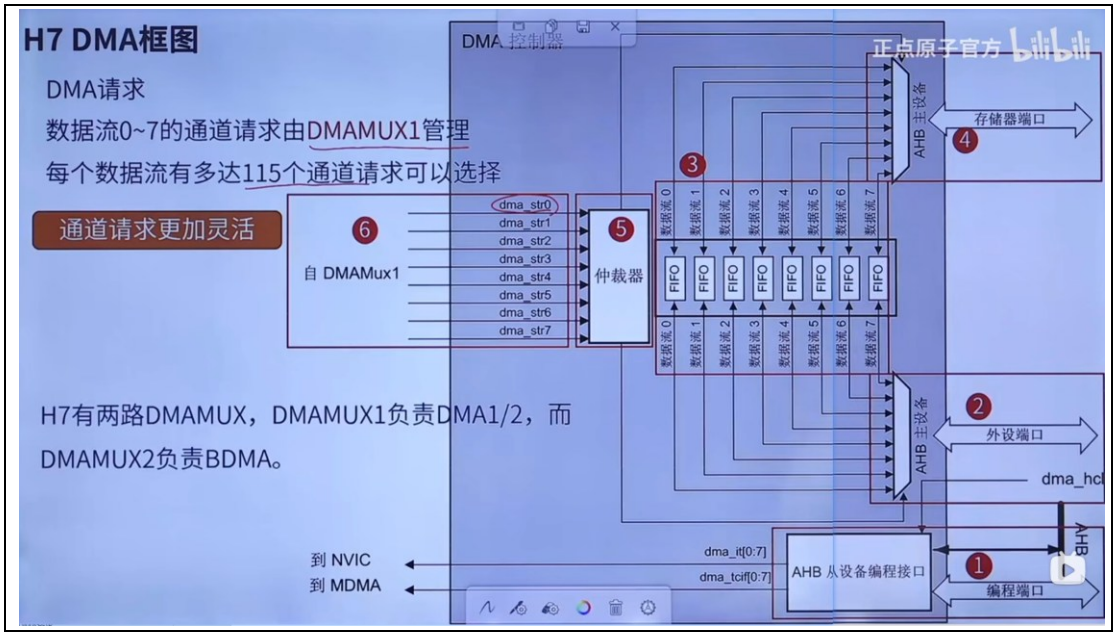
15.3.4 F4/F7DMA 请求

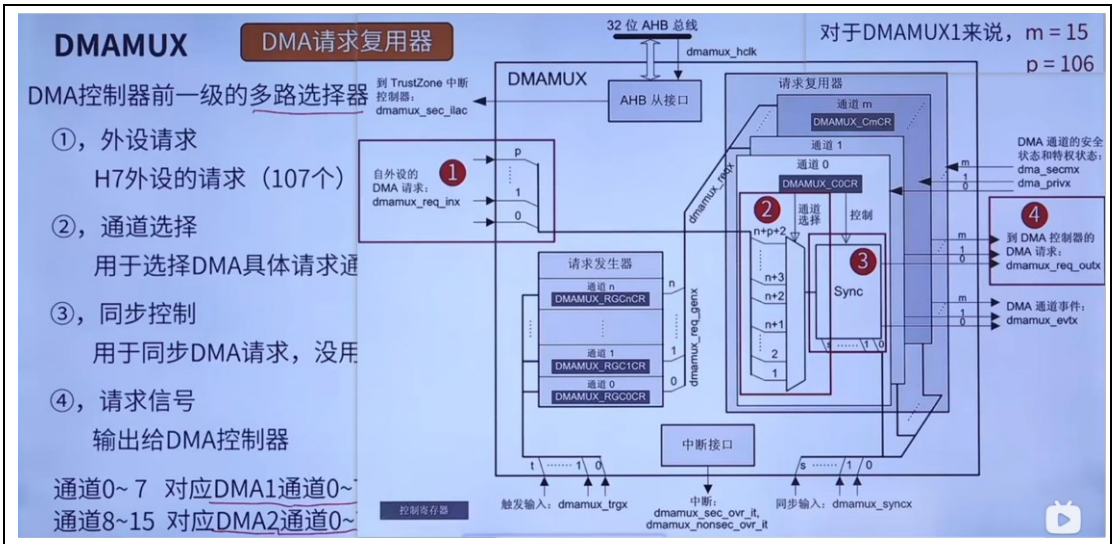


外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	SPI3_RX		SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX		SPI3_TX
通道 1	I2C1_RX		TIM7_UP		TIM7_UP	I2C1_RX	I2C1_TX	I2C1_TX
通道 2	TIM4_CH1		I2S3_EXT_RX	TIM4_CH2	I2S2_EXT_TX	I2S3_EXT_TX	TIM4_UP	TIM4_CH3
通道 3	I2S3_EXT_RX	TIM2_UP TIM2_CH3	I2C3_RX	I2S2_EXT_RX	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
通道 4	UART5_RX	USART3_RX	UART4_RX	USART3_TX	UART4_TX	USART2_RX	USART2_TX	UART5_TX
通道 5	UART8_TX(1)	UART7_TX(1)	TIM3_CH4 TIM3_UP	UART7_RX(1)	TIM3_CH1 TIM3_TRIG	TIM3_CH2	UART8_RX(1)	TIM3_CH3
通道 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2		TIM5_UP	
通道 7		TIM6_UP	I2C2_RX	I2C2_RX	USART3_TX	DAC1	DAC2	I2C2_TX

外设请求	数据流 0	数据流 1	数据流 2	数据流 3	数据流 4	数据流 5	数据流 6	数据流 7
通道 0	ADC1		TIM8_CH1 TIM8_CH2 TIM8_CH3		ADC1		TIM1_CH1 TIM1_CH2 TIM1_CH3	
通道 1		DCMI	ADC2	ADC2		SPI6_TX(1)	SPI6_RX(1)	DCMI
通道 2	ADC3	ADC3		SPI5_RX(1)	SPI5_TX(1)	CRYP_OUT	CRYP_IN	HASH_IN
通道 3	SPI1_RX		SPI1_RX	SPI1_TX		SPI1_TX		
通道 4	SPI4_RX(1)	SPI4_TX(1)	USART1_RX	SDIO		USART1_RX	SDIO	USART1_TX
通道 5		USART6_RX	USART6_RX	SPI4_RX(1)	SPI4_TX(1)		USART6_TX	USART6_TX
通道 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	
		TIM8_UP	TIM8_CH1	TIM8_CH2	TIM8_CH3	SPI5_RX(1)	SPI5_TX(1)	TIM8_CH4 TIM8_TRIG TIM8_COM

15.3.5 H7 DMA 框图

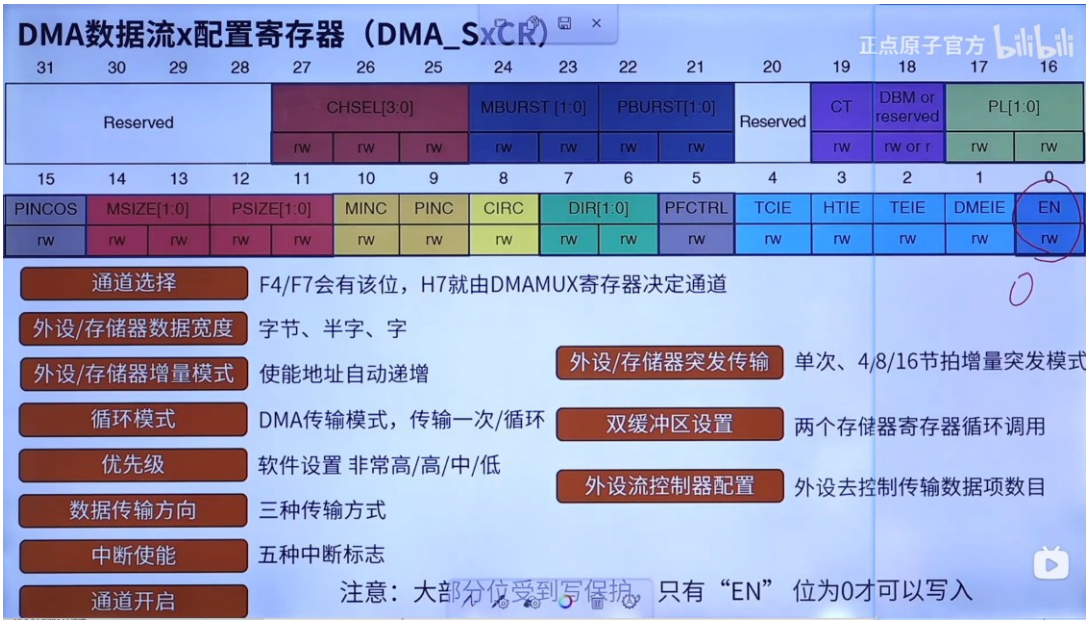




15.3.6 DMA 相关寄存器介绍 F4/F7/H7

寄存器	名称	作用
DMA_SxCR	DMA数据流x配置寄存器	用于配置DMA（核心控制寄存器）
DMA_(H/L)ISR	DMA(低/高)中断状态寄存器	用于查询当前DMA传输状态
DMA_(H/L)IFCR	DMA(低/高)中断标志清除寄存器	用来清除DMA_(H/L)ISR对应位
DMA_SxNDTR	DMA数据流x传输数量寄存器	用于控制DMA数据流x每次传输的数据量
DMA_SxPAR	DMA数据流x外设地址寄存器	用于存储STM32外设地址
DMA_SxM0AR	DMA数据流x存储器地址寄存器	用于存放存储器的地址
USART_CR3	USART控制寄存器3	用于使能DMA发送

对于H7 DMAMUX部分只需要知道DMAMUX请求发生器通道x配置寄存器的使用即可



DMA低中断状态寄存器（DMA_LISR）

管理数据流0~3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserved	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserved	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF1	HTIF1	TEIF1	DMEIF1	Reserved	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserved	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

DMA高中断状态寄存器（DMA_HISR）

管理数据流4~7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF7	HTIF7	TEIF7	DMEIF7	Reserved	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Reserved	FEIF6
				r	r	r	r		r	r	r	r	r		r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF5	HTIF5	TEIF5	DMEIF5	Reserved	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Reserved	FEIF4
				r	r	r	r		r	r	r	r	r		r

DMA有五种标志：TCIF 传输完成；HTIF 半传输；TEIF 传输错误；DMEIF 直接模式错误；FEIF FIFO错误

注意：当设置了允许中断时，将会产生中断

DMA低中断标志清除寄存器（DMA_LIFCR）

管理数据流0~3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
				w	w	w	w		w	w	w	w	w		w

DMA高中断标志清除寄存器（DMA_HIFCR）

管理数据流4~7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Reserved	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Reserved	CFEIF6
				w	w	w	w		w	w	w	w	w		w

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Reserved	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Reserved	CFEIF4
				w	w	w	w		w	w	w	w	w		w

DMA通道x传输数量寄存器（DMA_CNDTR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **NDT[15:0]**：要传输的数据项数目 (Number of data items to transfer)

要传输的数据项数目（0 到 65535）。只有在禁止数据流时，才能向此寄存器执行写操作。使能数据流后，此寄存器为只读，用于指示要传输的剩余数据项数。每次 DMA 传输后，此寄存器将递减。

传输完成后，此寄存器保持为零（数据流处于正常模式时），或者在以下情况下自动以先前编程的值重载：

- 以循环模式配置数据流时。
- 通过将 EN 位置“1”来重新使能数据流时

如果该寄存器的值为零，则即使使能数据流，也无法完成任何事务。

这里指的是数据项，并不是字节数

最大数据传输数目：65535

注意：当寄存器值为0表明数据传输已经全部发送完成

DMA数据流x外设地址寄存器（DMA_SxPAR） 串口1，0x40011028（&USART1_TDR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

DMA数据流x存储器0地址寄存器（DMA_SxM0AR） sendbuf[10]，&sendbuf/sendbuf

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

注意：H750的DMA支持双缓冲，存储器寄存器除了上面，还有一个DMA_SxM1AR（双缓冲模式有效

17.6.1 DMAMUX1 请求线复用器通道x配置寄存器 (DMAMUX1_CxCR)

DMAMUX1 request line multiplexer channel x configuration register

偏移地址：0x04 * x（x = 0 到 15）

复位值：0x0000 0000

通道0~7 对应DMA1通道0~7

通道8~15 对应DMA2通道0~7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SYNC_ID[4:0]					NBREQ[4:0]					SPOL[1:0]		SE
			rw					rw					rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	ege	SOIE	DMAREQ_ID[7:0]							
						rw	rw	rw							

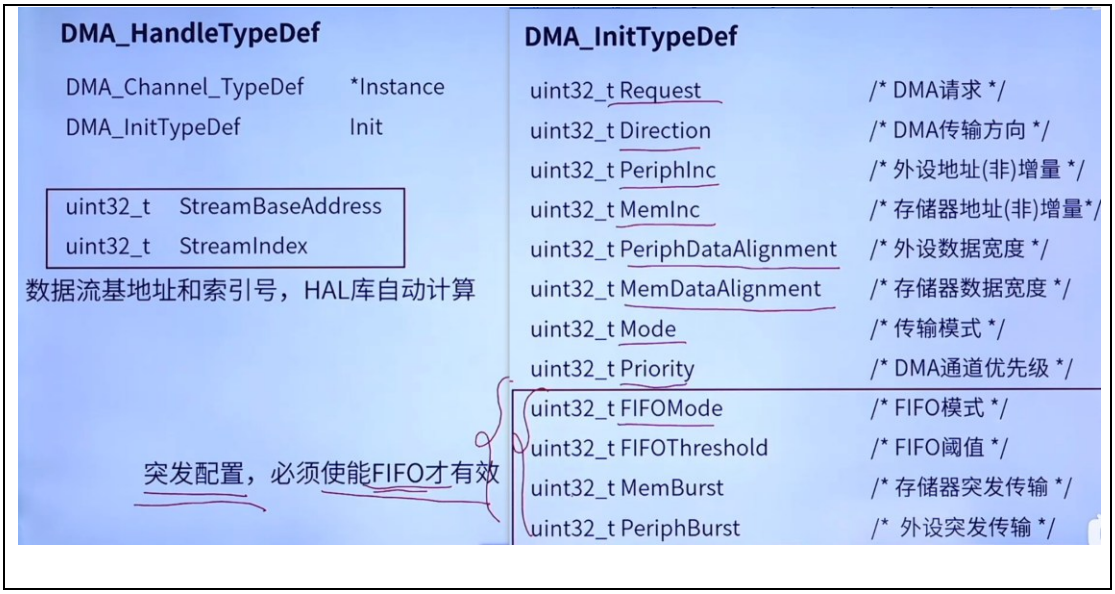
位 7:0 **DMAREQ_ID[7:0]**：DMA 请求标识 (DMA request identification)

选择输入 DMA 请求。（请参见表：DMAMUX - 复用器输入到资源的分配）

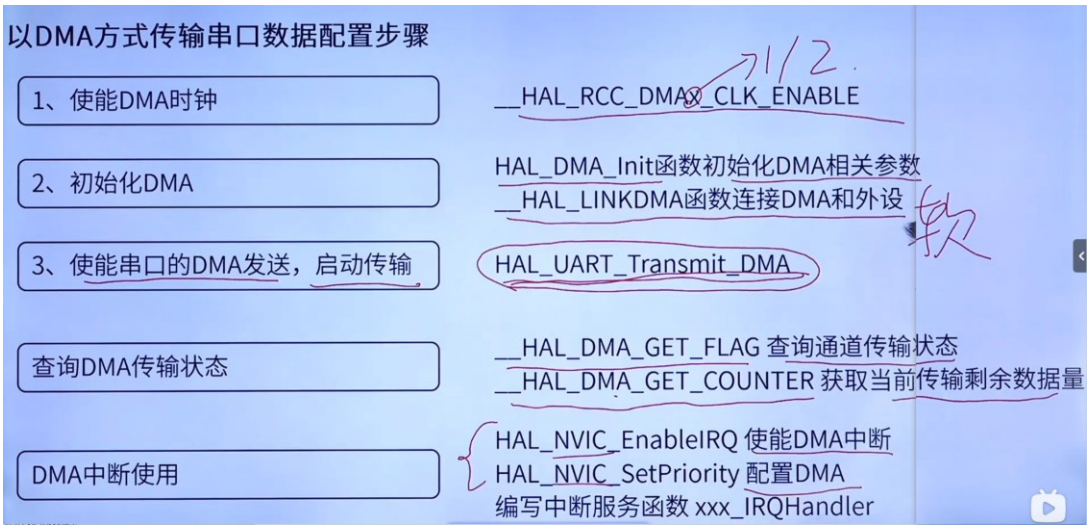
15.3.7 DMA 相关 HAL 库驱动介绍

驱动函数	关联寄存器	功能描述
__HAL_RCC_DMAx_CLK_ENABLE(…)	RCC_AHBENR	使能DMAx时钟
HAL_DMA_Init(…)	DMA_SxCR	初始化DMA
HAL_DMA_Start_IT(…)	DMA_SxCR/SxPAR/SxM0AR/SxNDTR	开始DMA传输
__HAL_LINKDMA(…)		用来连接DMA和外设句柄
HAL_UART_Transmit_DMA(…)	SxCR/SxPAR/SxM0AR/SxNDTR/USART_CR3	使能DMA发送，启动传输
__HAL_DMA_GET_FLAG(…)	DMA_ISR	查询DMA传输通道的状态
__HAL_DMA_ENABLE(…)	DMA_CCR(EN)	使能DMA外设
__HAL_DMA_DISABLE(…)	DMA_CCR(EN)	失能DMA外设
HAL_UART_DMAStop(…)	USART_CR3	停止DMA传输

DMA外设相关结构体：DMA_HandleTypeDef 和 DMA_InitTypeDef



15.3.8 F4/F7/H7 DMA 配置步骤



16 ADC

16.1 ADC 简介

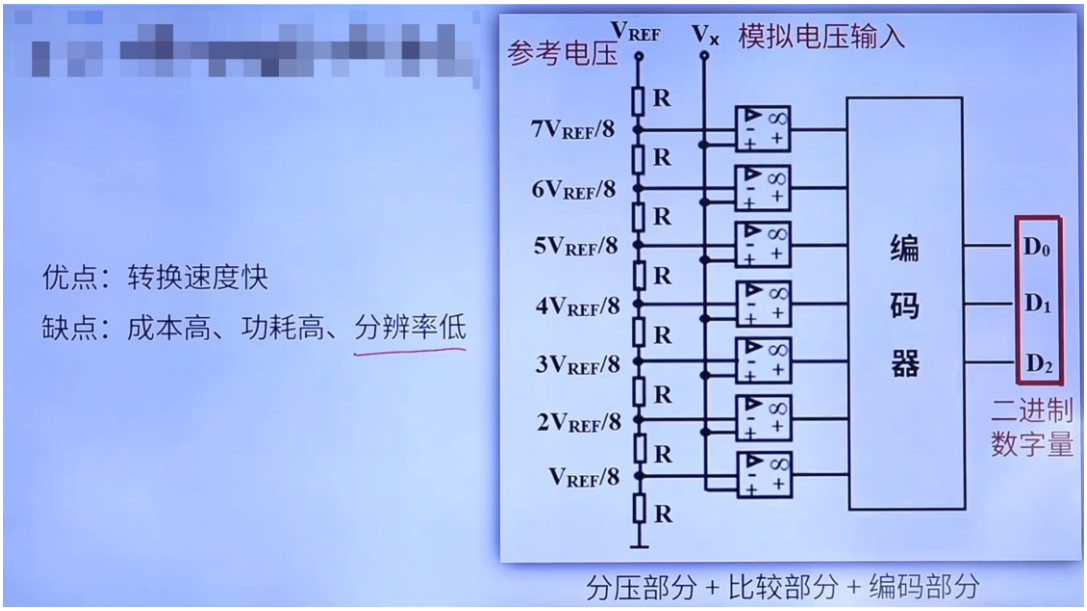
16.1.1 什么是 ADC

16.1.2 常见的 ADC 类型

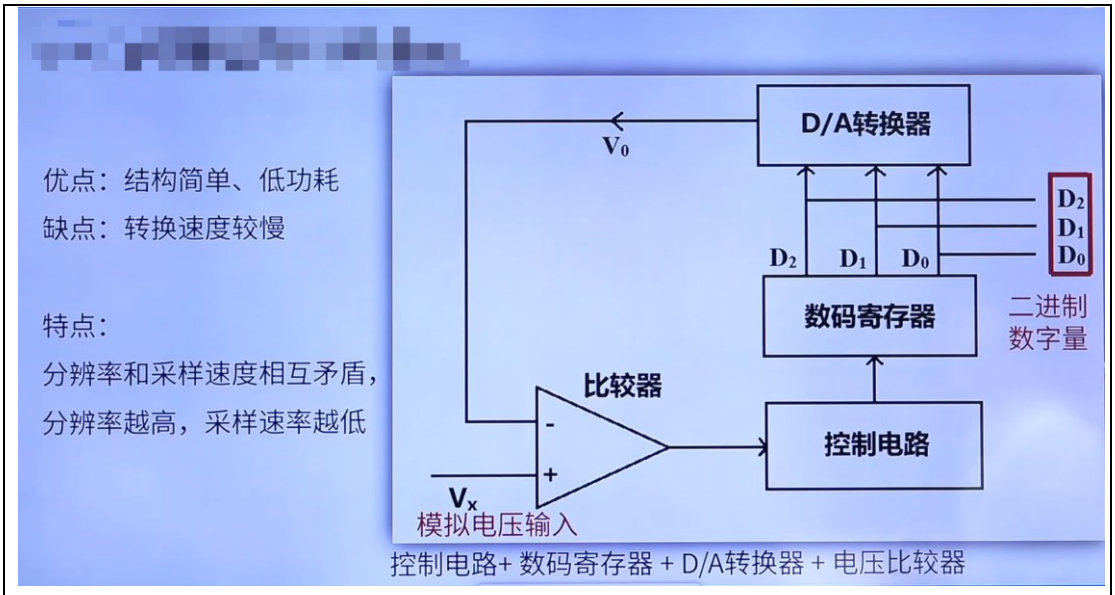
ADC电路类型	优点	缺点
并联比较型	转换速度最快	成本高、功耗高，分辨率低
逐次逼近型	结构简单，功耗低	转换速度较慢

常见ADC类型：https://blog.csdn.net/whereismatrix/article/details/81814431

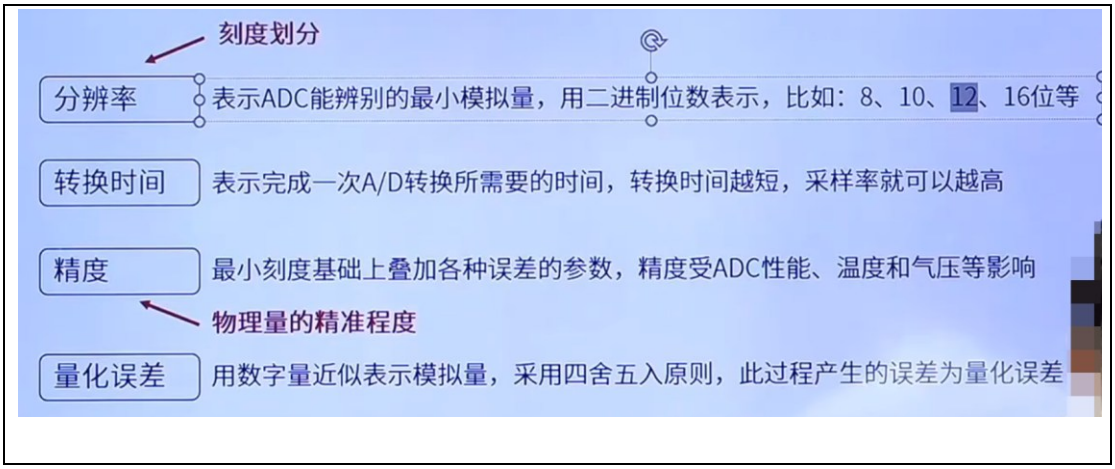
16.1.3 并联比较型工作示意图



16.1.4 逐次逼近型工作示意图



16.1.5 ADC 特性参数



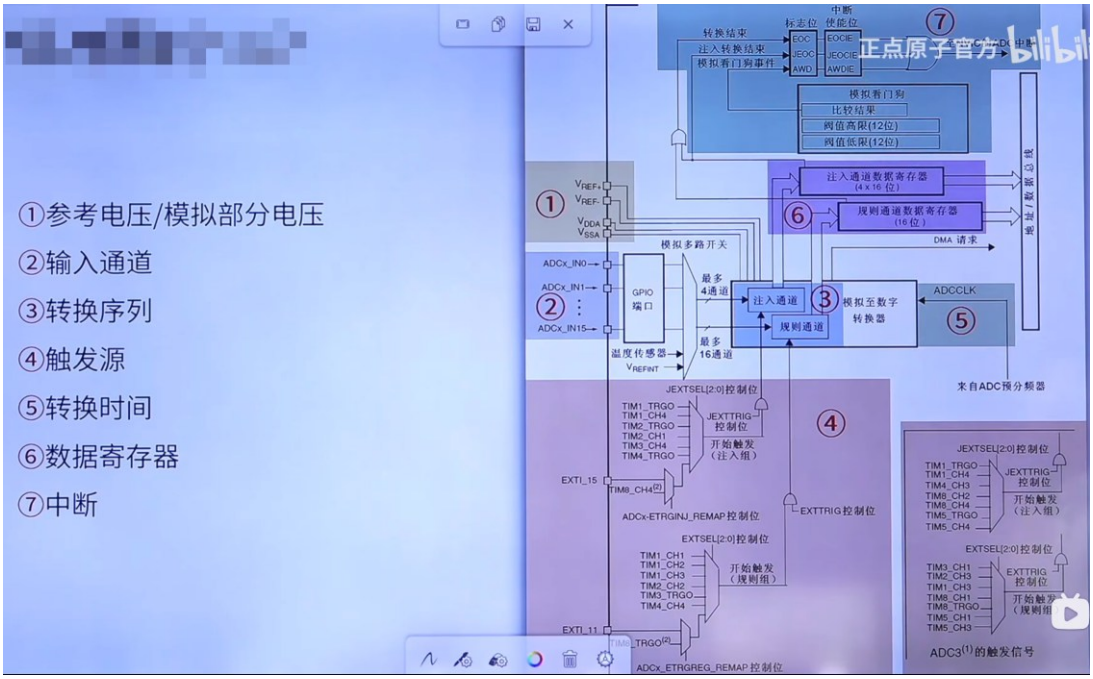
16.1.6 STM32 各系列 ADC 的主要特性

主要特性	F1	F4	F7	H7
ADC类型	逐次逼近型			
分辨率	12位	6/8/10/12位	6/8/10/12位	8/10/12/14/16位
ADC时钟频率	14MHz（max）	36MHz（max）		
采样时间	采样时间越长，转换结果相对越准确，但是转换速度就越慢			
转换时间	与ADC时钟频率、分辨率和采样时间等有关			
供电电压	V_{SSA}：0V，V_{DDA}：2.4V~3.6V（全速运行）			
参考电压	V_{REF-}：0V，V_{REF+}一般为3.3V			
输入电压	$V_{REF-} \leq V_{IN} \leq V_{REF+}$			

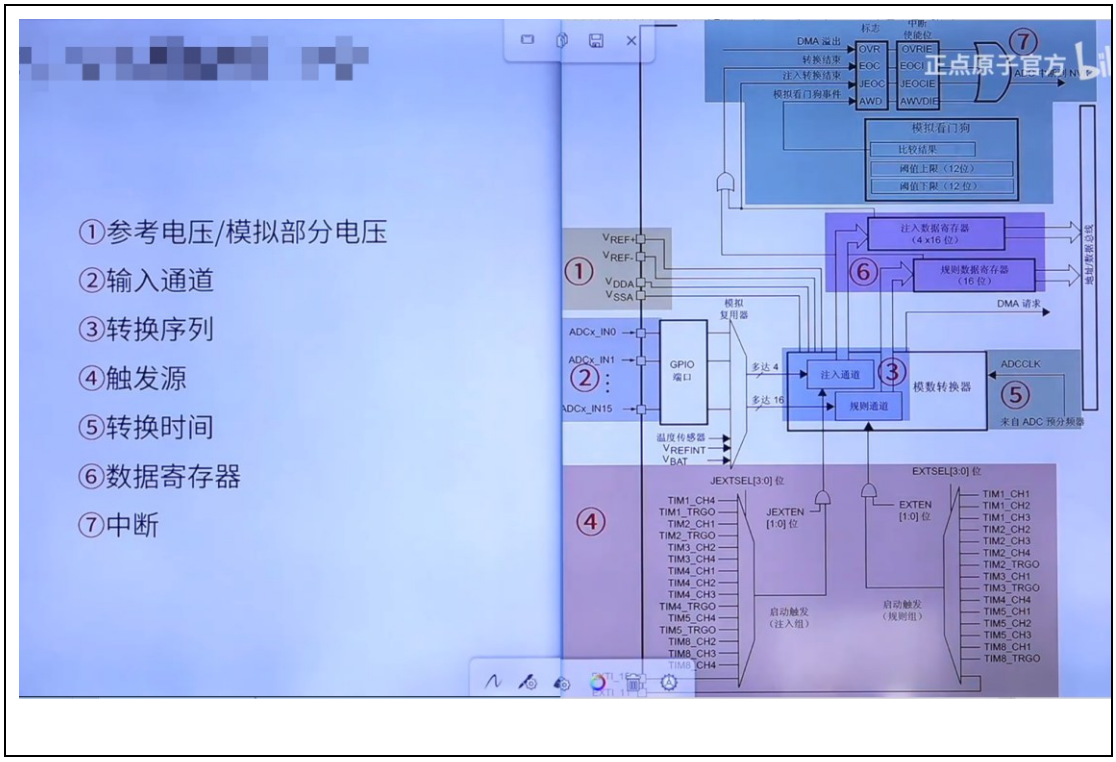
16.2 ADC 工作原理

16.2.1 ADC 框图简介（F1/F4/F7/H7）

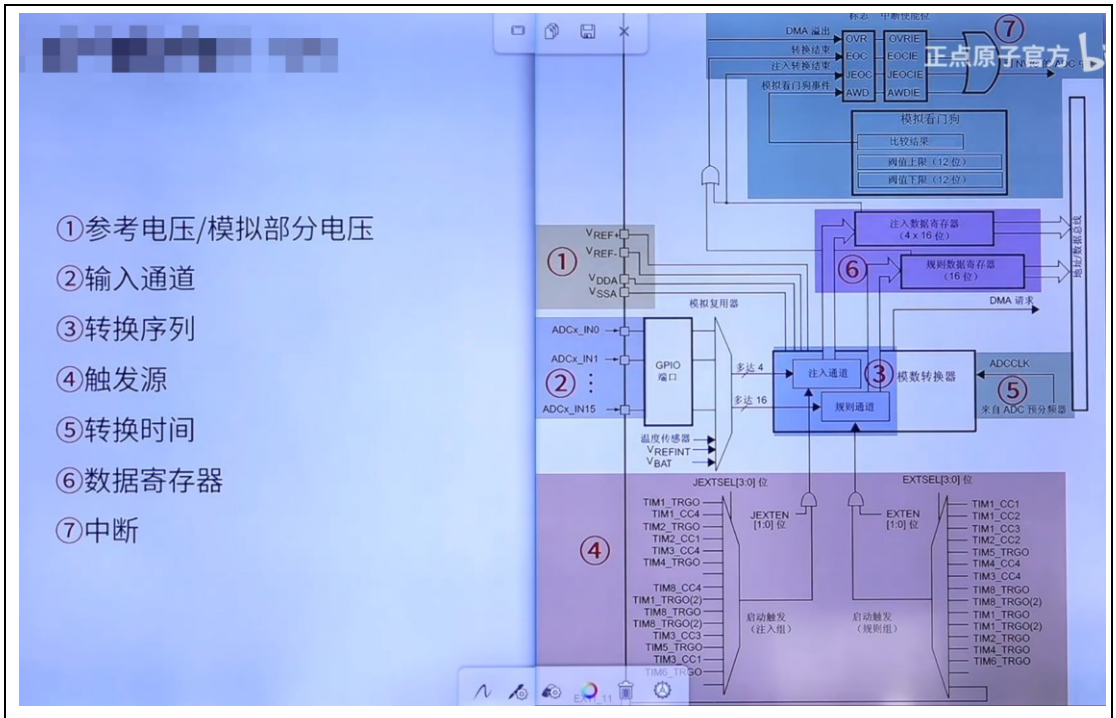
16.2.1.1 ADC 简介（F1）



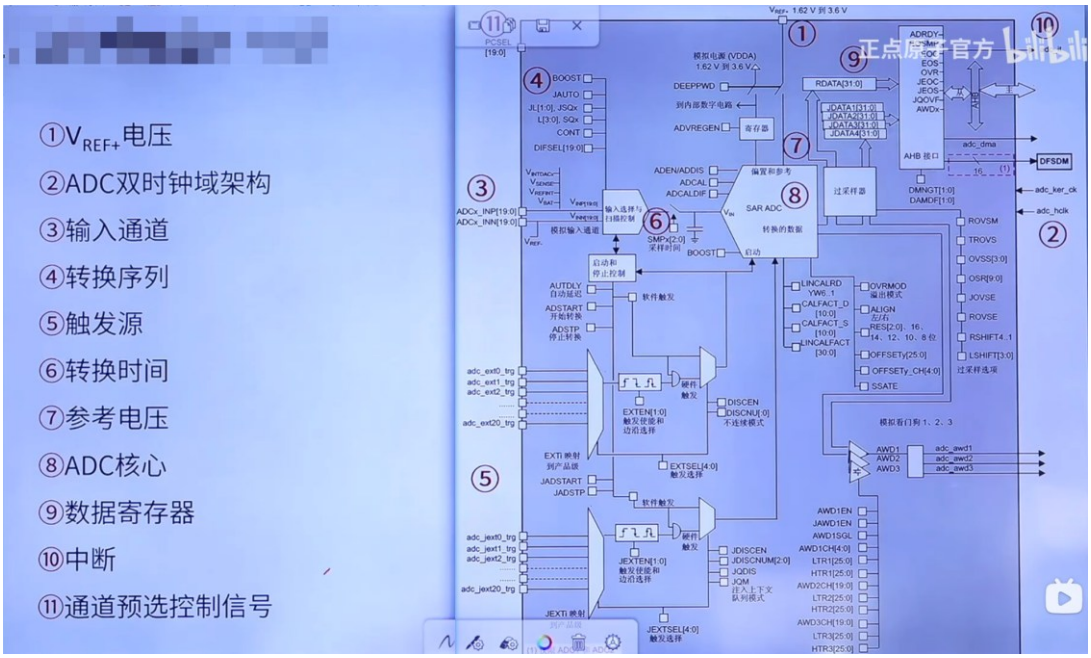
16.2.1.2 ADC 框图简介（F4）



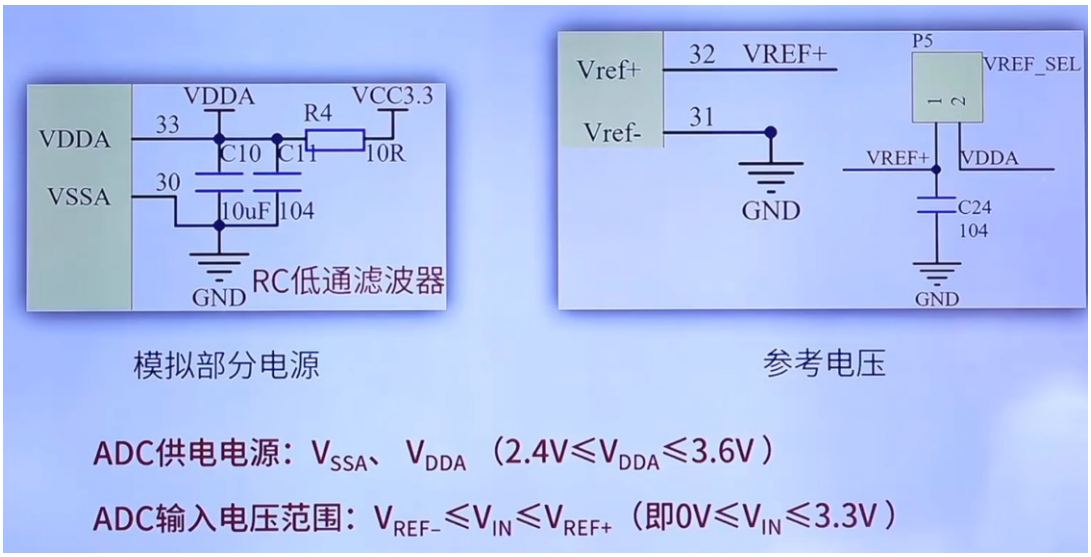
16.2.1.3 ADC 框图简介（F7）



16.2.1.4　ADC 框图简介（H7）



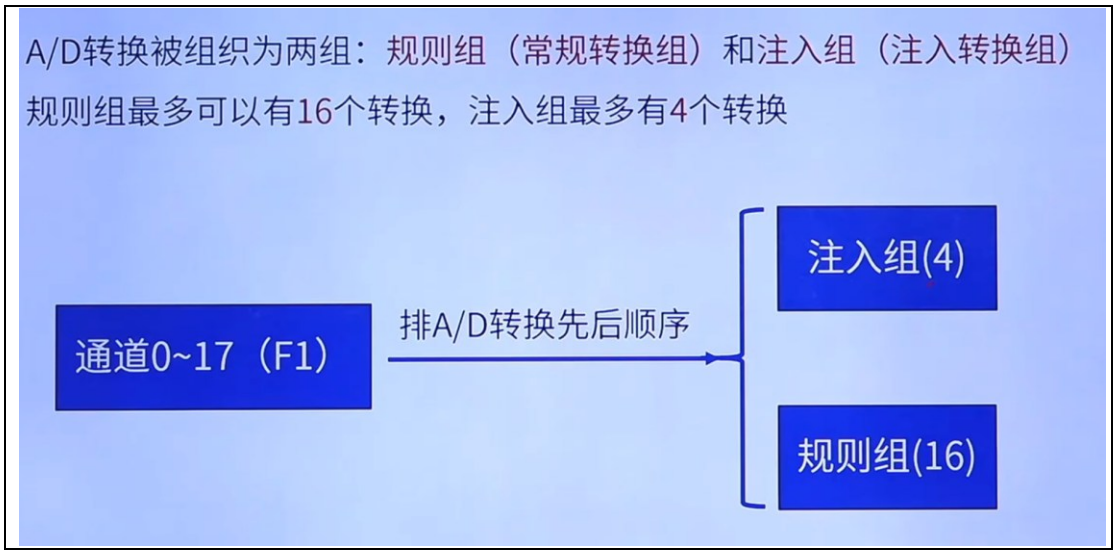
16.2.2 参考电压/模拟部分电压



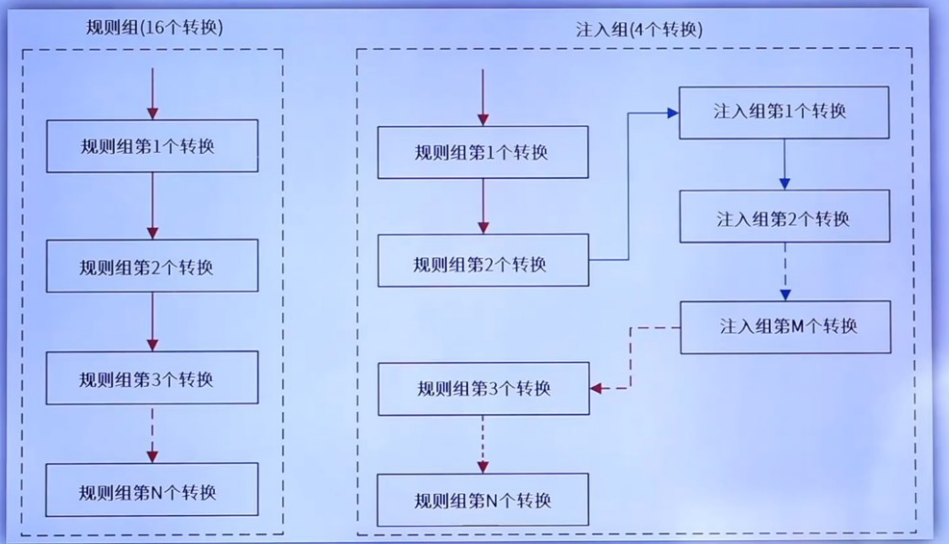
16.2.3 输入通道（F1）

ADC1	IO	ADC2	IO	ADC3	IO
通道0	PA0	通道0	PA0	通道0	PA0
通道1	PA1	通道1	PA1	通道1	PA1
通道2	PA2	通道2	PA2	通道2	PA2
通道3	PA3	通道3	PA3	通道3	PA3
通道4	PA4	通道4	PA4	通道4	PF6
通道5	PA5	通道5	PA5	通道5	PF7
通道6	PA6	通道6	PA6	通道6	PF8
通道7	PA7	通道7	PA7	通道7	PF9
通道8	PB0	通道8	PB0	通道8	PF10
通道9	PB1	通道9	PB1	通道9	连接内部VSS
通道10	PC0	通道10	PC0	通道10	PC0
通道11	PC1	通道11	PC1	通道11	PC1
通道12	PC2	通道12	PC2	通道12	PC2
通道13	PC3	通道13	PC3	通道13	PC3
通道14	PC4	通道14	PC4	通道14	连接内部VSS
通道15	PC5	通道15	PC5	通道15	连接内部VSS
通道16	连接内部温度传感器	通道16	连接内部VSS	通道16	连接内部VSS
通道17	连接内部Vrefint	通道17	连接内部VSS	通道17	连接内部VSS

16.2.4 转换序列(F1)



规则组和注入组执行优先级对比



规则序列和注入序列（F1为例）

规则序列寄存器控制关系汇总

寄存器	寄存器位	功能	取值
SQR3	SQ1[4:0]	设置第1个转换的通道	通道0~17
	SQ2[4:0]	设置第2个转换的通道	通道0~17
	SQ3[4:0]	设置第3个转换的通道	通道0~17
	SQ4[4:0]	设置第4个转换的通道	通道0~17
	SQ5[4:0]	设置第5个转换的通道	通道0~17
	SQ6[4:0]	设置第6个转换的通道	通道0~17
SQR2	SQ7[4:0]	设置第7个转换的通道	通道0~17
	SQ8[4:0]	设置第8个转换的通道	通道0~17
	SQ9[4:0]	设置第9个转换的通道	通道0~17
	SQ10[4:0]	设置第10个转换的通道	通道0~17
	SQ11[4:0]	设置第11个转换的通道	通道0~17
	SQ12[4:0]	设置第12个转换的通道	通道0~17
SQR1	SQ13[4:0]	设置第13个转换的通道	通道0~17
	SQ14[4:0]	设置第14个转换的通道	通道0~17
	SQ15[4:0]	设置第15个转换的通道	通道0~17
	SQ16[4:0]	设置第16个转换的通道	通道0~17
	SQL[3:0]	设置规则序列要转换的通道数	0~15

规则序列和注入序列（F1为例）

注入序列寄存器控制关系汇总

寄存器	寄存器位	功能	取值
JSQR	JSQ1[4:0]	设置第1个转换的通道	通道0~17
	JSQ2[4:0]	设置第2个转换的通道	通道0~17
	JSQ3[4:0]	设置第3个转换的通道	通道0~17
	JSQ4[4:0]	设置第4个转换的通道	通道0~17
	JL[1:0]	设置注入序列要转换的通道数	0~3

注入序列的转换顺序是从JSQx[4:0]（x=4-JL[1:0]）开始

16.2.5 触发源

触发转换的方法有两种：

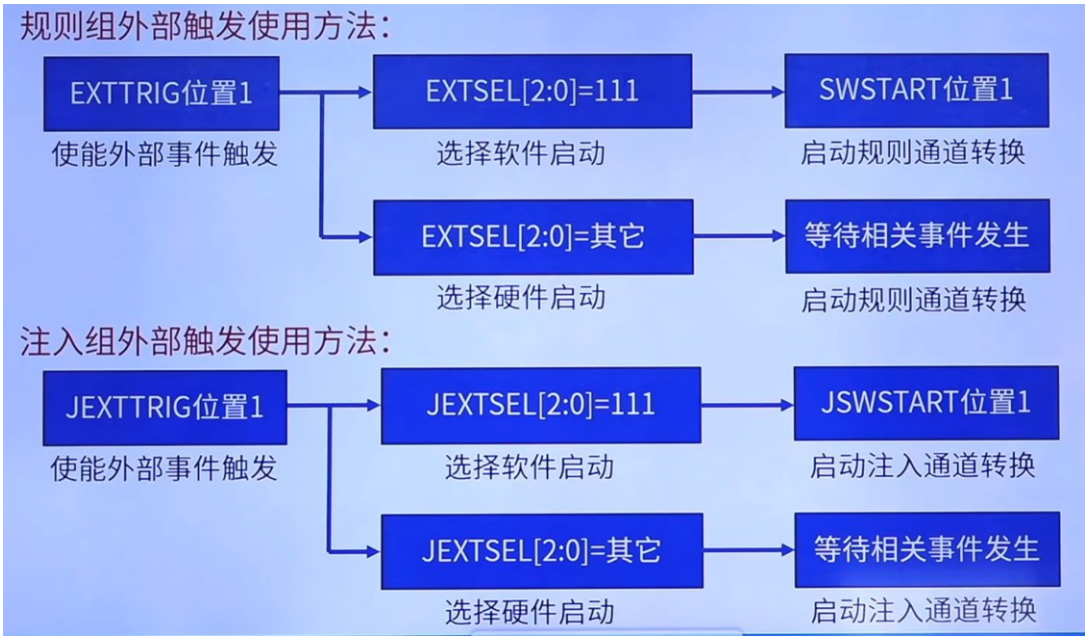
（1）ADON位触发转换(仅限F1系列)

当ADC_CR2寄存器的ADON位为1时，再单独给ADON位写1，只能启动规则组转换

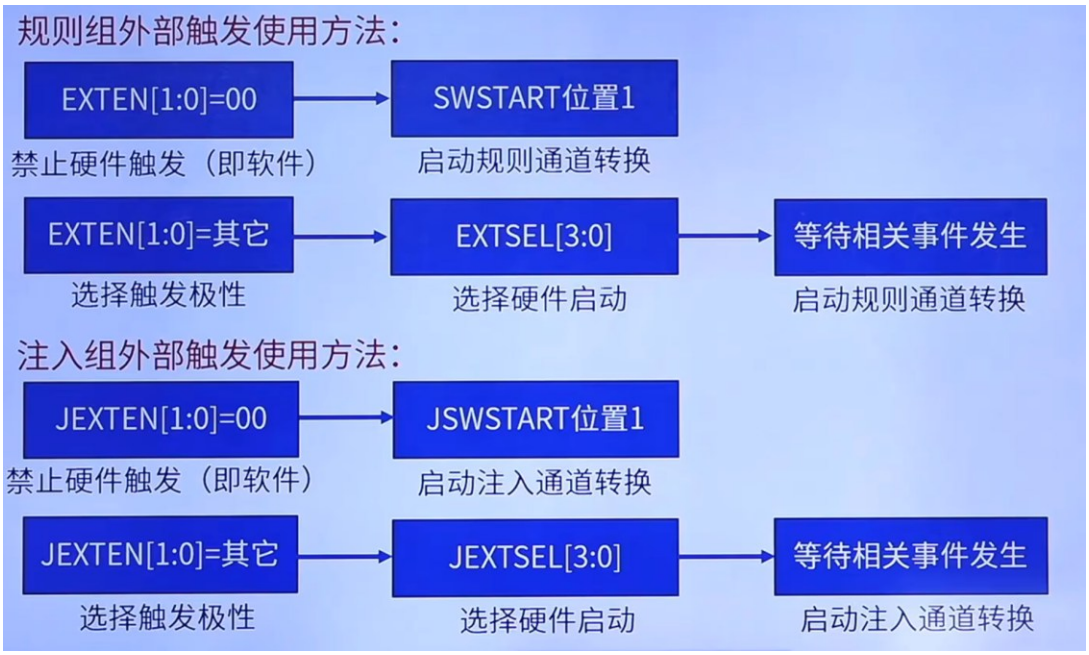
（2）外部事件触发转换

外部事件触发转换分为：规则组外部触发和注入组外部触发

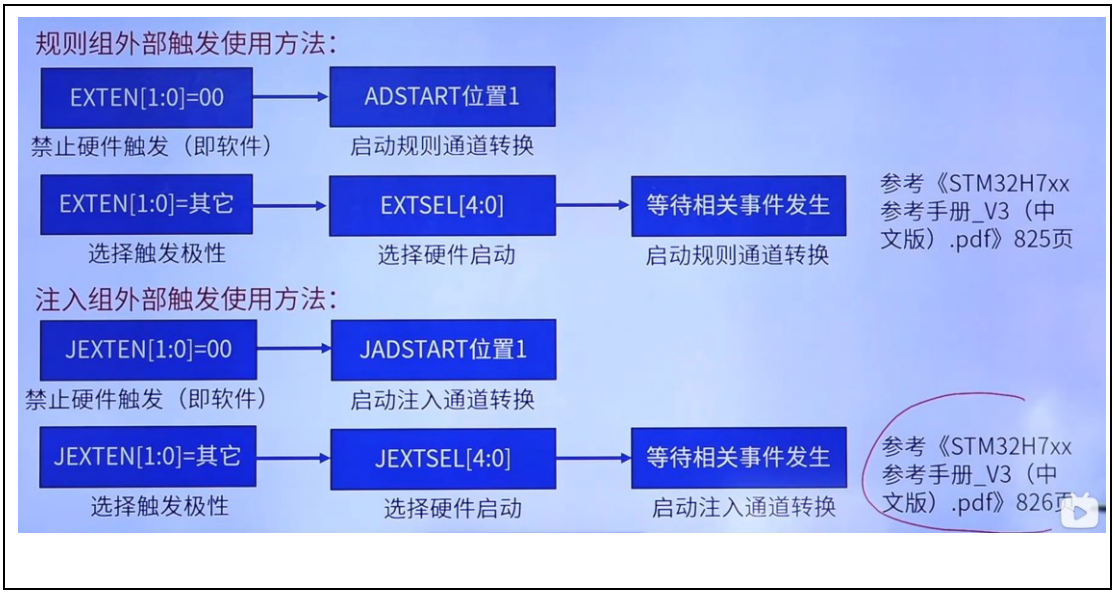
16.2.5.1 F1



16.2.5.2 F4/F7

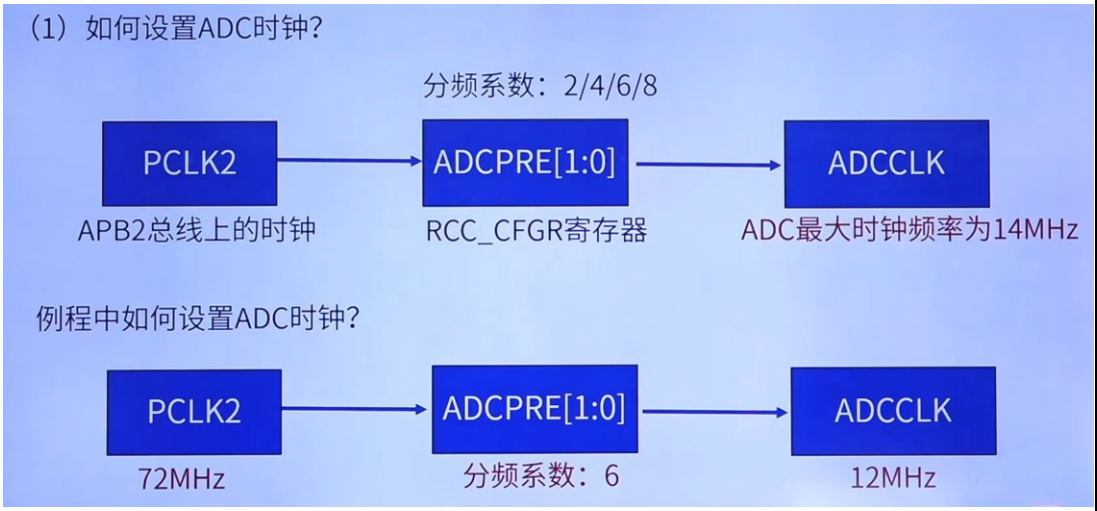


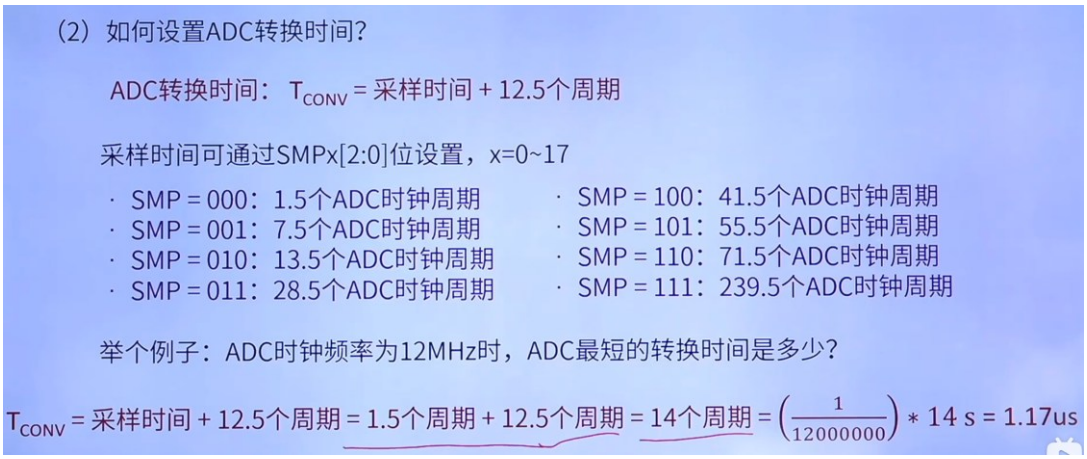
16.2.5.3 H7



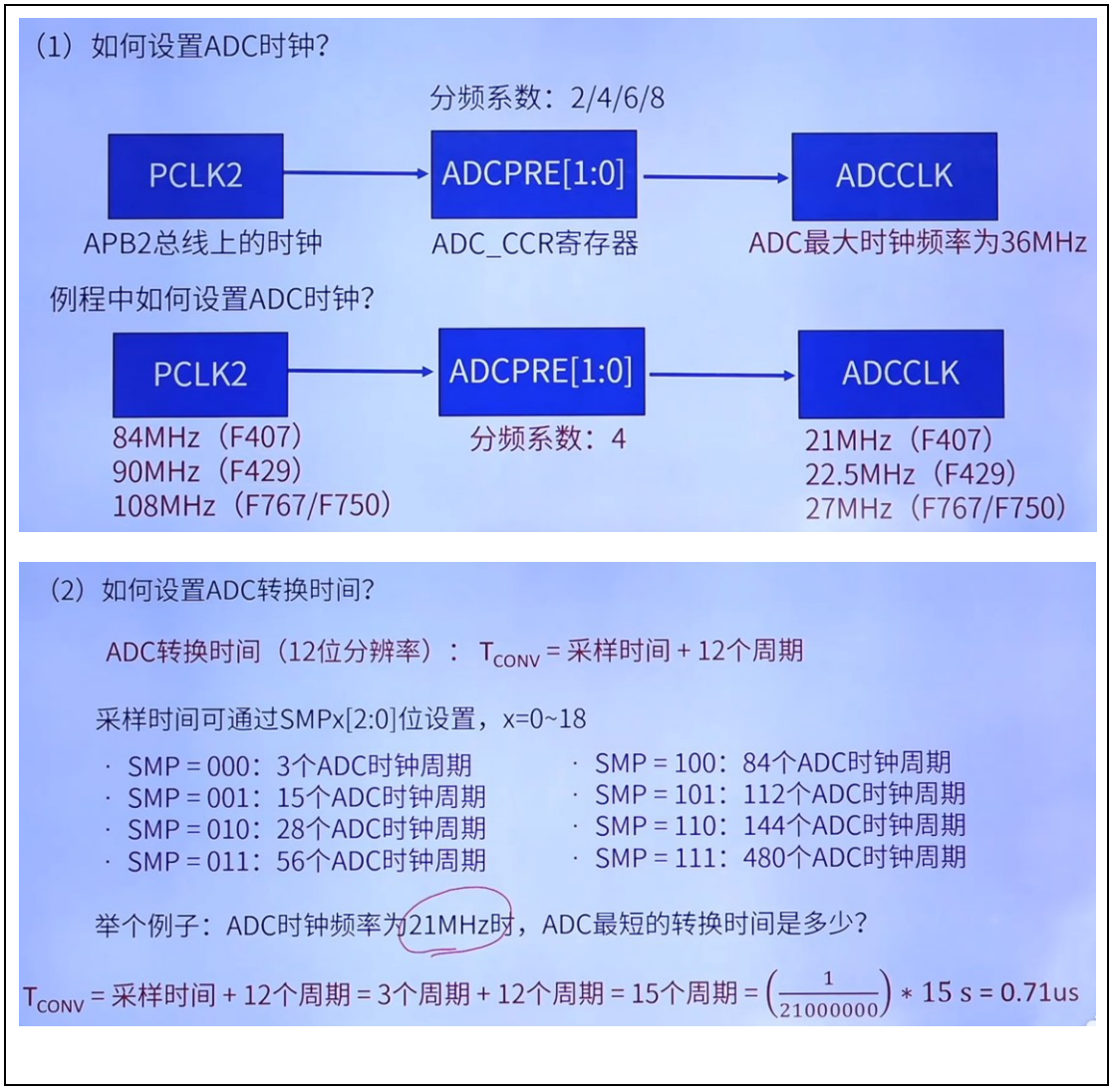
16.2.6 转换时间

16.2.6.1 F1

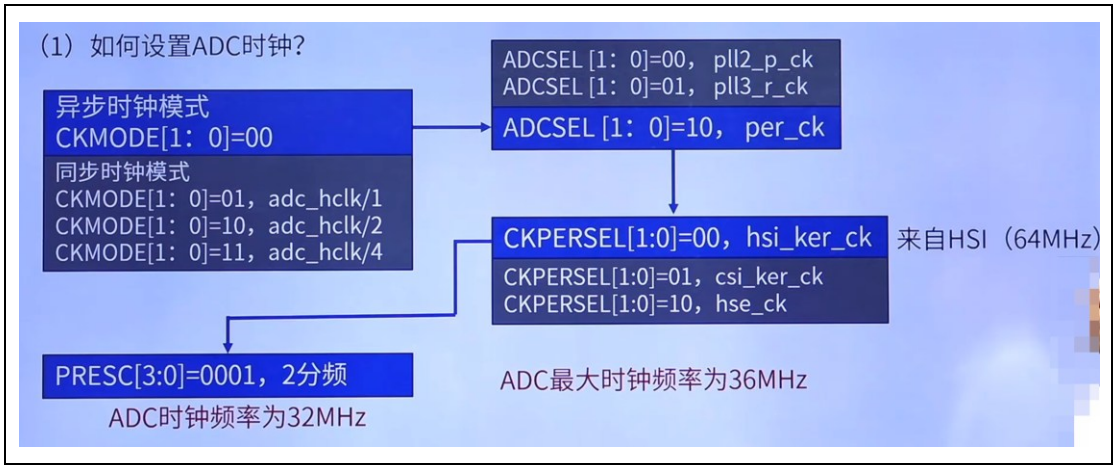




16.2.6.2 F4/F7



16.2.6.3 H7



（2）如何设置ADC转换时间?

ADC转换时间：T_{CONV} = 采样时间（T_{SMPL}） + 逐次逼近时间（T_{TSAR}）

采样时间可通过SMPx[2:0]位设置，x=0~19

- SMP = 000：1.5个ADC时钟周期
- SMP = 001：2.5个ADC时钟周期
- SMP = 010：8.5个ADC时钟周期
- SMP = 011：16.5个ADC时钟周期
- SMP = 100：32.5个ADC时钟周期
- SMP = 101：64.5个ADC时钟周期
- SMP = 110：387.5个ADC时钟周期
- SMP = 111：810.5个ADC时钟周期

逐次逼近时间（T_{TSAR}）和分辨率有关，RES[2:0]用于设置分辨率，如：16/14/12/10/8位

逐次逼近时间（T_{TSAR}）和分辨率对应关系表：

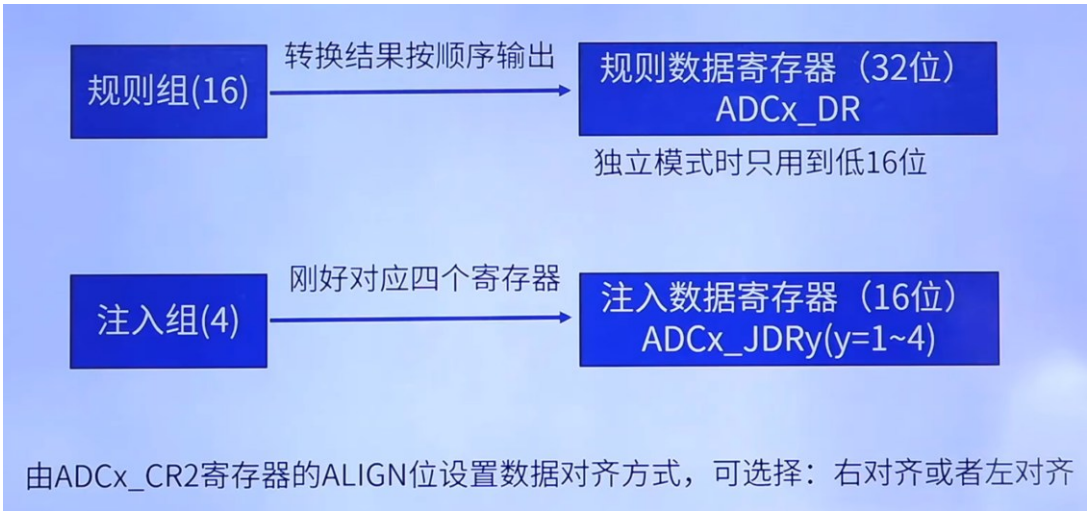
RES	T_{SAR} ADC 时钟周期	T_{SAR} (ns), F_{ADC}=24 MHz	T_{ADC}（ADC 时钟周期）（采样时间 = 1.5 个 ADC 时钟周期）	T_{ADC} (ns), F_{ADC}=24 MHz
16	8.5 个 ADC 时钟周期	354.2	10 个 ADC 时钟周期	416.7
14	7.5 个 ADC 时钟周期	312.5	9 个 ADC 时钟周期	375
12	6.5 个 ADC 时钟周期	270.8	8 个 ADC 时钟周期	333.3
10	5.5 个 ADC 时钟周期	229.2	7 个 ADC 时钟周期	291.7
8	4.5 个 ADC 时钟周期	187.5	6 个 ADC 时钟周期	250.0

举个例子：ADC时钟频率为32MHz时，分辨率为16位，ADC最短的转换时间是多少?

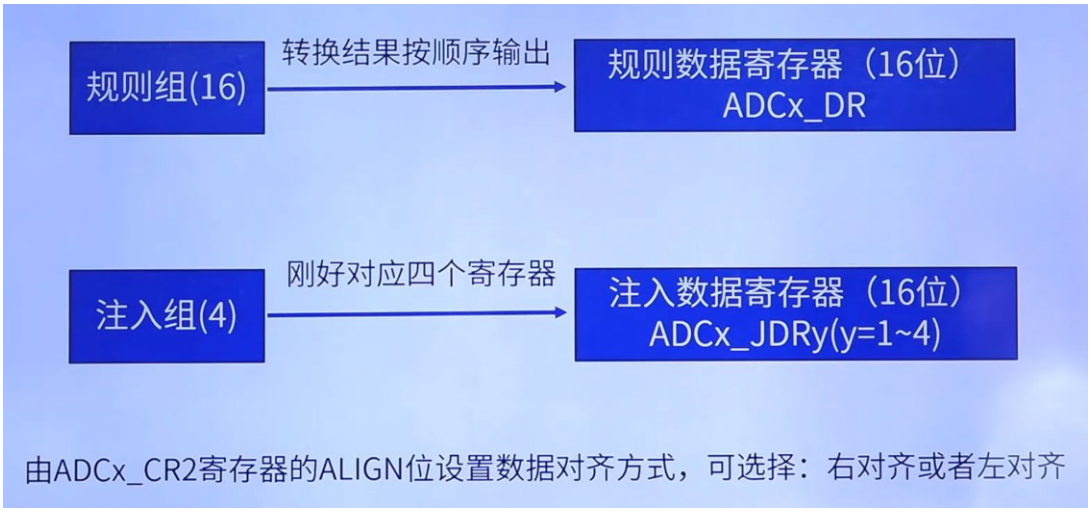
$T_{CONV} = T_{SMPL} + T_{TSAR} = (1.5 + 8.5)$个ADC周期 = 10个ADC周期 = $\left(\frac{1}{32000000}\right) * 10s = 0.31us$

16.2.7 数据寄存器

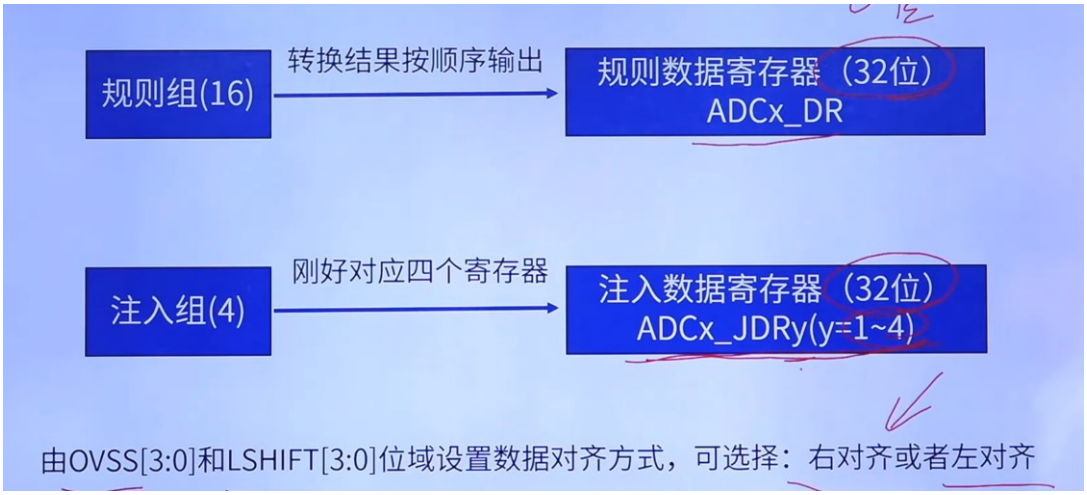
16.2.7.1 F1



16.2.7.2 F4/F7



16.2.7.3 H7



16.2.8 中断

16.2.8.1 F1/F4/F7

F1/F4/F7系列ADC中断事件汇总表：

中断事件	事件标志	使能控制位
规则通道转换结束	EOC	EOCIE
注入通道转换结束	JEOC	JEOCIE
设置了模拟看门狗状态位	AWD	AWDIE
溢出（F1没有）	OVR	OVRIE

DMA请求（只适用于规则组）

规则组每个通道转换结束后，除了可以产生中断外，还可以产生DMA请求，我们利用DMA及时把转换好的数据传输到指定的内存里，防止数据被覆盖。

16.2.8.2 H7

中断事件	事件标志	使能控制位
ADC 就绪	ADRDY	ADRDYIE
结束常规组的转换	EOC	EOCIE
常规组的转换序列结束	EOS	EOSIE
注入组的转换结束	JEOC	JEOCIE
注入组的转换序列结束	JEOS	JEOSIE
模拟看门狗 1 状态位置 1	AWD1	AWD1IE
模拟看门狗 2 状态位置 1	AWD2	AWD2IE
模拟看门狗 3 状态位置 1	AWD3	AWD3IE
采样阶段结束	EOSMP	EOSMPIE
上溢	OVR	OVRIE
注入上下文队列溢出	JQOVF	JQOVFIE

16.2.9 单次转换模式和连续转换模式

CONT位	0	1
转换模式	单次转换模式	连续转换模式

转换组/转换模式	单次转换模式(只触发一次转换)	连续转换模式(自动触发下一次转换) 注意：只有规则组才能触发该模式
规则组	转换结果被储存在ADC_DR EOC（转换结束）标志位被置1 如果设置了EOCIE位，则产生中断 然后ADC停止	转换结果被储存在ADC_DR EOC（转换结束）标志位被置1 如果设置了EOCIE位，则产生中断
注入组	转换结果被储存在ADC_DRJx JEOC（转换结束）标志位被置1 如果设置了JEOCIE位，则产生中断 然后ADC停止	转换结果被储存在ADC_DRJx JEOC（转换结束）标志位被置1 如果设置了JEOCIE位，则产生中断 自动注入：将JAUTO位置1

16.2.10　扫描模式

SCAN位	0	1
扫描模式	关闭扫描模式	使用扫描模式

关闭扫描模式	使用扫描模式
ADC只转换ADC_SQRx或ADC_JSQR选中的第一个通道进行转换	ADC会扫描所有被ADC_SQRx或ADC_JSQR选中的所有通道

16.2.11　不同模式组合的作用

不同模式组合的作用

单次转换模式（不扫描）	只转换一个通道，而且是一次，需等待下一次触发
单次转换模式（扫描）	ADC_SQRx 和ADC_JSQR 选中的所有通道都转换一次
连续转换模式（不扫描）	只会转换一个通道，转换完后会自动执行下一次转换
连续转换模式（扫描）	ADC_SQRx 和ADC_JSQR 选中的所有通道都转换一次，并自动进入下一轮转换

具体例子：

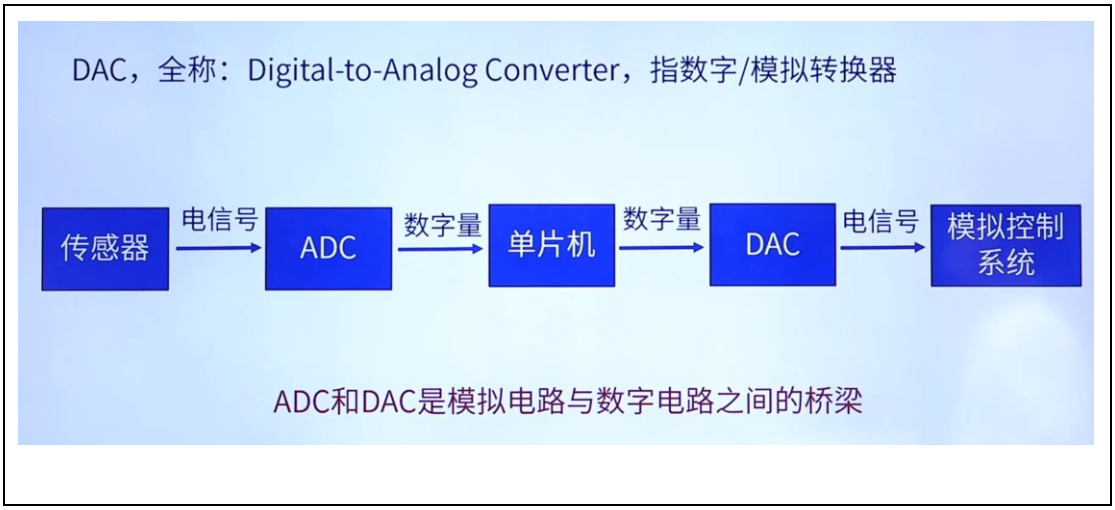
单次转换模式（不扫描）	使用ADC单通道，并要求进行一次转换
单次转换模式（扫描）	使用ADC多通道，并要求所有通道都转换一次就停止
连续转换模式（不扫描）	使用ADC单通道，并要求对该通道连续转换
连续转换模式（扫描）	使用ADC多通道，并要求所有通道都转换一次后，自动启动下一轮转换

一种比较少用的模式：不连续采样模式（F1手册称为：间断模式），只适用在扫描模式下。

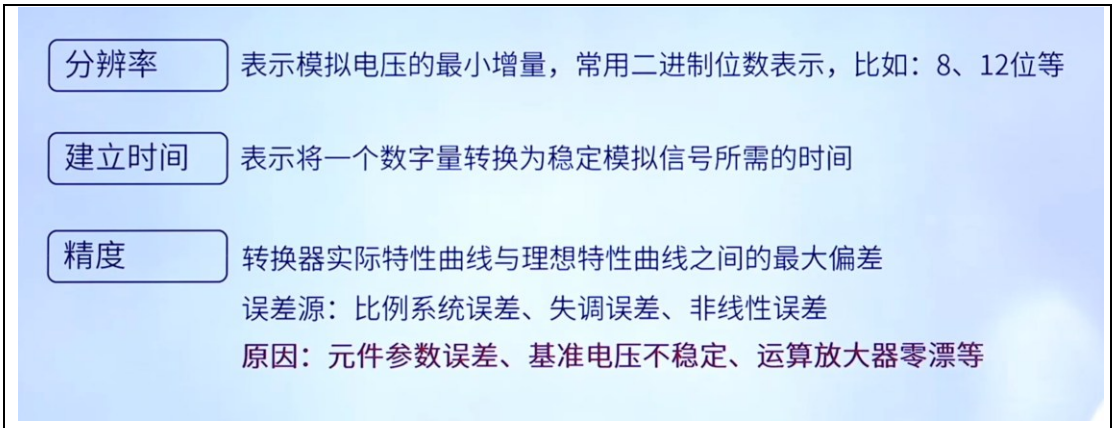
17 DAC

17.1 DAC 简介

17.1.1 什么是 DAC



17.1.2 DAC 的特性参数



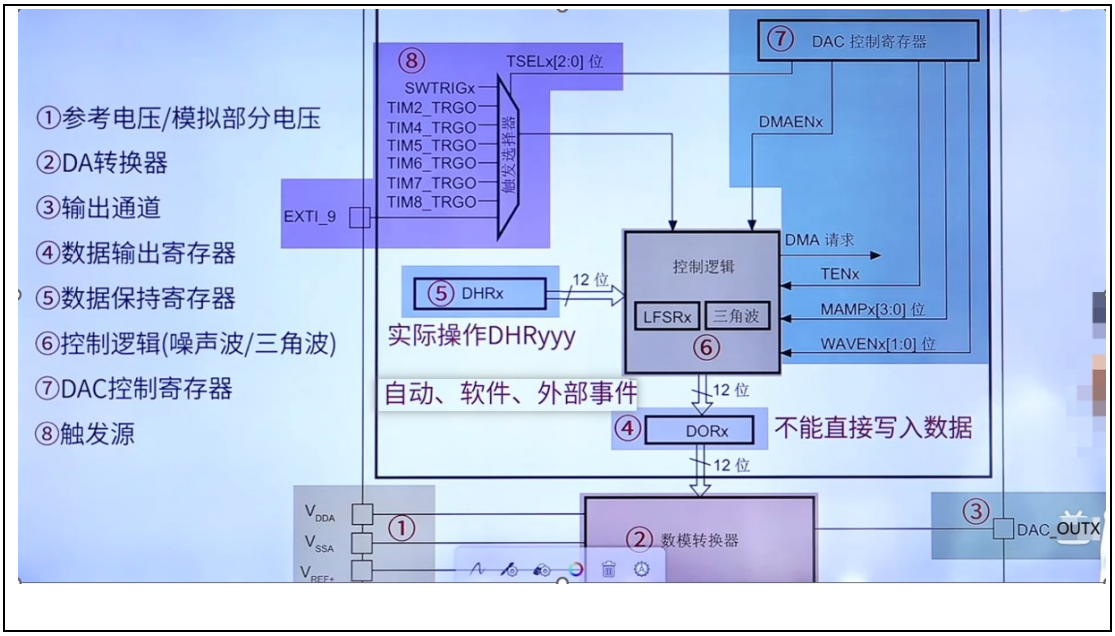
17.1.3 STM32 各个系列 DAC 的主要特性

主要特性	F1	F4	F7	H7
DAC输出类型	电压输出型			
分辨率	8/12位			
DAC时钟频率	36MHz(APB1)	42MHz(APB1) 45MHz(APB1)	54MHz(APB1)	120MHz(APB1)
建立时间	3us			1.7us
供电电压	V_{SSA}：0V，V_{DDA}：2.4V~3.6V			
参考电压	V_{REF-}：0V，V_{REF+}一般为3.3V			
输出通道	DAC_OUT1（PA4）、DAC_OUT2（PA5） 为了避免寄生电流消耗，PA4和PA5配置为模拟功能 每个通道都有单独的转换器，可同时或单独转换，每个通道都有DMA功能			

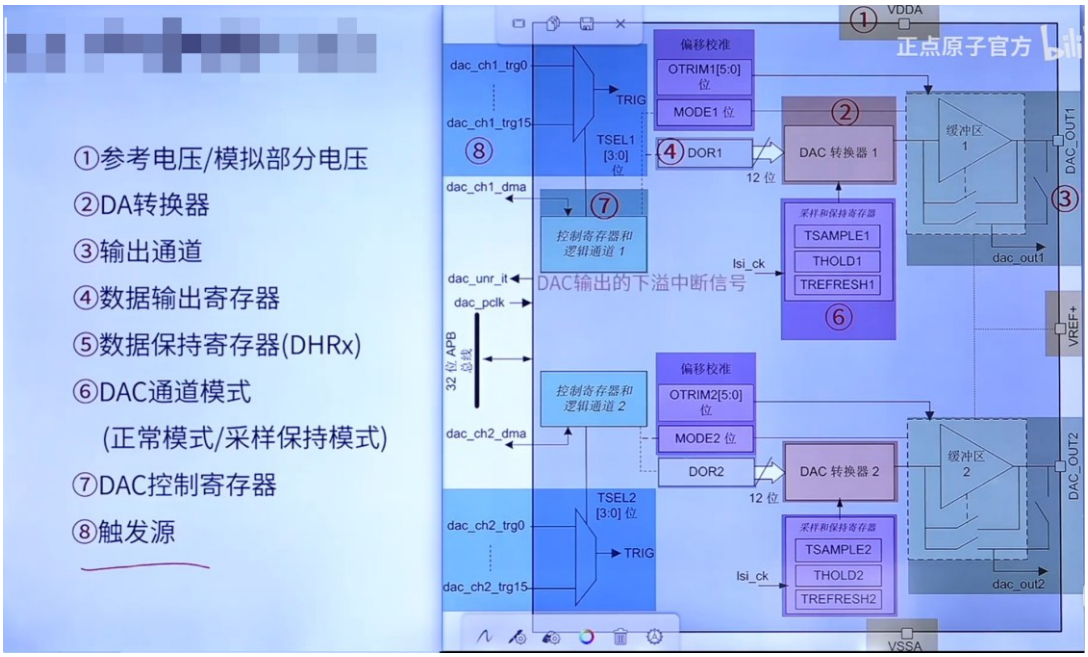
17.2 DAC 工作原理

17.2.1 ADC 框图简介（F1/F4/F7/H7）

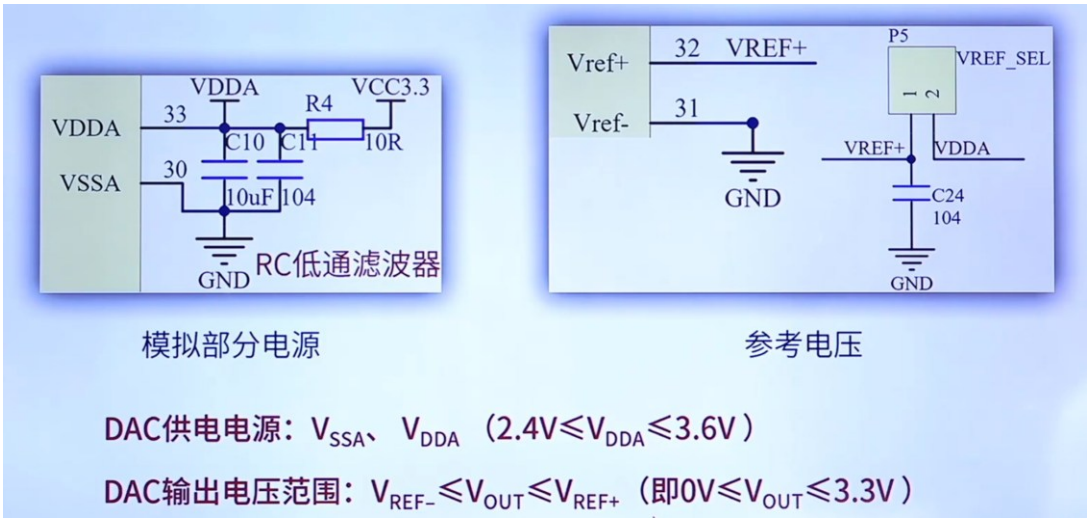
17.2.1.1 （F1/F4/F7）



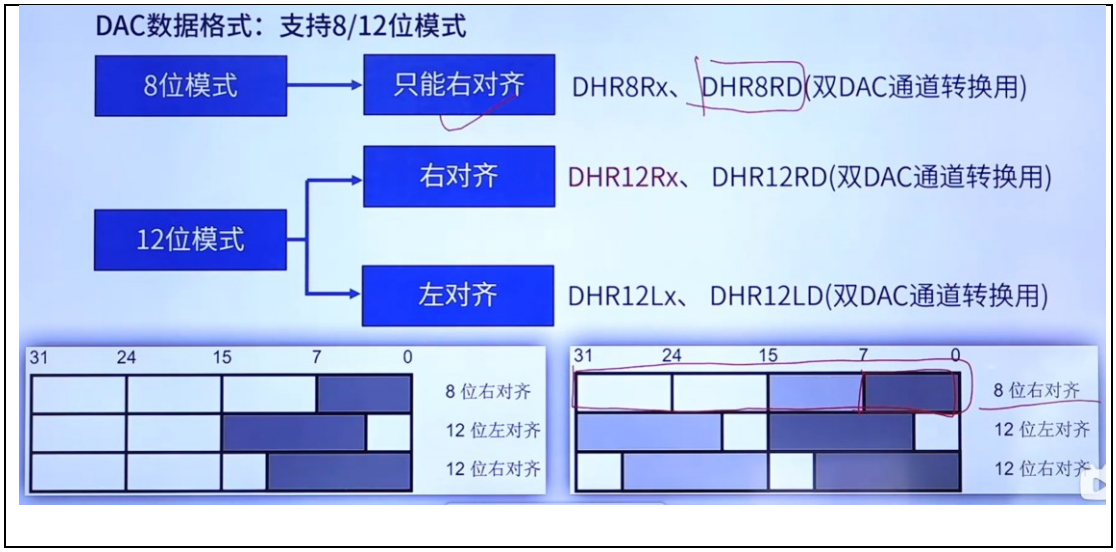
17.2.1.2 H7



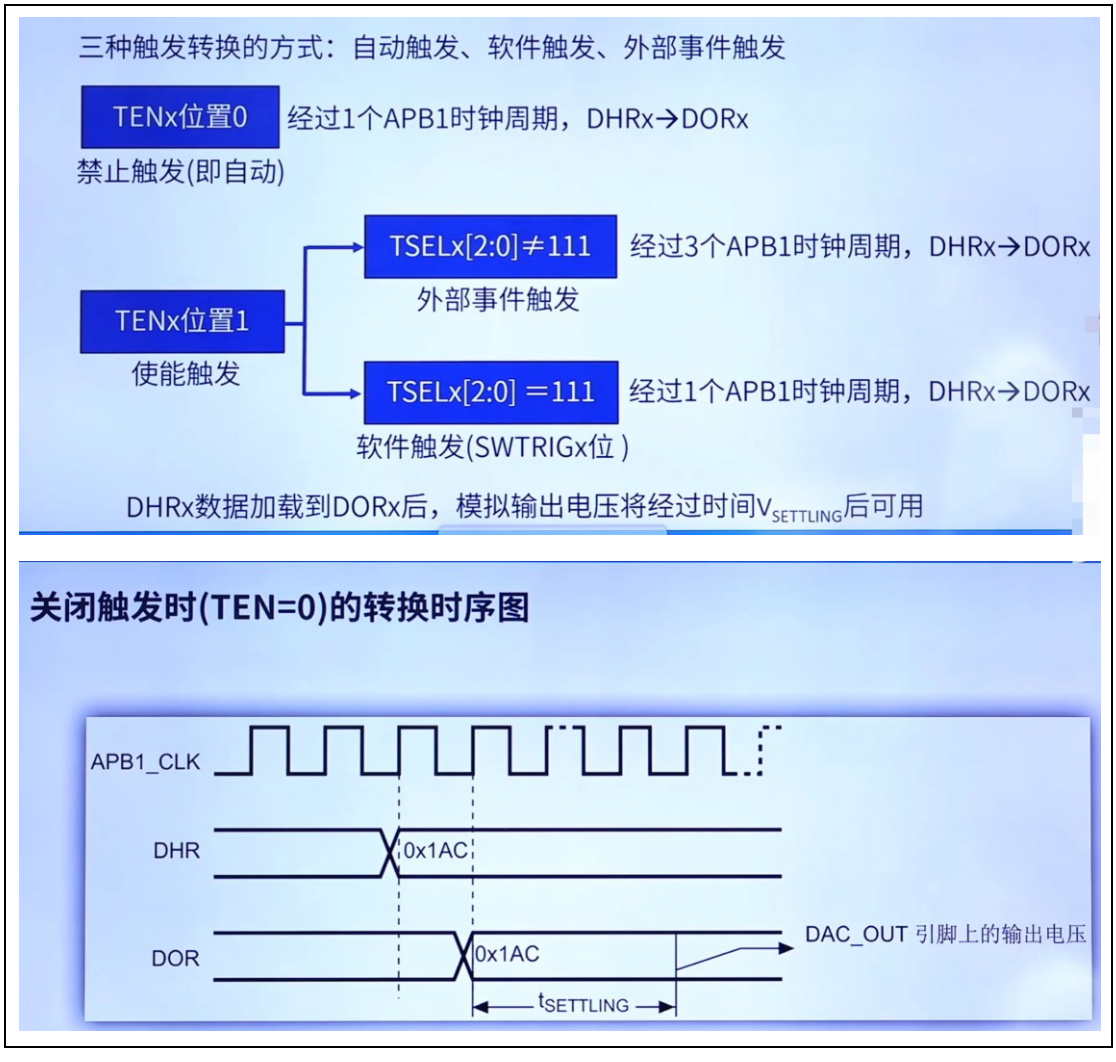
17.2.2 参考电压/模拟部分电压（F1）



17.2.3 DAC 数据格式



17.2.4 触发源



17.2.5 DNA 请求

17.2.6 DAC 输出电压

12位模式下，DAC输出电压计算方法：

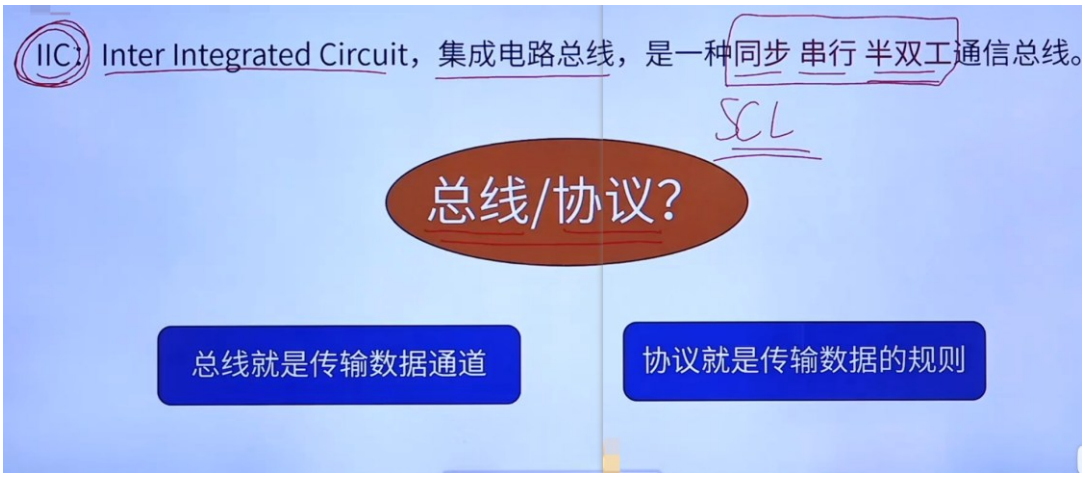
$$DAC输出电压 = \left(\frac{DORx}{4096}\right) * V_{REF+}$$

8位模式下，DAC输出电压计算方法：

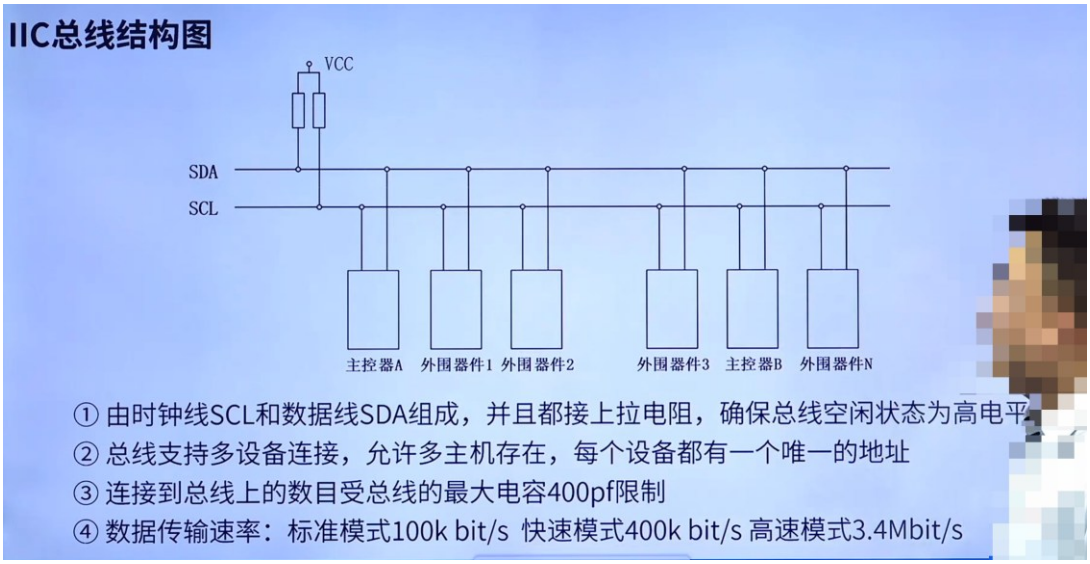
$$DAC输出电压 = \left(\frac{DORx}{256}\right) * V_{REF+}$$

18 IIC

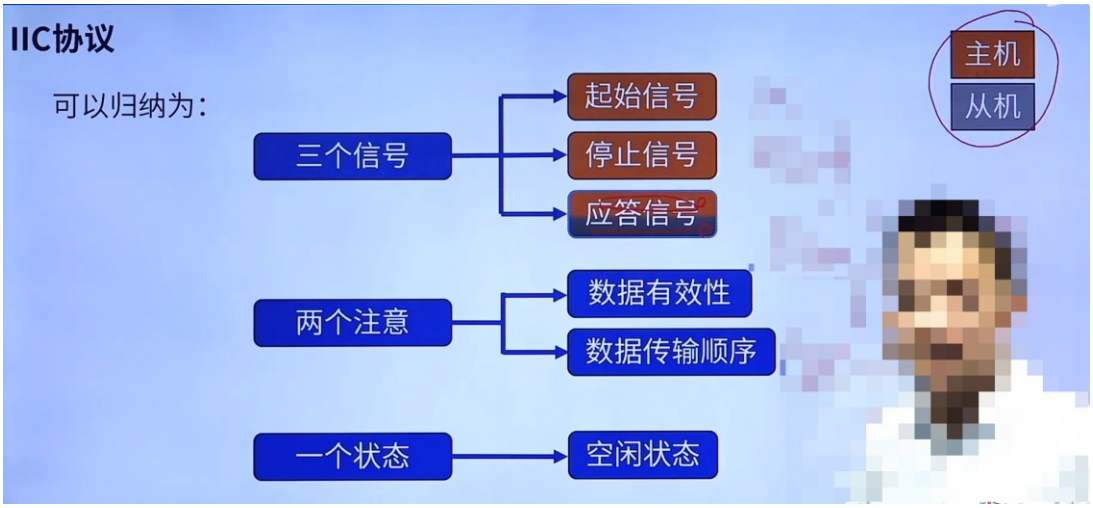
18.1 IIC 总线协议介绍



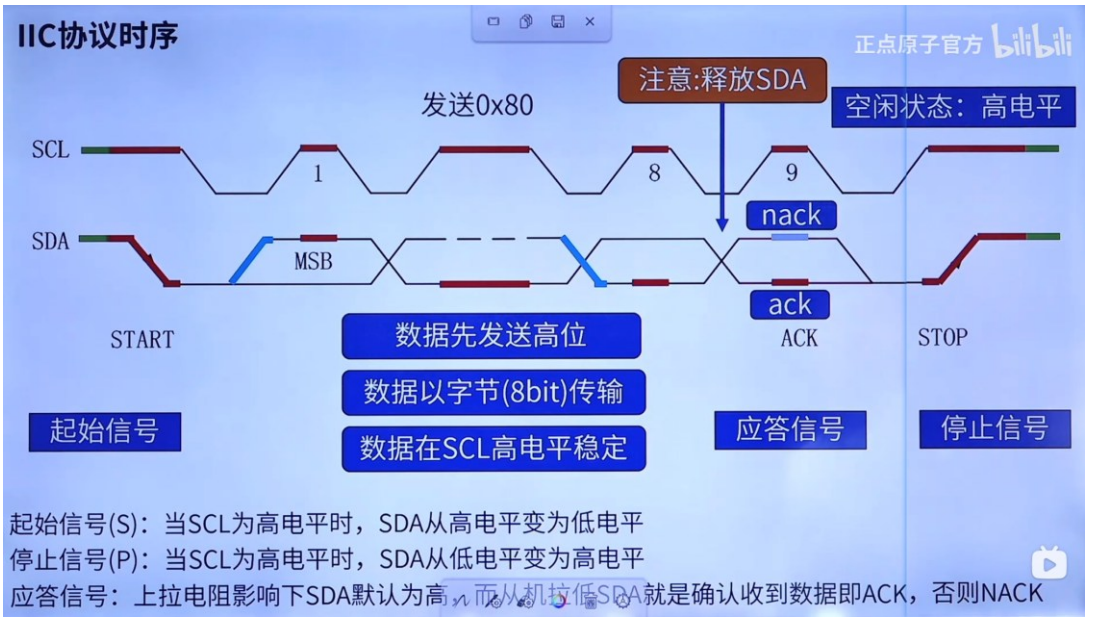
18.2 IIC 总线结构图

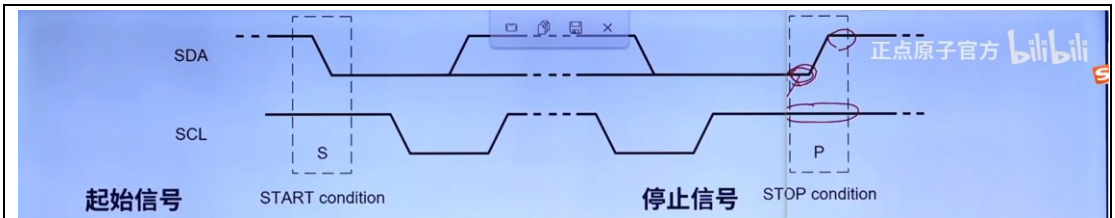


18.3 IIC 协议



18.4 IIC 协议时序





起始信号

```
void iic_start(void)
{ /* SCL为高电平期间, SDA从高电平往低电平跳变*/
    IIC_SDA ( 1 );
    IIC_SCL ( 1 );
    iic_delay( );
    IIC_SDA ( 0 );
    iic_delay( );
    IIC_SCL ( 0 );
    iic_delay( ); /* 钳住总线, 准备发送/接收数据 */
```

停止信号

```
void iic_stop(void)
{ /* SCL为高电平期间, SDA从低电平往高电平跳变*/
    IIC_SDA ( 0 );
    iic_delay( );
    IIC_SCL ( 1 );
    iic_delay( );
    IIC_SDA ( 1 );    /* 发送总线停止信号*/
    iic_delay( );
}
```

检测应答信号

```
uint8_t iic_wait_ack (void) /* return 1:fail 0:succeed*/
{
    IIC_SDA (1); /* 主机释放SDA线 */
    iic_delay( );
    IIC_SCL (1); /* 从机返回ACK*/
    iic_delay( );
    if ( IIC_READ_SDA ) /* SCL高电平读取SDA状态*/
    {
        iic_stop();    /* SDA高电平表示从机nack */
        return 1;
    }
    IIC_SCL(0); /* SCL低电平表示结束ACK检查 */
    iic_delay( );
    return 0;
}
```

发送应答信号

```
void iic_ack(void)
{
    IIC_SCL (0);
    iic_delay( );
    IIC_SDA (0); /* 数据线为低电平，表示应答 */
    iic_delay( );
    IIC_SCL (1);
    iic_delay( );
}
```

发送非应答信号

```
void iic_nack(void)
{
    IIC_SCL (0);
    iic_delay( );
    IIC_SDA (1); /* 数据线为高电平,表示非应答 */
    iic_delay( );
    IIC_SCL (1);
    iic_delay( );
```

发送1字节数据

```
void iic_send_byte(uint8_t data)
{
    for (uint8_t t = 0; t < 8; t++)
    {    /* 高位先发 */
        IIC_SDA((data & 0x80) >> 7);
        iic_delay( );
        IIC_SCL ( 1 );
        iic_delay( );
        IIC_SCL ( 0 );
        data <<= 1; /* 左移1位, 用于下一次发送 */
    }
    IIC_SDA ( 1 );    /* 发送完成,主机释放SDA线 */
}
```

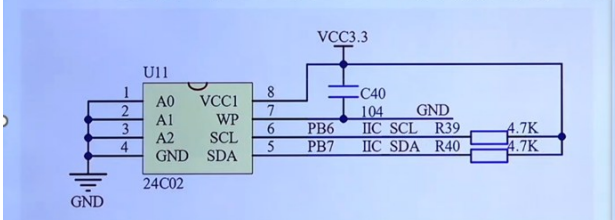
读取1字节数据

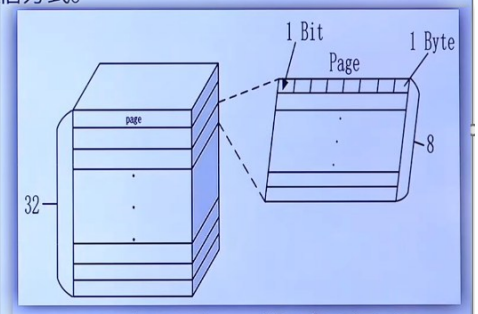
```
uint8_t iic_read_byte (uint8_t ack) /* 1:ack 0:nack*/
{
    uint8_t receive = 0 ;
    for (uint8_t t = 0; t < 8; t++)
    {    /* 高位先输出, 先收到的数据位要左移 */
        receive <<= 1;
        IIC_SCL ( 1 );
        iic_delay( );
        if ( IIC_READ_SDA ) receive++;
        IIC_SCL ( 0 );
        iic_delay( );
    }
    if ( !ack ) iic_nack();
    else iic_ack();
    return receive;
```

18.5 AT24C02 介绍

EEPROM是一种掉电后数据不丢失的储存器，常用来存储一些配置信息，在系统重新上电时就可以加载。

AT24C02是一个2K bit的EEPROM存储器，使用IIC通信方式。





A0/1/2 : 设备地址决定引脚

WP : 写保护引脚

SCL : 时钟线

SDA : 数据线

24C02由32页(page)组成且每页8Byte

AT24Cxx	容量（bit）	页数	页内字节数
AT24C01	1K bit (128 B)	16	8 Byte
AT24C02	2K bit (256 B)	32	8 Byte
AT24C04	4K bit (512 B)	32	16 Byte
AT24C08	8K bit (1024 B)	64	16 Byte
AT24C16	16K bit (2048 B)	128	16 Byte
AT24C32	32K bit (4096 B)	128	32 Byte
AT24C64	64K bit (8192 B)	256	32 Byte
AT24C128	128K bit (16384 B)	256	64 Byte
AT24C256	256K bit (32768 B)	512	64 Byte
AT24C512	512K bit (65535 B)	512	128 Byte

AT24C02通讯地址

通讯地址

不可编程部分 1010

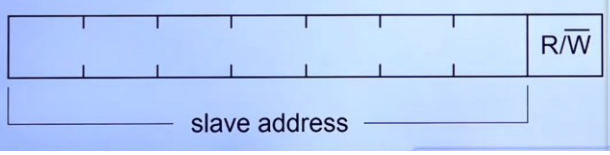
可编程部分 由硬件管脚A0/1/2决定

设备地址

数据传输方向 读数据‘1’还是写数据‘0’

写操作地址：0xA0 读操作地址：0xA1

MSB LSB

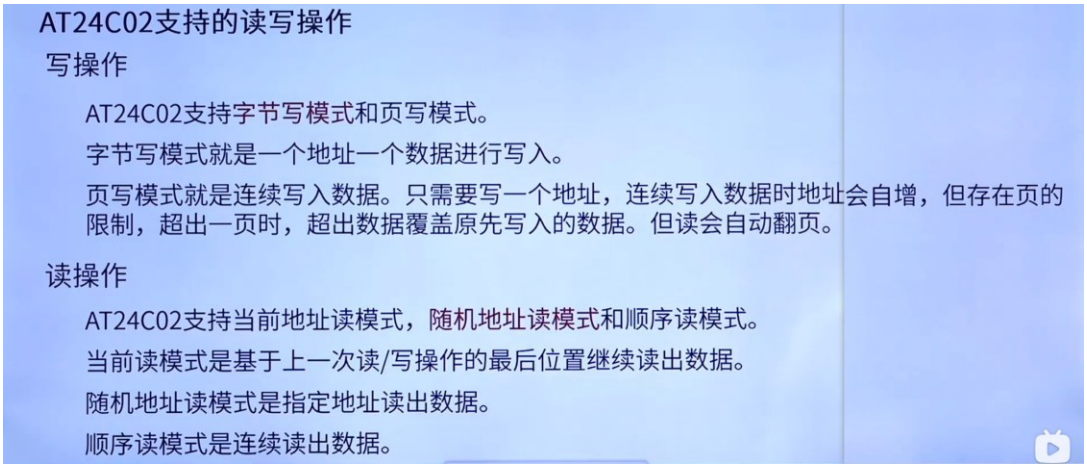


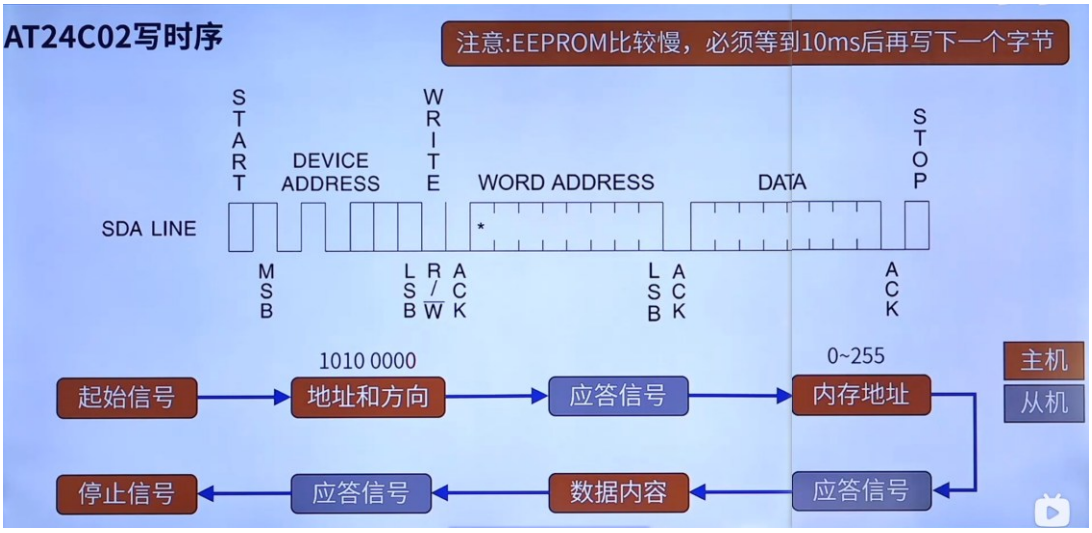
1K/2K	1	0	1	0	A2	A1	A0	R/W
4K	1	0	1	0	A2	A1	P0	R/W
8K	1	0	1	0	A2	P1	P0	R/W
16K	1	0	1	0	P2	P1	P0	R/W

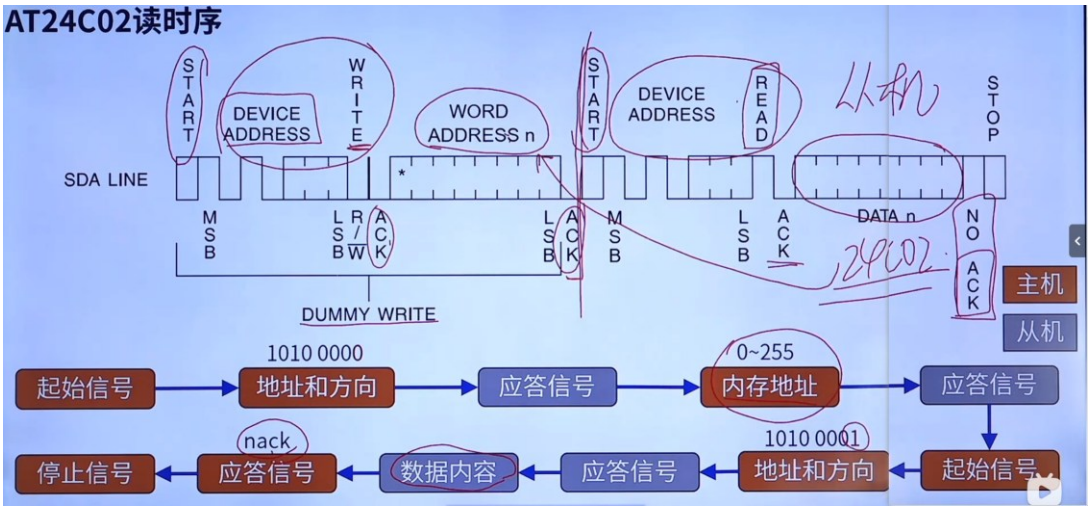
MSB LSB

P0, P1, P2 = Data word address bits

18.6 AT24C02 读写时序







18.7 硬件和软件 IIC 对比

硬件和软件IIC对比

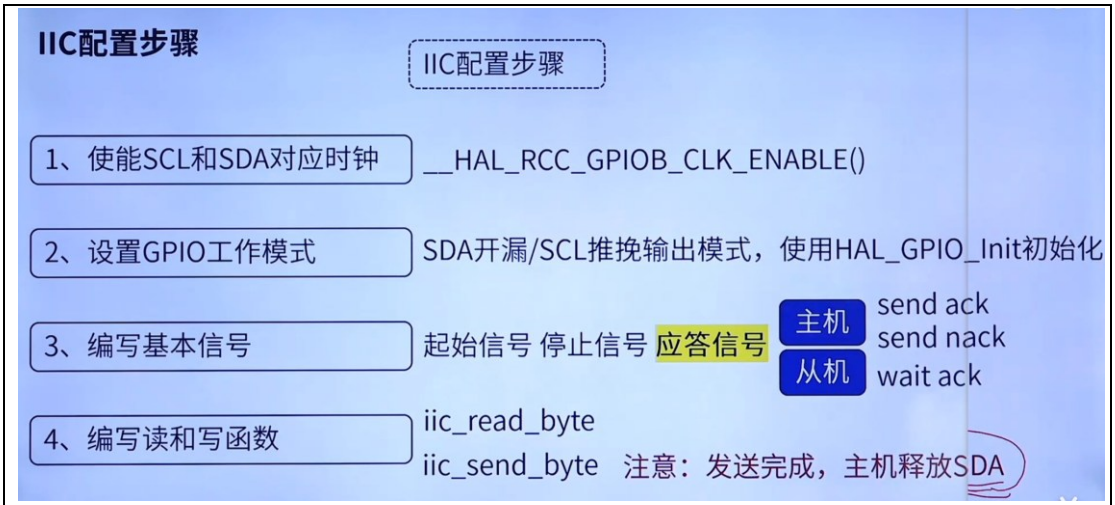
IIC	用法	速度	稳定性	管脚
硬件IIC	比较复杂	快	较稳定	需使用特定管脚
软件IIC	操作过程比较清晰	较慢	稳定	任意管脚，比较灵活

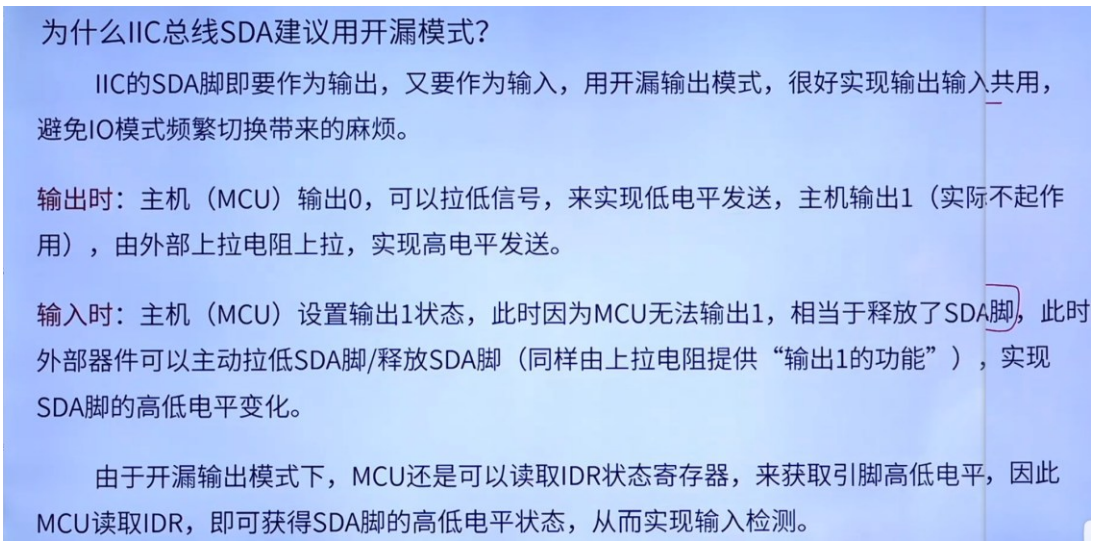
18.8 AT24C02 模块与各开发板引脚硬件连接说明

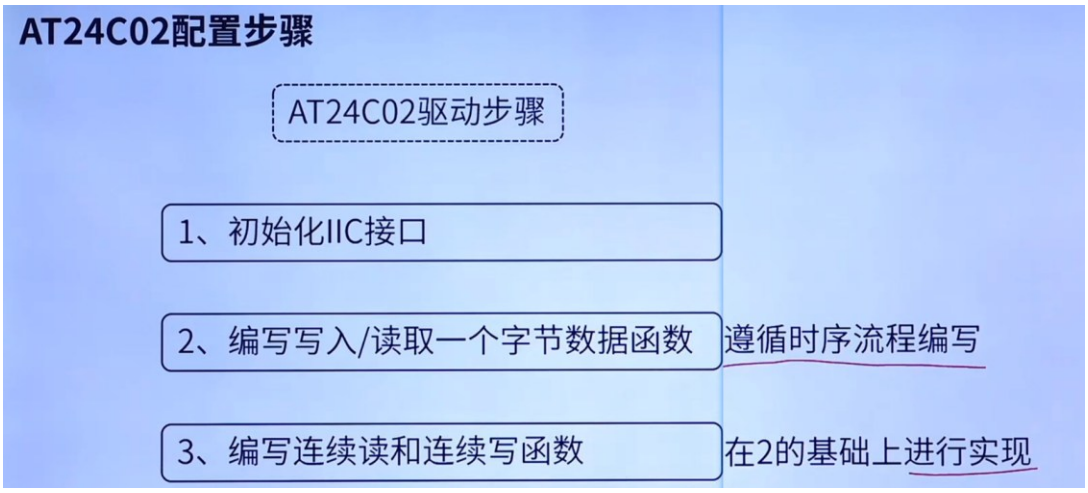
AT24C02模块与各开发板引脚硬件连接说明

分类	AT24C02模块引脚			
	供电脚（3.3V）	时钟线（SCL）	数据线（SDA）	地（GND）
MINI板	VCC	PC12	PC11	GND
精英板	VCC	PB6	PB7	GND
战舰板	VCC	PB6	PB7	GND
探索板	VCC	PB8	PB9	GND
阿波罗	VCC	PH4	PH5	GND
北极星	VCC	PH4	PH5	GND
MINIPro H750	VCC	PB10	PB11	GND

18.9 AT24C02 驱动步骤

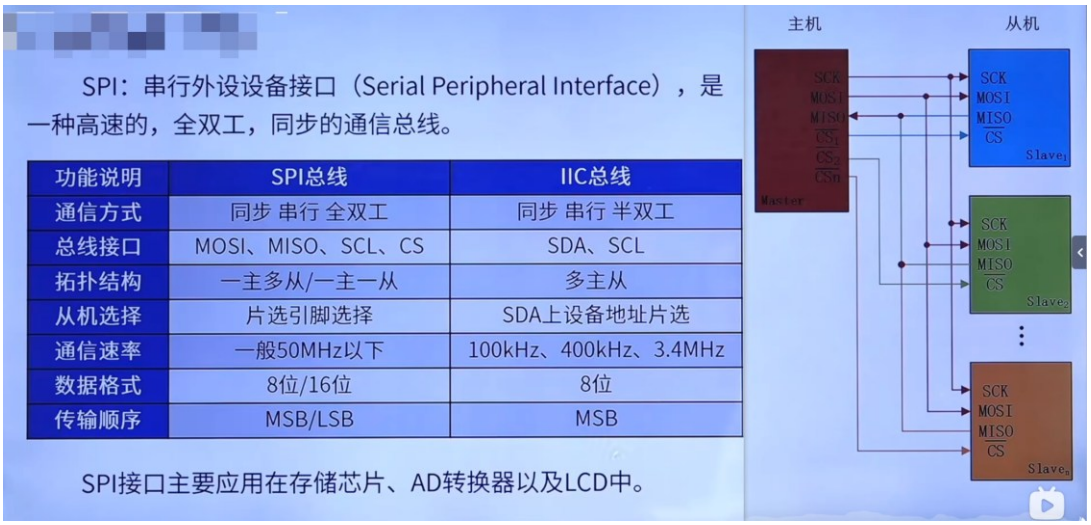




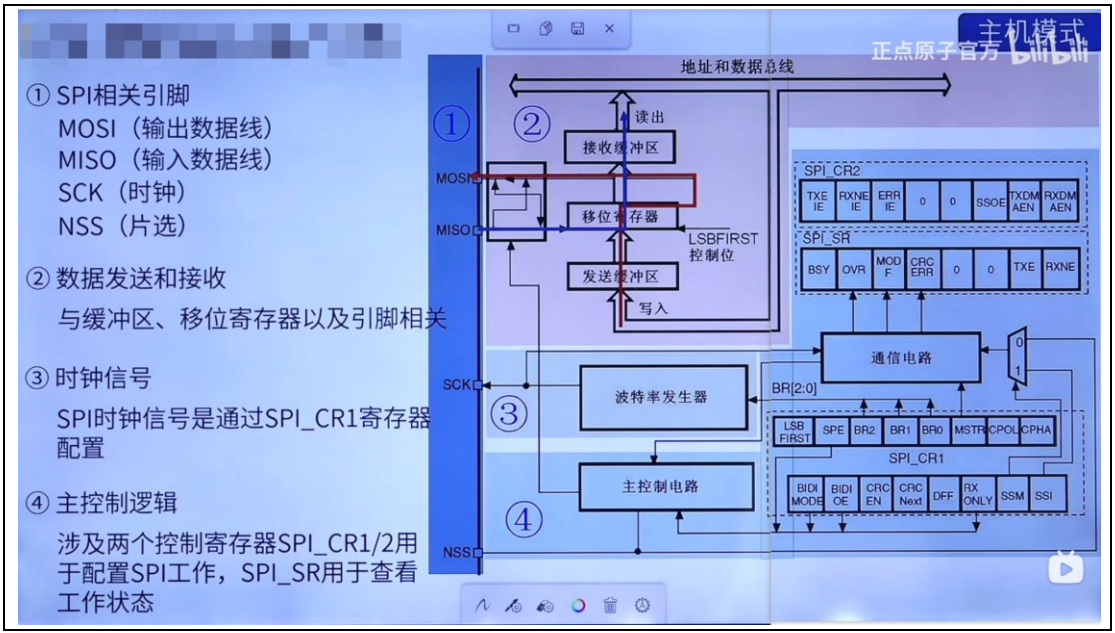


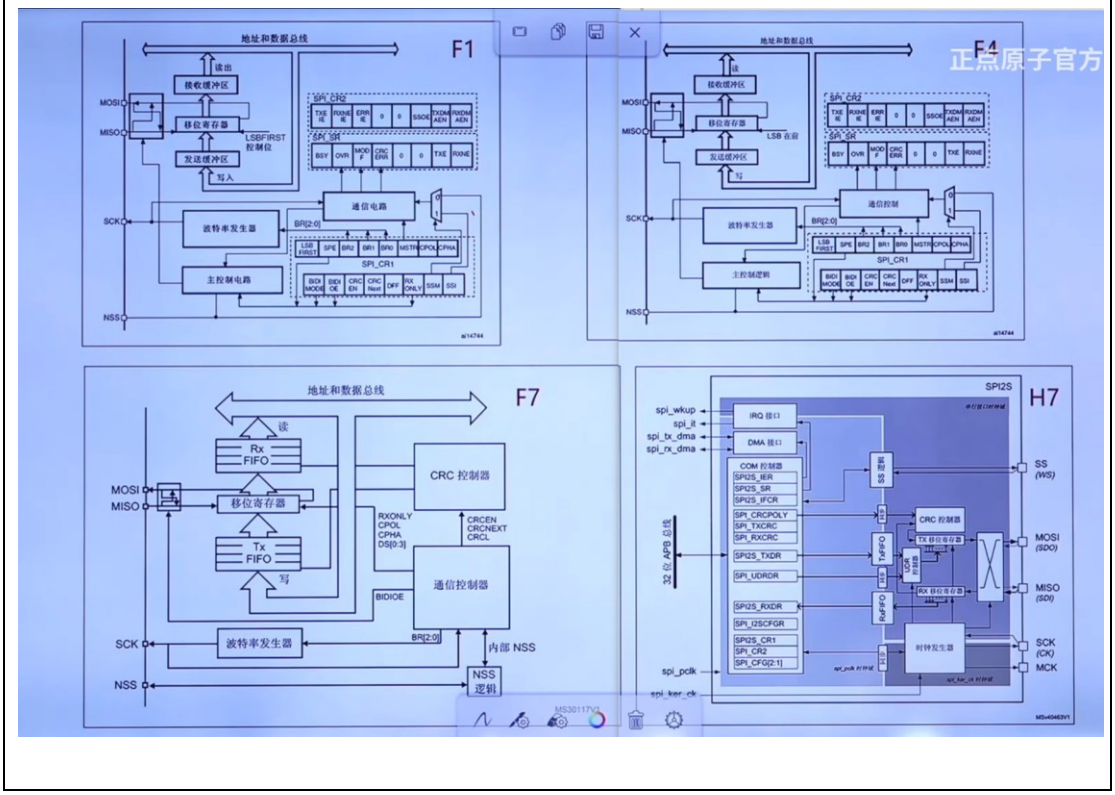
19 SPI

19.1 SPI 介绍



19.2 SPI 结构框图介绍





19.3 SPI 外设对应的引脚

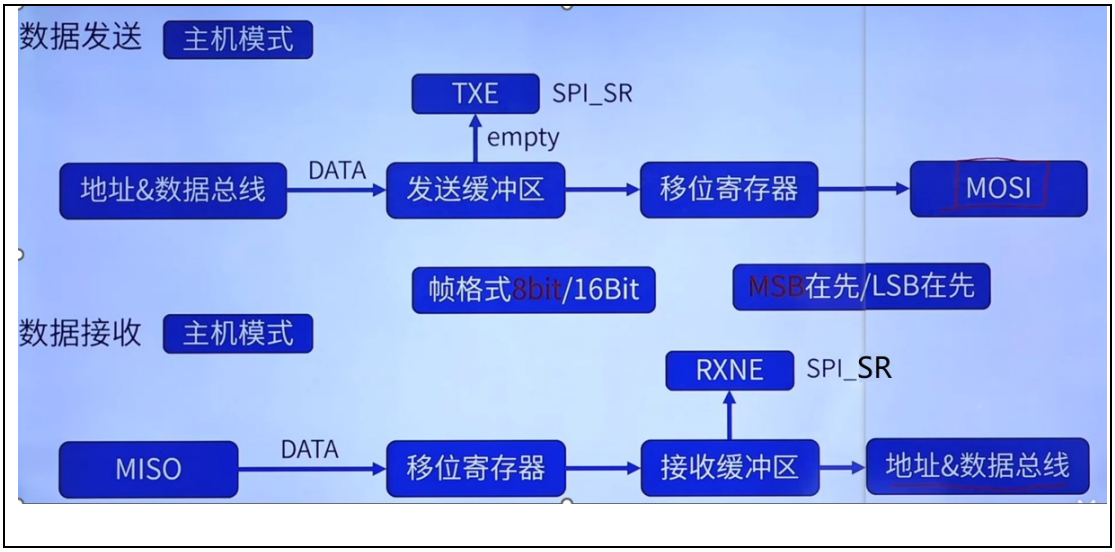
STM32芯片有多个SPI外设，每个SPI外设输出的信号会到不同的GPIO口。

STM32F1有三个SPI。

	引脚	SPI1	SPI2	SPI3
F1	NSS	PA4	PB12	PA15
	CLK	PA5	PB13	PB3
	MISO	PA6	PB14	PB4
	MOSI	PA7	PB15	PB5

F4 / F7 / H7 系列芯片SPI外设都有6个，分别为SPI1、SPI2、SPI3、SPI4、SPI5和SPI6。

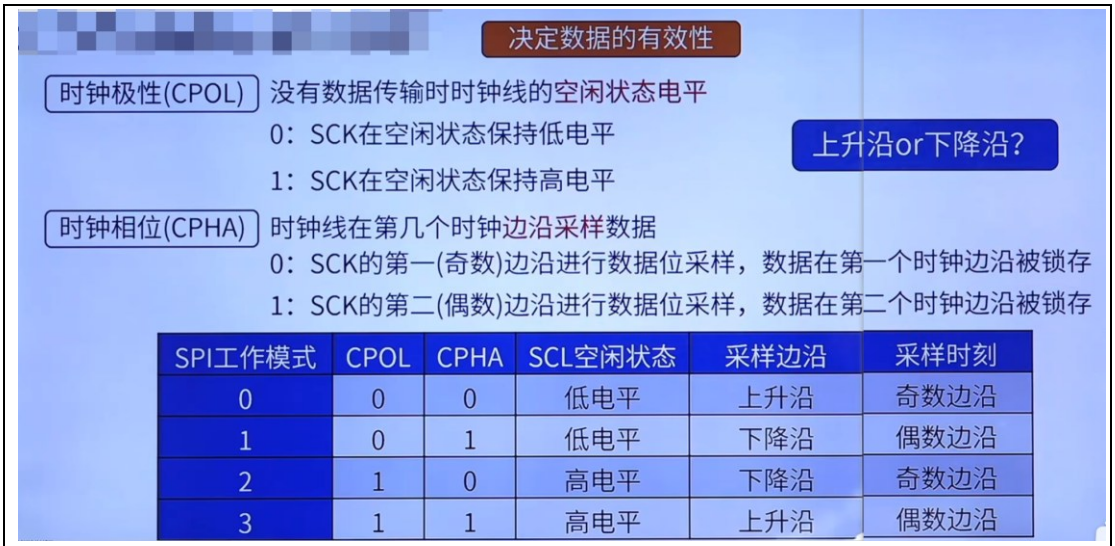
不同系列的SPI外设用到的引脚是不一样的，具体可以查看芯片的datasheet。


数据发送
主机模式
TXE
SPI_SR
empty
地址&数据总线
DATA
发送缓冲区
移位寄存器
MOSI
帧格式8bit/16Bit
MSB在先/LSB在先
数据接收
主机模式
RXNE
SPI_SR
MISO
DATA
移位寄存器
接收缓冲区
地址&数据总线

19.4 SPI 工作原理

SPI工作原理
MOSI
怎么移?
正点原子官方
MISO
MSB
主机
移位寄存器
0 1 0 1 0 1 0 0
0xAA
0x55
移位寄存器
1 1 0 1 0 1 0 1
从机
MSB
主机只向从机进行写操作，可以忽略接收到的从机数据。
主机要对从机进行读操作，需要发送一个空数据来引发从机发送数据。

19.5 SPI 工作模式介绍



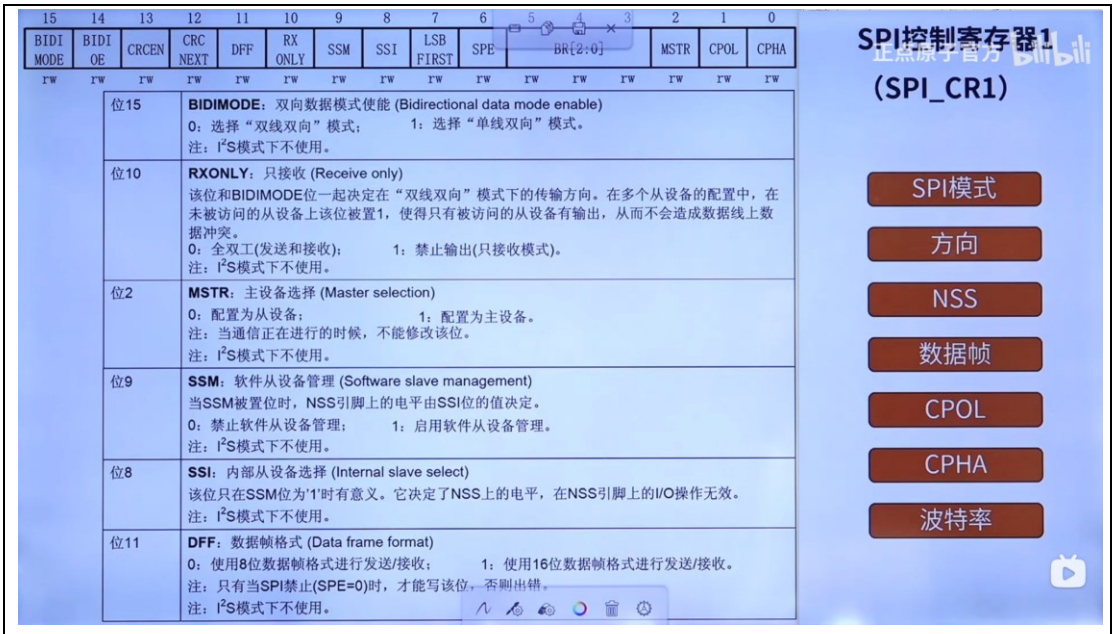
SPI工作模式	CPOL	CPHA	SCL空闲状态	采样边沿	采样时刻
0	0	0	低电平	上升沿	奇数边沿
1	0	1	低电平	下降沿	偶数边沿
2	1	0	高电平	下降沿	奇数边沿
3	1	1	高电平	上升沿	偶数边沿

19.6 SPI 相关寄存器介绍（F1/F4/F7）

寄存器	名称	作用
SPI_CR1	SPI控制寄存器1	用于配置SPI工作参数
SPI_SR	SPI状态寄存器	用于查询当前SPI传输状态（TXE、RXNE）
SPI_DR	SPI数据寄存器	用于存放待发送数据或接收数据，有两个缓冲区（TX/RX）

对于H7来说，用到的寄存器有所差异，相关寄存器如下：

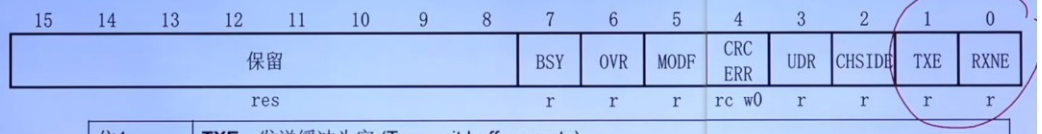
SPI时钟源选择	RCC_D2CCIP1R
使能SPI	SPI_CR1
配置SPI工作参数	SPI_CFG1（SPI传输速率以及帧格式）
配置SPI工作参数	SPI_CFG2（SPI工作模式 / NSS引脚 / 传输顺序等）
选择SPI模式	SPI_I2SCFGR（SPI模式）
查看SPI传输状态	SPI_SR
发送数据寄存器	SPI_TXDR（与TxFIFO连接）
接收数据寄存器	SPI_RXDR（与RxFIFO连接）



续上表

位7	**LSBFIRST**：帧格式 (Frame format) 0：先发送MSB； 1：先发送LSB。
位6	**SPE**：SPI使能 (SPI enable) 0：禁止SPI设备； 1：开启SPI设备。 注：当关闭SPI设备时，请按照第23.3.8节的过程操作。
位5:3	**BR[2:0]**：波特率控制 (Baud rate control) 000：$f_{PCLK}/2$ 001：$f_{PCLK}/4$ 010：$f_{PCLK}/8$ 011：$f_{PCLK}/16$ 100：$f_{PCLK}/32$ 101：$f_{PCLK}/64$ 110：$f_{PCLK}/128$ 111：$f_{PCLK}/256$
位1	**CPOL**：时钟极性 (Clock polarity) 0： 空闲状态时，SCK保持低电平； 1： 空闲状态时，SCK保持高电平。
位0	**CPHA**：时钟相位 (Clock phase) 0： 数据采样从第一个时钟边沿开始； 1： 数据采样从第二个时钟边沿开始。 注：当通信正在进行的时候，不能修改该位。

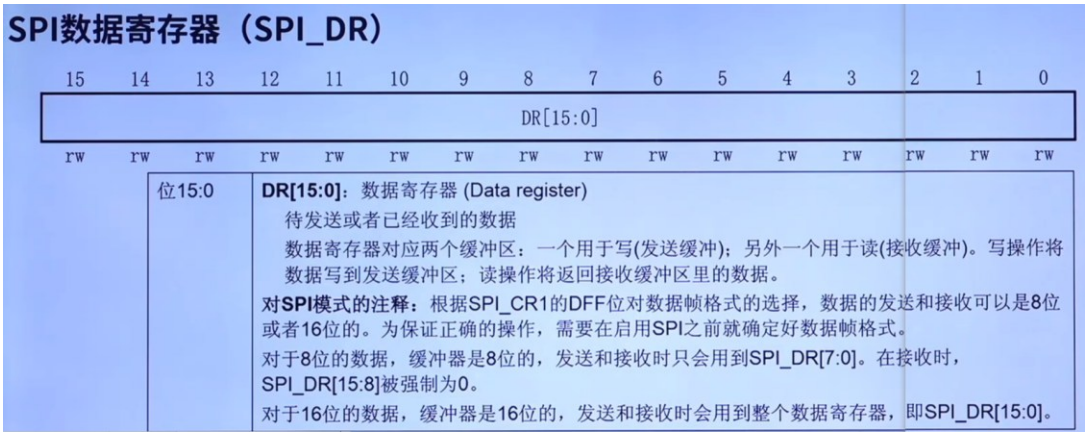
SPI状态寄存器（SPI_SR）



位1	**TXE**：发送缓冲为空 (Transmit buffer empty) 0：发送缓冲非空； 1：发送缓冲为空。
位0	**RXNE**：接收缓冲非空 (Receive buffer not empty) 0：接收缓冲为空； 1：接收缓冲非空。

当TXE位置1时，表示数据已经全部发送出去

当RXE位置1时，表示已经接收到数据



19.7 SPI 相关 HAL 驱动介绍(F1/F4/F7/H7)

驱动函数	关联寄存器	功能描述
__HAL_RCC_SPIx_CLK_ENABLE(…)	RCC_APB2ENR	使能SPIx时钟
HAL_SPI_Init(…)	SPI_CR1	初始化SPI
HAL_SPI_MspInit(…)	初始化回调	初始化SPI相关引脚
HAL_SPI_Transmit(…)	SPI_DR/SPI_SR	SPI发送
HAL_SPI_Receive(…)	SPI_DR/SPI_SR	SPI接收
HAL_SPI_TransmitReceive(…)	SPI_DR/SPI_SR	SPI接收发送
__HAL_SPI_ENABLE(…)	SPI_CR1(SPE)	使能SPI外设
__HAL_SPI_DISABLE(…)	SPI_CR1(SPE)	失能SPI外设

对于H7来说，还需要HAL_RCCEx_PeriphCLKConfig函数设置时钟源。

SPI外设相关结构体：SPI_HandleTypeDef 和 SPI_InitTypeDef

SPI_HandleTypeDef

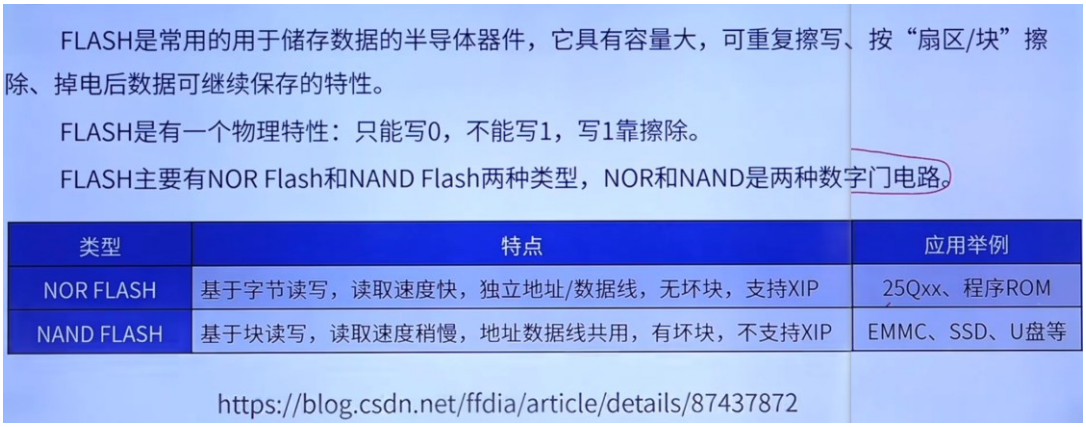
SPI_TypeDef	*Instance
SPI_InitTypeDef	Init
帧格式 DataSize	F1(8/16bit) F4(8/16bit) F7(4~16bit) H7(4~32bit)

SPI_InitTypeDef

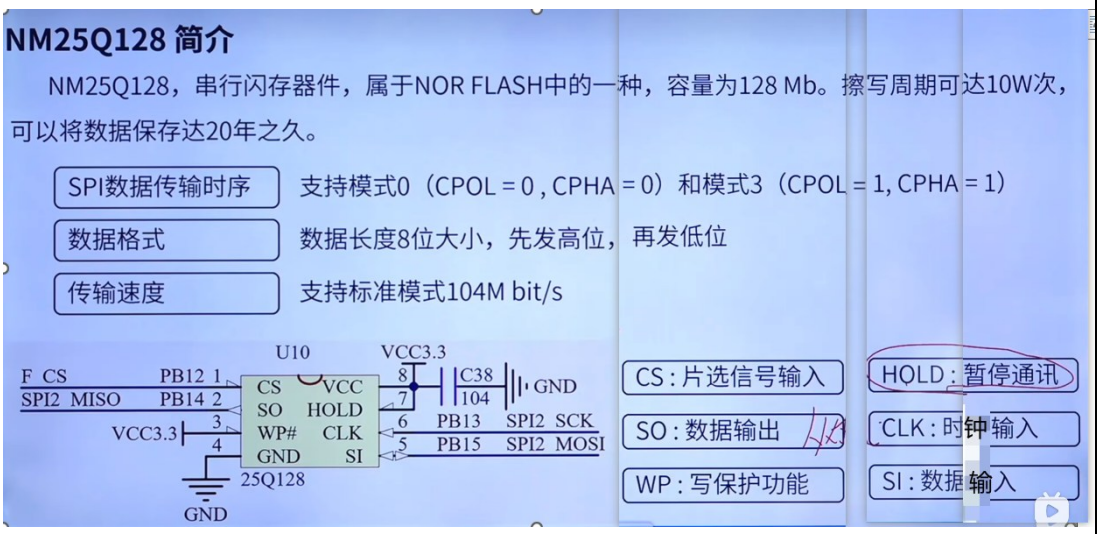
```
uint32_t Mode                /* SPI模式（主机） */
uint32_t Direction           /* 工作方式（全双工） */
uint32_t DataSize            /* 帧格式（8位） */
uint32_t CLKPolarity         /* 时钟极性（CPOL = 0） */
uint32_t CLKPhase            /* 时钟相位（CPHA = 0） */
uint32_t NSS                 /* SS控制方式（软件） */
uint32_t BaudRatePrescaler   /* SPI波特率预分频值 */
uint32_t FirstBit            /* 数据传输顺序（MSB） */
uint32_t TIMode              /* 帧格式：Motorola / TI */
uint32_t CRCCalculation      /* 设置硬件CRC校验 */
uint32_t CRCPolynomial       /* 设置CRC校验多项式 */
…
(对于F7/H7来说，还有一些附加功能相关成员(NSS/CRC/IOSwap))
```

19.8 NOR FLASH

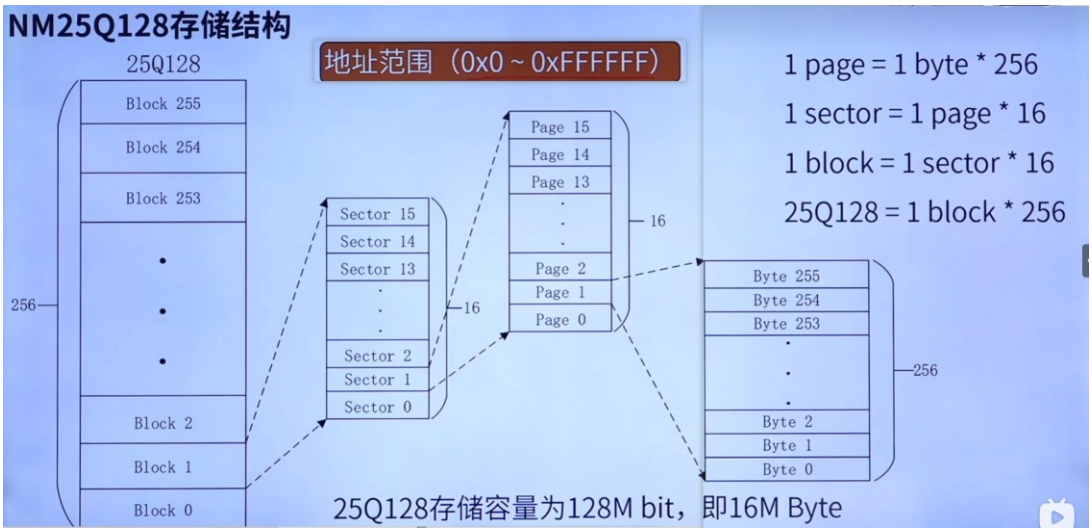
19.8.1 NOR FLASH 介绍



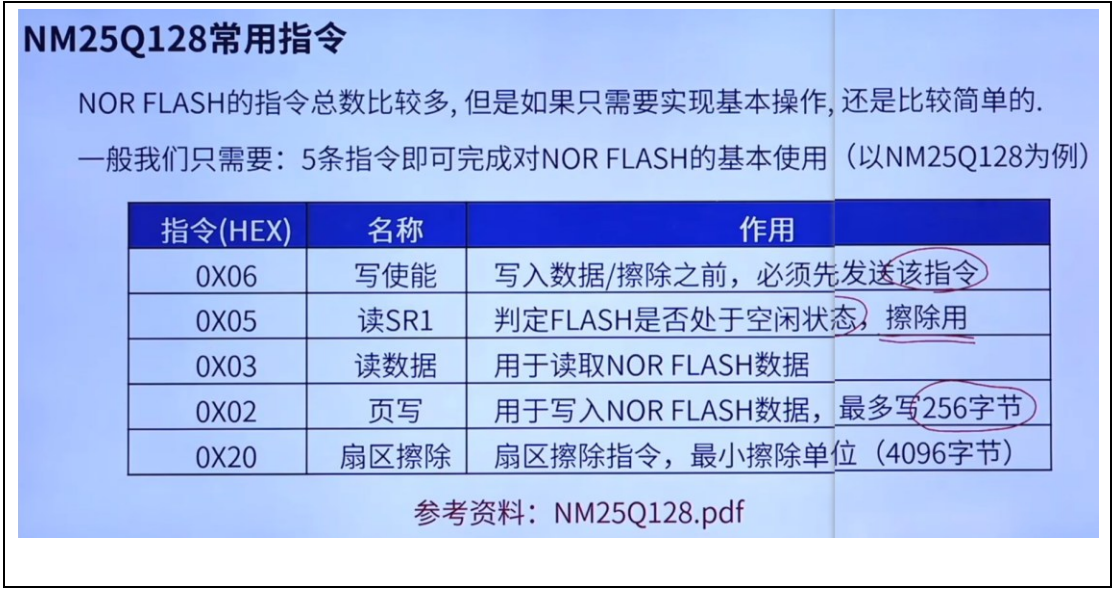
19.8.2 NM25Q128 简介



19.8.3 NM25Q128 存储结构

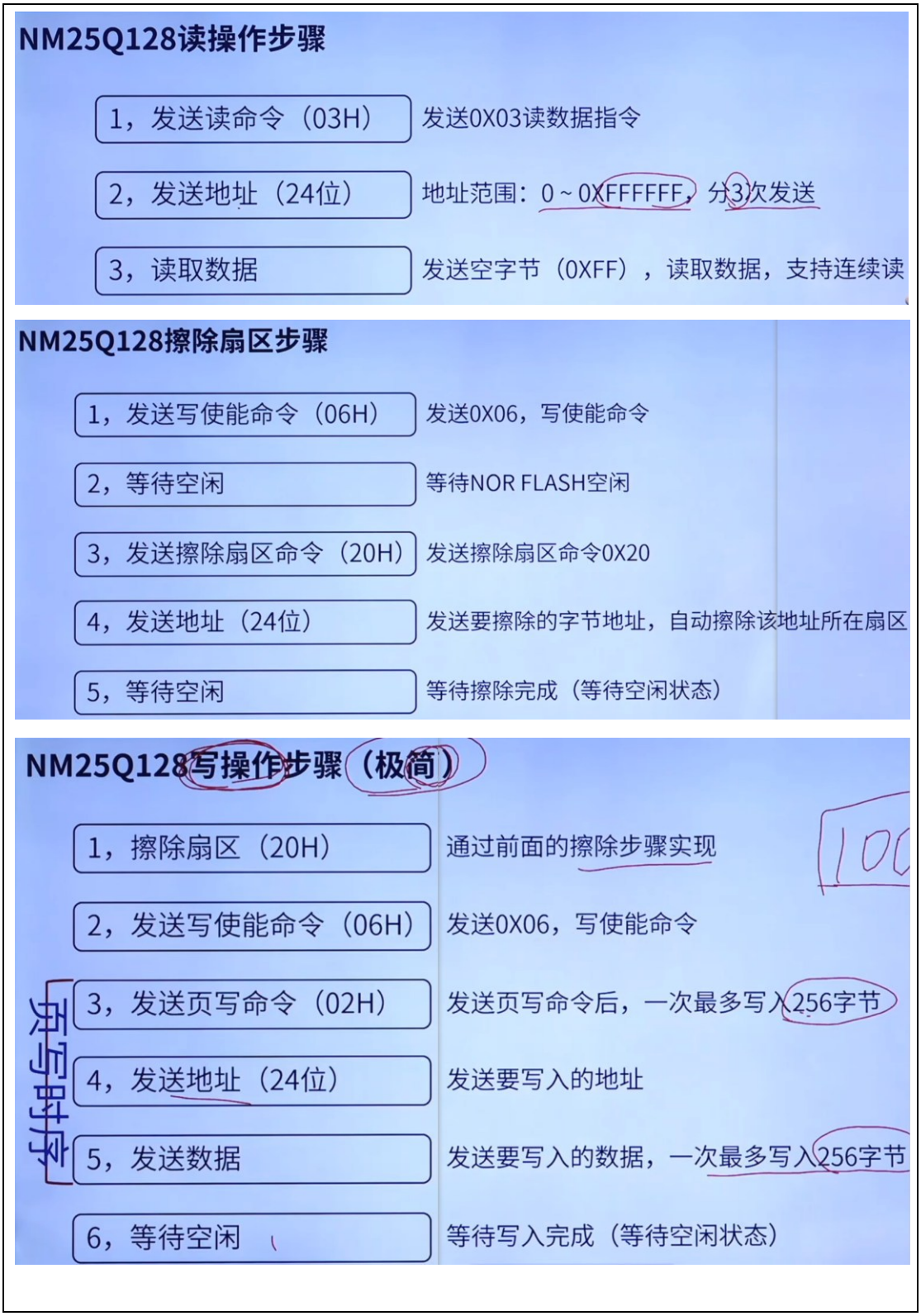


19.8.4 NM25Q128 常用指令

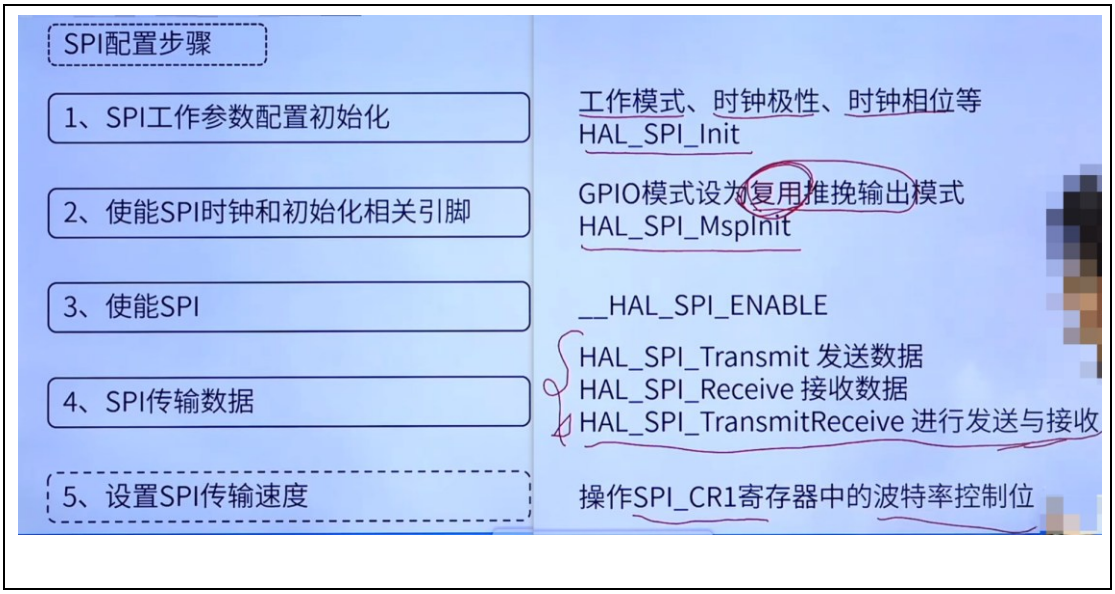


指令(HEX)	名称	作用
0X06	写使能	写入数据/擦除之前，必须先发送该指令
0X05	读SR1	判定FLASH是否处于空闲状态，擦除用
0X03	读数据	用于读取NOR FLASH数据
0X02	页写	用于写入NOR FLASH数据，最多写256字节
0X20	扇区擦除	扇区擦除指令，最小擦除单位（4096字节）

19.8.5 NM25Q128 读/擦除/写步骤

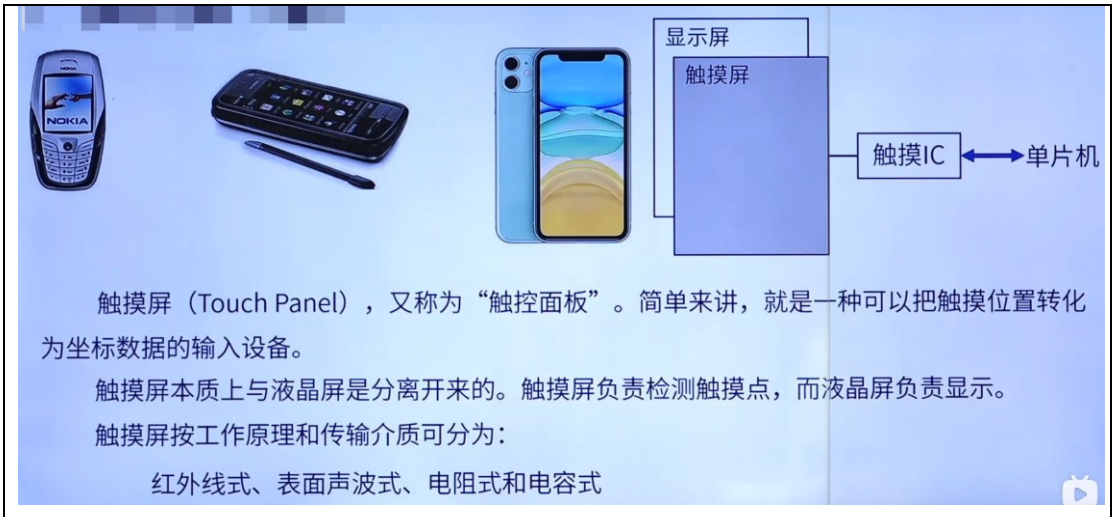


19.9 NOR FLASH 基本驱动步骤



20 触摸屏

20.1 触摸屏介绍



20.2 触摸分类

触摸屏分类

1）电阻式触摸屏

分类：四线，五线，七线和八线

2）电容式触摸屏

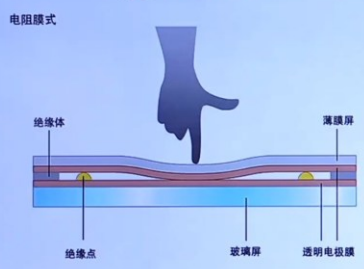
分类：① 表面电容式（利用电场感应感测屏幕触摸，只能识别一次触摸）

② 投射式（利用触摸屏电极发射出静电场线）

自我电容（扫描电极与地构成的电容）

交互电容（玻璃表面的横向和纵向的ITO电极的交叉处形成的电容）

电阻触摸屏

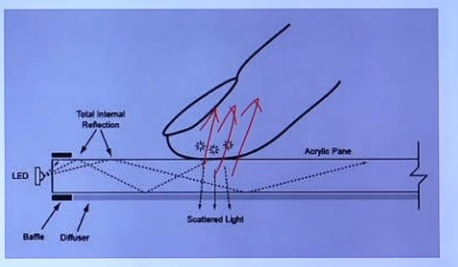


触摸点(X,Y)转换代表X坐标和Y坐标的电压

优点：精度高，价格便宜，抗干扰能力强，稳定性好

缺点：需校准，容易被划伤，透光性较差，不支持多点触摸

电容触摸屏

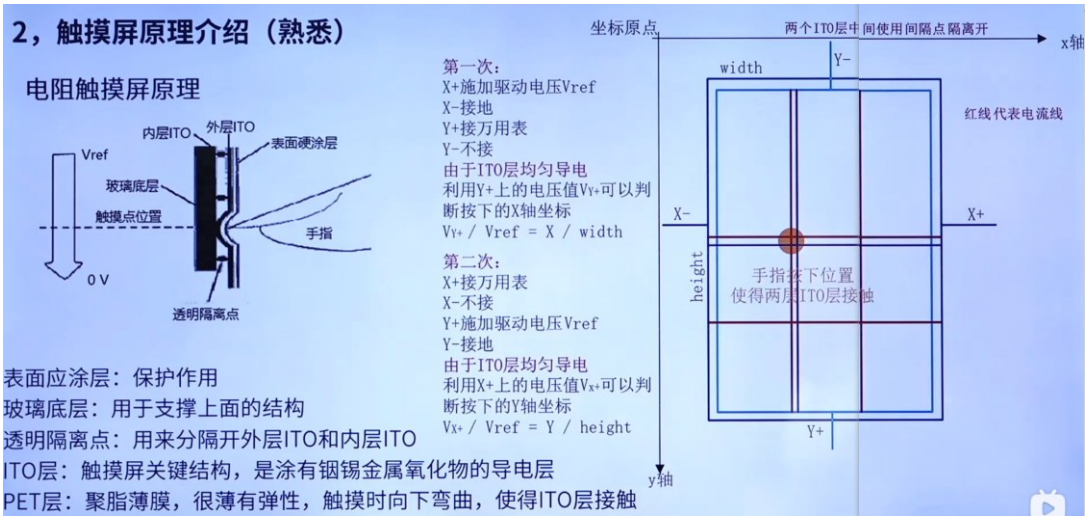


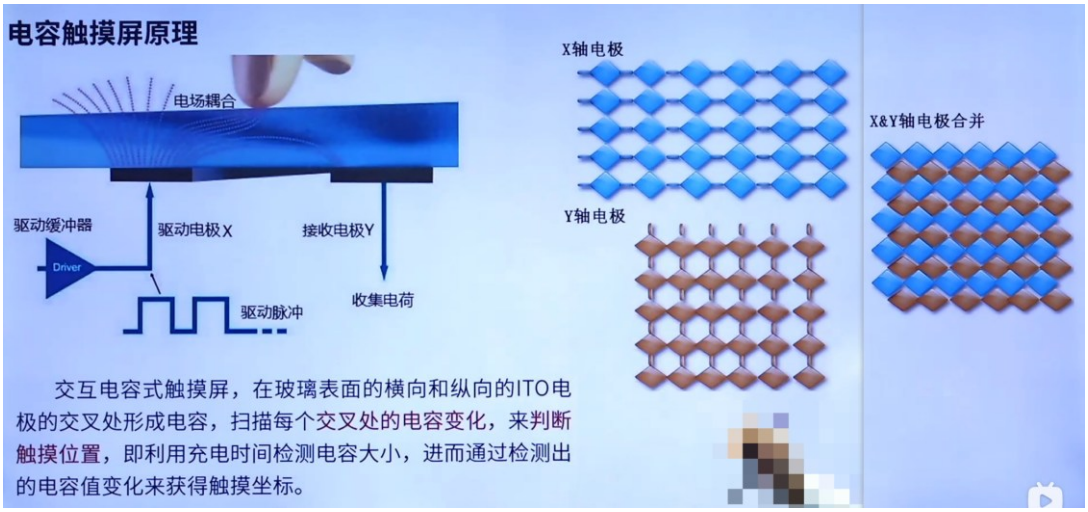
利用人体感应进行触点检测控制，只需要轻微接触，通过检测感应电流定位触摸坐标。

优点：手感好、无需校准、支持多点触摸、透光性好

缺点：成本高、精度不高、抗干扰能力差

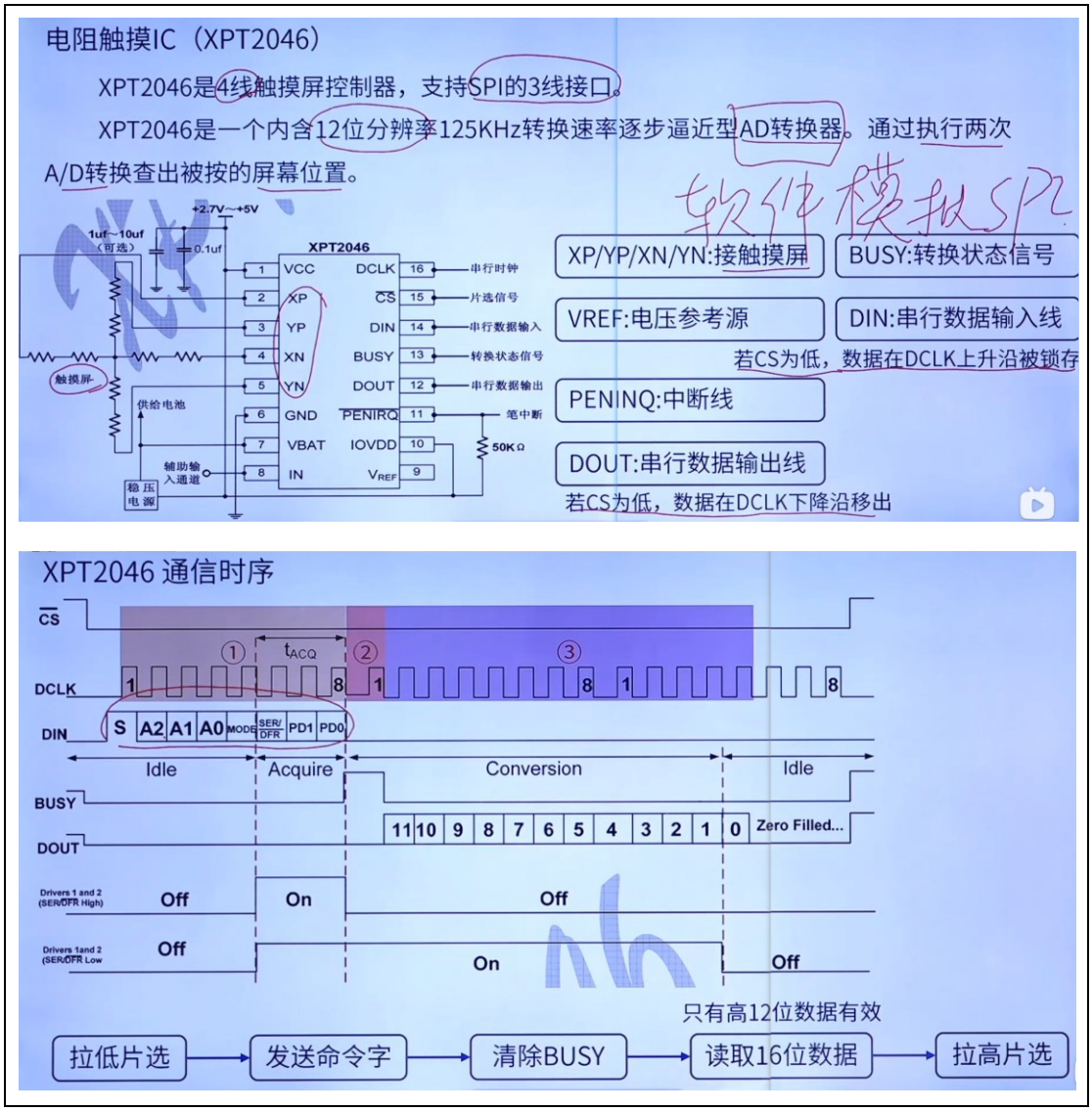
20.3 触摸屏原理介绍

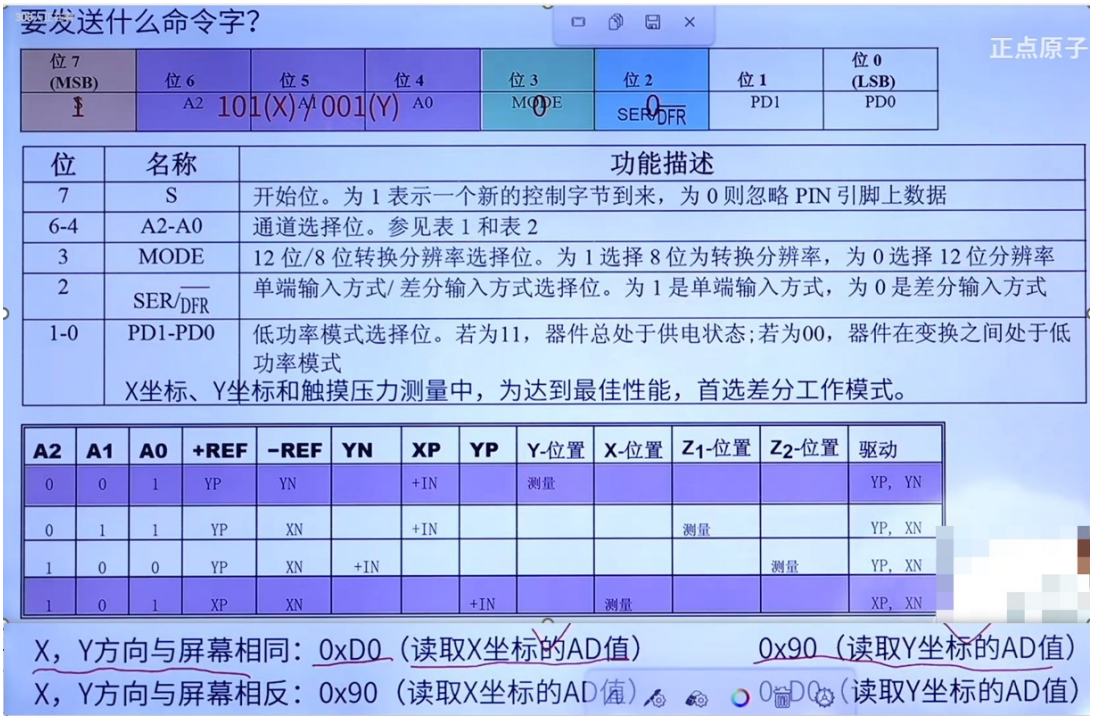

2，触摸屏原理介绍（熟悉）
电阻触摸屏原理
内层ITO
外层ITO
表面硬涂层
Vref
玻璃底层
触摸点位置
手指
0 V
透明隔离点
第一次：
X+施加驱动电压Vref
X-接地
Y+接万用表
Y-不接
由于ITO层均匀导电
利用Y+上的电压值VY+可以判
断按下的X轴坐标
VY+ / Vref = X / width
第二次：
X+接万用表
X-不接
Y+施加驱动电压Vref
Y-接地
由于ITO层均匀导电
利用X+上的电压值VX+可以判
断按下的Y轴坐标
VX+ / Vref = Y / height
坐标原点
两个ITO层中间使用间隔点隔离开
x轴
y轴
width
height
Y-
Y+
X-
X+
红线代表电流线
手指按下位置
使得两层ITO层接触
表面应涂层：保护作用
玻璃底层：用于支撑上面的结构
透明隔离点：用来分隔开外层ITO和内层ITO
ITO层：触摸屏关键结构，是涂有铟锡金属氧化物的导电层
PET层：聚脂薄膜，很薄有弹性，触摸时向下弯曲，使得ITO层接触

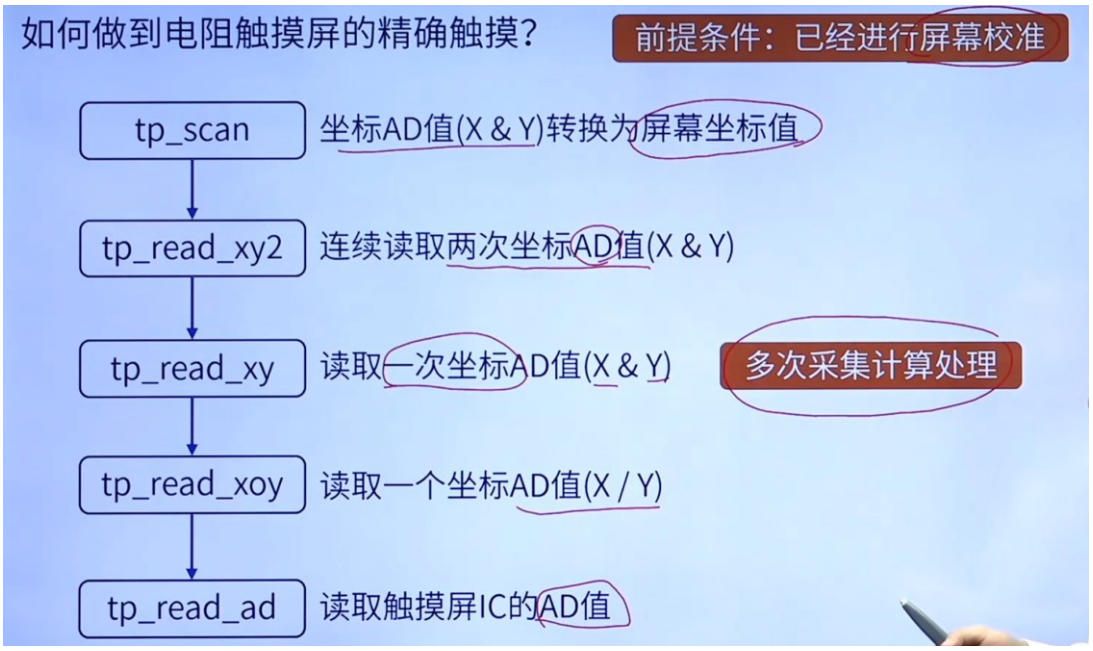

电容触摸屏原理
电场耦合
驱动缓冲器
Driver
驱动电极X
接收电极Y
驱动脉冲
收集电荷
X轴电极
Y轴电极
X&Y轴电极合并
交互电容式触摸屏，在玻璃表面的横向和纵向的ITO电极的交叉处形成电容，扫描每个交叉处的电容变化，来判断触摸位置，即利用充电时间检测电容大小，进而通过检测出的电容值变化来获得触摸坐标。

20.4 触摸屏 IC 介绍

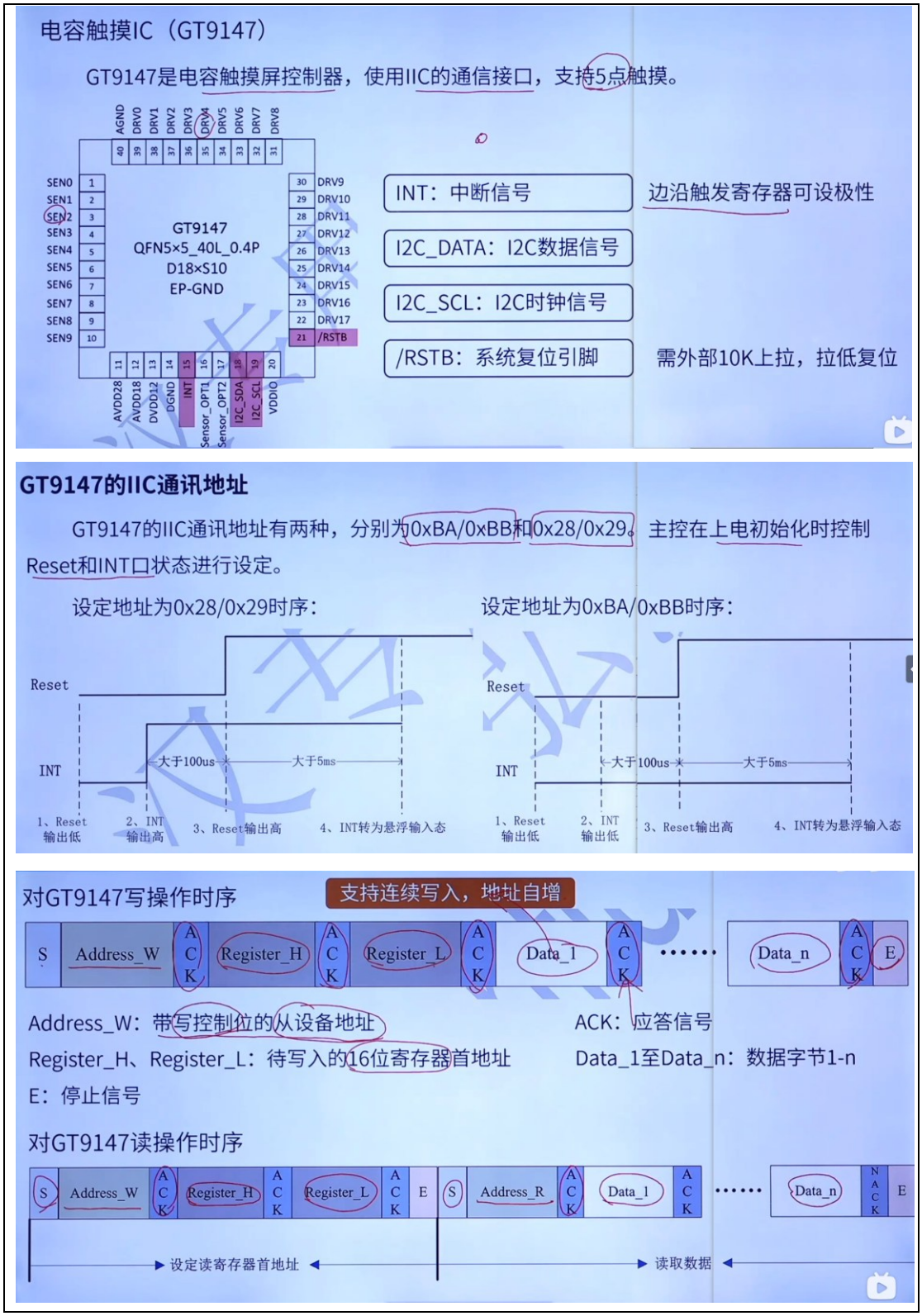
20.4.1 电阻触摸 IC




要发送什么命令字?
正点原子
位 7 (MSB) | 位 6 | 位 5 | 位 4 | 位 3 | 位 2 | 位 1 | 位 0 (LSB)
1 | A2 | A1 | A0 | MODE | SER/DFR | PD1 | PD0
101(X) / 001(Y)
0
位 | 名称 | 功能描述
7 | S | 开始位。为 1 表示一个新的控制字节到来，为 0 则忽略 PIN 引脚上数据
6-4 | A2-A0 | 通道选择位。参见表 1 和表 2
3 | MODE | 12 位/8 位转换分辨率选择位。为 1 选择 8 位为转换分辨率，为 0 选择 12 位分辨率
2 | SER/DFR | 单端输入方式/ 差分输入方式选择位。为 1 是单端输入方式，为 0 是差分输入方式
1-0 | PD1-PD0 | 低功率模式选择位。若为11，器件总处于供电状态;若为00，器件在变换之间处于低功率模式
X坐标、Y坐标和触摸压力测量中，为达到最佳性能，首选差分工作模式。
A2 | A1 | A0 | +REF | -REF | YN | XP | YP | Y-位置 | X-位置 | Z1-位置 | Z2-位置 | 驱动
0 | 0 | 1 | YP | YN | | +IN | | 测量 | | | | YP, YN
0 | 1 | 1 | YP | XN | | +IN | | | | 测量 | | YP, XN
1 | 0 | 0 | YP | XN | +IN | | | | | | 测量 | YP, XN
1 | 0 | 1 | XP | XN | | | +IN | | 测量 | | | XP, XN
X，Y方向与屏幕相同：0xD0（读取X坐标的AD值）
0x90（读取Y坐标的AD值）
X，Y方向与屏幕相反：0x90（读取X坐标的AD值）
0xD0（读取Y坐标的AD值）


如何做到电阻触摸屏的精确触摸?
前提条件：已经进行屏幕校准
tp_scan
坐标AD值(X & Y)转换为屏幕坐标值
tp_read_xy2
连续读取两次坐标AD值(X & Y)
tp_read_xy
读取一次坐标AD值(X & Y)
多次采集计算处理
tp_read_xoy
读取一个坐标AD值(X / Y)
tp_read_ad
读取触摸屏IC的AD值

20.4.2 电容触摸 IC


电容触摸IC（GT9147）
GT9147是电容触摸屏控制器，使用IIC的通信接口，支持5点触摸。
GT9147
QFN5×5_40L_0.4P
D18×S10
EP-GND
INT：中断信号
I2C_DATA：I2C数据信号
I2C_SCL：I2C时钟信号
/RSTB：系统复位引脚
边沿触发寄存器可设极性
需外部10K上拉，拉低复位
GT9147的IIC通讯地址
GT9147的IIC通讯地址有两种，分别为0xBA/0xBB和0x28/0x29。主控在上电初始化时控制Reset和INT口状态进行设定。
设定地址为0x28/0x29时序：
设定地址为0xBA/0xBB时序：
Reset
INT
大于100us
大于5ms
1、Reset输出低
2、INT输出高
3、Reset输出高
4、INT转为悬浮输入态
对GT9147写操作时序
支持连续写入，地址自增
S
Address_W
ACK
Register_H
Register_L
Data_1
Data_n
E
Address_W：带写控制位的从设备地址
Register_H、Register_L：待写入的16位寄存器首地址
E：停止信号
ACK：应答信号
Data_1至Data_n：数据字节1-n
对GT9147读操作时序
Address_R
NACK
设定读寄存器首地址
读取数据

GT9147关键寄存器

寄存器名称	寄存器地址	作用
控制命令寄存器	0x8040	设置设备的模式（0：读坐标状态） （0x02：软件复位）
配置寄存器组	0x8047~0x8100	配置信息(版本信息、X/Y输出最大值和支持触点数目等)
产品ID寄存器	0x8140~0x8143	读取产品ID
状态寄存器	0x814E	当前检测到的触摸情况
坐标数据寄存器	0x81(50/58/60/68/70)	5组坐标的起始地址

注意：配置寄存器共有168个寄存器，用于配置触摸IC的各个参数，这些配置一般由厂家提供给我们（一个数组），所以我们只需要将厂家给我们的配置，写入到这些寄存器中，即可完成对触摸IC的配置。

控制命令寄存器

0x8040	Command	0：读坐标状态　1、2：差值原始值 3：基准更新（内部测试）　4：基准校准（内部测试）　5：关屏 6：进入充电模式　7：退出充电模式　8：切换手势唤醒固件 0x20:进入从机接近检测模式　0x21：进入主机接近检测模式 0x22：进入数据传输模式　0x23:进入主机传输模式 0x28：退出从机检测模式　0x29：退出主机接近检测模式 0x2A：退出数据传输模式 0xAA:ESD 保护机制使用，由驱动定时写入 0xAA 并定时读取检查 其余值无效

对该寄存器写入不同值，实现不同的控制。一般硬复位后，需要写2进行软复位，然后写0结束软复位。

产品ID寄存器

0x8140	R	Product ID (first Byte，ASCII 码)
0x8141	R	Product ID (second Byte，ASCII 码)
0x8142	R	Product ID (third Byte，ASCII 码)
0x8143	R	Product ID (forth Byte，ASCII 码)

产品ID寄存器由4个寄存器组成，用于保存产品ID。
对于GT9147来说，这4个寄存器读出来就是9,1,4,7四个字符（ASCII码格式）。

状态寄存器

Addr	Access	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
0x814E	R/W	buffer status	large detect	Proximity Valid	HaveKey	number of touch points			

[0x814E]

Bit7: Buffer status，1 表示坐标（或按键）已经准备好，主控可以读取；0 表示未就绪，数据无效。
当主控读取完坐标后，必须通过 I²C 将此标志（或整个字节）写为 0。
Bit3~0: Number of touch points， 屏上的坐标点个数。

通常以Bit7和Bit0~3为判断依据，进而读取坐标信息。

坐标数据寄存器

共分为5组，每组6个寄存器存储数据，以触点1的坐标数据寄存器组为例。

0x8150	R	point 1 x coordinate (low byte)
0x8151	R	point 1 x coordinate (high byte)
0x8152	R	point 1 y coordinate (low byte)
0x8153	R	point 1 y coordinate (high byte)
0x8154	R	Point 1 size (W)
0x8155	R	point 1 size (H)

一般只用触点的x，y坐标，所以只需要读取0X8150~0X8153的数据，组合即可得到触点坐标。其他4组分别是：0X8158、0X8160、0X8168和0X8170等开头的16个寄存器组成，分别针对触点2~4的坐标。设备支持寄存器地址自增，我们只需要发送寄存器组的首地址，然后连续读取即可，地址自增，从而提高读取速度。

如何实现电容触摸屏的触摸检测？

rd State_Reg 读取触摸IC的状态寄存器值

rd point_reg 读取到四字节数据

get x/y 坐标数据整合(X和Y坐标)

illegal data 非法数据处理，丢弃

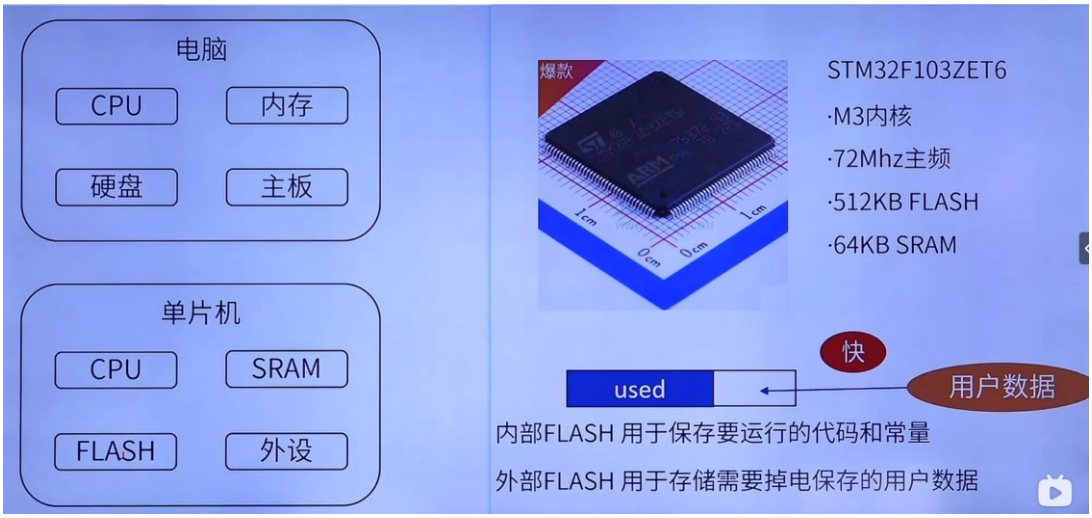
自定义触摸屏控制器结构体类型

```
typedef struct
{
    //(省略一些函数指针)
    uint16_t x[CT_MAX_TOUCH];    /* 当前坐标 */
    uint16_t y[CT_MAX_TOUCH];    /* 电容屏有最多10组坐标,x[9],y[9]存储第一次按下时的坐标
                                 */
    uint16_t sta;                /* 笔的状态
                                 * b15:按下1/松开0;
                                 * b14:0,没有按键按下;1,有按键按下.
                                 * b13~b10:保留
                                 * b9~b0:电容触摸屏按下的点数(0,表示未按下,1表示按下)
                                 */
    //(省略一些与电阻触摸屏相关成员和touchtype)
} _m_tp_dev;
```

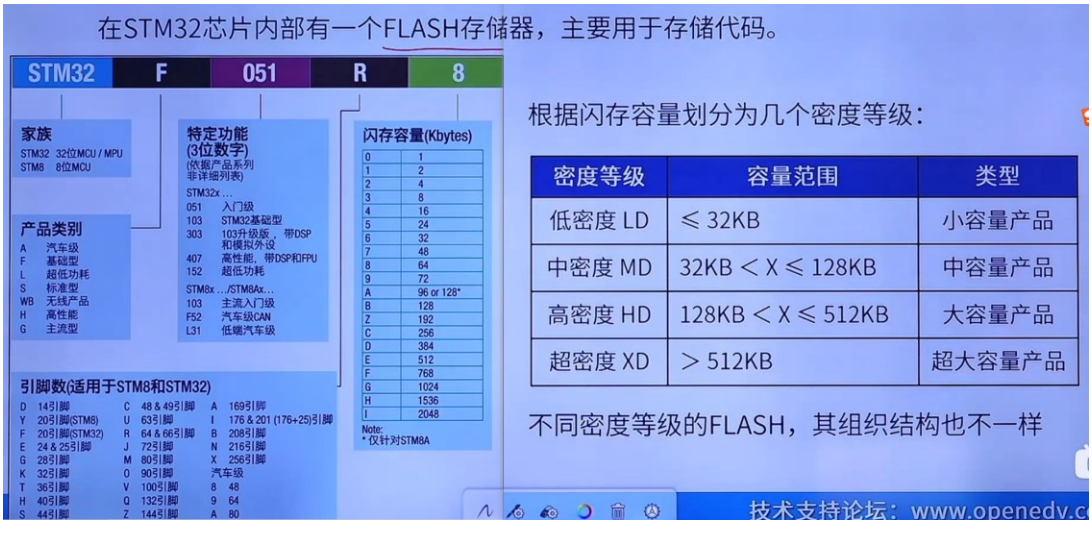
20.5 触摸屏驱动步骤

1、初始化通信接口与其IO口	XPT2046使用的是SPI、GT9147使用的是IIC
2、编写通信协议基础读写函数	在IIC驱动或SPI驱动基础上，编写基础读写函数
3、编写触摸IC的读写驱动函数	根据触摸IC的读写时序进行编写
4、编写坐标获取函数	查阅数据手册获取命令词/寄存器
触摸校准	对于电阻触摸屏需要实现

21　FLASH 模拟 EEPROM



21.1 STM32 内部 FLASH 简介



芯片型号	主频（MHz）	FLASH容量（B）	SRAM容量（B）
STM32F103RCT6	72	256K	48K
STM32F103ZET6	72	512K	64K
STM32F407ZGT6	168	1024K	192K
STM32F429IGT6	180	1024K	256K
STM32F76IGT6	216	1024K	512K
STM32H743IIT6	480	2048K	1060K
STM32F750N8H6	216	64K	320K
STM32H750XBH6	400	128K	1060K
STM32H750VBT6	480	128K	1060K

21.2 STM32 FLASH 构成（F1）

内部FLASH主要由三部分组成：主存储器、信息块、闪存存储器接口寄存器。

闪存模块的子部分	作用
主存储器	用来存放代码和数据常数（如const类型到的数据）
信息块	分为两个部分：系统存储（启动程序代码）、选项字节（用户选择字节）
闪存存储器接口寄存器	用于控制闪存读写等，是整个闪存模块的控制结构

主存储器：地址范围为0x0800 0000 ~ 0x0807 FFFF，分为256页，每页2KB（小/中容量为1K）。

当BOOT0接地，系统将从0x0800 0000地址处开始读取代码（从主存储器启动）

信息块：系统存储大小为2KB，用来存储ST自带的启动程序，用来串口下载代码。

选项字节大小为16B，一般用于设置内存的写保护、读保护

当BOOT0接VCC，BOOT1接GND （串口下载程序），系统运行的就是这部分代码

模块	名称	地址	大小(字节)
主存储块	页0	Coder	2K
	页1		2K
	页2		2K
	页3		2K
	…		…
	…	…	…
	页255	0x0807 F800 - 0x0807 FFFF	2K
信息块	系统存储器	0x1FFF F000 - 0x1FFF F7FF	2K
	选择字节	0x1FFF F800 - 0x1FFF F80F	16
闪存存储器接口寄存器	FLASH_ACR	0x4002 2000 - 0x4002 2003	4 闪存访问控制寄存器
	FALSH_KEYR	0x4002 2004 - 0x4002 2007	4 FPEC键寄存器
	FLASH_OPTKEYR	0x4002 2008 - 0x4002 200B	4 选择字节键寄存器
	FLASH_SR	0x4002 200C - 0x4002 200F	4 闪存状态寄存器
	FLASH_CR	0x4002 2010 - 0x4002 2013	4 闪存控制寄存器
	FLASH_AR	0x4002 2014 - 0x4002 2017	4 闪存地址寄存器
	保留	0x4002 2018 - 0x4002 201B	4
	FLASH_OBR	0x4002 201C - 0x4002 201F	4 选择字节寄存器
	FLASH_WRPR	0x4002 2020 - 0x4002 2023	4 写保护寄存器

512KB

大容量产品闪存模块组织

不同容量的，页数和页大小会有不同

控制 FLASH 写入&擦除

复位 后，FPEC模块有WP

21.3 FLASH 读写过程(F1)

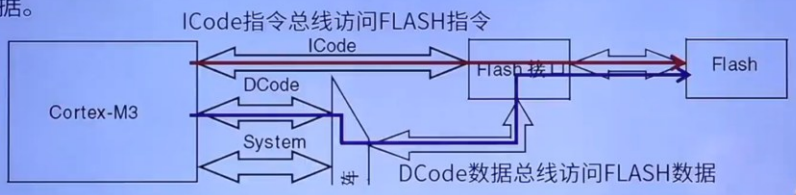
对FLASH的核心操作就是读和写。

FLASH的物理特性：只能写0，不能写1，写1靠擦除。

读操作　　写操作　　擦除操作

闪存的读取

直接在通用地址空间直接寻址，任何32位数据的读操作都能访问闪存模块的内容并得到相对应的数据。



CPU运行速度比FLASH快得多，STM32F103的FLASH最快访问速度≤24MHz，CPU频率超过这个速度，得加入等待时间，否则读写FLASH可能出错，导致死机等情况。

频率范围	等待周期数(LATENCY)
0 < SYSCLK < 24MHz	0个等待周期
24MHz < SYSCLK ≤ 48MHz	1个等待周期
48MHz < SYSCLK ≤ 72MHz	2个等待周期

正确设置好等待周期后，利用指针读取数据。

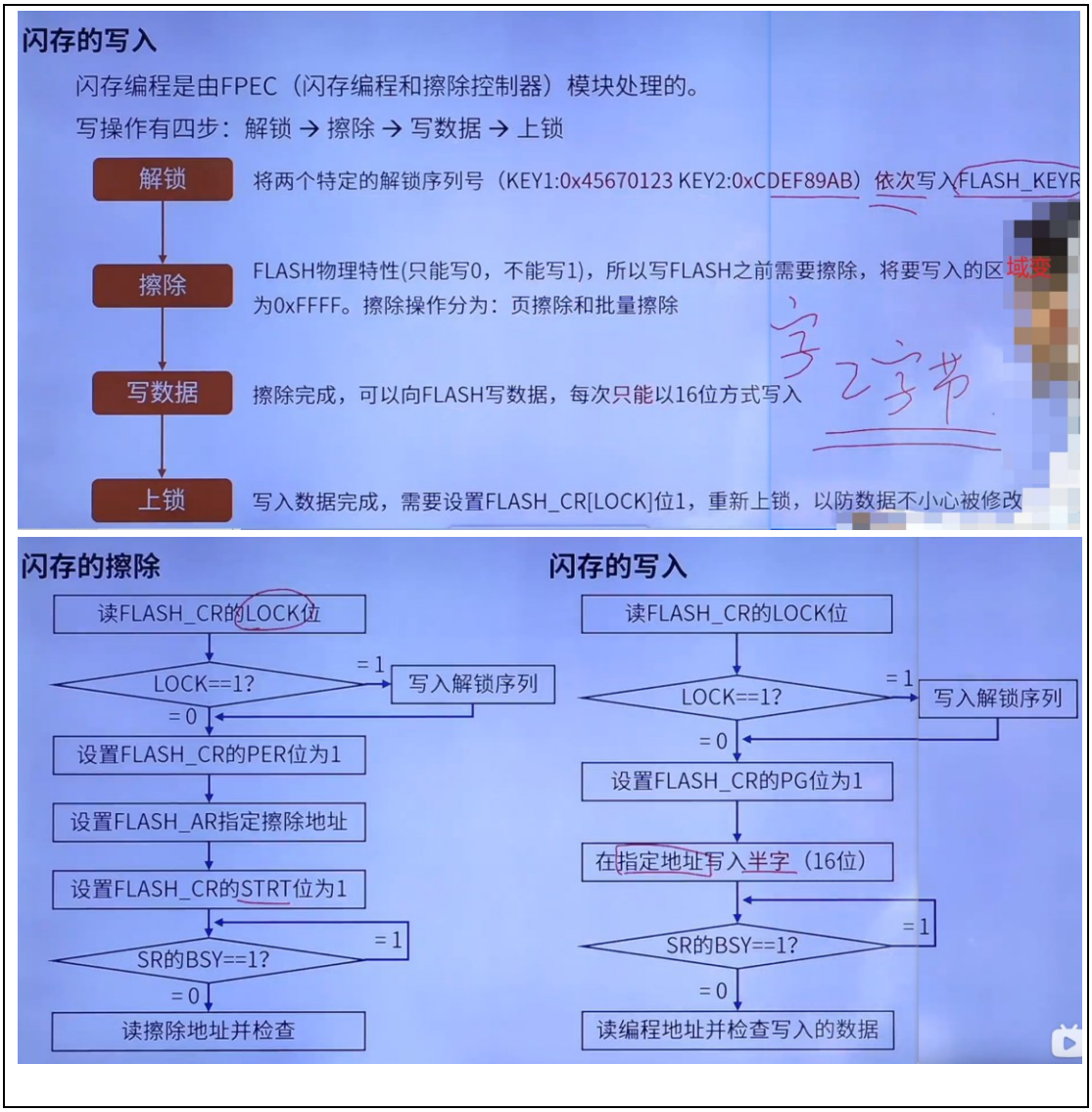
从地址addr，读取数据（字节为8位，半字为16位，字为32位）

```
data = *(volatile uint8_t *)addr;      /* 读取一个字节数据 */
data = *(volatile uint16_t *)addr;     /* 读取一个半字数据 */
data = *(volatile uint32_t *)addr;     /* 读取一个字数据 */
```

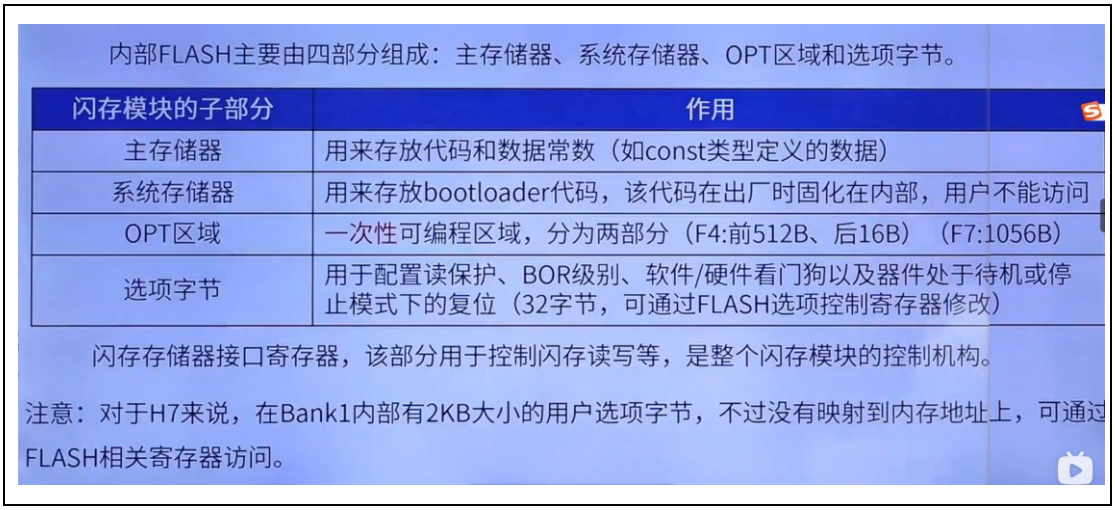
将addr强制转换为uintx_t指针，然后取该指针所指向地址的值，即可获得addr地址的数据。

注意： 在进行写或擦除操作时，不能进行代码或数据的读取操作。

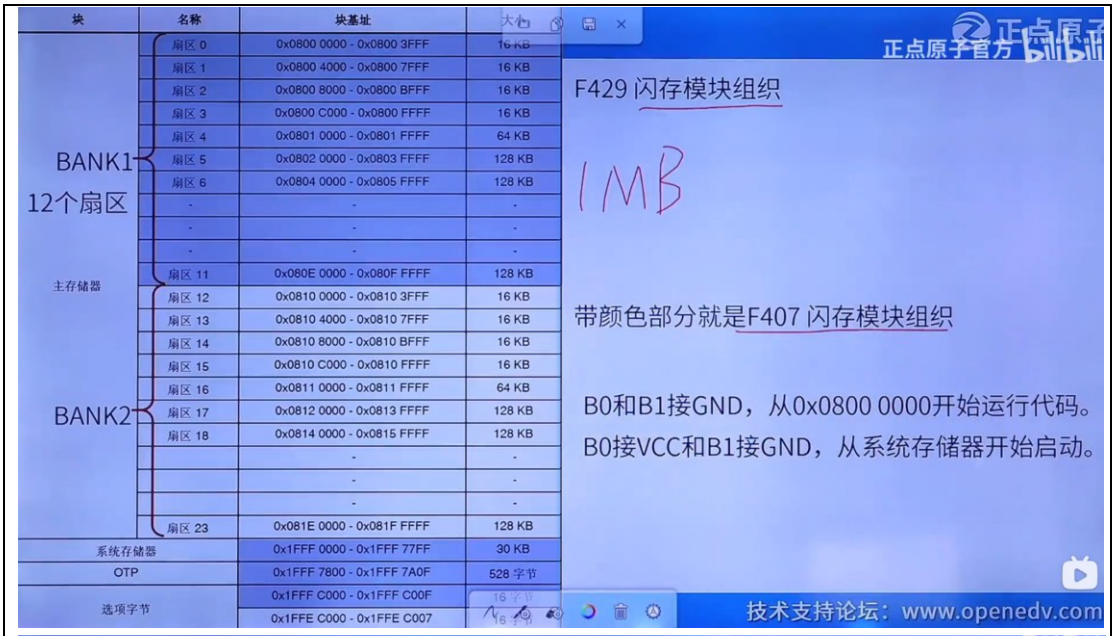
写操作： *(volatile uint16_t *)addr = data? 并不全是



21.4 内部 FLASH 构成（F4/F7/H7）



闪存模块的子部分	作用
主存储器	用来存放代码和数据常数（如const类型定义的数据）
系统存储器	用来存放bootloader代码，该代码在出厂时固化在内部，用户不能访问
OPT区域	一次性可编程区域，分为两部分（F4:前512B、后16B）（F7:1056B）
选项字节	用于配置读保护、BOR级别、软件/硬件看门狗以及器件处于待机或停止模式下的复位（32字节，可通过FLASH选项控制寄存器修改）



块	名称	块基址	大小
BANK1 12个扇区	扇区 0	0x0800 0000 - 0x0800 3FFF	16 KB
	扇区 1	0x0800 4000 - 0x0800 7FFF	16 KB
	扇区 2	0x0800 8000 - 0x0800 BFFF	16 KB
	扇区 3	0x0800 C000 - 0x0800 FFFF	16 KB
	扇区 4	0x0801 0000 - 0x0801 FFFF	64 KB
	扇区 5	0x0802 0000 - 0x0803 FFFF	128 KB
	扇区 6	0x0804 0000 - 0x0805 FFFF	128 KB
	-	-	-
	-	-	-
	-	-	-
主存储器	扇区 11	0x080E 0000 - 0x080F FFFF	128 KB
	扇区 12	0x0810 0000 - 0x0810 3FFF	16 KB
	扇区 13	0x0810 4000 - 0x0810 7FFF	16 KB
	扇区 14	0x0810 8000 - 0x0810 BFFF	16 KB
	扇区 15	0x0810 C000 - 0x0810 FFFF	16 KB
	扇区 16	0x0811 0000 - 0x0811 FFFF	64 KB
BANK2	扇区 17	0x0812 0000 - 0x0813 FFFF	128 KB
	扇区 18	0x0814 0000 - 0x0815 FFFF	128 KB
	-	-	-
	-	-	-
	-	-	-
	扇区 23	0x081E 0000 - 0x081F FFFF	128 KB
系统存储器		0x1FFF 0000 - 0x1FFF 77FF	30 KB
OTP		0x1FFF 7800 - 0x1FFF 7A0F	528 字节
选项字节		0x1FFF C000 - 0x1FFF C00F	16 字节
		0x1FFE C000 - 0x1FFE C007	16 字节

STM32F7的FLASH访问路径有两条：AXIM和ITCM，对应不同的地址映射。

F767 单Bank的1M闪存组织结构（默认）　　双Bank详看《STM32F7xx参考手册》

块	名称	AXIM 接口上的块基址	ICTM 接口上的块基址	大小
8个扇区 主存储器	扇区 0	0X0800 0000-0X0800 7FFF	0X0020 0000-0X0020 7FFF	32KB
	扇区 1	0X0800 8000-0X0800 FFFF	0X0020 8000-0X0020 FFFF	32KB
	扇区 2	0X0801 0000-0X0801 7FFF	0X0021 0000-0X0021 7FFF	32KB
	扇区 3	0X0801 8000-0X0801 FFFF	0X0021 8000-0X0021 FFFF	32KB
	扇区 4	0X0802 0000-0X0803 FFFF	0X0022 0000-0X0023 FFFF	128KB
	扇区 5	0X0804 0000-0X0807 FFFF	0X0024 0000-0X0027 FFFF	256KB
	扇区 6	0X0808 0000-0X080B FFFF	0X0028 0000-0X002B FFFF	256KB
	扇区 7	0X080C 0000-0X080F FFFF	0X002C 0000-0X002F FFFF	256KB
系统存储器		0X1FF0 0000-0X1FF0 EDBF	0X0010 0000-0X0010 EDBF	60KB
OTP 区域		0X1FF0 F000-0X1FF0 F41F	0X0010 F000-0X0010 F41F	1056B
选项字节		0X1FFF 0000-0X1FFF 001F		32B

1 MB

带蓝色部分是F750 闪存组织结构

块	名称	FLASH 起始地址	大小
8个扇区 Bank1	扇区 0	0X0800 0000-0X0801 FFFF	128K
	扇区 1	0X0802 0000-0X0803 FFFF	128K
	扇区 2	0X0804 0000-0X0805 FFFF	128K
	扇区 3	0X0806 0000-0X0807 FFFF	128K
	扇区 4	0X0808 0000-0X0809 FFFF	128K
	扇区 5	0X080A 0000-0X080B FFFF	128K
	扇区 6	0X080C 0000-0X080D FFFF	128K
	扇区 7	0X080E 0000-0X080F FFFF	128K
	系统存储器	0X1FF0 0000-0X1FF1 FFFF	128K
Bank2	扇区 0	0X0810 0000-0X0811 FFFF	128K
	扇区 1	0X0812 0000-0X0813 FFFF	128K
	扇区 2	0X0814 0000-0X0815 FFFF	128K
	扇区 3	0X0816 0000-0X0817 FFFF	128K
	扇区 4	0X0818 0000-0X0819 FFFF	128K
	扇区 5	0X081A 0000-0X081B FFFF	128K
	扇区 6	0X081C 0000-0X081D FFFF	128K
	扇区 7	0X081E 0000-0X081F FFFF	128K
	系统存储器	0X1FF4 0000-0X1FF5 FFFF	128K

H743 2MB flash闪存模块组织

1 MB

带颜色部分是H750 闪存模块组织

21.5 FLASH 读写过程（F4/F7/H7）

对FLASH的核心操作就是读和写。

FLASH的物理特性：只能写0，不能写1，写1靠擦除。

读操作　写操作　擦除操作

F4

FLASH_ACR [LATENCY]

准确读取FLASH数据需要设置等待周期数，跟CPU时钟频率和器件电源电压有关。

Wait states (WS) (LATENCY)	HCLK (MHz) Voltage range 2.7 V - 3.6 V	Voltage range 2.4 V - 2.7 V	Voltage range 2.1 V - 2.4 V	Voltage range 1.8 V - 2.1 V Prefetch OFF
0 WS (1 CPU cycle)	0 <HCLK ≤ 30	0 <HCLK ≤ 24	0 <HCLK ≤ 22	0 < HCLK ≤ 20
1 WS (2 CPU cycles)	30 <HCLK ≤ 60	24 < HCLK ≤ 48	22 <HCLK ≤ 44	20 <HCLK ≤ 40
2 WS (3 CPU cycles)	60 <HCLK ≤ 90	48 < HCLK ≤ 72	44 < HCLK ≤ 66	40 < HCLK ≤ 60
3 WS (4 CPU cycles)	90 <HCLK ≤ 120	72 < HCLK ≤ 96	66 <HCLK ≤ 88	60 < HCLK ≤ 80
4 WS (5 CPU cycles)	120 <HCLK ≤ 150	96 < HCLK ≤ 120	88 < HCLK ≤ 110	80 < HCLK ≤ 100
5 WS (6 CPU cycles)	150 <HCLK ≤ 180	120 <HCLK ≤ 144	110 < HCLK ≤ 132	100 < HCLK ≤ 120
6 WS (7 CPU cycles)		144 <HCLK ≤ 168	132 < HCLK ≤ 154	120 < HCLK ≤ 140
7 WS (8 CPU cycles)		168 <HCLK ≤ 180	154 <HCLK ≤ 176	140 < HCLK ≤ 160
8 WS (9 CPU cycles)			176 <HCLK ≤ 180	160 < HCLK ≤ 168

复位 — 3.3V供电 f_{CPU}为16M(RC振荡器)

LATENCY默认为0，1个CPU周期

正常工作 — 3.3V供电 设f_{CPU}为168/180MH

LATENCY为5，6个CPU周期

自适应实时存储器加速器 (ART Accelerator)

通过指令缓存存储器预取指令，实现相当于0 FLASH等待的运行速度。

F7

等待周期 (WS) (LATENCY)	HCLK (MHz) 电压范围 2.7 V - 3.6 V	电压范围 2.4 V - 2.7 V	电压范围 2.1 V - 2.4 V	电压范围 1.8 V - 2.1 V
0 WS（1 个 CPU 周期）	0 < HCLK ≤ 30	0 < HCLK ≤ 24	0 < HCLK ≤ 22	0 < HCLK ≤ 20
1 WS（2 个 CPU 周期）	30 < HCLK ≤ 60	24 < HCLK ≤ 48	22 < HCLK ≤ 44	20 < HCLK ≤ 40
2 WS（3 个 CPU 周期）	60 < HCLK ≤ 90	48 < HCLK ≤ 72	44 < HCLK ≤ 66	40 < HCLK ≤ 60
3 WS（4 个 CPU 周期）	90 < HCLK ≤ 120	72 < HCLK ≤ 96	66 < HCLK ≤ 88	60 < HCLK ≤ 80
4 WS（5 个 CPU 周期）	120 < HCLK ≤ 150	96 < HCLK ≤ 120	88 < HCLK ≤ 110	80 < HCLK ≤ 100
5 WS（6 个 CPU 周期）	150 < HCLK ≤ 180	120 < HCLK ≤ 144	110 < HCLK ≤ 132	100 < HCLK ≤ 120
6 WS（7 个 CPU 周期）	180 < HCLK ≤ 210	144 < HCLK ≤ 168	132 < HCLK ≤ 154	120 < HCLK ≤ 140
7 WS（8 个 CPU 周期）	210 < HCLK ≤ 216	168 < HCLK ≤ 192	154 < HCLK ≤ 176	140 < HCLK ≤ 160
8 WS（9 个 CPU 周期）	-	192 < HCLK ≤ 216	176 < HCLK ≤ 198	160 < HCLK ≤ 180
9 WS（10 个 CPU 周期）	-	-	198 < HCLK ≤ 216	-

设置LATENCY为7，即8个等待周期。

H7

R：H743/H750XB
B：H750VB

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;45 MHz]	[0;55 MHz]	[0;70 MHz]	[0;70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz;90 MHz]	]55 MHz;110 MHz]	]70 MHz;140 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz;135 MHz]	]110 MHz;165 MHz]	]140 MHz;185 MHz]	]140 MHz;185 MHz]
	10	-	-	]185 MHz;210 MHz]	]185 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz;180 MHz]	]165 MHz;225 MHz]	]210 MHz;225 MHz]	]210 MHz;225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz;225 MHz]	225 MHz	-	]225 MHz;240 MHz]

H743/H750XB Vcore电压范围为VOS1，频率一般设置200MHz，即等待周期为3。
H750VB　Vcore电压范围为VOS0，频率一般设置为240MHz，即等待周期为5。

21.6 闪存的读取

闪存的读取

正确设置好等待周期后，利用指针读取数据。

从地址addr，读取数据（字节为8位，半字为16位，字为32位）

```
data = *(volatile uint8_t *)addr;        /* 读取一个字节数据 */
data = *(volatile uint16_t *)addr;       /* 读取一个半字数据 */
data = *(volatile uint32_t *)addr;       /* 读取一个字数据 */
```

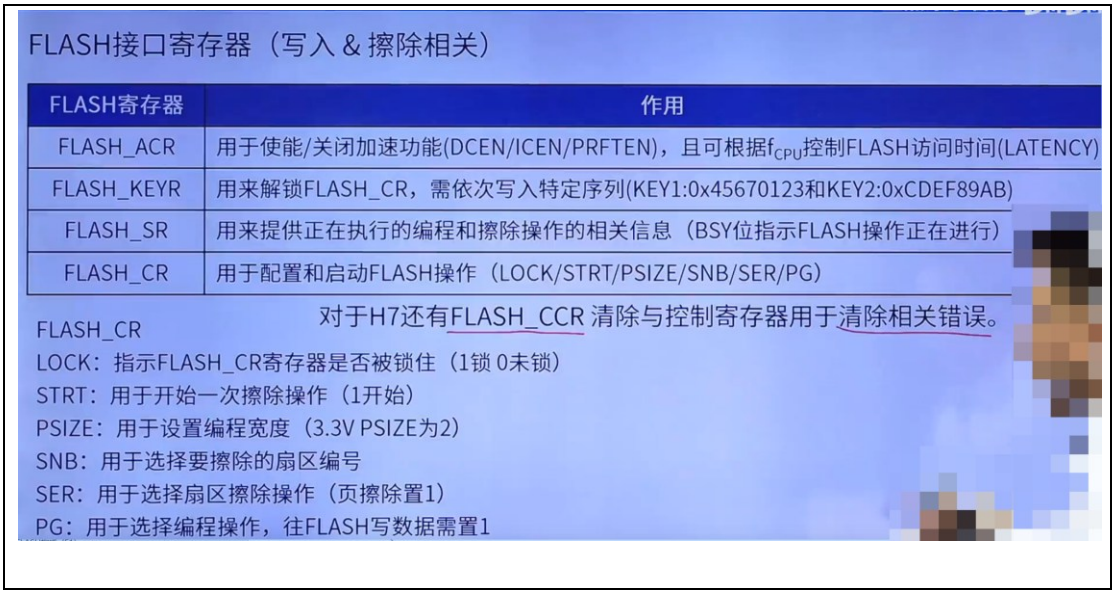
将addr强制转换为uintx_t指针，然后取该指针所指向地址的值，即可获得addr地址的数据。

注意：在进行写或擦除操作时，不能进行代码或数据的读取操作。

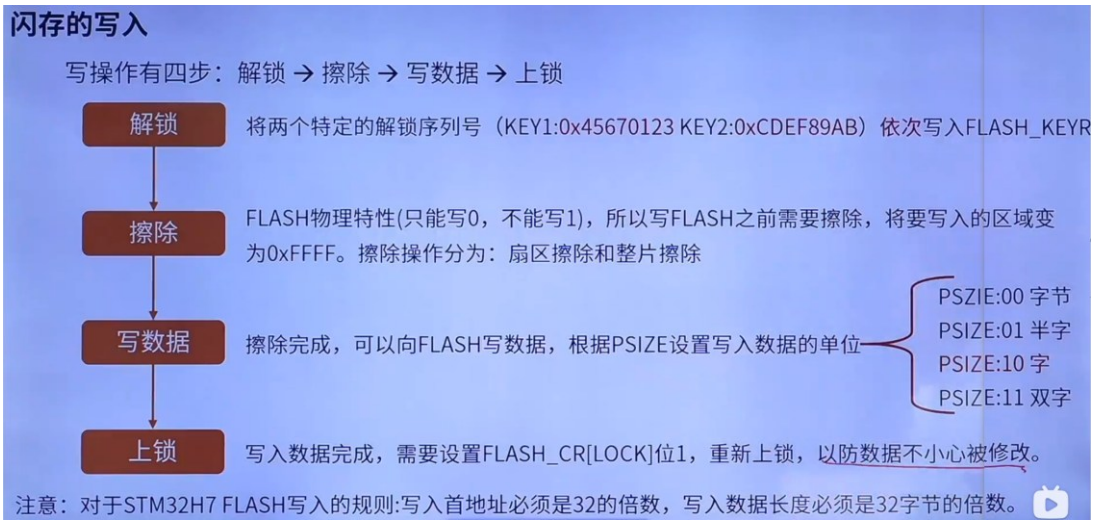
写操作：*(volatile uint16_t *)addr = data? 并不全是

21.7 闪存的写入

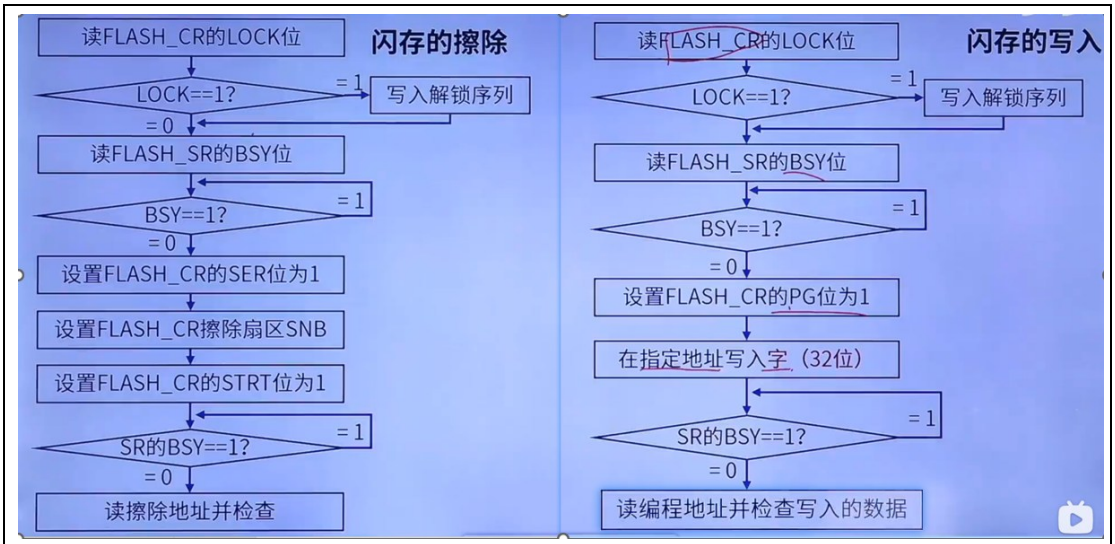
21.8 FLASH 接口寄存器（写入&擦除相关）



21.9 闪存的写入



21.10　闪存的擦除&写入



21.11　相关 HAL 库函数简介（F1/F4/F7/H7）

驱动函数	关联寄存器	功能描述
HAL_FLASH_Unlock(…)	FLASH_KEYR	用于解锁FLASH_CR的访问
HAL_FLASH_Lock(…)	FLASH_CR	用于锁定FLASH_CR的访问
HAL_FLASH_Program(…)	FLASH_CR	用于FLASH的写入
HAL_FLASHEx_Erase(…)	FLASH_CR/AR	用于大量擦除或擦除指定的闪存扇区
FLASH_WaitForLastOperation(…)	FLASH_SR	等待操作完成

FLASH相关结构体：FLASH_EraseInitTypeDef

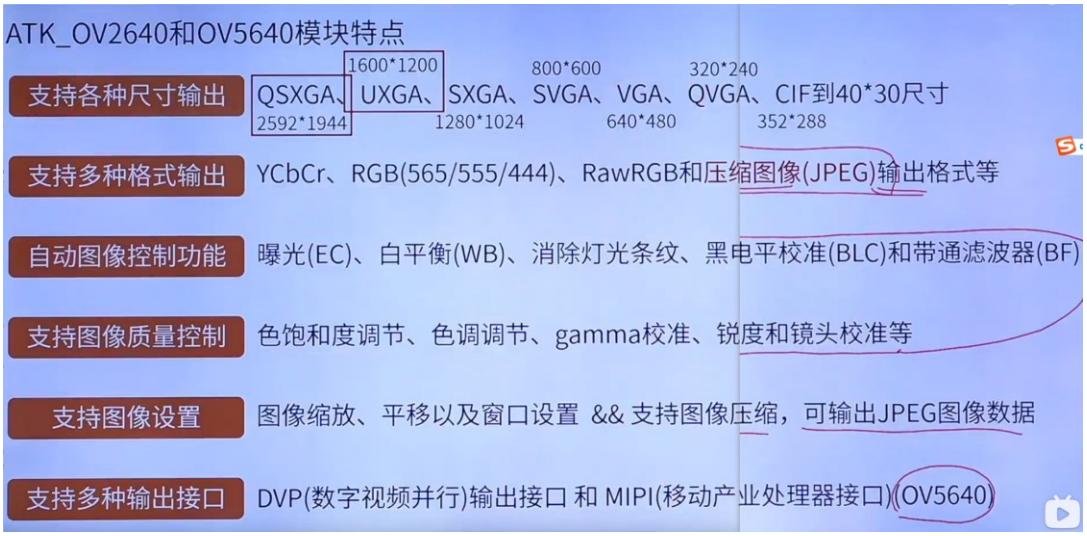
F1 **FLASH_EraseInitTypeDef**

```
uint32_t   TypeErase      /* 擦除类型 */
uint32_t   Banks          /* 擦除的bank编号（整片擦除） */
uint32_t   PageAddress    /* 擦除页面地址 */
uint32_t   NbPages        /* 擦除的页面数 */
```

22　摄像头实验

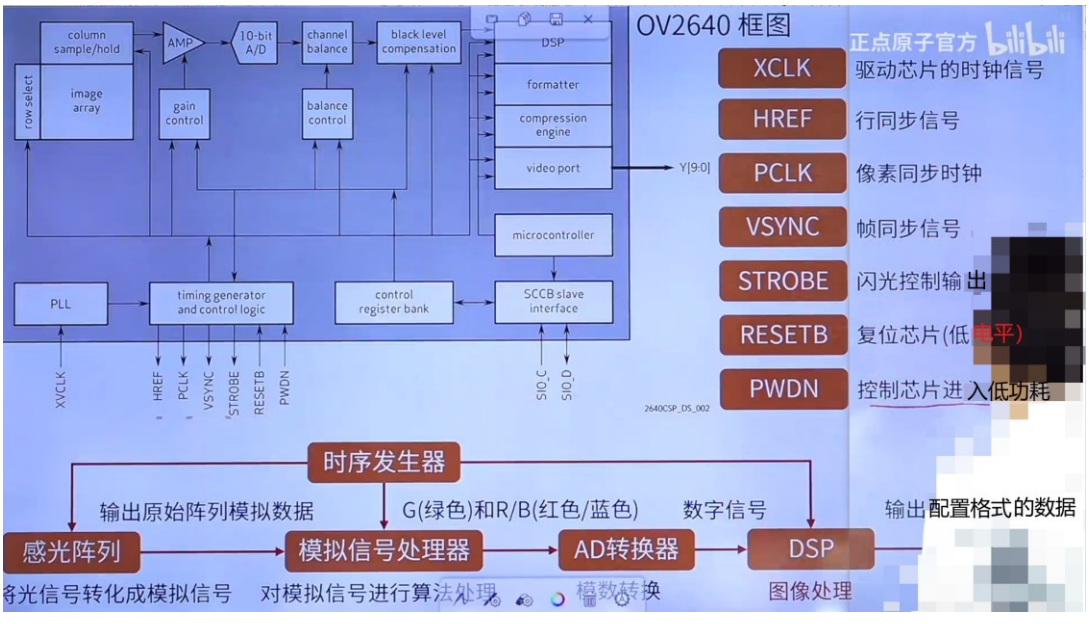
22.1 OV2640 和 OV5640 模块简介

22.2 ATK_OV2640 和 OV5640 模块特点

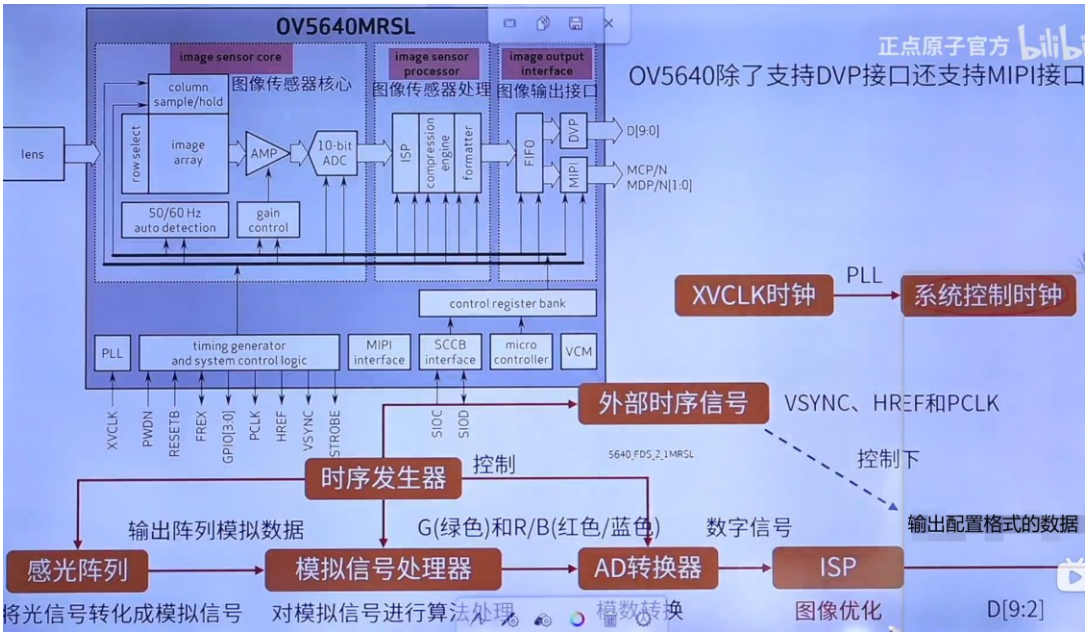


22.3 框图

22.3.1 OV2640 框图

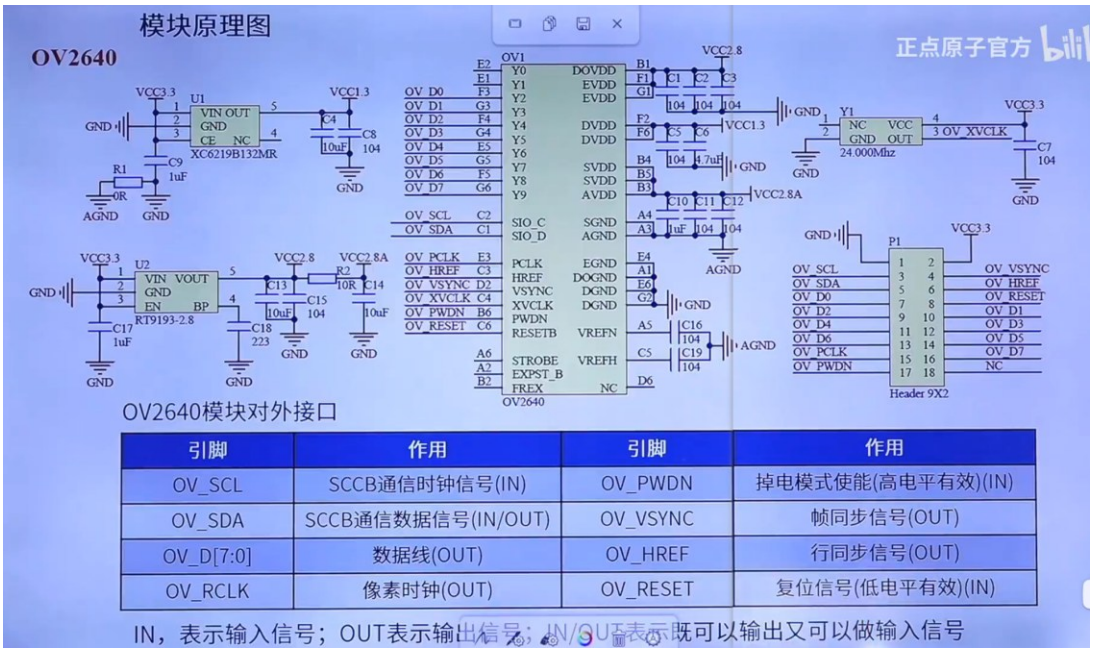


22.3.2 OV5640 框图



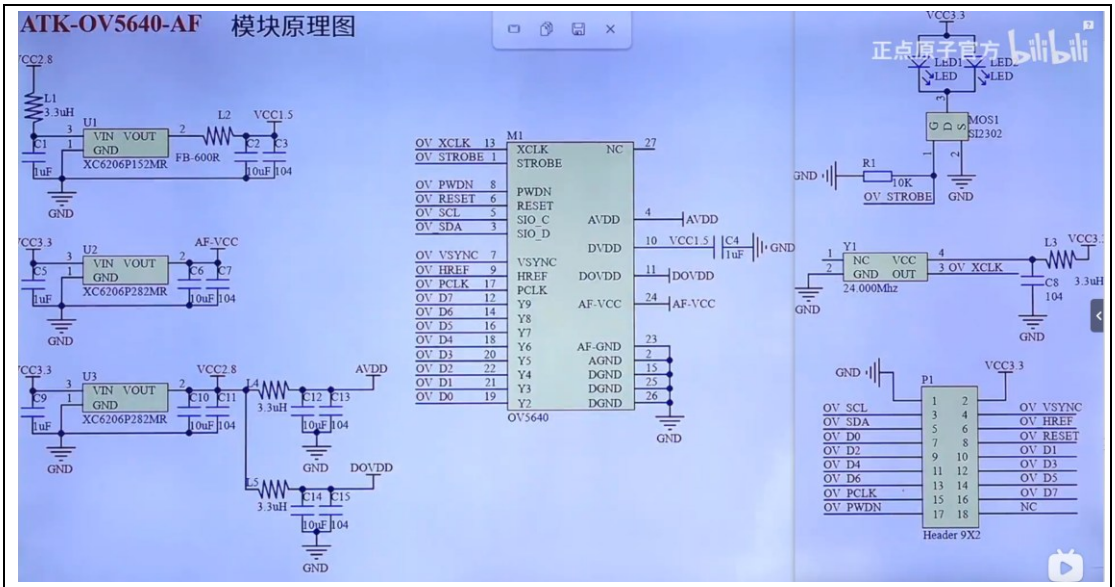
22.4 模块原理图

22.4.1 OV2640 原理图

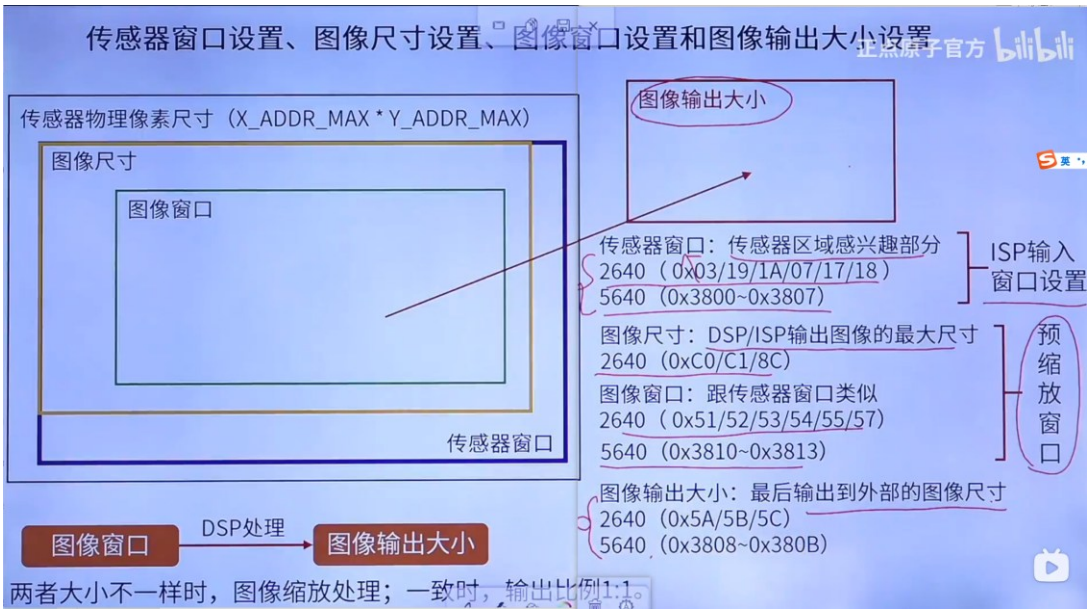


引脚	作用	引脚	作用
OV_SCL	SCCB通信时钟信号(IN)	OV_PWDN	掉电模式使能(高电平有效)(IN)
OV_SDA	SCCB通信数据信号(IN/OUT)	OV_VSYNC	帧同步信号(OUT)
OV_D[7:0]	数据线(OUT)	OV_HREF	行同步信号(OUT)
OV_RCLK	像素时钟(OUT)	OV_RESET	复位信号(低电平有效)(IN)

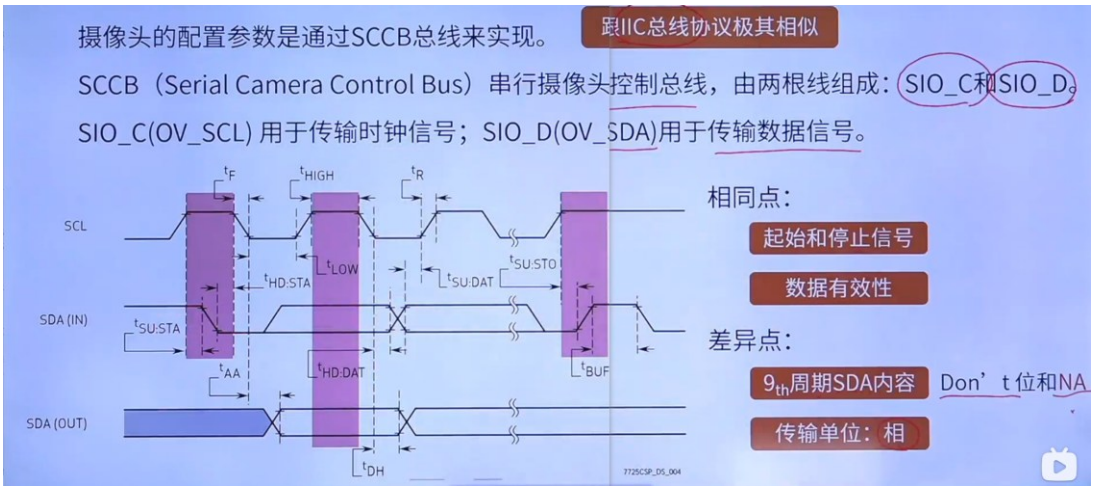
22.4.2 OV5640 原理图



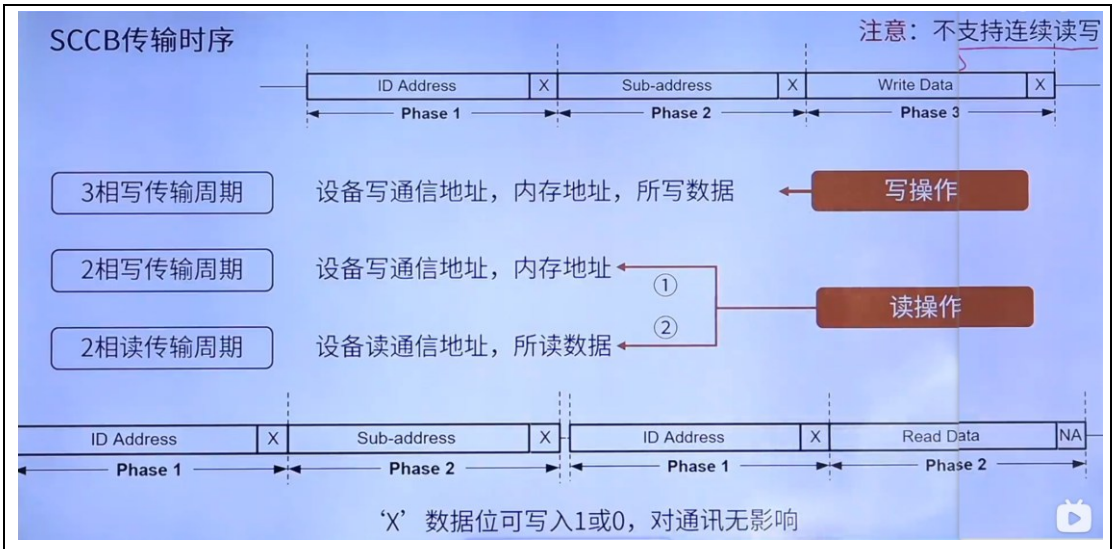
22.5 传感器，图像尺寸/图像窗口/图像输出大小设置



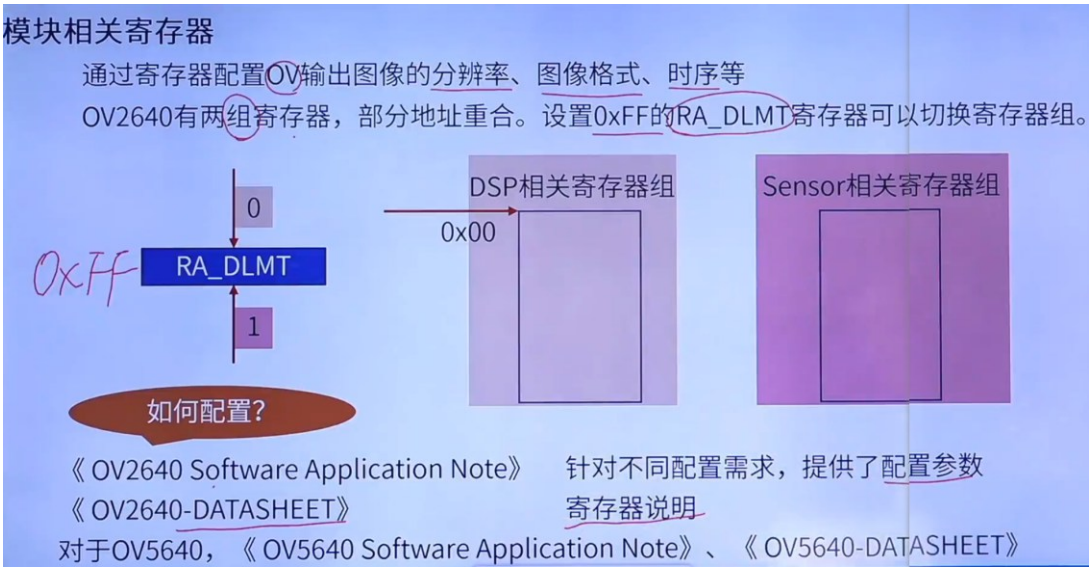
22.6 SCCB 通信协议介绍



22.7 SCCB 传输时序



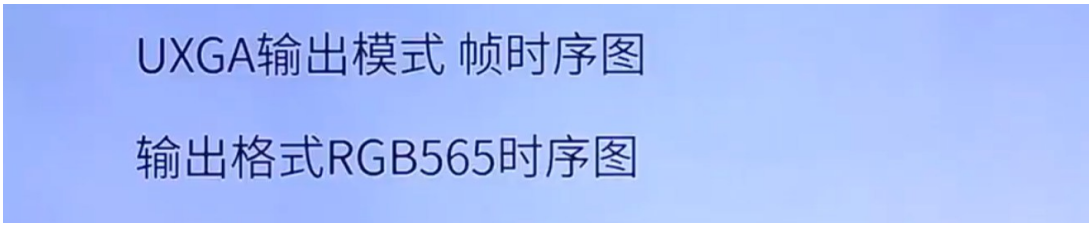
22.8 模块相关寄存器



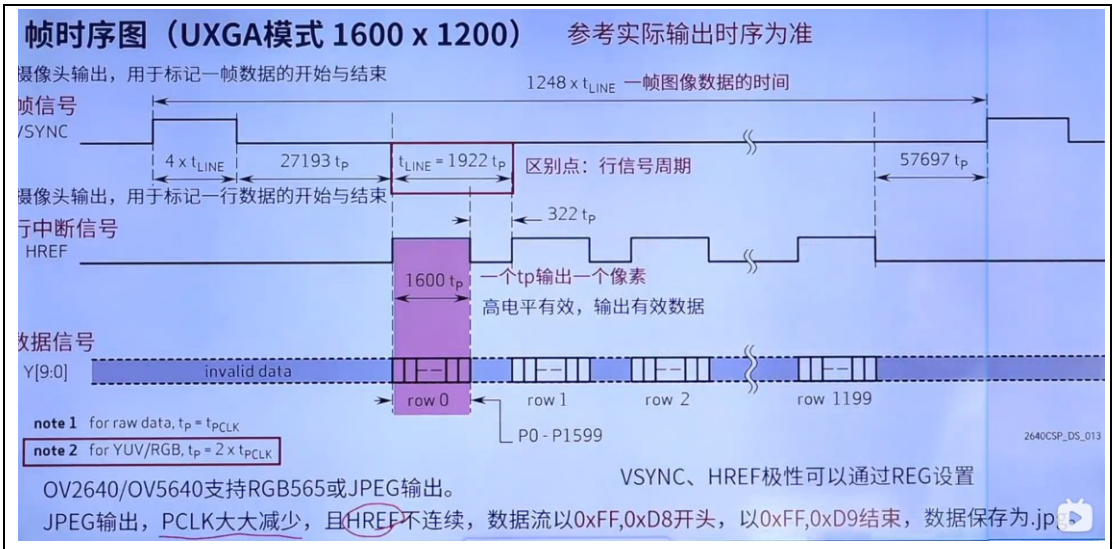
关键寄存器

寄存器功能	OV2640		OV5640
	DSP相关	Sensor相关	
设置图像格式	0xDA		0x4300
设置时序输出		0x15	0x4740
设置分辨率&复位		0x12	0x3008
设置传感器窗口		0x17/18/19/1A	0x3800~0x3807
设置图像尺寸	0xC0/C1/8C		0x3810~0x3813
设置图像窗口	0x51/52/53/54/55/57		
图像输出大小	0x5A/5B/5C		0x3808~0x380B
ID获取		0x0A/0B/1C/1D	0x300A/300B
组写			0x3212

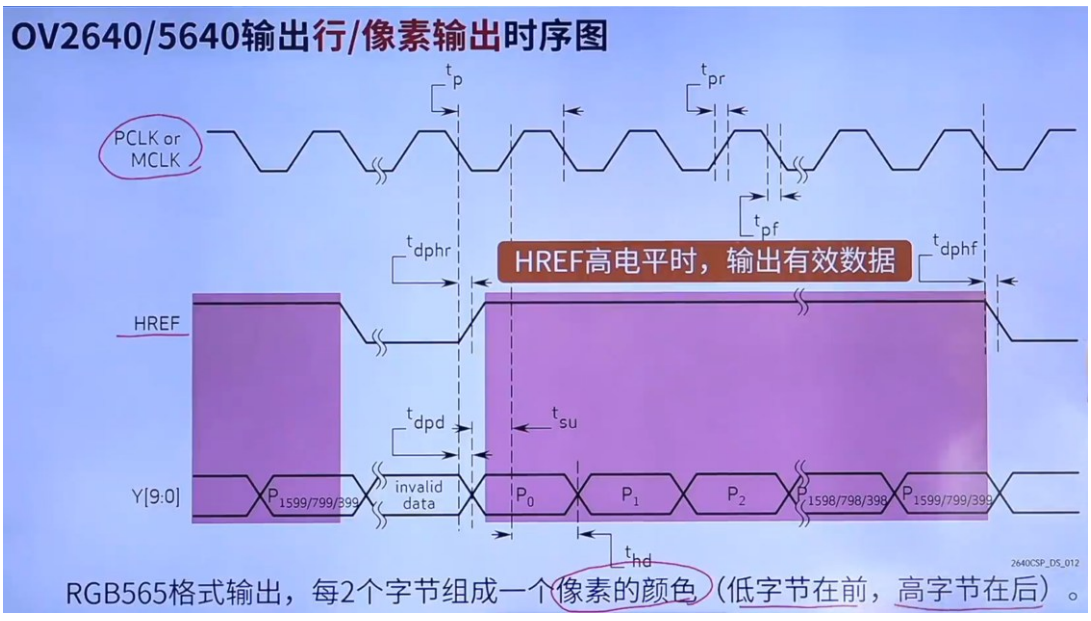
22.9 ATK_OV2640/OV5640 模块时序图介绍



22.9.1 帧时序图（UXGA 模式 1600 * 1200）

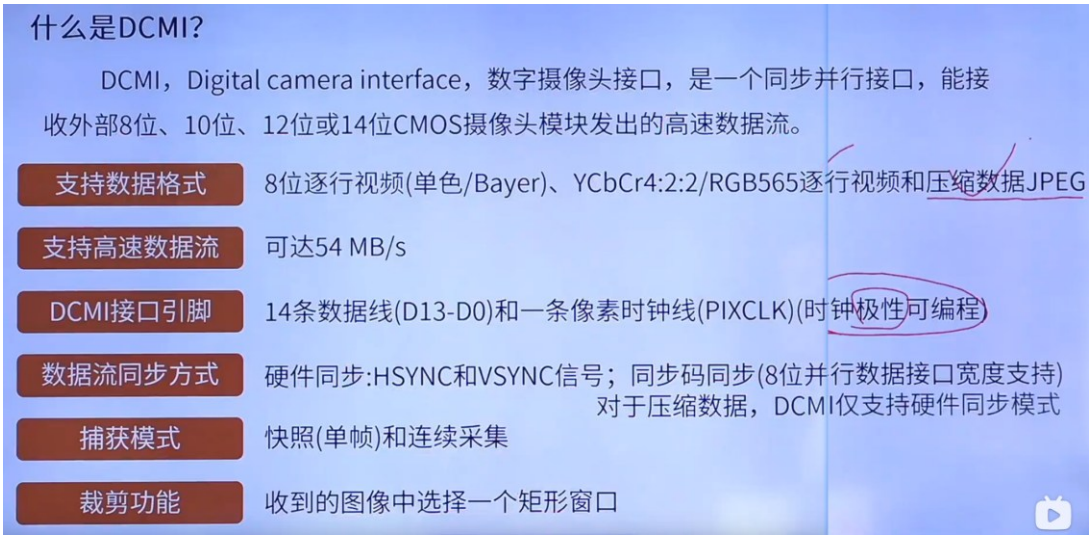


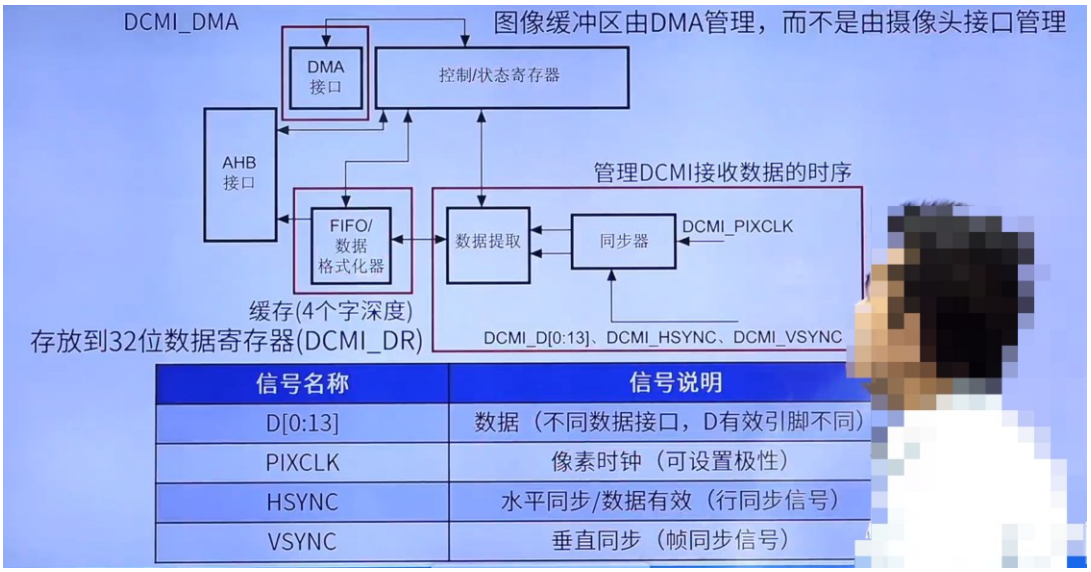
22.9.2 OV2640/5640 输出行/像素输出时序图



22.10 DCMI 外设介绍

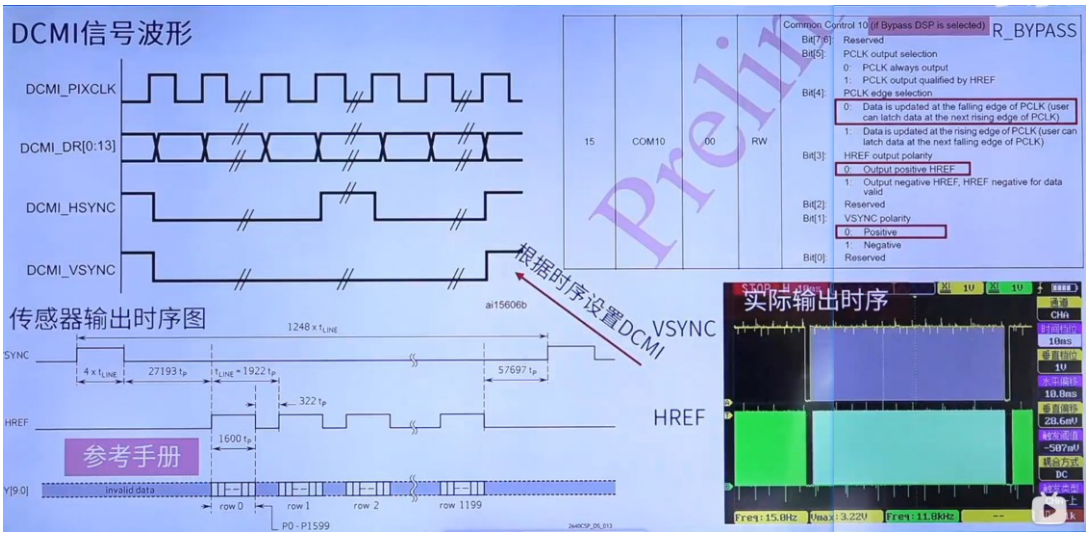
22.10.1 DCMI 介绍



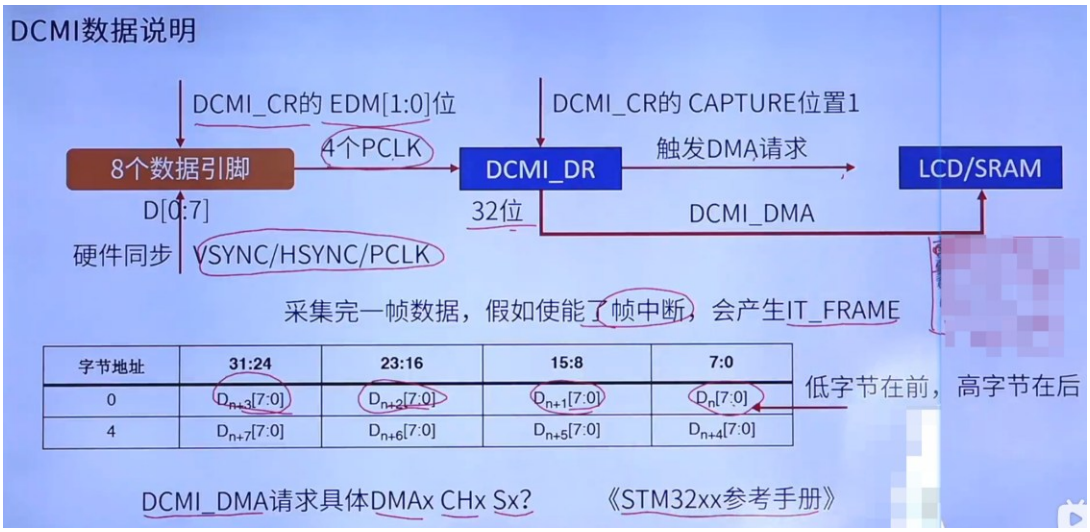


信号名称	信号说明
D[0:13]	数据（不同数据接口，D有效引脚不同）
PIXCLK	像素时钟（可设置极性）
HSYNC	水平同步/数据有效（行同步信号）
VSYNC	垂直同步（帧同步信号）

22.10.2 DMCI 信号波形



22.10.3 DCMI 数据说明



字节地址	31:24	23:16	15:8	7:0
0	D_{n+3}[7:0]	D_{n+2}[7:0]	D_{n+1}[7:0]	D_n[7:0]
4	D_{n+7}[7:0]	D_{n+6}[7:0]	D_{n+5}[7:0]	D_{n+4}[7:0]

22.10.4 DCMI 寄存器介绍(F4/F7/H7)

DCMI相关寄存器介绍 F4/F7/H7

寄存器	名称	作用
DCMI_CR	DCMI控制寄存器	用来设置DCMI相关配置参数
DCMI_IER	DCMI中断使能寄存器	用来使能DCMI相关中断（FRAME_IE）
DCMI_ICR	DCMI中断清零寄存器	用来清除中断状态（FRAME_ISC）
DCMI_MISR	DCMI屏蔽中断状态寄存器	用来返回相关中断的当前屏蔽状态（FRAME_MIS）

除了DCMI相关寄存器，还用到DMA相关寄存器(CCR/CR/LISR/FCR/PAR/M0AR/M1AR/NDTR)

DCMI控制寄存器 DCMI_CR

31–16	15	14	13–12	11–10	9–8	7	6	5	4	3	2	1	0
Reserved		ENABLE	Reserved	EDM	FCRC	VSPOL	HSPOL	PCKPOL	ESS	JPEG	CROP	CM	CAPTURE
		rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

ENABLE：使能DCMI　使能前，必须将其他配置设置好

FCRC[1:0]：帧捕获率控制　捕获所有帧，设置00即可

VSPOL：设置垂直同步极性，VSYNC引脚上数据无效时的电平状态　结合前面分析，设置0即可

HSPOL：设置水平同步极性，HSYNC引脚上数据无效时的电平状态　结合前面分析，设置0即可

PCKSPOL：设置像素时钟极性　上升沿捕获，设置1

CM：设置捕获模式　连续采集模式，设置0

CAPTURE：用于使能捕获　设置1　该位使能后，激活DMA，DCMI等待第一帧开始，然后生成DMA请求将收到的数据传输到目标存储器

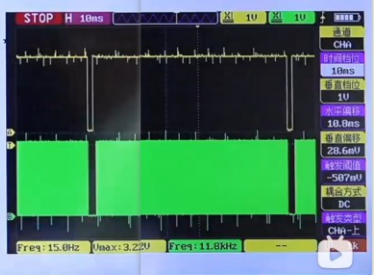
注意：完成其他配置后才设置！！！

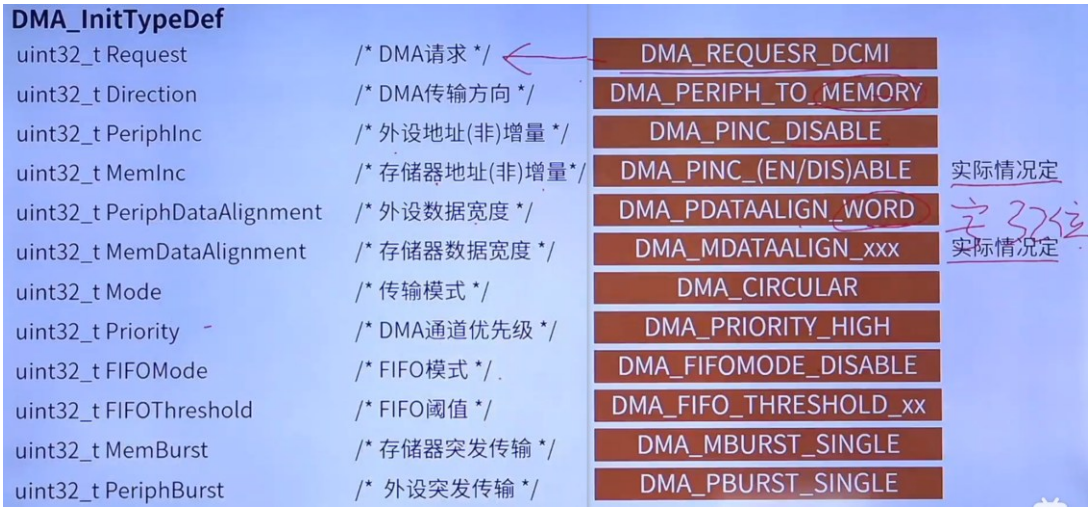
22.10.5 相关 HAL 函数

驱动函数	关联寄存器	功能描述
__HAL_RCC_DCMI_CLK_ENABEL(…)	RCC_AHB1ENR	使能DCMI时钟
HAL_DCMI_Init(…)	DCMI_CR	初始化DCMI
__HAL_DCMI_ENABLE_IT(…)	DCMI_IER	使能DCMI相关中断
__HAL_DCMI_ENABLE(…)	DCMI_CR	使能DCMI

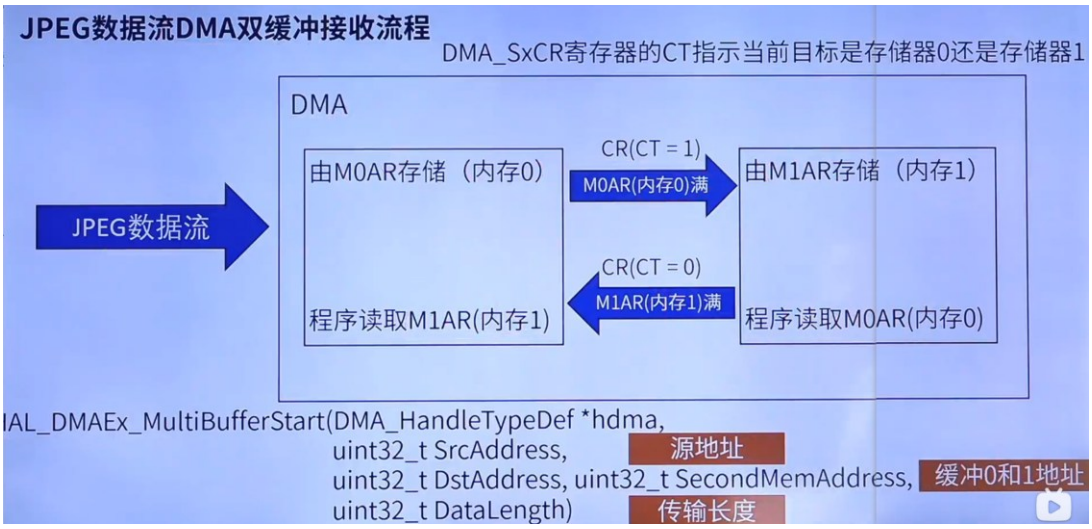
DCMI_InitTypeDef

```
uint16_t   SynchroMode        /* 数据同步模式(内嵌码同步/硬件同步) */
uint16_t   PCKPolarity        /* 像素时钟有效边沿 */      上升沿
uint16_t   VSPolarity         /* VSYNC的有效电平(无数据传输) */   低电平
uint16_t   HSPolarity         /* HSYNC的有效电平(无数据传输) */   低电平
uint16_t   CaptureRate        /* 捕获数据的帧率 */        全帧捕获
uint16_t   ExtendedDataMode   /* 数据线宽度 */            8位数据格式
uint16_t   JPEGMode           /* JPEG模式选择 */
...
```



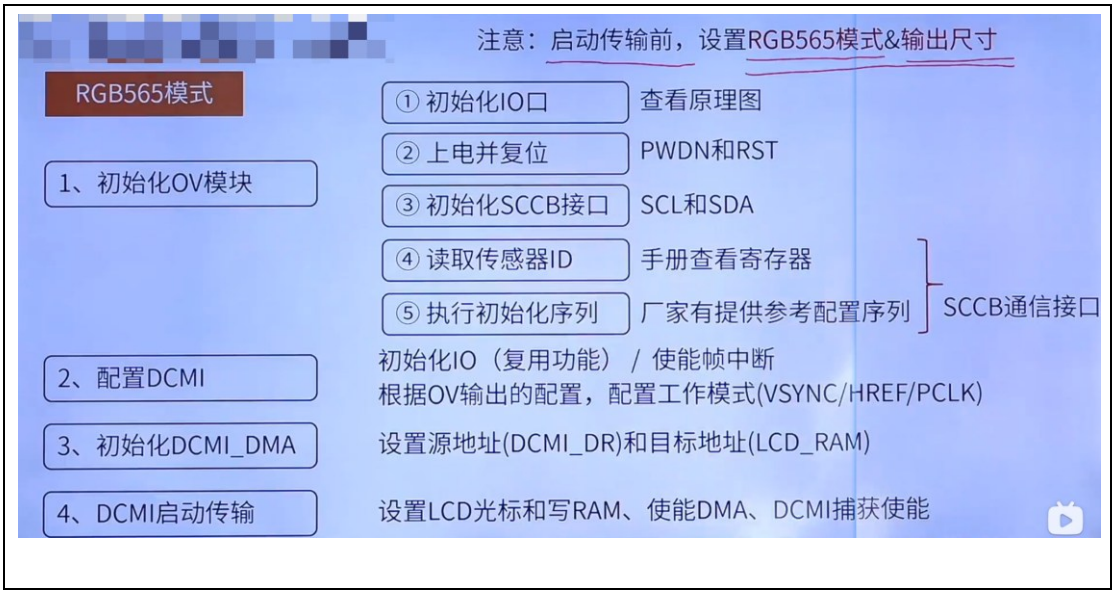


22.10.6 JPEG 数据流 DMA 双缓冲接收流程

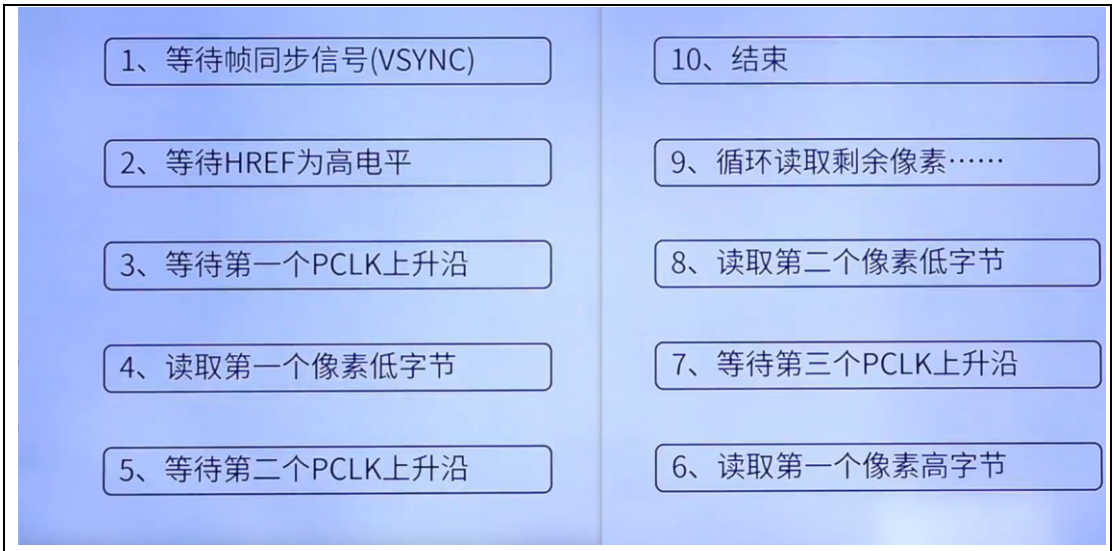


22.11 模块驱动步骤

22.11.1 RGB566 模式



22.11.2 读取图像数据的过程



22.11.3　JPEG 模式